
As per
Gujarat Technological
University
Syllabus

Compiler Design

The book covers:

- » Compiler Overview
- » Lexical Analyzer
- » Parsing Theory
- » Error Recovery
- » Intermediate Code Generation
- » Run time Memory Manager
- » Code Optimization
- » Code Generation

Compiler Design

**Rajesh Kumar Maurya
&
Kogent Learning Solutions Inc.**

Published by:



©copyright 2011 by Dreamtech Press, 19-A, Ansari Road, Daryaganj, New Delhi-110002.

This book may not be duplicated in any way without the express written consent of the publisher, except in the form of brief excerpts or quotations for the purposes of review. The information contained herein is for the personal use of the reader and may not be incorporated in any commercial programs, other books, databases, or any kind of software without written consent of the publisher. Making copies of this book or any portion for any purpose other than your own is a violation of copyright laws.

Limits of Liability/Disclaimer of Warranty : The author and publisher have used their best efforts in preparing this book. The author make no representation or warranties with respect to the accuracy or completeness of the contents of this book, and specifically disclaim any implied warranties of merchantability or fitness of any particular purpose. There are no warranties which extend beyond the descriptions contained in this paragraph. No warranty may be created or extended by sales representatives or written sales materials. The accuracy and completeness of the information provided herein and the opinions stated herein are not guaranteed or warranted to produce any particular results, and the advice and strategies contained herein may not be suitable for every individual. Neither Dreamtech Press nor author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

Trademarks : All brand names and product names used in this book are trademarks, registered trademarks, or trade names of their respective holders. Dreamtech Press is not associated with any product or vendor mentioned in this book.

ISBN: 978-93-5004-215-1

ISBN: 978-93-5004-962-4 (ebk)

Edition : 2011

Printed at: Himal Impressions, New Delhi

Preface

Modern computing machines have given an end user the independence of giving specifications and command in most abstract forms. Use of High-Level Languages (HLLs) has taken over the lower level languages. Although application programmers used HLL to write their programs and afforded to remain unaware about program translation and internal details of implementations, system programmers and designers need to know the detail of the program translation. Knowing the internals of program translation not only improves writing tools and system programs, but also enhances better programming skills that would lead to efficient code.

Compilers are special class of system programs that are translators, which convert the high-level program inputs to object code that can be directly executed. Compilation is a complex task, which involves many stages coordinating with each other before producing the final object code. Every stage or phase of the compiler has a significant role and contributes significantly in the process of program translation. Important phases of the compiler include lexical analysis, syntax analysis, semantic analysis, intermediate code generation, target code generation, and code optimization. In addition to these error detection and recovery, there are also other various phases during compilation.

In this book, we will study the role of modern compiler in program translation by exploring the design and implementation of each phase separately. Each of the chapters in the books is supplemented with adequate number of examples, illustration, and practice questions for better understanding of the concepts involved in the compilation of programs. Summary is provided at the end of each chapter to help the learner to refresh the concepts in most relevant manner.

The book is divided into 8 chapters, and the topics discussed in each chapter are briefly discussed as follows:

- ❑ **Chapter 1** gives the overview of the translation process and describes the working of a simple compiler. It also provides the differentiation between interpreter, assembler, and compiler. Further, it explores the concept of linker and loader and their role in compilation and execution process. The chapter introduces you to various types of compilers and explores the role of each phase of the compiler. Finally, it discusses in detail the pass structure of modern compiler.
- ❑ **Chapter 2** focuses on exploring the details of lexical analysis phase of the compiler and discusses about the concepts of input buffering, tokens, and lexemes. It explains in detail about design and implementation of lexical analyzers with suitable illustrative examples. The chapter ends with a discussion on regular expressions and finite automata and its optimization.
- ❑ **Chapter 3** concentrates on the study of syntax analysis or parsing phase of the compiler. In this chapter, we elaborate on concepts of top-down and bottom-up Parsing. It explains in detail about precedence parsing technique, LL parsers, and LR parsers with examples. It also explains the concept of ambiguous grammars and their effect in parsing in simple but effective manner. The later part of the chapter discusses on parser generators and automatic generation of parsers.
- ❑ **Chapter 4** presents concepts of syntax-directed translation and error detection. Then it discusses about syntax-directed definitions and translation schemes makes. The chapter explains the construction of syntax trees and bottom-up evaluation of attributed grammars. Later sections of the chapter elaborate on concept of error detection and recovery, types of errors, and methods of handling errors.
- ❑ **Chapter 5** contains topics describing the role of intermediate codes in compilation process and various forms of intermediate codes including three-address codes, Directed Acyclic Graphs (DAGs), and postfix forms. It also explains the concepts of attributed grammar and syntax-directed translation.

- ❑ **Chapter 6** concentrates on the study of run-time memory management in modern compilers. It discusses about source language issues, storage organization, storage-allocation strategies, and symbol tables in detail. Toward the end, it explains the methods of allocation and deallocation of program object at runtime.
- ❑ **Chapter 7** describes the code optimization phase of the compiler and brings out the significance of optimization in generation of good and efficient code. It explains in detail about global data flow analysis and a few selected optimizations, such as command sub-expression removal, loop invariant code motion, and strength reduction.
- ❑ **Chapter 8** finally describes the target code generation phase of the compiler and elaborates on design techniques and issues involved in target code generation. It describes the concepts of basic blocks and flow graphs, next-use information along with register allocation and assignment. Toward the end, it discusses about DAG and its use in code generation, optimization of final code, and topics of dynamic programming.

The book has been designed to enable beginners in the subject to graduate to advance level of understanding of the computing machine. The book follows algorithmic approach that enables better grasping of the concepts and systematic learning outcomes. It will be useful to students of computer science, engineering and technology, system programmers involved in core developments and other professionals involved in language translation.

Acknowledgements

This book is an outcome of a decade of teaching of the subject. Lecture notes produced during this duration have finally taken shape of this book. Through this book, I would like to acknowledge Dr. V. N. Joshi, Dr. M. A. Sanglikar, and Dr. Ambuja Salgoankar at the Department of Computer Science, University of Mumbai, for being catalyst in inspiring and extending their valuable support throughout their interaction over these years. I would also like to acknowledge all my student, colleagues, and friend who have always motivated me in designing this book.

Last but not the least, my family members have played the most important role in accomplishing this task of completing the book by allowing me to remain away on many important occasions.

Rajesh K. Maurya

Table of Contents

Preface	iii
Chapter 1: Introduction to Compiler	1
1.1 Overview of Translation Process	2
1.2 A Simple Compiler	3
1.3 Difference between Interpreter, Assembler, and Compiler	4
1.4 Overview and Use of Linker and Loader	7
1.5 Types of Compilers	7
Single-pass compilers	7
Multi-pass compilers	8
Load and go compiler	9
Link and load compiler	9
Optimizing compilers	9
1.6 Analysis of the Source Program	10
1.7 The Phases of a Compiler	11
Lexical Analysis	11
Syntax Analysis	12
Semantic Analysis	13
Intermediate Code Generation	14
Code Optimization	14
Code Generation	15
Symbol Table Management	16
Error Handling (Detection and Reporting)	16
1.8 Cousins of the Compiler	17
Preprocessors	17
Assembler	19
Loader and Linkers	20
1.9 The Grouping of Phases: Front End and Back End of Compiler	21
Front End of Compiler: The Analysis Phase	21
Back End of Compiler: The Synthesis Phase	22
1.10 The Pass Structure of Compiler	23

1.11	Compiler Writing Tools.....	24
	Summary.....	25
	Review Questions	26
	Multiple Choice Questions	28
Chapter 2: Lexical Analyzer.....		29
2.1	Introduction to Lexical Analyzer	29
2.2	Input Buffering	30
2.3	Specification of Tokens	31
	Tokens, Patterns, and Lexemes.....	32
	Elements of Token Design.....	33
	Chomsky Classification of Grammar.....	35
	Regular Expression.....	37
	Arden's Rule.....	38
2.4	Recognition of Tokens.....	39
	Transition Diagrams	41
2.5	A Language for Specifying Lexical Analyzer	42
2.6	Finite-State Automata	45
2.7	Finite Automata from Regular Expressions.....	47
	Converting NFA to DFA	49
	Building DFAs.....	53
2.8	Designing a Lexical Analyzer Generator.....	53
	Recognizer for Identifiers	54
	Pattern Matching Using NFA	55
	DFA's for Lexical Analyzers.....	56
2.9	Optimizing of DFA tion of DFA	57
	Summary.....	61
	Review Questions	62
	Multiple Choice Questions	64
Chapter 3: Parsing Theory		65
3.1	Types of Parsing: Top-Down and Bottom-Up Parsing Algorithms	66
3.2	Top-Down Parsing-Naïve Approach	67
	Top-Down Parsing Based on Prediction and Backtracking	68
	Recursive Productions	69
	Left Factoring	70

	Recursive Descent and Predictive Parser	72
	LL Parsers.....	72
3.3	Bottom-Up Parsing-Naïve Approach.....	78
	Shift-Reduce Parsers.....	78
	Precedence Parsers.....	80
	Simple Precedence Grammar and Parsing	81
	Operator Precedence Parsing	87
	Precedence Functions	94
3.4	LR Parsers.....	100
	LR Configuration and Tracing.....	102
	LR Parsing Algorithm.....	102
	Types of LR Parsers.....	104
	LR(0) Parsing Table	105
	SLR(1) Parsers.....	110
	LR(1) Parsers.....	113
	LALR(1) Parser	117
3.5	Parser Generators.....	120
3.6	Using Ambiguous Grammar	123
	Summary.....	127
	Review Questions	128
	Multiple Choice Questions	132
Chapter 4:	Syntax-Directed Translation and Error Recovery.....	133
4.1	Syntax-Directed Program Evaluation	134
4.2	Syntax-Directed Definitions	134
	Analysis of SDD	138
4.3	Construction of Syntax Trees.....	140
4.4	Bottom-Up Evaluation of S-Inherited Attributes	143
4.5	L-Attributed Definitions	143
4.6	Syntax-Directed Translation Scheme.....	145
4.7	Error Detection and Recovery	147
	Error Detection	147
	Error Reporting.....	147
4.8	Adhoc and Systematic Methods.....	148
	Adhoc Methods for LR Parsers.....	148
	Systematic Methods.....	150

Summary.....	150
Review Question.....	151
Multiple Choice Questions	152
Chapter 5: Intermediate Code Generation	153
5.1 Different Intermediate Forms	153
High-Level IRs	154
Low-Level IRs	159
5.2 Syntax-Directed Translation Mechanism and Attributed Mechanisms.....	163
Three-Address Code Generation for Assignment Statements.....	163
Three-Address Code Generation for Arrays	165
Three-Address Code Generation for Boolean Expressions.....	166
Translation of Conditional and Iterative Control Flow	170
Computing l-and r-Values	172
Code Generation for Function Calls	173
Translation of Mixed-Mode Expressions.....	174
Summary.....	175
Review Questions	176
Multiple Choice Questions	178
Chapter 6: Runtime Memory Management.....	179
6.1 Source Language Issues.....	180
6.2 Storage Organization	180
The Runtime Storage Organization.....	181
Allocation of Simple Data Objects	181
Activation and Activation Trees	182
Activation Records	184
Activation Record for Procedure Calls	182
6.3 Storage Allocation Strategies.....	185
Static Allocation	186
Stack Allocation.....	186
Heap Allocation.....	187
Handling Recursive Calls	187
6.4 Management of Variable-Length Blocks.....	189
Runtime Allocation Using Stacks.....	189
Runtime Allocation Using Heaps	190

6.5	Passing Parameters	191
6.6	Symbol Table.....	191
	Uses of Symbol Table.....	192
	Content of the Symbol Table	192
	Data Structures for Symbol Table.....	193
6.7	Language Facilities for Dynamic Allocation	195
	Dangling References.....	195
	Garbage.....	195
	Garbage Collection Algorithms	196
	Compacting Collectors	196
	Generational Collectors	198
	Advantages and Disadvantages of Garbage Collection	198
6.8	Dynamic Storage Allocation Techniques	199
	Managing Implicit Deallocation	199
	Explicit Allocation of Fixed-Sized Blocks	199
	Explicit Allocations of Variable-Sized Blocks	199
	Summary.....	199
	Review Questions	201
	Multiple Choice Questions	202
Chapter 7: Code Optimization		203
7.1	Classification of Compiler Optimization	203
7.2	Important Optimizing Transformations	204
	Compile-Time Evaluation.....	204
	Common Sub-Expression Elimination.....	205
	Code Movement.....	206
	Dead Code Elimination.....	207
	Loop Optimization.....	207
	Strength Reduction	210
7.3	Global Data Flow Analysis.....	210
	Available Expressions.....	211
	Reaching Definitions	211
	Data-Flow Equations	212
	Use-Definition (U-D) Chaining	212
7.4	Optimizations Based on Data-Flow Analysis	212
	Loop Detection	213

	Invariant Code Motion.....	213
	Common Sub-Expression Elimination for Data-Flow Analysis	214
	Dead Code Elimination.....	215
	Loop Unrolling	215
	Induction Variable Elimination.....	215
	Busy Expressions.....	216
	Summary.....	217
	Review Questions	218
	Multiple Choice Questions	220
Chapter 8:	Code Generation	221
8.1	Major Tasks in Code Generation	221
8.2	Design Issues of a Code Generator.....	223
	Input for Code Generators	223
	Output from the Code Generator	223
	Memory and Reference Management	224
	Choice of Instruction to Generate	224
	Register Allocation	227
	Choice of Evaluation Order	227
8.3	The Target Machine.....	227
	Addressing Modes	227
	Program and Instruction Cost	228
8.4	Runtime Storage Management.....	229
	Stack Allocation.....	229
	Caller Calling Callee.....	229
	Callee Returning	230
8.5	Basic Blocks and Flow Graphs	230
	Basic Blocks	230
	Flow Graphs	231
8.6	Next-Use Information.....	233
8.7	A Simple Code Generator.....	235
	Code Generation Algorithm.....	235
	Register Allocation and Assignments	235
	Machines Implementation of Conditional Jumps	236
8.8	The DAG Representation of Basic Blocks	237

8.9	Peephole Optimization.....	238
	Eliminating Redundant Loads and Stores	239
	Identifying Unreachable Instructions.....	240
	Simplify Algebraic Expressions	240
	Strength Reduction	241
	Use Faster Machine Instructions.....	241
	Elimination of Multiple Jumps	242
8.10	Generating Code from DAGs	242
	Rearranging Order of the Code.....	242
	A More Practical Code Generation from DAG.....	244
8.11	Dynamic Programming Code-Generation Algorithm	246
8.12	Code-Generator Generators	247
	Summary.....	247
	Review Questions	249
	Multiple Choice Questions	250
	Index	251

01

Introduction to Compiler

OBJECTIVES

- ↳ Learning about the overview of translation process
- ↳ Understanding and differentiating compiler, interpreter, and assembler
- ↳ Exploring the need and use of linker and loader
- ↳ Learning about structure of a compiler
- ↳ Understanding the significance of compiler phases and passes
- ↳ Gaining knowledge front end and back end of the compiler

A computer is a machine that performs calculations automatically and serves different purposes to different people. The speed and accuracy of calculation, and its behavior as black box for a common user have made it work more than just a calculator. It is capable of solving problems and manipulating data using a systematic approach. The computer is a stored-program machine based on Von Neumann architecture that carries out the desired operations. It requires us to write sequence of codes called *programs* to perform desired computations.

Programs are of two types--system programs and application programs. Programs that control some aspects of the operation of a computer directly or indirectly are called *system programs*. Operating system, compiler, interpreter, and similar utility programs belong to this category. System programming encompasses activity of programming to develop software that provides services to the computer hardware. On the other hand, *application programs* aim at developing software that provides services to the user. Word processor, banking, and inventory systems are examples of application programs. Programmers who develop system programs are known as system programmers. They have a great role in developing and successfully implementing the computing systems and require greater knowledge of functioning of computing machines.

Every program is written in a Programming Language (PL) that the computer can understand to give reliable output. PL can be classified into machine, assembly, and high-level languages. *Machine language* is the lowest in the hierarchy of PLs and most transparent to computers. This is what a computer in reality sees and understands. It uses binary sequences to understand any instruction or command. Initially, all programs were written in machine language. However, machine language being difficult to memorize, it was simplified to some extent using *Assembly Language* (AL), which uses symbolic letters or sequences of letters in place of large binary sequences. It is easier to remember these symbolic letters or sequences than to remember only ones and zeros. Assembly programs are more readable and easier to memorize, but they still require you to work with the computer itself. Modern PLs have come a long way and evolved themselves in specifying instruction in a more abstract and general manner. The developers aimed at developing PLs that are closer to the languages humans speak or write, so called High-Level Language (HLL). *HLL* is the highest in the language hierarchy and makes programming much easier because it allows you to describe the program in a more natural language. With higher and easier readability, an HLL command is equivalent to several commands in an AL and hundreds of binary bits of machine language. Most modern programming is done using HLL.

Although HLLs have made lives of application programmers quite simple, it invites a greater challenge for system programmers for developing language processors, especially language translators that make machines

understand the high-level specifications written in variety of languages including imperative languages such as Pascal and C; object-oriented language C++; functional language such as LISP; and visual languages including Visual BASIC and Delphi. A computer must translate these into machine-oriented representations.

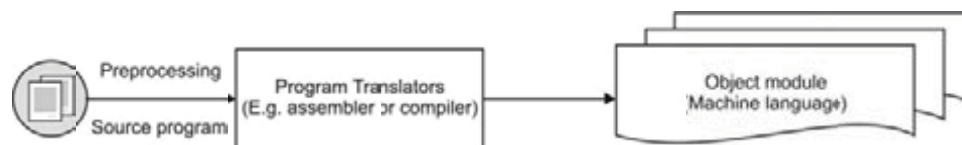
This chapter explores the process of language translation and introduces some important translators including compilers. It also explores the difference between interpreter, assembler, and compiler. Further, the chapter discusses about linker and loader and their use. It explores the types of compiler, wherein you learn about multipass compiler, load and go compiler, link and load compiler, and optimizing compiler. Then it presents the analysis of source program and the phases of a compiler. The chapter discusses in brief about the cousins of the compiler. Finally, you learn about grouping of phases, pass structure of compiler, and compiler writing tools.

1.1 Overview of Translation Process

The programs written in HLLs or the symbolic ALs are not executable; they must be translated into machine language before they are run. The computer itself performs this translation process under the control of a special program called *language translator*. Interestingly, programmers rarely write instructions using machine-level language due to the complexity involved with them. Rather, they write a set of instruction called the source program in one of the many symbolic PLs. This source program cannot be executed directly or operated upon by the computer until it has been translated into a machine language. The translation process uses the source program itself as input and produces (in most cases) a machine language equivalent called an *object program*, which is then executable.

The specification languages used for writing programs and machine languages differ in the level of abstraction. For execution purpose, a computer needs machine language that deals with memory locations and registers. These resources are never accessed or are difficult to do so, especially in HLLs. Since the level of abstraction differs from language to language and some languages are farther from machine code than others, we use language processors to bridge the specification and execution gaps. The language processing activities to bridge these gaps are performed by language translators. A *translator* is a language processor (program) that translates a given source program into an equivalent object program. The source program and the object program use Source Language (SL) and object language, respectively. In most cases, machine language forms the object language.

A *program translation model* uses language translators to bridge the execution gap by translating one (source) PL instruction(s) into another (target) PL instruction(s) without the loss of the intended meaning. The source program written in a language from the PL domain is translated into an equivalent target program in the machine or AL of the computer. The primary advantage of translation model is that the translated target program can be saved in a file and executed repeatedly. Note that a program must be retranslated following any modifications in the source PL. This model of execution is preferred when program modifications are least and the generated program follows the target machine structure very well. Assemblers and compilers are examples of such translators. A program translation model is illustrated in Figure 1:



▲ Figure 1: Program Translation

The translation process performed by the translators can be viewed as a series of phases, wherein each phase carries out some specific aspect of translation. During translation to final program in target language, the interfaces to these phases generate and work on intermediate forms of the source program and produce information for execution. Broadly speaking, the translation process decodes

the specifications in the SL and re-encodes their meaning onto the target language. It is an intensive language processing activity involving analysis and synthesis processes as described.

The *advantages* of the translator model are as follows:

- ❑ Enables efficient execution of machine language program generated by the translator
- ❑ Ensures that the object/binary program can be rerun many times without spending time in going through the source specification on each run

The *disadvantages* of the translator model are as follows:

- ❑ Prevents portability of the generated machine language programs to other systems
- ❑ Requires recompiling the source program following every modification in it

Output interface of the translator model is determined by its operating environment that consists of the operating system, runtime library, and target machine architecture. The operating system and the architecture determine the Object Module (OM) format used by linker and loader. The runtime library determines how certain functions are performed during execution. The compiler works on the principles of this model.

The next section discusses about a simple compiler.

1.2 A Simple Compiler

A *compiler* is a language translator that translates a high-level program into a low-level equivalent form, which the computer can more easily understand. The low-level output of a compiler is generally called object program. As a computer can only understand binary representations at the most basic level, the output of the compiler is expected to be in binary form. Earlier, compilers were simple and designed for doing simple computations. Later, these compilers evolved and became capable of doing complex operations and were able to handle large programs. Later versions of compiler in most cases generate assembly version of the original source programs, wherein the inbuilt assembler processes and generates the target object program in binary form.

A compiler is a translator of problem-oriented languages or machine-independent languages. Apart from translating the input program, it is also responsible for checking the lexical and syntactical compositions of the program; it checks the program before translating to make sure that it is grammatically correct. The translation is performed only on error-free programs. Detection of even a single error in either case stops the generation of output object program and reports the error. Unless the error is resolved, the output object file cannot be produced.

A *compiler* overcomes the semantic gap between PL domain and execution domain. Primarily, it has to perform the following tasks:

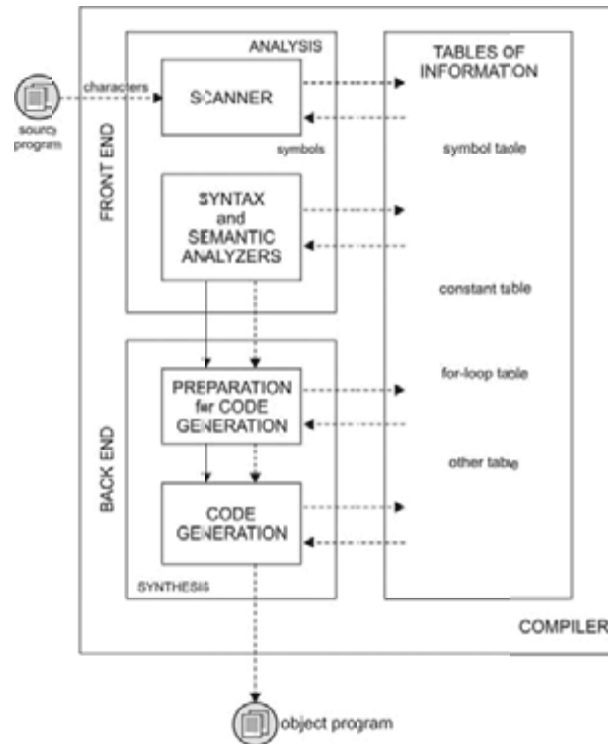
- ❑ Generate target program in the execution domain, keeping in mind the architectural details of the computer where it is implemented. It must preserve the meaning of specifications across translation from source to target language.
- ❑ Possess diagnostic features to recognize any violations in program specification at syntactic as well as semantic levels.
- ❑ Capable of generating compact and efficient target code and optimize the use of system resources.

In order to achieve the preceding task, the compiler goes through several phases, and passes and produces the object file that is linked and loaded for execution.

Generally, the task of a compiler can be divided into front end and back end (Figure 2). The front end of the compiler is responsible for doing the analysis and producing the information, while the back

end of the compiler performs allocations and synthesis of the programs using the information produced by the front-end.

The functions of each of these phases are described later in the chapter. Figure 2 shows the structures of a simple compiler:



▲ Figure 2: The Simple Compiler Structures

Let's explore the difference between interpreters, compilers, and assemblers in the next section.

1.3 Difference between Interpreter, Assembler, and Compiler

Assemblers, compilers, and interpreters are important language processors that are used nowadays. Evidently, when the input language is an HLL, such as FORTRAN, ALGOL, or C, and the object language is the AL or machine language of some computer, the translator is known as a *compiler*. Machine language is often called code; hence, the object program is referred to as object code. The translation of the source program into the object program occurs at compile time, while the actual execution of the object program occurs at runtime.

An *assembler* is a system program that transforms (translates) the source program written in AL into corresponding machine language of a computer. The use of AL imposes fixed format on program statements, thereby enabling easy analysis. The assembly program uses symbolic representation of machine language statements and hence is closer to machine language. Assembly statements generally contain no nested statements and blocks.

An *interpreter* is system software that translates a given HLL program into a low-level one, but differs from compiler and assembler. Interpretation is a real-time activity where an interpreter takes the program, one statement at a time, and translates each line before running it. In contrast to compiler and assembler, interpreters do not produce any target output program; instead, it translates the line and runs it. A successful translation of each line is followed by execution. The major difference between a compiler and an interpreter is that the interpreter does not produce an object program to be executed; it executes the source program itself. A pure interpreter will analyze a source program statement each time it is to be executed in order to discover how to perform the execution. This, of course, is very inefficient and not used very often. The usual method is to program the interpreter in two phases. The first phase analyzes the complete source program, much the way a compiler does, and translates it into an internal form. The second phase then interprets or executes this internal form of the source program. The internal form is designed to minimize the time needed to "decode" or analyze each statement in order to execute it.

Assembler and compiler are based on translation-linking-loading model, while an interpreter at its core reads a statement and then executes it, reads another statement and executes it, and so on. One must acknowledge that despite the different orientations in program processing and evaluation, they share some common features. The interpretation cycle shares close similarity with the instruction cycle of the Central Processing Unit (CPU). Hence, lexical, syntax, and semantic analyses are the same as in compilation since the analysis tasks only depend on the SL.

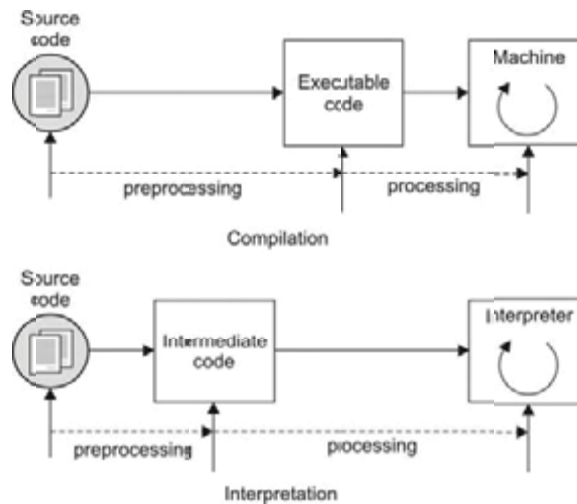
Note that a compiler is itself just a program written in some language -- its input is a source program and its output is an equivalent object program. The early day compilers were written in the AL of the computer itself. The use of HLLs for writing compiler reduces the amount of programming time and debugging time, and the readability of the compiler when finished. Languages are also available explicitly for compiler writing. Such Translator Writing Systems (TWS) are known as "compiler-compilers."

A brief comparison of assembler, compiler, and interpreter is presented in Table 1:

Table 1: Comparison between Assembler, Compiler, and Interpreter			
S. No.	Assemblers	Compilers	Interpreters
1.	Are language processors based on language translation-linking-loading model	Are language processors based on language translation-linking-loading model	Are a class of language processors based on interpretation model
2.	Generate object program as an output, which can be run independently from the source program written in AL	Generate object program as an output, which can be run independently from the source program written in SL	Do not generate any output program; rather, they evaluate the source program at each time for execution
3.	Program execution is separate from assembly process and performed only after the entire output program (object code) is produced	Program execution is separate from compilation and performed only after the entire output (object code) program is produced	Program execution is a part of interpretation and is performed on a statement-by-statement basis
4.	Object code executes independently and generally does not need the presence of assembler in the memory	Target program (object code) executes independently and does not need the presence of compiler in the memory	Exist in the memory during interpretation, i.e., it coexists with the source program to be interpreted

Table 1: Comparison between Assembler, Compiler, and Interpreter			
S. No.	Assemblers	Compilers	Interpreters
5.	Erroneous program will not generate object code	Presence of errors in the source program will prohibit generation of object code	Evaluate and execute program statement until an error is found
6.	Need re-assembly of the program for generating afresh object code after each modification in the source program	Need recompilation for generating afresh output program in target language after each modification in the source program	Independent of program modification issues as it processes the source program each time during execution
7.	Suitable for production environment for full utilization of system recourses.	Suitable for production environment	Suited for a program development environment
8.	Bounded to specific target machine with limited number of instruction set	Bounded to a specific target machine, but more flexible in program development than assembler	Made portable by carefully coding them in a higher level language
9.	8086 is AL for writing programs that are processed by Turbo Assembler (TASM) and Microsoft Macro Assembler 8.0 (MASM) assemblers	C and C++ are examples of PLs that use compilation model	Visual basic, LISP, and MATLAB [®] use interpreters

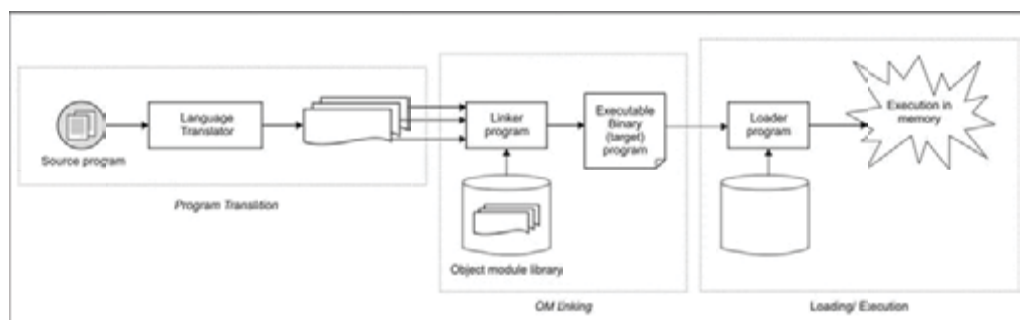
A schematic difference between compilation and interpretation models is shown in Figure 3:



▲ Figure 3: Difference between Compilation and Interpretation Models

1.4 Overview and Use of Linker and Loader

The translation model of execution of a program makes use of linkers and loaders to complete the execution purpose (Figure 4). A system program called *linker* processes the OM's generated by the translator and, along with information from the library, produces an executable form of the program called a binary program. The library of the OM helps in performing many runtime functions. The binary program is loaded into the memory for execution by another system program called *loaders*. Often, in modern computing environment, linker and loaders are clubbed together. Loading a program in the memory is called execution of a program. Figure 4 shows the translation, linking, and loading models of program execution:



▲ Figure 4: Translation, Linking, and Loading Models of Program Execution

Two important functions performed by loader/linker include relocating the OM's, i.e., allocation of load time address, such that they can co-exist in the memory and linking of the OM's to resolve mutual references. The different forms of the programs, namely, source form, OM, and the binary program, can be stored in the libraries of the computer for repeated use. An absolute loader loads the binary form of the program in the memory of the computer and organizes its execution.

Next, let's learn about the types of compilers.

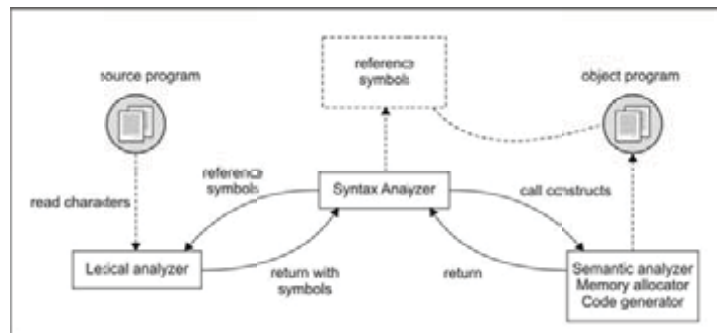
1.5 Types of Compilers

The compiler of modern age has improved significantly over last few decades. Generally, compiler can be classified based on the number of its passes, its simplicity in target code generation, and its optimization capabilities. The most evident types of compilers are explained as follows:

- ☐ Single-pass compilers
- ☐ Multi-pass compilers
- ☐ Load and go compiler
- ☐ Link and load compiler
- ☐ Optimizing compilers

■ Single-pass compilers

In general, compilation is a complex process that comprises many parts. In some simple compiler implementations (single-pass compiler), not all the parts need be used. A single-pass compiler implementation does not require the internal form of the program; it prepares and generates code by fusing the routines, especially with the semantic analyzer. A typical single-pass compilation scheme is illustrated in Figure 5:



▲ Figure 5: A Single-Pass Compilation Scheme

The syntax analyzer would call the lexical analyzer (scanner) when it needs a new symbol and calls a routine when it recognizes some construct. The single-pass translation performs semantic checking, storage allocation, and code generation for the construct before returning to the syntax analyzer (parser).

Compiling involves performing lots of work, and earlier computers did not have enough memory to contain one program that did all of the work. Therefore, compilers were split up into smaller programs, which each made a pass over the source (or some representation of it) performing some of the required analysis and translations.

Some of the advantages of using single-pass compilation are as follows:

- ❑ It simplifies the job of writing a compiler
- ❑ Single-pass compilers are generally faster than multi-pass compilers
- ❑ Many languages were designed in earlier decades including Pascal, which could be compiled in a single pass

Some of the disadvantages of single-pass compilation are as follows:

- ❑ All PLs are not structured so that they can be translated by a pass-pass compiler
- ❑ Use of complex programming skills and requirements
- ❑ Difficulty to perform several optimizations that are desirable to generate a high-quality code

■ Multi-pass compilers

It has been aforementioned that not all languages can be translated by a single-pass compiler. This can be attributed to the complex structure of the languages and program writing. Certain languages have feature that may require a compiler to perform multiple passes over the source program. One such feature is forward and intersegment references. For instance, a reference to a variable may be done in the program before the declaration of that variable. To handle such cases, the first pass would be required to collect information about declarations appearing after program statements that they affect. Subsequent passes would perform the required translation.

Multi-pass compiler generates machine code from its final pass. The lexical analyzer and syntax analyzers have significant role in compilation and provide the much needed information to generate final code. Syntax analyzers can be largely automated. The later phase of the compiler that includes semantic analysis, program preparation, and code generation are interdependent and must be designed carefully.

ALCOR Illinois 7090 compiler is an example of multi-pass compiler for IBM 7090-7040 systems that used four pass implementation. Another example of multi-pass compiler is IBM 360 FORTRAN IV H compiler, which has the passes presented in Table 2:

Table 2: Passes of IBM 360 FORTRAN IV H Compiler and the Task Performed by Them

Pass	Task performed
1	Does scanning and syntax analysis (produce internal source program and construct symbol table).
2	Comprises a set for three passes in itself. It processes simple statements, generates quadruples (operator, operand, operand, result), and performs runtime storage allocation.
3	Does code optimization, removes redundant operations, and performs loop optimizations. It also performs runtime register allocation and optimization of control statements.
4	Generates final code in target language.

Multi-pass compilers were designed with the objectives of optimization of source and target program and generation of efficient code.

■ Load and go compiler

A compiler that produces absolute code, which is executed immediately at conclusion of the compilation is called load and go compiler.

The advantages of load and go compiler are as follows:

- ☐ It can reduce the necessity for file communication between passes by using internal buffers that store the intermediate representations of the source program during the compilation passes
- ☐ It can get rid of the need for linking by producing absolute code that resides in the core
- ☐ It can be used during program development

■ Link and load compiler

Link and load compilers are compilers wherein the object code is transformed by a linking loader into absolute code.

The advantages of link and load compiler are as follows:

- ☐ It generates better absolute code compared to load and go compiler
- ☐ It can be used after program development is completed to minimize execution cost

The disadvantage of link and load compiler is that it suffers from increased compilation time.

■ Optimizing compilers

Optimizing compilers apply a series of transformations during the compiler passes and improve the space and time efficiency of the target code. Optimizing compiler compile itself to get a faster compiler. Peephole optimization and global optimization are two principle classes of optimization used in optimizing compilers. Peephole optimizations, which are local optimizations, produce a set of new instructions by combining or expanding on a small number of consecutive instructions, achieving space/time benefits in execution. Global optimizations are concerned basically with reordering, removing, or adding instructions to transform the structure of generated code to see the benefits. Importantly, an optimizing compiler must produce target programs that are semantically equivalent to their source versions.

The advantages of optimizing compilers are as follows:

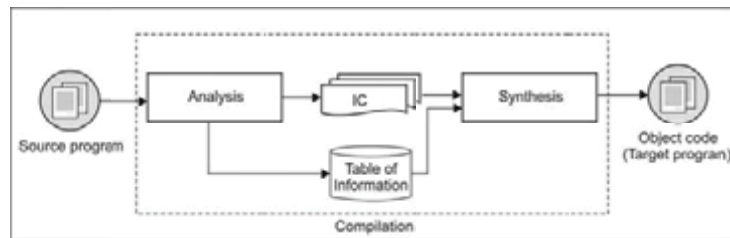
- ☐ Execution efficiency is enhanced by utilizing either less space, time or both
- ☐ Full utilization of underlying architectural benefits including registers and addressing modes

The disadvantage of optimizing compilers is that larger development time is required to develop such compilers.

Other compiler classifications include source-to-source compiler, stage compiler, and just-in-time compiler. The input and output of the *source-to-source compiler* are HLL. An example of such implementation is parallelizing compiler. *Stage compiler* compiles to AL of a theoretical machine, while the *just-in-time compiler*, used by Smalltalk, Java systems, and Common Intermediate Language (CIL) from Microsoft, deliver bytecode that are compiled to native machine code just before the execution.

1.6 Analysis of the Source Program

During the analysis stage, the compiler breaks up the source program into constituent pieces, imposes a grammatical structure on them, determines the meaning, and generates an Intermediate Representation (IR) of the source program. On the other hand, the synthesis stage is concerned with the construction of the desired target output program from these IRs. The IRs are also referred to as Intermediate Code (IC). Of the two stages, synthesis requires the most specialized techniques. Figure 6 shows the basic analysis-synthesis model of compilation:



▲ Figure 6: The Analysis-Synthesis Model of Compilation

During *analysis*, the specification for operations written in the source program is determined and recorded/translated into a suitable intermediate form to simplify the rest of the activities in the compilation process. As aforementioned, analysis breaks the program into pieces and creates an IR of the source program. This stage also collects information about the constituents of the source program and stores it in a data structure called a symbol table, which is passed along with the IR to the synthesis stage. The intermediate forms may be stored in a variety of data structure, including table organization and hierarchical structure such as tree. Basically, analysis is altogether concerned with gathering information about the constructs, structure, and compositions of the source program and preparing them for use in the synthesis stage. In general, the analysis of source program consists of the following three steps:

1. Determining the lexical constituents in a source program (lexical analysis)
2. Determining the structure of the strings in the source program (syntax analysis)
3. Determining the meaning of the source string (semantic analysis)

The syntax and semantics of a language are defined in its standards that define a unique meaning for every source string for a given language. Analysis is an important activity in language processors, including pretty printer, interpreters, structure editors, and static checkers. *Structure editors* are useful to fill in syntax units while typing; *pretty printers* assist in program development by highlighting the comments and indenting the code; *static checkers* discover programming bugs without actually running the program; and *interpreters* just perform the requested operations (e.g., MATLAB). All such language processing activities are possible in the analysis-synthesis model of compilation.

Now that you have learned about the types of compilers, let's move on and learn about the phases of a compiler.

1.7 The Phases of a Compiler

Conceptually, a compiler operates in *phases*, each of which transforms the source program from one representation to another. A typical decomposition of a compiler is shown in Figure 7:



▲ Figure 7: Phases of a Compiler

Each of the *front* and *back ends* is itself divided into multiple *phases* with each phase receiving its input from the output of the previous phase. Some of the phases in the sequence of compilation can change the representation of the input, while it remains unchanged in some phases.

In practice, some of the phases may be grouped together and the IRs between the grouped phases need not be explicitly constructed. The first three phases, namely, lexical, syntax, and semantic analyses, form the bulk of the analysis portion of a compiler. Two other activities, that is, symbol-table management and error handling (Figure 7), continually interact with the phases during the compilation process. Invariably, symbol-table manager and error handler are also phases of the compiler, which are accompanied with all other phases. We briefly describe each of these phases in the following subsections.

■ Lexical Analysis

Lexical analysis, also known as *linear analysis* or *scanning*, processes characters from left to right to group the character stream into meaningful units called *lexemes*, which are mapped into *tokens*. The input to this phase is a character stream and the output is a series of tokens that represent the building blocks of a program. The tokens may be the keywords or some elementary constructs of the PL and are also called uniform symbols. In other words, this phase extracts and identifies the lowest level

lexical elements from a source stream and removes non-grammatical elements from the stream, i.e., spaces and comments. These lexical elements are as follows:

- ❑ **Reserved words:** Include for, if, switch, print, and repeat
- ❑ **Identifiers:** Include strings such as “i,” “j,” “table,” and “age”
- ❑ **Constants/numeric literals:** Include 3.14159, 17, and “%d\n”
- ❑ **Punctuation and operator symbols:** Include “,” “(,” “),” “+,” “*,” “/,” and “^”

For the expression

```
result = cost - rate * 66
```

the lexical analysis groups it into the lexemes `result`, `=`, `cost`, `-`, `rate`, `*`, and `66`, which form tokens of the source inputs. For practical reasons, token is represented by a pair <token-name, attribute-value>. For the preceding example, lexeme “`result`” could be mapped to a token such as <id,#1> or similar variants, where `id` represents a class known as identifier and #1 is the index of the entry for “`result` in the symbol table”; similarly, other lexemes result into tokens as output of the phase.

This phase takes each of the characters in turn, and as soon as it recognizes a legitimate token, it reports to the next stage of the process. Non-significant blanks within the inputs are normally removed during scanning with the exception of blanks inside strings, which are handled separately. Each token has a label and a type, so a variable “`cost`” in a program would have the label corresponding to its name (“`cost`”) and type “variable name.” We use regular expressions to define token strings which are recognized by lexical analyzers. Lexical analyzer is also known as *scanner* and can be easily implemented as finite-state machines. Lex and flex are scanner generator programs that recognize lexical patterns in the input text.

■ Syntax Analysis

Syntax analysis is all about recognizing the structural composition of the PL. Often referred to as *hierarchical analysis* or simply *parsing*, it involves grouping the tokens of the source program into grammatical phrases that are used by the compiler to synthesize output. Syntax analysis identifies and notifies the syntactic errors in the program once the program has been broken into tokens. It is governed by the rules of grammar and language theory and recognizes valid grammatical phrases. Many compilers try to guess and suggest the possible errors and their cause committed by the programmers.

The hierarchical structure of the source program can be represented by a parse tree, as shown in Figure 8:



▲ Figure 8: Syntactical Structure of Source Program Using Parse Tree

The parse tree is based on data structures such as trees and represents the syntactical constructs, which define the rules of precedence for operation during recognition of valid and erroneous constructs used in the program.

In general, if a syntactic unit can be recognized in a linear scan, we convert it into a token during lexical analysis. It must be noted that more complex syntactic units, especially recursive structures, are normally processed during syntactic analysis (parsing), as lexical analysis may not be capable enough in doing so. Lexical constructs do not require recursion, while syntactic constructs often do. Context-Free Grammars (CFGs) are formalization of recursive rules and are used to determine the syntactic structure of the program constructs.

Syntax means grammar, and the syntax analyzer in a compiler checks that the right tokens appear in the right order to make grammatically correct instructions. For instance, in C++, the instruction `abc--;` is syntactically correct, but the instruction `--abc;` is not. The order of the tokens is wrong. Identifiers (such as user-defined variables) can be recognized easily in a linear scan. These identifiers are tokenized during lexical analysis; on the other hand, their processing order is determined during syntax analysis, wherein they use defined rules of grammar for deciding the order of operations. Yacc and Bison are tools for generating parsers: programs that recognize the grammatical structure of programs. Bison is a faster version of Yacc.

■ Semantic Analysis

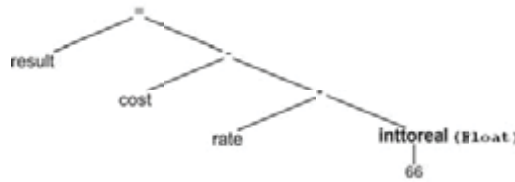
The semantic analysis phase is all about interpreting and determining the meaning of the constructs recognized in the syntax analysis phase. The word semantics refers to meaning, and the semantic analyzer checks the meaning of the program. This phase checks the source program for semantic errors and gathers ‘type information’ about the program elements for subsequent use in the code-generation phase. It comprises action routines that help to create intermediate form of the sources program and add information to the identifier table. Semantic analysis uses the hierarchical structure determined by the syntax-analysis phase to identify the operators and operands of expressions, and calls the appropriate action routines. The main feature of semantic analysis is to examine that variables have been declared and type check the expressions. For example, `abc--;` may be syntactically correct; however, if the variable `abc` has not been declared, then it is semantically incorrect.

An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands. Every operation has some rules of operation and the compiler must account for it. In this phase, the compiler checks if each operator has operands that are permitted by the SL specification and, if required, performs type conversion and type casting operations before evaluating the source specifications. For example, many PL implementations may need to convert the integer to real before performing the actions. Syntax-directed translation activities are the most important outcome of this phase.

For example, let’s again consider the following expression:

```
result = cost - rate * 66
```

If `rate` is a float, the integer literal `66` should be converted to a float before multiplying. Type float is also known as real in some modern languages. Figure 9 shows the type conversion during semantic phase:



▲ Figure 9: Type Conversion during Semantic Phase

The various distinct actions to be performed by this phase can be summarized as follows:

- ❑ Checks definition of identifiers and ascertains that the usage is correct
- ❑ Determines the operators and operands of expressions
- ❑ Disambiguates overloaded operators
- ❑ Translates from source to IR

The semantic analyzer not only checks variable declarations and scope, but also determines whether the program has entered or left loops or subroutines and whether classes are accessed correctly. Interestingly, syntax and semantic phases are only logically distinct phases. Most compiler implementation often view these two as companions, wherein syntax phase recognizes syntactic constructs that have an associated meaning, while semantic phase interprets the precise meaning and translates then into more simplified IRs.

■ Intermediate Code Generation

Followed by the syntax and semantic analyses, most compilers generate an explicit IR of the source program. This form of generating codes is an idealized machine version of the semantic analysis for an abstract machine. IC generation in most compilers would initially assume that the target has an unlimited number of registers, which are used for any operation. Further assumption that is adopted is machine operations would take (up to) three operands: two sources and one target. This IR should have two important properties: it should be easy to produce and easy to translate into the target program.

Most common representation is *three-address code* in which all memory locations are treated as registers, and most instructions apply an operator to two operand registers and store the result to a destination register. For example, the simple expression $a=b+c*d$ generates the following three-address code (assuming no type conversion required here):

- ❑ $T_1 = c*d$
- ❑ $T_2 = b+T_1$
- ❑ $a = T_2$

Note that a , b , and c are recognized and maintained as identifiers and T_i is the temporary memory assignment. As per the requirement of execution, the complex expression is split into series of operations. Other IRs include *polish notations*, *quadruple*, *triple*, *indirect triple*, and Directed Acyclic Graph (DAG) representations.

■ Code Optimization

The technique that a compiler can employ in an attempt to produce a better language program than the most obvious for a given source program is referred to as *code optimization*. Code optimization phase improves the IC to make it run faster. For a source program, there can be infinitely many object programs that implement the same computation, i.e., they produce the same output when presented with the same input. This phase tries to generate optimized code, satisfying the essential principles of

optimization. The optimization must be corrected, that is, preserve the meaning of the compiled program, improve the performance of many programs kept reasonable, and the engineering effort required must be manageable.

The quality of the program is measured by the size of the program (usually for small program) and running time of the program (for large programs). The objective of optimization is to reduce one or both of them. Optimization techniques are generally applied after syntax analysis, usually both before and during code generation. There are two categories of optimization, namely, machine-independent and machine-dependent optimizations. Both the techniques consist of detecting patterns in the program and replacing these patterns by equivalent but more efficient constructs. As these patterns may be local or global, the replacement strategy may be machine dependent or machine independent.

Machine-independent optimizations are usually IR optimizations. They are performed before code generation and do not require the use of internal machine registers. On the other hand, *machine-dependent* optimization involves the registers and requires the machine architecture to be known. Machine-independent optimization is performed on the code generated by inbuilt assemblers. Machine-dependent optimization is also sometimes referred to as assembler optimization. Constant folding, common sub-expression elimination, strength reduction, loop optimization, and code movement are examples of machine-independent optimizations. Optimization involving improving register usage and allocation and choice of appropriate addressing mode are some issues addressed by machine-dependent optimization.

■ Code Generation

Code generation, being the final stage, takes the IR (such as three-address code) and converts it to the object code in target language. This phase carefully picks memory locations for variables and allocates registers for them. Machine-dependent code optimization may also be performed on the generated code to produce more efficient target program (object files). These object files are then linked by linker programs and loaded into the memory by loaders (often relocating loaders) for execution.

For example, the following sequence of three address code:

$$\begin{aligned} T_1 &= c * d \\ T_2 &= b + T_1 \end{aligned}$$

may generate final code (some arbitrary formats are used here and no specific machine architecture is used here for instance), shown as follows:

```
MOV R2, c
MUL R2, d
MOV R1, b
ADD R2, R1
MOV R1, T1
```

Note that the example variables b, c, and d represent identifiers and have been used here for simplicity. This code moves the contents of the address variable c into register 2 and then multiplies it with the content of address variable d. The third instruction moves the content of address variable b into register 1 and the fourth statement adds to it the value previously computed in register 2. Finally, the value in register 1 is moved into the address T₁.

Code generation phase also determines whether the intermediate results are to be preserved. If yes, then what would be the location to store these results? These intermediate results can either occupy memory or some machine registers. It requires familiarity with target architecture with the knowledge of instructions and addressing modes to select the most efficient instruction.

■ Symbol Table Management

The essential function of object code generation by a compiler cannot be achieved unless it records the identifiers used in the source program and collects information about the various attributes of each of these identifiers. These attributes provide the much needed information to generate the efficient target output code. They contain information such as storage allocated for an identifier, its type, and its scope, and in the case of procedure names, the number and types of its arguments, the method of passing each argument (e.g., by reference), and the type returned, if any.

A compiler records all such information in a special data structure called *symbol table* with fields for the attributes of the identifier. The data structure allows us to find the record for each identifier quickly and store or retrieve the data from that record. It is contributed and used by almost every phase of the compiler.

Figure 7 reveals that symbol table is fairly used by every phase of the compiler. As soon as an identifier in the source program is identified by the scanner, an entry is made for the identifier in the symbol table together with its associated attributes. However, it is difficult to determine all of the attributes of the identifier during lexical analysis. The later phases of the compiler update information about identifiers into the symbol table and then use this information in various ways.

A phase of compiler uses the information stored in the data structure and writes or updates the same. For example, when doing semantic analysis and IC generation, we need to know the type of identifier. The type may either be a simple variable, a keyword, or a program name. This is useful in checking that the source program uses them in valid ways and generates the proper operations on them. The code generator typically enters and uses detailed information about the storage assigned to each of the identifiers. Let's understand this through the following example:

Example 1: Consider a code snippet that declares variables during a programming task. Table 3 shows the partially filled symbol table for the following code sequence:

```
int a;
float b,c;
b=a+b;
```

Table 3: Partially Filled Symbol Table					
Entry No.	Symbol	Type	Length	...	Address
1	a	Integer	2	...	...
2	b	Float	4	...	...
3	c	Float	4	...	...
4	a*	Float	4	...	...
5	T ₁	Float	4	...	...

The entries in the symbol table contain information about variables as encountered during the language processor pass. It contains information such as name, type, length, scope, and address of the variables. Note that all operation should confirm to rules by syntax-directed translation and semantics of the operations. Therefore, the compiler typecasts variable a to a* of type float, which could be the requirements of semantic translation. Moreover, new temporaries may be introduced and assigned intermediate results, which also have entries in the symbol table.

■ Error Handling (Detection and Reporting)

Errors are inevitable and each phase of the compiler can encounter errors. Therefore, we need each phase to detect these errors and somehow deal with them if possible. Before generating the final

object code, the compiler must detect and report all errors present in the source specifications either at lexical, syntactical, or semantic level in the program. The compilation can proceed and allow further detection of errors in the source program, but does not generate final object code until the entire program is error-free in every aspect. A compiler that stops when it finds the first error is not as helpful as it could be; it should find the presence of all erroneous specification in the program.

The syntax and semantic analysis phases usually handle a large fraction of the errors that are detectable by the compiler. The lexical phase can detect errors in cases where the characters remaining in the input do not form any token of the language. Syntax analysis can identify the errors that result due to violation of structure rules (syntax) by the token streams in the language. During semantic analysis, the compiler tries to detect constructs that may have the right syntactic structure, but do not adhere to any meaningful operation. For example, adding two identifiers, one of which is the name of an array and the other the name of a procedure, will result into semantic error.

1.8 Cousins of the Compiler

Other than compiler, there are many other system programs that assist in complete translation of the source program to the target code. These system programs are known as cousins of compiler. Each of these performs specialized functions and makes the intermediate form suitable for efficient target program. The most popular cousins of compiler are as follows:

- ❑ Preprocessors
- ❑ Assemblers
- ❑ Linker and loaders

We describe them in brief in the following subsections.

■ Preprocessors

Source programs written by the programmers are rarely in the required form that can generate the output code. Before an attempt is made to generate the target code, it must be made suitable in its form and structure so that the task becomes easier. This is achieved through a special program called a *preprocessor*. It is a program that processes its input data to produce output that is used as input to another program.

The output from the preprocessor is known as preprocessed form of the input data, which is then used by subsequent programs translators such as compiler. These preprocessor programs are executed before the actual program translation activity begins (including compilation and assembling). The preprocessor therefore understands and assimilates all the directives used in the program before attempting to generate the code for the statements.

Preprocessor may perform various functions depending on the kind of input to it. Some of the important preprocessor functions are explained as follows

Macro processing

A *macro* is a rule or pattern that defines how a certain sequence of characters in the input is mapped to an output sequence according to a defined procedure. It is single-line abbreviations for groups of instructions. Macro instructions are nothing but units of specifications that are used for program generation through expansion. For every occurrence of this one-line macro instruction within the program, the instruction must be mapped by the entire block. The process through which a macro is transformed into a specific output sequence is known as macro expansion.

Macros can be used in HLL as well as at AL level. An example of macro definitions is as follows:

```
#define identifier replacement_strings
```

During the initial program traversing, as soon as the preprocessor encounters `#define` directive, it replaces all occurrences of the identifier in the rest of the program code by `replacement_strings`. In the macro definition, `replacement_strings` can be a statement, a block, an expression, or simply anything else. The macro preprocessor is not concerned with the meaning of the statements nor required to understand the SL in part or completely. A macro preprocessor would simply replace all occurrences of identifier in the code by `replacement_strings`. For the macro definition

```
#define ARRAY_SIZE 50
```

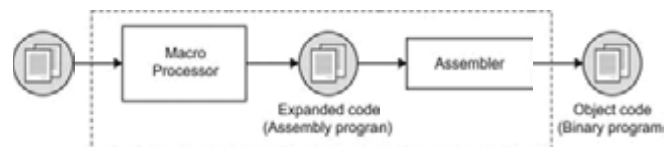
and the program statements

```
int array1[ARRAY_SIZE];
int array2[ARRAY_SIZE];
```

the preprocessor would replace `ARRAY_SIZE` in the body of the source code and results in the following equivalent expanded code:

```
int array1[50];
int array2[50];
```

Macro processors are in essence a program generator that enables to define and use macros in programs. When working with AL, a macro definition associates group of assembly codes (the instructions) with a macro name to perform a specified task. Whenever such instructions are needed in the assembly program, the programmer can use macro names instead of writing sequence of codes. The assembler must replace the occurrences of these macro names with the body contained in the macro definition. Figure 10 shows the placement of macro processors in assembly process:



▲ Figure 10: Placement of Macro Processors in Assembly Process

File inclusion

Preprocessor programs are also used to include header files into the program text. These header files include the standard definitions of routines and actions used in the programs using of functions or procedures. When the preprocessor encounters an `#include` directive in the program text, it would immediately replace it by the entire content of the specified file. Consider the following sample file inclusion statements in a C program to specify a file to be included in the program:

```
#include "filename"
#include <filename>
```

The preprocessor would include the content of the file `'filename'` in the source program. In the first statement, the file with `'filename'` is searched first in the current directory that includes the file containing the `#include` directive. If not found, the compiler searches the file in the default

directories where it is configured to look for the standard header files. The later `#include` would directly search the file in the directory where the compiler is configured to look for the standard header files. In either case following the successful search of the specified 'filename', it is replaced by the entire content of the specified file.

Rational preprocessors

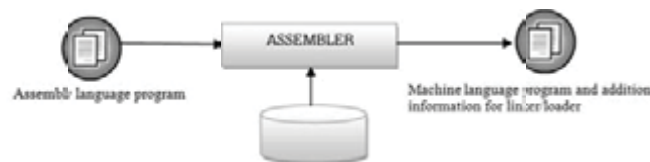
Rational preprocessors enhance the capabilities of older languages by providing the programmer with built-in macros for constructs that support flow of control and data structuring facilities. Some older languages did not have constructs such as while-statements or if-statements in them; these can be easily augmented through rational preprocessors.

Language extension

Language extensions are preprocessing facilities that enhance the capabilities of the language through concepts such as built-in macros. The language extension statements are translated into procedure calls on routines to perform desired operations such as database access or similar activities.

■ Assembler

An *assembler* is a language translator whose SL is AL and target language is binary machine language. An assembler converts the input specification written in AL to its equivalent binary program that the computer's processor can use to perform its operations. AL is a low-level PL, which is close approximation to machine language. AL uses symbolic equivalent of the numeric machine codes (called op-codes) needed to program under specific CPU architecture. In the earliest computers, programmers wrote programs in machine code (machine language). ALs replaced binary programming to speed up the programming process and used only where very efficient control over processor operations is needed and complete architectural details with instruction set is available. Figure 11 shows the function of an assembler:



▲ Figure 11: Function of an Assembler

An assembler for IBM 360/370 may not work for Intel x86 chips as they have distinct instruction sets, addressing modes, and unit of operation. There are many assemblers available today, which include MASM from Microsoft, TASM from Borland, and A86 Assemblers. The following program shows an AL program written for Intel x86 based machines that will interchange two bytes at the two memory locations represented by OPR1 and OPR2 using XCHG instruction:

```

DATA_HERE    SEGMENT
    OPR1      DB      20H
    OPR2      DB      16H
DATA_HERE    ENDS
CODE_HERE    SEGMENT
    ASSUME    CS: CODE_HERE, DS: DATA_HERE
    MOV       AX, DATA_HERE
    MOV       DS, AX
    MOV       AL, OPR1
    MOV       AH, OPR2
    XCHG      AL, AH
CODE_HERE    ENDS
END
  
```

The output of the assembler is a machine code listing. For every instruction in assembly, there is a fixed format in binary listing. For example, an instruction such as `MOV CX, COUNT` will generate the code in binary as `10001011000011100110111100000000`, and in hexadecimal, it is equivalent to `8C0E6F00`.

The output listing file (object code) that can be directly (linked if more than one modules present) loaded by loader in the memory for execution. Every instruction in assembly has corresponding fixed code that is generated using instruction templates and formats specified for that architecture. For hand coding purpose, the programmer must remember these formats.

Generally, based on how many passes they go through the source to produce the target program, assemblers can be classified into one-pass and two-pass assemblers, which are explained as follows:

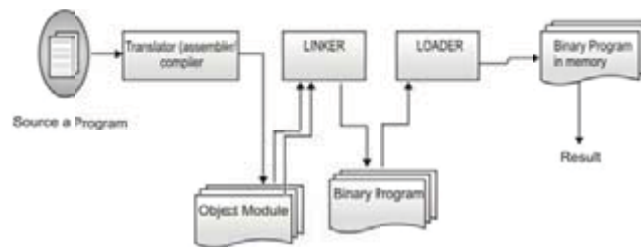
- ❑ **One-pass assemblers:** Traverse the source program only once to produce the final code. It is assumed that the program structure is symbol and name variables are defined ahead of their usage in the program.
- ❑ **Two-pass assemblers:** Use for writing more complex assembly programs involving forward references and subroutine calls. The first pass creates symbol table, which is then used in the second pass to generate code. The first pass gathers information about type and length of name variable while, so that the addresses of variables can be calculated.

One-pass assemblers are simpler to implement and offer more speed, while two-pass assemblers can resolve symbols defined anywhere in the source program.

■ Loader and Linkers

The user source programs are converted into object program in machine language by assemblers and compilers. A *loader* is a system program that takes object program files and prepares them for execution by the computer and initiates the execution. Loaders are responsible for program loading, allocation, and relocation. When a program is made up of multiple subprograms and one subprogram references another, most implementation executing them requires specialized program called *linker* that works in coordination with loaders. Linker is a system program that combines various pieces of object code and data together to form logically a single executable program, which can be loaded in the memory for execution (Figure 12). External references and their definition in the program control linking. Based on how references are resolved, linking can be done at compile time, load time (by loaders), or runtime (by application programs).

Modern linkers make use of complex features that support runtime behavior and use dynamic linked shared libraries. In general, a linker is responsible for resolving symbols. A linker finds the memory location of referenced variable, subroutine, or program and generates integrated logical code such as “.exe” file. Figure 12 shows the linking and loading in execution process:



▲ Figure 12: Schematic of Linking and Loading in Execution Process

The functions of a linker are as follows:

- ❑ Looks for the library routines or functions used by program

- ❑ Determines and relocates the instructions to the desired memory locations by adjusting absolute references
- ❑ Resolves references among multiple files

The functions of loader are as follows:

- ❑ Reads executable file's header to determine the size of text and data segments
- ❑ Creates a new address space and copies program instructions and data into it.
- ❑ Pushes the arguments passed to the program on the stack and initializes the registers including the stack pointer
- ❑ Jumps to a startup routine responsible for loading arguments from the stack to registers and calls the main routine

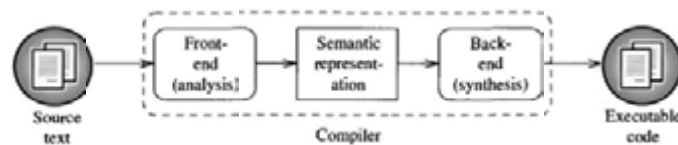
The basic job of any linker or loader is simple and remains the same, regardless of the operating system, processor architecture, or object file format being used. Linker binds more abstract references to more concrete addresses, while loader finds best suitable memory location to initiate the execution. Such flexibility permits programmers to write code using the more abstract names without worrying about managing memory and storage issues.

1.9 The Grouping of Phases: Front End and Back End of Compiler

It has been aforementioned that a language processor comprises two distinct phases: analysis and synthesis phases. During the *analysis phase*, the source program is analyzed and table of information is produced, which is followed by the synthesis of the target program. Analysis phase is also referred to as the front end of the compiler, while synthesis phase is known as back end of the compiler. In general,

$$\text{Compilation of Program} = \text{Front End Processing (Analysis)} + \text{Back end Processing (Synthesis)}$$

During language processing, the front end and back end perform logically cohesive operation and transform the representation of the input source program to another representation in the output program. Figure 13 shows the conceptual interaction of front end and back end of compiler:



▲ Figure 13: Conceptual Interaction of Front End and Back End of Compiler

The front end and back end of a compiler are explained as follows:

■ Front End of Compiler: The Analysis Phase

The front end of the compiler is more generously known as *analysis phase*, which looks for the specification of the SL and forms the foundation for the source program analysis. The analysis of a source program includes lexical, syntax, and semantic analyses. Each analysis performs specific task and provides necessary information for the synthesis phase to generate target program.

The *lexical analysis*, also known as *scanning* phase, scans through the program and helps to identify and form valid lexical units (called tokens) in the SL and classify them. Lexical analyzers are usually

automatically generated from patterns called regular expressions. They are also known as scanners. **Lex** is an example of scanner.

The *syntax analysis*, also known as *parsing* phase, governs the formation of valid statements in the SL and builds the ICs to represent the structure of the statement. Syntax analyzers (also known as *parsers*) impose structure (high level) on token streams. The structure is usually represented with an Abstract Syntax Tree (AST). Parsers can be automatically generated from CFGs. Some examples of parsers are Yacc, Bison, Cup, Javacc, and Sablecc.

Finally, the *semantic analysis* determines the meaning of the valid statements and identifies the sequence of actions necessary to implement the meaning of a source statement.

Consider a statement: `Salary = Basic + DA + HRA.`

Lexical analyzer identifies `Basic`, `DA`, and `HRA` as tokens of class identifier, while `'='` and `'+'` are classified as operators. Syntax analysis phase verifies its structure and recognizes it as assignment statement with `Salary` as Left-Hand Side (LHS) and expression involving binary operator `'+'` as Right-Hand Side (RHS). Semantic analysis determines the meaning of the construct and assigns the value of expression `Basic + DA + HRA` to identifier `Salary`.

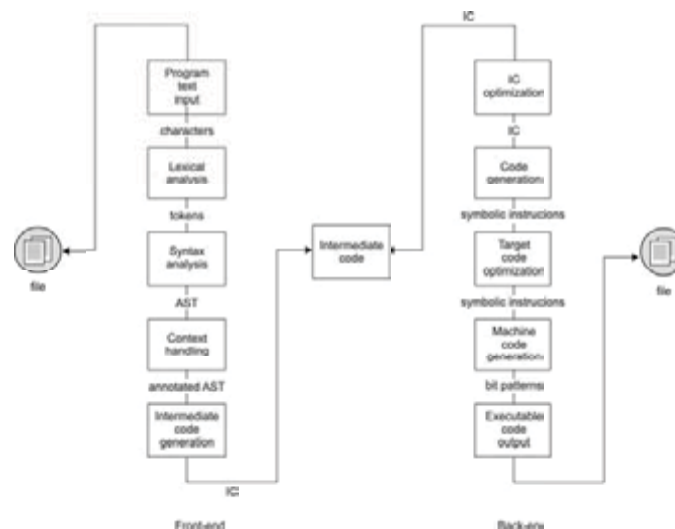
■ Back End of Compiler: The Synthesis Phase

The back end of the compiler also known as synthesis phase is concerned with the generation of target language statement(s) that has the same meaning as a source statement. The synthesis phase uses the information produced by analysis phases and consists of the following two main activities:

- ❑ **Memory allocation:** Creates data structures in the target program
- ❑ **Code generation:** Creates program code in the target language (specification language of the machine)

The synthesis phase has much to do with the architectural details of the machine and hence the target program may differ across architectures.

The conceptual detailed structure of front end and back end and their interaction during compilation is shown in Figure 14:

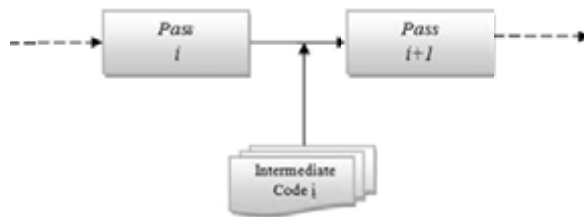


▲ Figure 14: Structure and Function of Front End and Back End of a Compiler

1.10 The Pass Structure of Compiler

While designing a language translator such as a compiler, a fundamental consideration is to decide the unit of source program for applying the fundamental translation model. To generate the object code, the source program needs to be analyzed. Compiler programs can also be specialized, and logically, each stage in the compilation can be viewed as a separate program that reads input and produces output for the next stage, such as a pipeline. Moreover, certain language structures are better suited for a particular task than others, so specific compilers were developed for specific tasks or languages.

Since all computing models process instructions sequentially, it may not be possible to obtain all the required information at the start of translation. Therefore, most compilers are multistage or multiple pass. The compiler (language processor) has to make passes over the entire program to gather information about the program components that would be required for generating target code. Note that if we implement each stage separately and use multiple passes for some of them, the compiler has to undertake a large number of Input/Output (I/O) operations, which prove to be an expensive endeavor. Remarkably, techniques have been developed to reduce the number of passes for practical reasons (say two-pass or so). If we attempt to combine stages (phases) to reduce the overheads, an internal form of the entire program will need to be stored in the memory. This could also introduce further problems as the later stage may need some information early in their execution, which the earlier stage produces only late in its execution. This can be solved using symbol tables and a two-pass approach. The first pass could take a very natural language and make it closer to a computer-understandable language. The second or even a third pass could take it to the final stage, the executable file (Figure 14). Alternatively, one can also address the issues using back patching. Figure 15 shows passes in a multi-pass compiler:



▲ Figure 15: Passes in a Multi-Pass Compiler

A *compiler pass* is a complete scan of the source program or its equivalent representation. A compiler may have to make multiple passes for several reasons, including forward references, storage limitations, and optimization. The pass of a compiler generates the IC (or pseudo code) stored in a disk file, which reflects the effects of its processing. The next pass reads it from the disk for further processing. Since the parts of the program in different passes overlay each other, the overall memory requirement is reduced. Subsequent passes tend to generate more information, updating the table of information and simplifying the program structure. Splitting up the task in this way made it easier to write more sophisticated compilers, as each sub-task is different from the other. The front end of the compiler incorporates source-language-dependent issues and maps legal source code into IR, while the back end incorporates the pragmatics of compilation (i.e., target-machine-dependent issues) and maps IR into target machine code. It can be realized that a portion of the memory is occupied by the program of the pass with its code and data structures, and the remainder is used for temporary purposes during the processing of the source program or IC input to the pass. The analysis-synthesis model of compilation heavily relies on this pass structure.

Types of compiler have been discussed earlier in the chapter. Despite this apparent complexity, the basic tasks that any compiler must perform are essentially the same.

1.11 Compiler Writing Tools

Nowadays, the task of writing a compiler has so much. Programmers make use of many tools that simplify and speed up the process of development and reduce the burden that would be experienced when writing a compiler from scratch. Software systems have been developed to help with the compiler-writing process. These systems are known as *compiler-compilers*, *compilers-generators*, or *translator-writing systems*. The compiler writer is a system programmer, but similar to any programmer, it can advantageously use software tools such as debuggers, version managers, profilers, and so on for generating compilers of the respective language model.

Profoundly, it is seen that the lexical analyzers for all languages are essentially the same, except for the particular keywords and signs recognized by them. Therefore, most compiler generators share fixed lexical analysis routines for their use in compiler generation process, wherein routines differ only in the list of keywords they recognize, usually supplied by the user but only for standard set of tokens. Generic tools have been designed and developed to help the automatic design of specific compiler components. Most of these tools hide the details of the sophisticated generation algorithm they use and produce components that can be easily integrated into the remainder of a compiler. The following is a list of some useful compiler-construction tools:

- ❑ **Parser generators:** These produced syntax analyzers automatically, normally from input that is a specification of language syntax based on a CFG. The output is a code to build the syntax tree from the token sequence. These parser generators make use of powerful parsing algorithms that are too complex to be carried out by hand otherwise. Parser generators have simplified the syntax analysis phase that consumes not only a large fraction of the running time of a compiler, but a large fraction of the intellectual effort of writing a compiler.
- ❑ **Scanner generators:** These tools (such as LEX) automatically generate lexical analyzers, normally from an input specification of the tokens of a language based on regular expressions. The output is a code to break the SL into tokens. It removes non-grammatical elements from the stream – i.e., spaces and comments. The basic organization of the resulting lexical analyzer is in effect a finite automaton.
- ❑ **Syntax directed translation engines:** Implemented as a Push-Down Automaton (PDA), they are a framework for subsequent semantic processing (e.g., YACC). These tools perform semantic translation to produce collections of routines that travel the syntax/parse tree used for generating intermediate code. The basic idea is that one or more "translations" are associated with each node of the parse tree and each translation is defined in terms of translations at its neighbor nodes in the tree. These syntax directed translation are essential features of compiler writing for generating efficient target programs.
- ❑ **Automatic code generators:** Automated systems for code synthesis take a collection of rules that define the translation of each operation specified in the intermediate representations and convert them into the machine language (object code) for the target machine. The basic technique is "*template matching*" using *code-production tables*. These code-production tables contain the templates of the target code equivalent to operations in the source/intermediate programs. The intermediate code statements are replaced by "templates" that represent sequences of machine instructions in such a way that the assumptions about storage of variables match from template to template. As there can be many ways to perform the same operation, it is important to select the best template that optimizes the target program.
- ❑ **Data-flow engines:** These tools assist code optimization for generating better and faster codes through gathering of information about how values are transmitted from one part of a program to other part. This is called *data-flow analysis* or *flow-graph analysis*. It provides the necessary information about variable usage and execution behavior to determine when a transformation is legal/illegal. As most programs use similar constructs, data-flow analysis can be performed by

essentially the same routine, with the user supplying details of the relationship between intermediate code statements and the information being gathered.

It must be recognized that modern compilers have been implemented with the help of compiler generators without which the task is much complex and will surely take larger development time. These are programs that take a formal description of the syntax and semantics of a PL as input and produce major parts of a compiler for that language as output.

Summary

In this chapter, we have learned about the importance of program translation and language processing in modern programming environment. PL is classified into machine, assembly, and HLL. The developers prefer PLs that are closer to the languages humans speak. HLL is the highest in the language hierarchy and makes programming much easier and allows describing the program in a more natural language. With higher and easier readability, an HLL command is equivalent to several commands in an AL and hundreds of binary bits of machine language.

A *translator* is a language processor (program) that translates a given source program into an equivalent object program. The source program and the object program use SL and object language, respectively. In most cases, machine language forms the object language. The language translators bridge the execution gap by translating one (source) PL instruction(s) into another (target) PL instruction(s) without the loss of the intended meaning. The primary advantage of translation model is that the translated target program can be saved in a file and executed repeatedly.

A *compiler* is a language translator that translates a high-level program into a low-level equivalent form that the computer can more easily understand. The low-level output of a compiler are called object program. A compiler is a translator of problem-oriented languages or machine-independent languages. Apart from translating the input program, it is also responsible for checking the lexical and syntactical compositions of the program; it also checks the program before translating it to make sure that it is grammatically correct. The translation is performed only for error-free programs. A compiler overcomes the semantic gap between PL domain and execution domain. The task of a compiler can be divided into front end and back end. The front end of the compiler is responsible for performing the analysis of the source program and producing the information, while the back end of the compiler performs allocations and synthesis of the programs using the information produced by the front end.

An *assembler* is a system program that transforms (translates) the source program in AL into corresponding machine language of a computer. The assembly program uses symbolic representation of machine language statements and hence is closer to machine language. An *interpreter* is system software that translates a given HLL program into a low-level one, but differs from compiler and assembler. Interpretation is a real-time activity where an interpreter takes the program, one statement at a time, and translates each line before running it. In contrast to compiler and assembler, interpreters do not produce any target output program; instead they translate the line and runs it. A successful translation of each line is followed by execution. The major difference between a compiler and an interpreter is that the interpreter does not produce an object program to be executed; it executes the source program itself. Assembler and compiler are based on translation-linking-loading model, while an interpreter at its core reads a statement and then executes it, reads another statement and executes it, and so on.

The translation model of execution of the program makes use of linkers and loaders to complete the execution purpose. *Linker* program processes the OMs generated by the translator and, along with information from the library, produces an executable form of the program called a binary program. The library of the OM helps in performing many runtime functions. The binary program is loaded into the memory for execution by another system program called *loaders*. In modern computing

environment, linker and loaders are clubbed together. Loading a program in the memory is called execution of a program.

Compilers can be of many types depending upon their objective and structure of operation. Broadly, compilers can be classified as single-pass, multi-pass, load and go, link and load, and optimizing compilers. *Single-pass compiler* implementation does not require the internal form of the program; it prepares and generates code by fusing the routines, especially with the semantic analyzer. *Multi-pass compilers* require a compiler to perform multiple passes over the source program before generating the target code. *Load and go* compilers produce absolute code that is executed immediately at conclusion of the compilation. *Link and load* compilers are those compilers wherein the object code is transformed by a linking loader into absolute code. *Optimizing compilers* apply a series of transformations during the compiler passes and improve the space and time efficiency of the target code. Peephole optimization and global optimization are two principle classes of optimization used in optimizing compilers.

Analysis-synthesis model of compilation is the most practical model. During *analysis*, the specification for operations written in the source program is determined and recorded/translated into a suitable intermediate form to simplify the rest of the activities in the compilation process. A compiler operates in *phases*, each of which transforms the source program from one representation to another. Important phases of compiler implementation include lexical analysis, syntax analysis, semantic analysis, machine independent and dependent optimizations, and code generation. Symbol-table management and error handling continually interact with the phases during the compilation process.

Preprocessors, assemblers, linker, and loaders are known as cousins of compiler, which have significant contributions in achieving program translation and its execution. Preprocessors are program that processes its input data to produce output that is used as input to another program. An *assembler* is a language translator whose SL is AL and target language is binary machine language. An assembler converts the input specification written in AL to its equivalent binary program that the computer's processor can use to perform its operations. A *loader* is a system program that takes these object program files and prepares them for execution by computer and initiates the execution. These loaders are responsible for program loading, allocation, and relocation. Linker is a system program that combines various pieces of object code and data together to form logically a single executable, which can be loaded in the memory for execution. External references and their definition in the program control linking. Based on how references are resolved, linking can be done at compile time, load time (by loaders), or runtime (by application programs).

A *compiler pass* is a complete scan of the source program or its equivalent representation. A compiler may have to make multiple passes for several reasons, including forward references, storage limitations, and optimization. The analysis-synthesis model of compilation heavily relies on this pass structure. Software systems have been developed to help the compiler-writing process. These systems are known as compiler-compilers, compilers-generators, or translator-writing systems.

Review Questions

- Q1. Differentiate between system program and application program.
- Q2. "Compiler is a system program." Comment and justify.
- Q3. Discuss the translation-linking and loading models with suitable illustration. What are the advantages of this model over interpretation model?
- Q4. What is a compiler? How is it different from an interpreter?
- Q5. Describe the analysis-synthesis model of program compilation with a suitable diagram.
- Q6. What is the role of front end and back end of the compiler in program processing? Explain.

- Q7. What are the advantages of using a pass structure for compilation?
- Q8. List and describe in brief the various phases of compiler and the activities they perform during the process of compilation.
- Q9. What is the need and role of IR in compilation model of language processing? Does an interpreter use or produce IR?
- Q10. What is the role of symbol table management in compilers?
- Q11. What are compiler writing tools? List and describe them in brief.
- Q12. What are the advantages of (a) a compiler over an interpreter and (b) an interpreter over a compiler?
- Q13. Compare and contrast compilers and interpreters.
- Q14. Explain under what conditions an interpreter is proved to be better than using a compiler for language processing.
- Q15. Write a short note error reporting during compilation.
- Q16. Comment on: Interpreter is a language processor but not a language translator.
- Q17. What advantages are there in a language-processing system in which the compiler produces assembly language rather than machine language?
- Q18. What is the need of program translation? Briefly describe the process of program translation. State the advantages and disadvantages of translation model. Give examples of program translators.
- Q19. Briefly describe the structure of a compiler with suitable illustration.
- Q20. What are linker and loader? Clearly explain the significance of loading and linking in execution of programs.
- Q21. What is object code?
- Q22. List and describe the various types of compilers.
- Q23. Compare single verses multi-pass compilers.
- Q24. Differentiate between load and go compiler from link and load compilers.
- Q25. What are optimizing compilers? State its advantages and disadvantages.
- Q26. Explain with suitable block diagram the analysis-synthesis model of compilation.
- Q27. List and describe the functions of various phases of a typical compiler.
- Q28. What are intermediate codes? Why are they important?
- Q29. List and describe the popular cousins of compiler.
- Q30. Write short note on:
 - a. Preprocessors
 - b. Functions of loader and linkers
 - c. Front-end of compiler
- Q31. Describe the front-end and back-end of compiler. Draw a suitable diagram to explain the interaction between front-end and back-end of compiler.
- Q32. What is a compiler pass? Explain.
- Q33. What is lexical analysis? What is the outcome of this phase?

■ Multiple Choice Questions**Q1. State True or False**

- i. The translation model allows target program to be saved in a file and executed repeatedly.
 - ii. The front end is also known as analysis phase of the compiler.
- | | |
|----------------|-----------------|
| a. True, true | b. False, false |
| c. True, false | d. False, true |

Ans. a

Q2. _____ is suitable for development environment, while _____ is suited for production environment.

- | | |
|----------------------------|--------------------------|
| a. Compiler, interpreter | b. Compiler, debugger |
| c. Interpreters, compilers | d. interpreter, debugger |

Ans. c

Q3. _____ produces binary executable target program.

- | | |
|------------|-------------|
| a. Loader | b. Linker |
| c. Monitor | d. Debugger |

Ans. b

Q4. Load and go compiler produce _____ code.

- | | |
|-----------------|--------------------|
| a. Absolute | b. Reloadable |
| c. Both a and b | d. Neither a nor b |

Ans. a

Q5. _____ phase performs type conversions and type casting operations.

- | | |
|----------------------|--------------------|
| a. Lexical analysis | b. Syntax analysis |
| c. Semantic analysis | d. None of these |

Ans. c

02 Lexical Analyzer

OBJECTIVES

- ✎ Understanding the role of lexical analyzers in compilation process
- ✎ Learning about input buffering, specification of tokens, and their recognition
- ✎ Exploring the language for specifying lexical analyzers
- ✎ Introducing the concepts of finite automata and regular expressions
- ✎ Studying the design of a lexical analyzer generator
- ✎ Learning about the technique of optimization DFA

Computing machines are planned to carry out computations and solve complex problems that can be well formulated. Carrying out such computation requires specifying operations using programs. Programs are nothing but unit of specification to carry out computational task to solve a problem. Computer makes use of language processors to understand programs for determining what actions are to be carried out. Compilers are one such language translator, which take high-level language programs containing instructions to carry out instruction by the computer in phases. The compiler in its early stage of translation, namely, the *lexical analysis*, sees an input program as a string of characters and classifies the program text into a sequence of *basic symbols*. Comments in the program text are identified and ignored as they are not to be translated. Basic symbols, also called *tokens*, are character sequences that are formed using most basic symbols called terminal symbols of the grammar. Tokens are classified as keyword, identifiers, constants, and operators. Lexical analysis does the job of tokenizing and identifying any invalid symbols or strings used in the language.

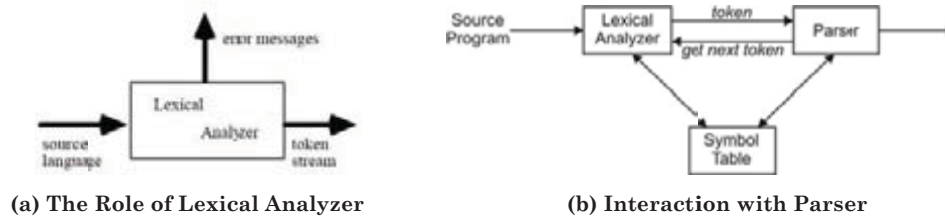
Finite automata can be used to represent the tokens and help in designing lexical analyzers. To implement a lexical analyzer by hand, we make use of diagram and other description of each token and write code to identify occurrence on the input and return information and class about the token identified. The lexical analyzer can also be automatically produced using lexical analyzer generators by specifying the lexeme patterns to them and compiling patterns into code that functions as a lexical analyzer. Regular expressions are used to specify token and help recognize tokens defined in these expressions.

This chapter introduces you to lexical analysis and explores the details of activities performed by lexical analyzers. It discusses about input buffering and specification of tokens. Then you learn about recognition of tokens and the language for specifying lexical analyzer. Further, the chapter explains about finite-state automata and finite automata from regular expressions. Finally, you learn about designing of a lexical analyzer generator and optimizing Deterministic Finite Automaton (DFA).

2.1 Introduction to Lexical Analyzer

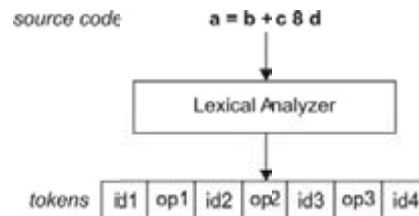
Lexical analysis is the first phase of the compilation process that recognizes patterns in input text. Lexical analyzer is also called *scanner* as it scans the text for classification into tokens. In this book, scanner is used to denote lexical analyzer at many places. Writing a scanner requires knowing valid program symbols, keywords, and different classes for tokenizing. Scanner can be custom

handwritten or it can be automatically generated using a general-purpose analyzer generator that takes descriptions of the patterns to be recognized (Figure 1(a)). The use of regular expression for description of pattern under recognition will provide great help. Lexical analysis could be performed as an independent pass, wherein the outputs of the pass are maintained in a set of information table that forms the input for the parser (the syntax analyzer). Importantly, the scanner for practical implementation is a co routine that works often with the parser and is called whenever the parser needs new token (Figure 1(b)). Figure 1(a) shows the role of lexical analyzer and Figure 1(b) shows its interaction with parser:



▲ Figure 1: Lexical Analyzer and Its Interaction with Parser

The lexical analyzer has to interact constantly with the symbol table to update it. When the lexical analyzer discovers a lexeme (an instance of a token) constituting an identifier, it needs to enter that lexeme into the symbol table, which contains information about identifiers and their types. Splitting the analysis phase into lexical (scanning) and syntactical (parsing) phases simplifies the design of a compiler and hence provide opportunities for using more specific token recognizers. Efficient implementation of scanning phase forms the basis of good parsers and hence better output code. Therefore, the compiler efficiency and portability are enhanced. Figure 2 shows *tokenization* by a lexical analyzer on the input $a=b+c*d$:



▲ Figure 2: Tokenization by Lexical Analyzer

In the preceding figure, $id\#$ represents identifiers and $op\#$ represents operators.

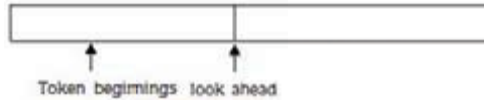
Concisely, the lexical analyzer (the *lexer*) is called by the parser when it is ready to process another token. Moreover, lexical phase performs housekeeping tasks such as eliminating whitespace and comments from the input program that would be overlooked by the code generator. Once tokenized, individual characters will no longer be examined by the compiler; instead, tokens are used for all subsequent tasks.

Let's learn about input buffering in the next section.

2.2 Input Buffering

Many a times, the scanner need to examine many characters beyond the next character before the next token itself can be determined. Usually, by default, the scanner scans the characters of the source program one at a time to discover tokens. It is advantageous for the lexical analyzer to read its input from an input buffer, which is the input area or input block that is used by the lexical

analyzers for identifying most viable tokens. Figure 3 shows a sample illustration of input buffering:



▲ Figure 3: Input Buffering

It can be seen in the preceding figure that we use two pointers, one pointer marking the beginning of the token being discovered and the other called look-ahead pointer, which scans ahead of the beginning point until the token is discovered. For practical purposes, the buffering concept adopts one convention -either a pointer is at the symbol last read or the symbol it is ready to read. In most cases, the latter is preferred.

Input buffering is extremely useful when tokenizing strings of large length. The distance that the look-ahead pointer may have to travel may be large. For example, in a program, we may come across

```
DEFINE (ARG1, ARG2,..., ARGn)
```

where it may not be possible to know whether DEFINE is a keyword or a user-defined array name unless character after the right parenthesis is analyzed. Note that in either case, the token ends at the second E in the word DEFINE. If the look-ahead pointer travels beyond the buffer half in which it began, the other half must be loaded with the next characters from the source file. It must also be noted that the buffer is of limited size; there is a constraint on how much look ahead can be used before the next token is discovered. In particular, for the preceding example, if the look ahead traveled to the left half and all the way through the left half to the middle, we could not reload the right half because we would lose characters that had not yet been grouped into tokens. We can make the buffer larger by choosing another buffering scheme, but we cannot ignore the fact that overhead is limited.

For all scanners, the key issue is the speed at which it can create token and other listings. Once it groups the input character sequences into lexical words, it associates them to a token, namely, number, keyword, identifier, etc., by performing a number of string comparisons. This can become quite expensive as the number of keywords and/or many special lexical patterns increase for a given language. Flex for C and JLex for Java are some automated scanner generators, which build a fast scanner automatically according to specifications described by regular expressions and finite automata. In the subsequent discussions, we explore the element of language design, automata, and regular expressions---the essential elements that a scanner generator uses during tokenizing.

2.3 Specification of Tokens

A formal discussion about computation and programs requires a clear understanding of the elements and relation between the basic elements that make up these programs. Analogous to the human language, all programming languages have grammar and language implemented in them. Lexical analyzer converts an input sequence of characters into a sequence of tokens. Specifying and recognizing tokens require defining basic elements such as alphabets, strings, language, etc.; further, we will see that regular expressions are an important notation for specifying lexeme patterns.

■ Tokens, Patterns, and Lexemes

All lexical analyzers take a stream of characters in the source program and read from left-to-right and classify them into tokens. Formally, a *token* is defined as a set of strings over the source alphabet which has a collective meaning. A *pattern* is a rule that describes the set of strings, and a *lexeme* is a sequence of characters matching the pattern. A lexeme can be taken as an instance of the token. For example, the following declaration in C language:

```
float pi = 3.1416;
```

reveals that substring `pi` is a lexeme for the token class called `identifier`. The token is the general class that a lexeme will belong to. It should be noted that a tokens may have exactly one lexeme such as the character `'<'` or there can be many (e.g., integer constants). Table 1 presents examples of token, pattern, and lexemes that are used in most programming languages:

Table 1: Example of Token, Lexeme, and Patterns		
Token category	Lexeme	Description of pattern
Relation	<code>=, <, >, <=, >=, !=</code>	<code>= or < or > or <= or >= or !=</code>
Identifier	<code>age, count, name, i, j, pi, result, str1</code>	Any letter followed by letters and digits
Number	<code>11, 13, 3.1416, 0, 5.02E11</code>	Any numeric constant value
Literals	<code>"Hello world"</code>	Any characters enclosed between "and"
Keywords	<code>if, then, else, while, return</code>	<code>if or then or else or while or return</code>

The complete set of tokens taken together forms the set of all possible terminal symbols that can be used in the grammar. The most widely used token categories include keywords, operators, identifiers, constants, literal strings, and punctuation. For ease of handling purposes, most compiler implementations represent token as an integer. The scanner and parser must arrive at consensus on which the integers are used for each token.

If the analyzer finds more than one lexeme for a token, we must save additional information about that token. For example, the token number matches lexemes 10 and 20. During the code generation, the compiler needs the actual value of the token to produce the target code. The value of the token and other related information are stored in the symbol table as token attributes, which can be accessed through a pointer into the symbol table. For simple tokens, the scanner uses simple integer codes for simple management. On the other hand, for complex tokens, most implementations maintain a paired representation containing an integer code and a pointer to the table storing the tokens such as identifier or constant. For example, for C source code that calculates the following area of circle of radius `R`:

```
area = Pi * R * R
```

We have the following token/attribute pairs:

- `<ID, pointer to symbol table entry for Area>`
- `<assign_operator, NULL>`
- `<ID, pointer to symbol table entry for Pi>`
- `<MUL_operator, NULL>`
- `<ID, pointer to symbol table entry for R>`

- ❑ <MUL_operator, NULL>
- ❑ <ID, pointer to symbol table entry for R>

During the lexical analysis, the compiler builds and populates the entries into the symbol table---maintaining information about identifiers, constants, literals, and so on. The scanner identifies and separates reserve keywords and user-defined variables as identifiers, thereby recognizing any conflicts, if any. Note that for tokens corresponding to keywords, we do not need attributes as the name of the token contains much of the information. For tokens other than keywords such as integer constants, subsequent stages of the compiler need to know more information, such as value, location, type, base, etc., of the constant. Identifiers much have information describing them, which can be used to distinguish one identifier from another. The use of symbol table entry for these identifiers and integer constants provides much of the information needed for use by the later phases of the compiler.

We must note that the error discovery starts from the very first phase of the compiler. Presence of an error could lead to a crash. Hence, it is advisable to print an error message then continue. Various possibilities to continue on error include:

- ❑ Deleting a character
- ❑ Inserting a missing character if any
- ❑ Replacing characters to correct errors
- ❑ Swapping adjacent characters

To summarize, the lexical analyzer is responsible for the following tasks:

- ❑ Reading in an input stream of characters and symbols (from the input program)
- ❑ Generating as output a sequence of tokens.
- ❑ Ensuring that upon generate-next-token request from the parser, the scanner reads in a string of characters called *lexeme* to and generates token
- ❑ Stripping out comments and white space from the source code
- ❑ Correlating parser errors with the source code location (the parser does not know at which line of the file it is at, but the scanner does)

The lexical analyzer works cooperatively with the syntax analyzer, wherein the scanner reads characters from the input stream to construct a token whenever needed by a syntax analyzer. Although the scanner may not be smart enough to determine about which token should come where, it is able to detect occurrences of invalid symbols very easily. In addition, scanners have limited scope and localized view in determining errors. Simple errors without any context such as characters which do not form valid tokens (e.g., an illegal or unrecognized symbol) and a few other malformed entities (illegal characters within a string constant, un-ended comments, etc.) can be easily identified; however, some critical errors including occurrences of garbled sequences, misspelled keywords, and mismatched types cannot be determined in this phase. Identification of the errors such as tokens out of place and undeclared identifiers is also not within the scope of the lexical phase. When a valid token is formed, it is returned to the syntax analyzer, which checks for syntax error, if any.

In the next section, let's discuss about elements of token design.

■ Elements of Token Design

In this section, we briefly explore the concepts such as alphabet, string, and language for specifying token for a programming environment. Some of the elements required for specifying token are explained as follows:

- **Alphabets:** To develop any language, a non-empty finite set of elements called alphabets, denoted by 'T', is required. An alphabet is a finite, nonempty ordered set of symbols, or characters with primitive graphical representations. For example, the set {a, b} is a *binary alphabet* containing two elements *a* and *b*, respectively.
- **Strings:** These are finite sequence of symbols $S = \{a_1 a_2 \dots a_n\}$ from a given alphabet set T. A string that consists of a sequence of symbols $a_1, a_2, \dots, a_n$ will be simply denoted by $a_1 a_2 \dots a_n$. In theoretical computer science, the terms sentence and word are synonyms for the term string. ...
- **Length of a string:** The number of alphabets in a string is called length of the string. For example, the string fruit has length 5.
- **Concatenation:** The operation of writing one string $V = v_1 v_2 v_3 \dots v_n$ after the string $U = u_1 u_2 u_3 \dots u_m$ to form a string $UV = u_1 u_2 u_3 \dots u_m v_1 v_2 v_3 \dots v_n$ is called concatenation of V with U. The length of the resultant string is $m+n$
- **Empty string:** Strings that have length zero (i.e., zero symbols) are called empty strings. Empty string is usually denoted by ϵ or λ . We will use ϵ for convenience. It has the following properties.
 1. $U\epsilon = \epsilon U = U$
 2. In general $\epsilon UV = UV = UV\epsilon \quad \forall \text{ strings } U, V$
- **Transitive closure:** It is a set of all possible strings over the alphabet set 'T' of length one or more. It is generally denoted by T^+ . For example, if $T = \{a\}$, the possible transitive closure can be as follows:

$$T^+ = \{a, aa, aaa, \dots\}$$
 and if $T = \{a, b\}$, the possible transitive closure can be as follows:

$$T^+ = \{a, b, ab, ba, aab, baa, \dots\}$$
 Note that transitive closure is a string of all possible lengths in any combination, except zero.
- **Kleene's closure:** It is set of all possible strings over the alphabet set 'T' of length zero or more. It is generally denoted by T^* ,

$$\text{i.e., } T^* = \{\epsilon\} \cup T^+$$
 if $T = \{a\}$ and $T^* = \{\epsilon, a, b, ab, ba, aab, baa, \dots\}$.
 This closure provides a universe of possible strings, which are useful means for the representation and interpretations of information carried by the strings.
- **Grammar:** More conveniently, languages can be specified in terms of grammars. The advantage in doing so arises mainly from the usage of a small number of rules for describing a language with a large number of sentences. A grammar is a set of four entities denoted by G: (N, T, P, S), where
 1. N is a finite set called the *non-terminal symbol*, often from meta language.
 2. T is a finite set called a set of *terminal symbol* such that $N \cap T = \emptyset$.
 3. P is a nonempty set of *product rule*, each of which has the following properties:

$$U \rightarrow V, \text{ where } U = (NUT)^+ \text{ and } V = (NUT)^*$$
 i.e., U is replaced by V. U is the *Left-Hand Side (LHS)* of the production rule and V is said to be the *Right-Hand Side (RHS)* of the production rule. In $U \rightarrow V$, it must have at least one non-terminal symbol in U.

4. S is a *distinct symbol*, which is a member of nonterminal set N . S is also called a *start state*, or *exclusive symbol*, which is always a set of nonterminal symbols. For practical purposes, if no start state is explicitly designated, the LHS of the first production is the start symbol.

The language L generated by grammar G is denoted by $L(G)$. Note that a language of the grammar $L(G)$ is set of all possible word that is valid for the given grammar. In other words, the language L generated by a grammar G is defined as follows:

$$L(G) = \{\alpha \mid S \Rightarrow^* \alpha \text{ and } \alpha \in T^*\}$$

A language ' L ' over the set ' T ' is a subset of ' T^* ', i.e., $L \subseteq T^*$. Each element of L is said to be a sentence or word or string of the language. $L(G)$ is the set of all programs in that language that is correct in a context-free sense.

■ Chomsky Classification of Grammar

A grammar is a mathematical system that defines universe of valid strings called language. Modern computer system use classification of languages and grammars named after linguist N. Chomsky. Depending upon the restriction on the rules of production, there are four types of grammar, which are explained as follows:

- **Type-0 grammar:** Resides at the top in the Chomsky classification of grammar. *Type-0 grammars* generate languages that can be recognized by any algorithmic procedure at all. Type-0 grammar is also known as *natural grammar* and does not impose restrictions on production rule. It has the following form:

$$U \rightarrow V$$

In other words, type-0 languages are those languages that can be recognized by Turing machine. These are also known as *phase structure grammar* and represent the most general class of grammars, which require the most powerful automata to recognize them. .

- **Type-1 grammar:** Refers to as *context-sensitive grammar* or *context-dependent grammar*. This is a class of restricted grammar, which has a production rule of the following form:

$$U \rightarrow V \quad U, V \in (N \cup T)^+$$

where:

$\text{length}(U) \leq \text{length}(V)$ if it is not for $S \rightarrow \epsilon$

If $S \rightarrow \epsilon$ is in P , then S does not appear on the RHS of any production rule

Type-1 languages can be recognized by Linear Bounded Automata (LBA) and validity of the substitution of symbols depends on the context.

- **Type-2 grammar:** Refers to as *context-free grammar*. All the production rules have the following form:

$$A \rightarrow V$$

where $A \in N$ & $V = (N \cup T)^*$.

The type-2 languages are same class of languages that are recognized by Push-Down Automata (PDA) with unbounded stack available to them. For example:

$G : (A, \{a, b, c\}, P, A)$

$P : \{A \rightarrow aAb$

$A \rightarrow c\}$

- **Type-3 grammar:** Refers to the most restricted class of languages also known as *regular grammar* or *linear grammar*. A grammar is called linear grammar if it is either a *left linear* or *right linear* grammar, which are explained as follows:

- **Right linear grammar:** In this type of grammar, all the productions are of the form:

$$A \rightarrow tB \text{ or } A \rightarrow t$$

where t is a terminal string, i.e., $t \in T^*$ & $A, B \in N$.

This grammar generates sentences only by adding terminal symbols to the right, one at a time in any substitution. This property makes it most suitable for implementing programming languages as this is what is required for a finite automaton to be able to recognize the given language.

For example,

$$G : (\{O, E\}, \{a\}, P, O)$$

$$P : \{O \rightarrow a$$

$$O \rightarrow aE$$

$$E \rightarrow aO\}$$

The preceding example is a type of right linear grammar.

- **Left linear grammar:** In this type of grammar, all the productions are of the form:

$$A \rightarrow Bt \text{ or } A \rightarrow t$$

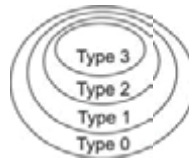
where t is a terminal string, i.e., $t \in T^*$ & $A, B \in N$.

This grammar again generates sentences only by adding terminal symbols to the left, one at a time in any substitution.



Type-2 and type-3 grammars are most widely used for programming language implementations.

Figure 4 shows the hierarchy of the various types of grammar:



▲ Figure 4: Chomsky Hierarchy of Grammar

The languages generated by each type of grammar have properties and make them suitable for different applications. Language generated by type-0 grammar is called natural language (or spoken language), while type-1 generates language called context-sensitive languages. Languages generated by type-2 grammar are called context-free language and type-3 grammar generates regular languages more often known as regular expressions.

Example 1: Consider an alphabet set $\{a, b\}$. Then $\{a, bb, aab\}$, $\{\epsilon, ba\}$, and $\{a, b\}^*$ are subsets of $\{a, b\}^*$ and represent valid possibilities over the alphabet set $\{a, b\}$.

While working with recognizers, we can use parts of a string such as prefix, suffix, substring, or subsequence to simplify recognition. Properties of a set theory and operations such as union, concatenation, and closure also apply to languages.

The *union* of two languages L_1 and L_2 is the language that contains of all the strings that are either in L_1 or in L_2 . It is denoted by denoted $L_1 \cup L_2$. Mathematically, it is represented as follows:

$$L_1 \cup L_2 = \{x \mid x \in L_1 \text{ or } x \in L_2\}$$

The *intersection* of L_1 and L_2 is the language that consists of all the strings that are both in L_1 and L_2 . It is denoted by $L_1 \cap L_2$. Mathematically, it is represented as follows:

$$L_1 \cap L_2 = \{x \mid x \in L_1 \text{ and } x \in L_2\}$$

Example 2: Consider the languages $L_1 = \{\epsilon, a, b\}$ and $L_2 = \{\epsilon, ab, bb\}$. The union and intersection of L_1 and L_2 are given as follows:

$$L_1 \cup L_2 = \{\epsilon, a, b, ab, bb\} \text{ and}$$

$$L_1 \cap L_2 = \{\epsilon\},$$

The *difference* of L_1 and L_2 is the language that consists of all the strings that are in L_1 but not in L_2 . Mathematically, it can be given by:

$$L_1 - L_2 = \{x \mid x \in L_1 \text{ but } x \notin L_2\}$$

The *cross product* of L_1 and L_2 is the set of all the pairs (x, y) of strings such that x is in L_1 and y is in L_2 . Mathematically, it is represented as follows:

$$L_1 \times L_2 = \{(x, y) \mid x \in L_1 \text{ and } y \in L_2\}$$

The *concatenation* of L_1 with L_2 , denoted $L_1 L_2$, is given by $L_1 L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

Example 3: Consider languages $L_1 = \{\epsilon, b, ab, bb\}$ and $L_2 = \{b, ab, bab\}$. Then $L_1 - L_2 = \{\epsilon, bb\}$ and $L_2 - L_1 = \{bab\}$. On the other hand, if $L_1 = \{\epsilon, a, b\}$ and $L_2 = \{ab, bb\}$, then the cross product of these languages is $L_1 \times L_2 = \{(\epsilon, ab), (\epsilon, bb), (a, ab), (a, bb), (b, ab), (b, bb)\}$, and their concatenation is $L_1 L_2 = \{ab, bb, aab, abb, bab, bbb\}$.

□ **Power of a language:** L^i will also be used to denote the concatenation of language L i times, where L^0 is defined as $\{\epsilon\}$. The set $L^0, L^1, L^2, L^3, \dots$, called the *Kleene closure* or just the closure of L , is denoted by L^* . The set $L^1, L^2, L^3, \dots$, called the *positive (transitive) closure* of L , will be denoted by L^+ . L^+ consists of those strings that can be obtained by concatenating i strings from L . L^* consists of those strings that can be obtained by concatenating an arbitrary number of strings from L .

Let's now learn about regular expression.

■ Regular Expression

The design and implementation of language processor (language translators in particular) require a clear understanding of the program entities and the relation between them. It has been aforementioned that analogous to the human language, all programming languages have grammar and language implemented in them. Lexical analyzer converts a sequence of input characters into a sequence of tokens. Specifying and recognizing tokens require the use of regular expressions.

Regular expressions can be a convenient form of representing (possibly infinite) sets of strings called *regular sets*. They are used to specify tokens and help to recognize tokens during scanning. A *regular expression* over the alphabet set T and the language denoted by that expression are defined recursively as follows:

1. $\emptyset$ is a regular expression denoting empty set
2. ϵ is a regular expression denoting $\{\epsilon\}$
3. a is a regular expression denoting $\{a\}$
4. If P and Q are regular expressions denoting the languages L_P and L_Q , respectively, then
 - a. $P + Q$ (also written sometimes as $P|Q$) is a regular expression denoting $L_P \cup L_Q$
 - b. $P.Q$ is a regular expression denoting $L_P.L_Q$

c. (P^*) is a regular expression denoting $\{\epsilon\} \cup L_P^1 \cup L_P^2 \cup L_P^3 \dots$

Nothing else is a regular expression. Note that ϵ represents empty string. This section discusses about regular expressions, their representation, and brings out their significance in scanner implementations.

Consider the problem of constructing identifiers for C language consisting of characters and digits. It can be easily designed using the regular expression $\text{letter}(\text{letter} \mid \text{digit})^*$, where the identifiers for C language can be described by the following regular definition:

```

CIdentifier  $\rightarrow$  letter(letter|digit)*
letter  $\rightarrow$  A | B | ... | Z | a | b | ... | z
digit  $\rightarrow$  0 | 1 | ... | 9

```

Some axioms that are important when working with regular expressions are as follows:

1. $(\alpha + \beta) + \vartheta = \alpha + (\beta + \vartheta)$
2. $(\alpha.\beta).\vartheta = \alpha.(\beta.\vartheta)$
3. $\alpha + \beta = \beta + \alpha$
4. $\alpha + \alpha = \alpha$
5. $\alpha.(\beta + \vartheta) = \alpha.\beta + \alpha.\vartheta$
6. $(\alpha + \beta).\vartheta = \alpha.\vartheta + \beta.\vartheta$
7. $\alpha + \emptyset = \alpha$
8. $\alpha.\emptyset = \emptyset = \emptyset.\alpha$
9. $\epsilon.\alpha = \alpha = \alpha.\epsilon$
10. $\alpha^* = \epsilon + \alpha\alpha^*$
11. $\alpha^* = (\epsilon + \alpha)^*$
12. $\alpha^+ = \alpha\alpha^*$

■ Arden's Rule

To find the regular expression from a given set of production rules, we can use *Arden's rule* and simplify the expressions. The rule states that if $X = \alpha X + \beta$, then $X = \alpha^* \beta$ is the solution.

To prove Arden's rule, let us suppose $X = \alpha X + \beta$. Then X has two productions, that is,

$$X \rightarrow \alpha X \text{ and } X \rightarrow \beta$$

where $\alpha, \beta \in T^*$. The language $L(X)$ using the preceding production rules is obtained as follows:

$$L(X) = \beta + \alpha\beta + \alpha\alpha\beta + \dots + \alpha^n\beta$$

Equivalently, it can be rewritten as follows:

$$L(X) = \{\beta, \alpha\beta, \alpha\alpha\beta, \dots, \alpha^n\beta\}$$

From the theory of regular expression, this set can be written as $\alpha^* \beta$. Since non-terminal X denotes language in itself, we can use X to denote $L(X)$. This leads to $X = \alpha^* \beta$.

Let us consider the following example to generate regular expression:

Example 4: Find the regular expression for the right linear grammar with the following productions:

$$\begin{array}{ll} S \rightarrow aS & S \rightarrow bR \\ S \rightarrow \epsilon & R \rightarrow aS \end{array}$$

Solution: In place of the four productions, we can use simultaneous equations in $L(S)$ and $L(R)$ as:

$$L(S) = \{a\}.L(S) \cup \{b\}.L(R) \cup \{\epsilon\} \quad \dots 2.1$$

$$L(R) = \{a\}.L(S) \quad \dots 2.2$$

In regular expressions, we dispense {and} and abbreviate $L(R)$ and $L(S)$ as R and S , respectively. Therefore, we can rewrite equations 2.1 and 2.2 as:

$$S = aS + bR + \epsilon \quad \dots 2.3$$

$$R = aS \quad \dots 2.4$$

Now considering equation 2.3, we have

$$\begin{aligned} S &= aS + bR + \epsilon \\ &= aS + baS + \epsilon && [\text{where } R = aS \text{ from equation 2.4}] \\ &= (a + ba)S + \epsilon && [\text{using axiom 6}] \\ &= (a + ba)^*\epsilon && [\text{using Arden's rule: } \alpha X + \beta = \alpha^*\beta] \\ &= (a + ba)^* \end{aligned}$$

Alternatively,

$$\begin{aligned} S &= aS + bR + \epsilon \\ &= aS + baS + \epsilon && [\text{where } R = aS \text{ from equation 2.4}] \\ &= aS + (baS + \epsilon) \\ &= a^*(baS + \epsilon) && [\text{using Arden's rule: } \alpha X + \beta = \alpha^*\beta] \\ &= a^*baS + a^*\epsilon \\ &= a^*baS + a^* \\ &= (a^*ba)^*a^* \end{aligned}$$

The study of regular languages is justified since it models the lexical analysis stage of the compiler. Note that the regular languages are simple and non-trivial subclass of CFGs. Regular grammar is preferably called regular expression due to the property and structure of the languages generated by using it.

2.4 Recognition of Tokens

We have discussed what the composition of a token is. Next let's discuss how to recognize the tokens. The concept of recognizing a token can be explained using the following example. Consider the following productions of the grammar fragment:

$$\begin{array}{l} \langle \text{statement} \rangle \rightarrow \text{if } \langle \text{expression} \rangle \text{ then } \langle \text{statement} \rangle \\ \langle \text{statement} \rangle \rightarrow \text{if } \langle \text{expression} \rangle \text{ then } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle \\ \langle \text{statement} \rangle \rightarrow \epsilon \end{array}$$

```

<expression>→ <term> rel_operator <term>
<expression>→ <term>
<term>→ id
<term>→ number

```

The terminals *if*, *then*, *else*, *rel_operator*, *id*, and *num* in the given grammar can generate sets of strings defined by the following regular definitions:

```

if→ if
then→ then
else→ else
rel_operator→ = | < | > | <= | >= | !=
id→ letter(letter|digit)*
number→ digit+ (.digit+)?(E(+|-)?digit+)?

```

The lexical analyzer needs to recognize the keywords *if*, *then*, and *else*. These keywords are reserved strings and cannot be used as simple identities. It also recognizes lexemes denoted by *rel_operator*, *id*, and *number* for the given language fragment. Languages such as Pascal represent unsigned integer and real numbers by *num*. For practical purposes, all instances of token (lexemes) are separated by white space, tabs, and newlines. While recognizing tokens, it will remove separators or white space (*space*) by comparing a string with the regular definition for *space*, which is given as follows:

```

separator → blank | tab | newline
space → separator+

```

The match for *space* in the recognition process indicates end of a sequence that forms the token. If the lexical analyzer again finds space, it will not return any token to the parser and proceed to find a token following the white space and returns that to the parser once done. This helps to identify the lexeme for the next token in the input buffer and enables the lexical analyzer to produce a pair of appropriate token and attribute value as output. The attribute value may use some symbolic integer or character constants as attribute values for the relational operators. Table 2 presents example of regular expressions, corresponding token, and their attribute values.

Table 2: Regular Expression with Token and Attribute Values		
Regular Expression	Token	Attribute Value
space		
if	if	
then	then	
else	else	
id	id	Pointer to table storage entry
num	num	Pointer to table storage entry
<	rel_operator	LT
<=	rel_operator	LE
=	rel_operator	EQ
!=	rel_operator	NE
>	rel_operator	GT
>=	rel_operator	GE

■ Transition Diagrams

Graphical representation is a convenient way of representing information. During the construction of a lexical analyzer, we first convert patterns in diagrammatic form called transition diagrams. These *transition diagrams* are used to keep track of recognized sequence of characters as the forward pointer scans the input. These transition diagrams contain collection of nodes or circles, which are called *states*. All states in the transition diagrams are drawn as circles. These states represent a stage in the process of searching for a lexeme in the input, which can match one of several patterns. The states in the diagram are connected to each other using arrows, which are usually called *edges*. All edges in the diagram are associated with labels. The outgoing labeled edge from a state in the diagram indicates the input characters that can next appear in the sequence from the current state. On every successful match with the characters of the input token, we move to another state in the transition diagram.

The general rules to represent a given grammar (right linear here) in diagrammatic form (also called transition diagrams) are as follows:

1. There is a node for each non-terminal symbol in the grammar and also a single final node. (The final node is also indicated by two circles.) Additional nodes may also be included by steps 3, 4, and 5.
2. For each production of the form $A \rightarrow B$, there is an arc labeled as ϵ from node A to node B.
3. For each production of the form $A \rightarrow tB$, where $t \in T^+$, there is an arc labeled as t from node A to node B. (Also see step 5.)
4. For each production of the form $A \rightarrow t$, there is an arc labeled from node A to the final node. (Also see step 5.)
5. Whenever steps 3 and 4 would have the effect of introducing an arc labeled by a string t , where $\text{length}(t) > 1$, the total of $\text{length}(t) - 1$ new nodes are added to the diagram and the arcs connecting these nodes are labeled with individual letters of t .

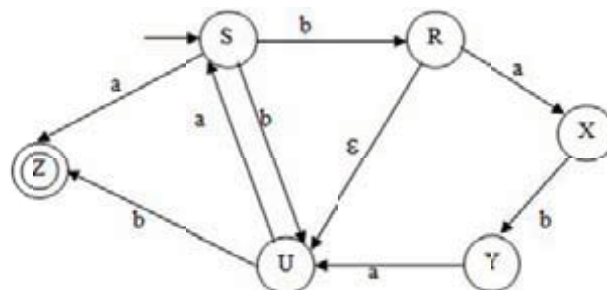
Let us consider the following example:

Example 5: Draw the diagram for the following grammar:

G : { {S, R, U}, {a, b}, P, S }

P : { S $\rightarrow$ a S $\rightarrow$ bU
 S $\rightarrow$ bR R $\rightarrow$ abaU
 R $\rightarrow$ U U $\rightarrow$ b
 U $\rightarrow$ aS }

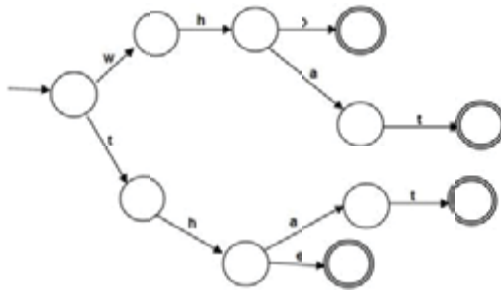
Solution: Figure 5 shows the transition diagram of the given example:



▲ Figure 5: Transition Diagram for Example 5

Note that the state Z is introduced by step 1 and state symbols X and Y are introduced by step 5.

For recognizing reserve words (keywords) such as `main`, `then`, `else`, and `new` using transition diagram, additionally you need to lookup into the symbol table entries to distinguish them from general identifiers. There are a few ways that a collection of transition diagrams can be used to build a lexical analyzer for generating tokens. In all the implementations of transition diagram, each state in the diagram represents a piece of code. Exploiting the structural arrangement and behavior of transition diagrams, we can imagine a state in the diagram associated with code, such as a switch statement or branch. This code will determine the next state by reading and examining the next input character. While designing the lexical analyzer, we could arrange the transition diagrams for each token to be tried sequentially and take the longest prefix of the input that matches any pattern. Figure 6 illustrates an example of drawing a simplified transition diagram (with final state for each path) to recognize strings `the`, `that`, `who`, and `what`:



▲ Figure 6: Transition Diagram Recognizing Strings: *the*, *that*, *who*, and *what*

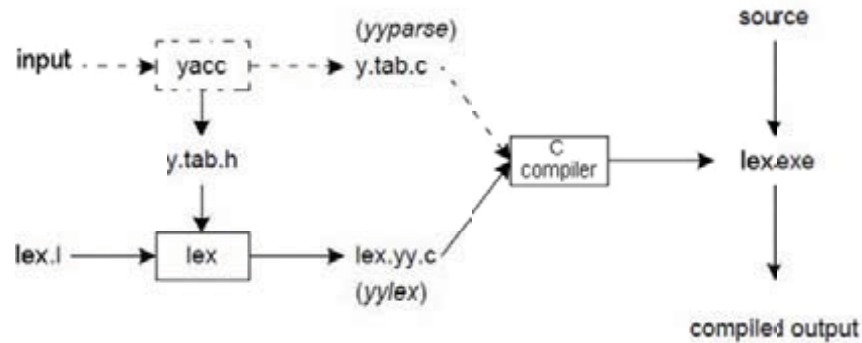
The next section discusses about finite-state automata, wherein you learn about construction of transition diagrams, converting Nondeterministic Finite Automaton (NFA) to DFA, building DFAs, and minimization of DFA.

2.5 A Language for Specifying Lexical Analyzer

Lexical analysis is the first phase of the compiler, which reads the input source and transforms strings from the source to tokens. **Lex** is a tool that generates C code for a lexical analyzer, or scanner. It uses patterns that match strings in the input and converts the strings to tokens. Regular expressions can be used to specify patterns to Lex. These patterns, which have associated actions, are scanned by the Lex program, which matches the strings in the input. The default action is to return a token for the subsequent use of parser.

As soon as the Lex discovers identifiers in the input stream, it would enter them into a symbol table that contains other information as well, including data type and address in the memory. The input specification to the Lex is known as *lex language*. The specifications are prepared by creating a program in Lex language, say, `lex.l`. When run through the Lex compiler, it produces `lex.yy.c`, which contains the representations of regular expressions (usually in tabular form) of `lex.l` and routines that recognize lexemes.

Lex reads the pattern descriptions (see Figure 7) in `lex.l`, includes file `y.tab.h` (obtained from parser during interaction), and generates a lexical analyzer, function `yylex`, in the file `lex.yy.c`. The lexer is compiled (together with parser) and linked to form the executable file, `lex.exe`, which generates the tokens from the stream of inputs. Figure 7 shows the creation of lexical analyzer with Lex:



▲ Figure 7: Creation of Lexical Analyzer with Lex

Note that the Lex program translates the regular expressions to a computer program that behaves as a Finite State Automaton (FSA). Table 3 gives some examples of the pattern matching primitives:

Table 3: Pattern Matching Primitives	
Metacharacter	Matches
.	Any character except new line
\n	Newline
*	Zero or more copies of the expression
+	One or more copies of the expression
?	Zero or one copy of the expression
^	Beginning of line
\$	End of line
x y	x or y
(xy) ⁺	One or more copies of xy(concatenation)
"x+y"	Literal "x+y"
[]	Represents a character class

The metacharacters are used by regular expressions in the Lex. Note that when two patterns match for some string, the longest match is considered, and when more than one match are of the same length, the first pattern listed is used. Table 4 describes the predefined variables and their functions that are used by Lex:

Table 4: Lex Predefined Variables	
Name	Function
int yylex (void)	call to invoke lexer, returns token
char *yytext	pointer to matched string
yylen	length of matched string
yyval	value associated with token
Int yywrap (void)	wrapup, return 1 if done, 0 if not done
File * yyout	output file

Table 4: Lex Predefined Variables	
Name	Function
File *yyin	input file
INITIAL	initial start condition
BEGIN	condition switch start condition
ECHO	write matched string

The input to Lex is partitioned into the following three segments, with `%%` dividing the segments:

... definitions ...

`%%`

... rules ...

`%%`

... subroutines ...

The input essentially requires the first `%%` to be written, as there must always be a rules section. However, if any rules are not specified, then match everything and copy them to the output. This is the default action in the absence of any rule. Defaults for input and output are `stdin` and `stdout`, respectively. The following program is an example that gives the fair idea of working of Lex and defines digits and letters:

```
digit [0-9]
letter [A-Za-z]
%{
  int count;
%}
%%
/* match the identifier */
{letter}({letter}|{digit})* count++;
%%
int main(void) {
  yylex();
  printf("number of identifiers = %d\n", count);
  return 0;
}
```

The following specification is a simple example of a program that does nothing at all.

```
%%
.
\n
```

In the above specification, all the input matches, but since no actions are specified with any pattern, there will be no output.

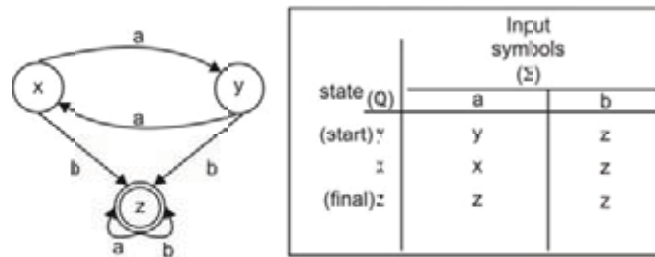
One of the limitation of Lex program is that it cannot recognize nested structures including parentheses. The reason is that it comprises only states and transitions between them. Efficient handling of nested structures requires stacks. Since Lex has no stack, it is not suited for parsing nested structures. For such things, Yacc is used as it augments an FSA with a stack. Noticeably, Lex is suitable for pattern matching and Yacc is good for more complicated tasks.

2.6 Finite-State Automata

Finite automata are graphs that decide whether a sentence is in the language (set of valid strings generated by regular expression) or not. These are *recognizers* of a given language that can be considered much similar to a machine in its behavior. Finite automaton can be used to recognize tokens defined by regular expressions. The forms of automaton known as *finite automata* are used as recognizers to output success or failure during string recognition. In fact, these are formalization of transition diagrams that include a designation of a start state and one or more accepting states, set of intermediate states, input characters, and transitions among the states. In simple sense, an automaton can be equated to a machine, which means that it is always in one or the other known states, and on some inputs, it transits to another state. An automaton forms the basis for construction of token recognizers. Therefore, *finite automata* contains the following components:

1. A finite set of states, which includes an *initial state* (or *start state*) and some (maybe none) designated as final states
2. A finite alphabet set Σ , which comprises all possible input symbols used for transition between the states
3. A finite set of transitions that specifies for each state the symbols of the input alphabet and the next state to move in.

Finite automata can be described using diagrammatic transition diagram, transition function, or transition tables. In some representations, the start state is explicitly represented by an incoming arrow, while the final states may be enclosed within double circles or prefixed by some special symbols. Example of a finite automaton is shown in Figure 8:



▲ Figure 8: Example Showing Representation of Finite Automata

A recognizer for a given language is any program that takes as input a string and outputs success if it can be completely described by a sequence of paths, which is a path from start to the final state. If no path exists from start to final state, the recognizer outputs a “failure.” We compile a regular expression into a recognizer by constructing a transition diagram called finite automation. A finite automation can be of two types: *deterministic finite automata (DFA)* and *non-deterministic finite automata (NFA)*. NFA means there can be more than one transition out of a state for the same input symbol. On the other hand, DFA has a unique transition for every state-character combination. The formal definitions of both are as follows:

- **NFA:** A finite automaton denoted by N is a mathematical model consisting of five tuples $(Q, \Sigma, \delta, S, F)$, where:
 - i. Q is a finite non-empty set of elements, which are called its states.
 - ii. Σ is a set of input alphabets of possible input symbols.

- iii. δ is a mapping (transition function) from $Q \times (\Sigma \cup \{\epsilon\})$ into the set of subset of Q . This mapping controls and defines the moves from one state to another along the edges labelled with symbols from the alphabet.
- iv. $S \in Q$ is a start (exclusive) state.
- v. $F \subseteq Q$ is a non-empty set of final state.

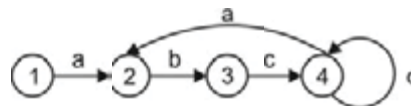
It can be seen from the definition that NFA can be found in two forms: ϵ -NFA with transition edges labeled with the empty string and another that has transition edges labeled with symbols from the alphabets (but not ϵ). The ϵ -NFA allows transition to the next state without using any input symbols. Automata can be completely described by the transition function δ . In general, NFAs do not have restrictions on the edges leaving a state; there can be many edges with the same label and some edges can be labeled with ϵ . This implies that there can be several possible next states from a given state and a current symbol of transition.

Transition table is another form of describing an automaton. The transition table for Example 5 is presented in Table 5:

Table 5: Transition Table for Example 5			
Q	Σ		
	a	b	ϵ
(start) S	{Z}	{R, U}	$\emptyset$
R	{X}	$\emptyset$	{U}
U	{S}	{Z}	$\emptyset$
X	$\emptyset$	{Y}	$\emptyset$
Y	{U}	$\emptyset$	$\emptyset$
(final) Z	$\emptyset$	$\emptyset$	$\emptyset$

- **DFA:** It is a special case of finite automata in which no state has ϵ transition. For each state and input symbol a , there is a unique transition (at most one edge labeled a leaving the said state), i.e., for a given combination of state and transition symbol, the next state is uniquely determined. Formally, a deterministic (finite-state) automata denoted by D is a mathematical model consisting of five tuples $(Q, \Sigma, \delta, S, F)$, where:
- i. Q is a finite non-empty set of elements, which are called its states
 - ii. Σ is a set of input alphabet of possible input symbols
 - iii. δ is a mapping (transition function) from $Q \times \Sigma$ into Q
 - iv. $S \in Q$ is a start (exclusive) state
 - v. $F \subseteq Q$ is a non-empty set of final state

DFA's are faster recognizers than NFAs, but can be much larger than equivalent NFAs. A DFA is equivalent to a finite-state machine that recognizes a given regular expression. For example, Figure 9 defines a DFA recognizing the expression $(abc^+)^+$:



▲ Figure 9: DFA Recognizing the Expression $(abc^+)^+$

The transition table for the DFA shown in Figure 9 is presented in Table 6:

Table 6: Transition Table for the DFA Shown in Figure 9			
Q	Σ		
	a	b	c
(start) 1	{2}	$\emptyset$	$\emptyset$
2	$\emptyset$	{3}	$\emptyset$
3	$\emptyset$	$\emptyset$	{4}
(final) 4	{2}	$\emptyset$	{4}

In Figure 9, it can be seen that there is a unique transition for any symbol from any state, that is, only one start and one final state. A DFA is said to accept a string if and only if starting from the exclusive start state, transiting from state to state in the arrow direction with symbols of input sequence, it reaches a final state and the entire input string is consumed. Otherwise, it rejects the string.

Essentially, the job of a scanner generator is to convert the regular expressions into a single DFA for efficient implementations. It does this in two steps: firstly, it converts the regular expressions into an NFA, and secondly, it converts the NFA into a DFA.

In the next section, let's discuss about finite automata from regular expressions.

2.7 Finite Automata from Regular Expressions

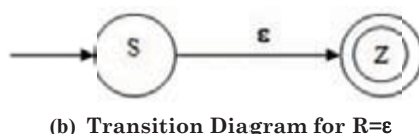
We can easily find that all types of finite automaton generate the language equivalent to that of regular expressions. In simple terms, if for a given regular expression R , L is a language corresponding to R , then there exists a finite automaton that recognizes L . Conversely, for finite automaton M that recognizes a language L , there is a regular expression R that recognizes L .

Thompson's algorithm gives the construction of a transition diagram from a given regular expression R such that the language recognized by the transition diagram equals the language denoted by R . The transition diagram for a given expression R has a single final node as well as a single initial node. To proceed with the construction, first parse R into its constituent sub-expressions and then construct NFAs for each of the basic symbols in R . The rules governing such construction are defined as follows:

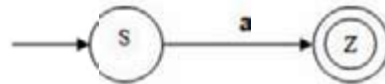
1. If $R = \emptyset$, then the transition diagram is shown in Figure 10(a):



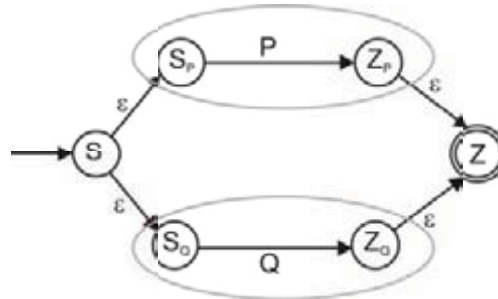
2. If $R = \epsilon$, then the transition diagram is shown in Figure 10(b):



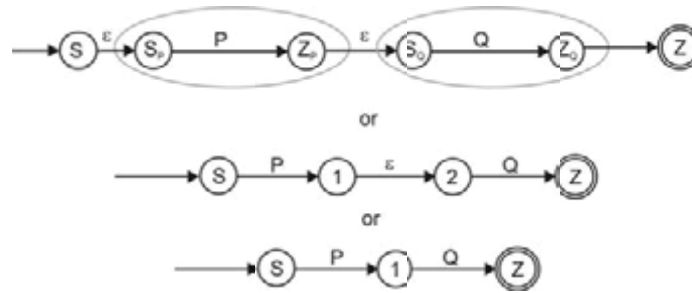
3. If $R = a$, then the transition diagram is shown in Figure 10(c):


(c) Transition Diagram for $R=a$

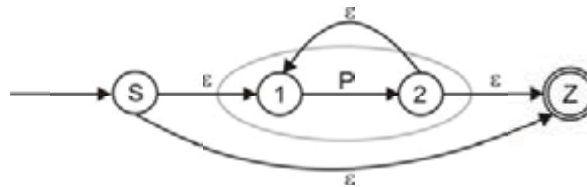
4. If $R = P + Q$, then the transition diagram is shown in Figure 10(d):


(d) Transition Diagram for $R=P+Q$

5. If $R = P.Q$, then the transition diagram is shown in Figure 10(e):


(e) Transition Diagram for $R=P.Q$

6. If $R = P^*$, then the transition diagram is shown in Figure 10(f):

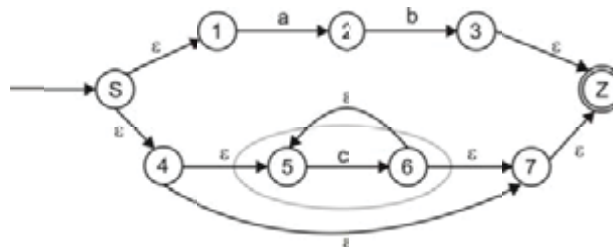
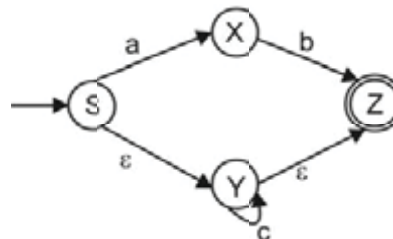

(f) Transition Diagram for $R=P^*$

▲ Figure 10: Transition Diagrams for Regular Expressions (R)

Now, let us consider the following example.

Example 6: Draw the transition diagram for $R = ab + c^*$.

Solution: The transition diagram for $R = ab + c^*$ can be drawn as shown in Figure 11:

(a) Transitioning Diagram for $R = ab + c^*$ 

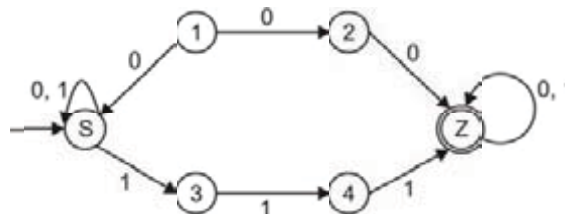
(b) Simplified Diagram for the Transition Diagram Shown in Figure 11(a)

▲ Figure 11: Transition Diagrams for $R=ab+c^*$

The transition diagram shown in Figure 11(a) can be simplified and equivalently drawn as Figure 11(b).

Example 7: Draw the transition diagram describing an NFA accepting language $(0+1)^*(000+111)(0+1)^*$.

Solution: The transition diagram describing the NFA accepting $(0+1)^*(000+111)(0+1)^*$ is shown in Figure 12:



▲ Figure 12: Transition Diagram Describing an NFA (from Regular Expressions to NFA)

Note that in Figure 12, we can find strings with more than one path through the machine. For example, string **1111** can reach the final state by following more than one path. This leads to non-determinism in the choice of paths to recognize string **1111**. Remember that an automaton recognizes a string if one can find possible paths for the given string leading to a final state.

■ Converting NFA to DFA

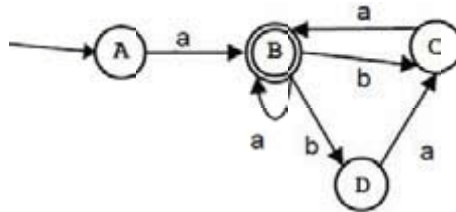
Using the methods described earlier (also known as Thompson's construction), we can build an NFA for a given regular expression. NFAs are not practical in most cases because non-determinism implies a choice that requires expensive and exhaustive backtracking algorithms. Therefore, for many practical purposes, we require DFA definitions rather than NFAs. We use an algorithm called *subset construction* algorithm, which constructs the DFA that would recognize the same language as of NFA. The highlights of the construction are as follows:

- ❑ The states in the newly constructed DFA have a set of states originally present in NFA. The start state NFA remains as start state in the newly constructed DFA.
- ❑ The alphabet set for possible input is identical in both cases.
- ❑ New state of DFA is constructed from the current state (start with exclusive state) in the original NFA. The transition symbol labeled *symb* generates a new state in DFA, which is a union of the original states that one can get using the symbol *symb* from this current state.
- ❑ The newly constructed state in DFA is analyzed with its definition of the original states. Build new states in the DFA for each possible input symbol from this state. This process generates states of the DFA that are subsets of Q . These new states are, in fact, a set of original states of NFA that would have been reached by a given input symbol.
- ❑ Any state in the DFA that contains a final state of the original NFA is also a final state in DFA.

To construct a DFA for recognizing tokens described by a regular expression, we first construct an NFA using Thompson's algorithm. Then the subset construction algorithm generates DFA from NFA. These DFAs are now ready in place of NFAs for recognizing the same token and used in implementing efficient non-backtracking scanner.

Let us consider the following example:

Example 8: Consider the automaton shown in Figure 13:

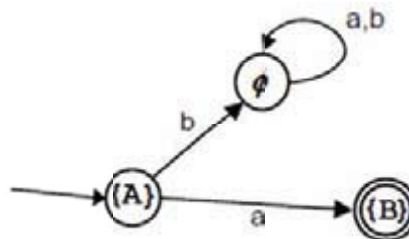


▲ Figure 13: Automata with Four States and Two Transition Symbols

Find if it is non-deterministic? If yes, then convert the same to equivalent DFA.

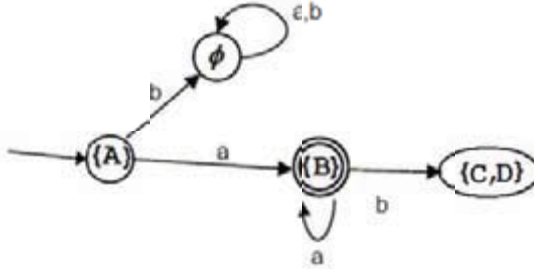
Solution: The automaton shown in Figure 13 is non-deterministic since there are two transitions on *b* coming out of the final state *B*. To convert this NFA to an equivalent DFA, we follow the steps of subset construction algorithm. We first create exclusive start state, which is similar to that of NFA, and then analyze the outgoing labeled edges from the states in the NFA. This will create a set of states for symbols from alphabet sets wherever applicable. If there were no transitions for a symbol, it goes to a sink hole (ϕ state) for such transition symbols. All the states of new DFA with final state will also become a final state. Note that the subset construction generates unique states and no duplicate states are allowed. The steps of conversion are as follows:

- ❑ Creating start state and its transition, as shown in Figure 14:



▲ Figure 14: Start State A and Its Transitions

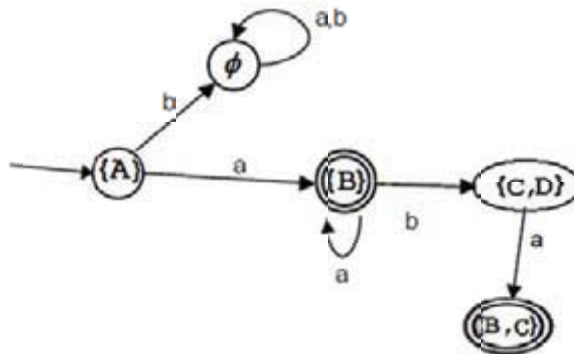
- Generating transition from $\{B\}$ state, as shown in Figure 15:



▲ Figure 15: Transition from State B

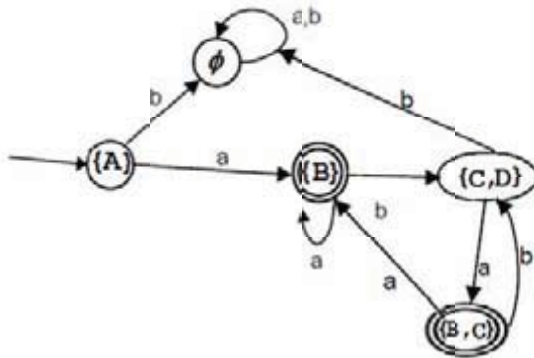
For every newly generated set of states, we continue the process and determine where we go in the NFA from each state on all the symbols of the alphabet and generate further states of the DFA.

- Generating transitions from $\{C,D\}$ state, as shown in Figure 16:



▲ Figure 16: Transitions from States C and D

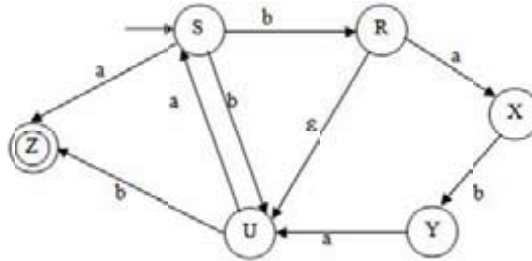
- Analyzing state $\{B,C\}$ generates the final DFA, as shown in Figure 17:



▲ Figure 17: DFA Equivalent to Figure 13

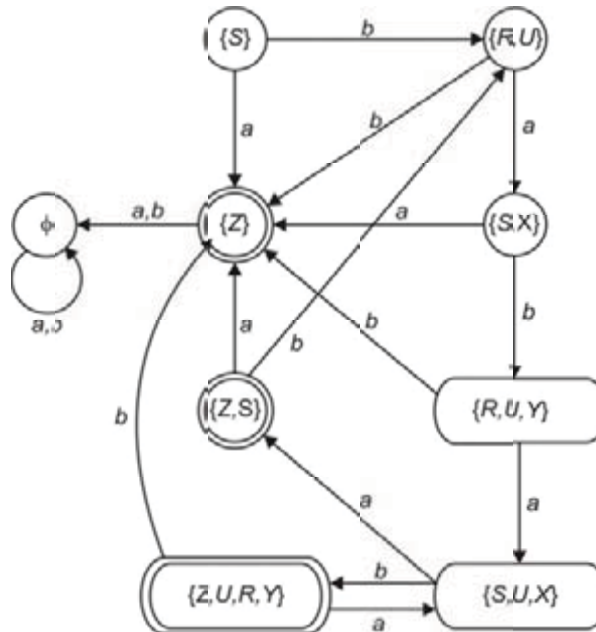
This completes the construction of the DFA for the given NFA that accepts the same language. Note that further new states cannot be generated. The newly constructed DFA has five states, namely, $\{A\}$, $\{B\}$, $\{C, D\}$, $\{B, C\}$, and $\{\phi\}$. States $\{B\}$ and $\{B, C\}$ are new final states as these contain final state B from the original NFA.

Example 9: Convert the given NFA to DFA and also construct the transitions (δ) from the transition diagram shown in Figure 18:



▲ Figure 18: An Example of NFA

Solution: Using the subset construction algorithm, we obtain the equivalent DFA shown in Figure 19:



▲ Figure 19: Equivalent DFA of the Given Example

In Figure 19, states $\{S\}$ and $\{Z\}$ are start and final states of the original NFA. The corresponding DFA contains final states $\{Z, U, R, Y\}$, $\{Z, S\}$, and $\{Z\}$.

Table 7 presents the transitions from the states of the newly constructed DFA:

Table 7: Transition Table for DFA		
Q	Σ	
	a	b
{S}	{Z}	{R, U}
{R,U}	{S,X}	{Z}
{S,X}	{Z}	{R,U,Y}
{R,U,Y}	{S,U,X}	{Z}
{S,U,X}	{S,Z}	{Z,U,R,Y}
{Z,U,R,Y}	{S,U,X}	{Z}
{Z,S}	{Z}	{R,U}
{Z}	$\emptyset$	$\emptyset$
$\{\emptyset\}$	$\emptyset$	$\emptyset$

■ Building DFAs

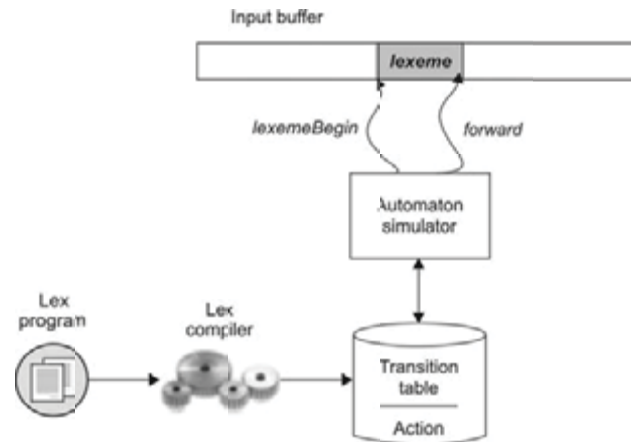
Some of the final states may lose their importance when converted to DFA that simulates NFA. Although the analyzer reaches a termination, it continues to simulate DFA until it reaches a set of states from which for input symbol, there exists no further transitions. Since the DFA has states that are eventually subset of NFA's states, look for the last DFA state that recognizes one of the patterns in NFA N_i . Remember that a valid condition will never entirely fill the input buffer without having the NFA reached termination for the lexical analyzer. To determine the correct match for the pattern, we record the input position and pattern p_i corresponding to the accepting state when added to the current set of states. Record the pattern p_i only if the accepting state is already in the current state. Moreover, the transitions must be recorded till the accepting state. Once reached, retrace back the forward pointer to a location at which the last match occurred in the buffer. The token that matches the pattern is found, and lexeme is the string between the lexeme beginning and forward pointers. The lexical analyzer must call and transfer control to some default error recovery routine if no such pattern match is found.

2.8 Designing a Lexical Analyzer Generator

Writing a lexical analyzer by hand begins with constructing a diagram of what a lexemes look like. One can then write a piece of code that follows the diagram and return the corresponding token with other information. Going this way may not be preferred in typical situation, although reporting/correction is better undertaken with handwritten method. Other alternative would be to feed the patterns describing the desired lexemes to a "lexical analyzer generator" such as Lex and flex.

The recognizer is usually implemented using a finite automaton. There is an input buffer with two pointers to it, a lexeme-beginning pointer and a forward pointer. Forward pointer helps in identifying the viable token prefixes. These lexical analyzer generators implement the generator specifications provided in the form of regular expressions as transition table. The lexical analyzer simulator behaves as finite automaton simulator, thereby using the transition table to look for the regular expression patterns in the input buffer and returning appropriate token.

Figure 20 shows the structure of the generated analyzer:

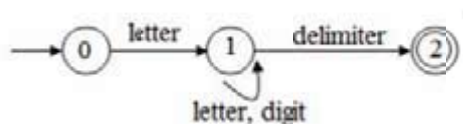


▲ Figure 20: Structure of the Generated Analyzer (Based on Lex)

Figure 20 shows the general structure of a lexical analyzer generator based on Lex. The program that serves as the lexical analyzer includes a fixed program that simulates an automaton. The rest of the lexical analyzer consists of components (transition table, functions passed directly through Lex to output, and actions from the input program) that are created from the Lex program by Lex itself. To construct the automaton, we begin by taking each regular-expression pattern in the Lex program and convert them to NFA. Since we need a single automaton that will recognize lexemes matching any of the patterns in the program, we combine all the NFA's into one by introducing a new start state with ϵ -transitions to each of the start states of the NFA's N_i for pattern p_i . We explain how it can be used for recognizing tokens in the following discussions.

■ Recognizer for Identifiers

Theoretical computer science uses diagrammatic representations of automaton as a convenient way to represent the design model of any tool in computing. Figure 21 shows an example of graphical representation of lexical analyzers for recognizing identifiers, which is defined to be a letter followed by any number of digits:



▲ Figure 21: Recognizer for Identifier

To design this lexical analyzer, we follow the rules listed as follows:

- Initially, we move to start state 0
- Move to state 1 on encountering *letter* in the input
- Now, if we come across a letter or a digit in the input, we remain in the same state 1
- If next in the sequence is a delimiter, we move to state 2, the final state

The design of the lexical analyzer forces the first element of token to be always a letter and allows the following characters to be letters or digits. Finally, on encountering a delimiter, the analyzer stops since it is neither a letter nor a digit.

While creating a lexical analyzer, the lexical analyzer generator uses a set of specifications usually in the format

$$P_i \{action_i\}$$

where p_i is a regular expression describing the pattern that is associated with action $action_i$. The $action_i$ are program fragments that are executed on match of a lexeme by p_i in the input. We choose the longest lexeme match that matches pattern, and if two or more patterns match the longest lexeme, select the first listed matching pattern.

Lexical analyzer that simulates NFA starts reading inputs beginning at the point on its input referred to as lexeme begin. It moves the forward pointer ahead in the input and calculates the set of states it is in at each point until it reaches a point on the input where there are no next states. At this condition, there is no possibility that any longer prefix of the input would ever lead the NFA to an accepting state. At this point, we can recognize the longest prefix of the lexeme that matches some pattern. Now traverse backward in the sequence of sets of states, until a set that includes one or more accepting states is found. Since there can be several accepting states in that set, choose the one associated with the earliest pattern p_i in the list. Move the forward pointer back to the end of the lexeme and perform the action A_i associated with pattern p_i .

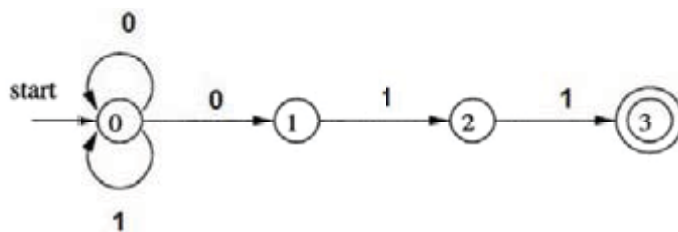
The method to convert a single regular expression to an NFA is known. However, a program such as Lex will have several regular expressions. The formulation of the recognizer would contain the following steps:

1. Produce an NFA corresponding to each regular expression
2. Introduce a new start state
3. Introduce a ϵ transition from the new start state to the start of each NFA constructed in step 1

At each of the accepting states corresponding to each NFA in step 1, the simulator will exercise the actions specified in the Lex program for the corresponding patterns.

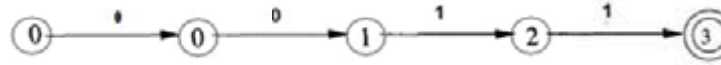
■ Pattern Matching Using NFA

An NFA accepts any input string s if and only if there is some path in the transition graph from the start state to one of the accepting states, such that the symbols along the path generate s on reaching the final state. ϵ -labels along the path are effectively ignored as they do not contribute to the string constructed along the path. The language defined (or accepted) by an NFA is the set of all those strings that are labeled over some path starting from start to final state. Figure 22(a) shows an example of nondeterministic automaton that recognizes all strings of 0's and 1's ending in particular string 011:



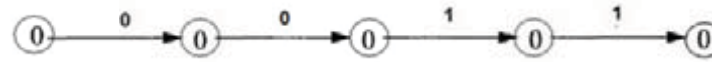
(a) NFA to Recognize a String of 0's and 1's Ending with String 011

It can be seen in Figure 22(a) that the string 0011 is accepted by the NFA. The path labeled by 0011 from state 0 to state 3 demonstrating this fact is shown in Figure 22(b):



(b) NFA Path 0011 from State 0 to State 3 Accepting String 0011

It can be clearly seen in Figure 22(b) that more than one paths labeled by the same string lead to different states. For instance, path shown in Figure 22(c) gives one more path from state 0 labeled by the same string 0011:



(c) NFA Path Accepting 0011

▲ Figure 22: Example of Nondeterministic Automaton

This path leads to state 0, which is not accepting. However, remember that an NFA accepts a string as long as *some* path labeled by that string from the start state terminates in some accepting state. The existence of other paths leading to a non-accepting state can be forgotten without much worry.

Conversion of NFA to DFA revealed that transition table for an NFA is considerably smaller than that for a DFA, although DFA have higher performance. In order to construct a lexical analyzer that behaves much similar to finite automaton, we can construct an NFA N_i for each token pattern p_i and join them with new start state. Equivalently, the transition table for the NFA can be constructed for the composite token p_i . Note that the NFA recognizes the longest prefix of the input that is matched by a pattern. Since each NFA had its own final state, there is an accepting state for each pattern p_i in the composite final NFA.

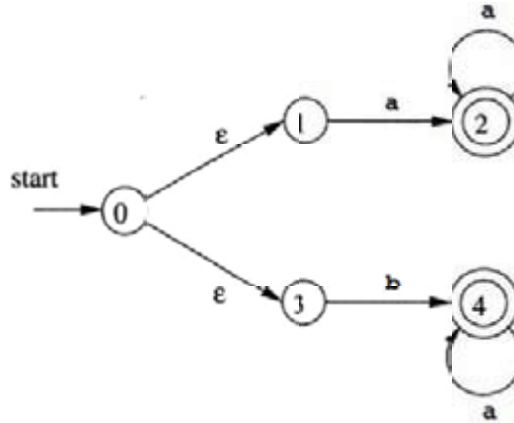
To put it in simple terms, the simulator will read sequence of characters and calculate the set of states it goes through. There are situations during the token recognition using NFA at which the input symbols do not lead to any state in the automaton. We say end-of-file is reached. Now, we scan through in backward direction from the current point until we find an accepting state. This is done so because we wish to find the *longest* lexeme that matches the pattern. Note that each accepting state symbolizes a matched pattern, and when a lexeme matches multiple patterns, we need to select the pattern listed first in the Lex-program.

■ DFA's for Lexical Analyzers

When converted to DFA that simulates NFA, some of the final states may lose their significance. Upon reaching a termination, the lexical analyzer continues to simulate DFA until it reaches a set of states from which for input symbol, there exist no further transitions. Since the DFA has states that are eventually subset of NFA's states, look for the last DFA state that recognizes one of the patterns in NFA N_i . Remember valid condition will not entirely fill the input buffer without having the NFA reach termination for the lexical analyzer. In order to determine the correct match for the pattern, we record the input position and pattern p_i corresponding to the accepting state when added to the current set of states. Record the pattern p_i only if the accepting state is already in the current state. Moreover, the transitions must be recorded till the accepting state. Once reached, retrace back the forward pointer to a location at which the last match occurred in the buffer. The token found matches the pattern and lexeme is the string between the lexeme beginning and forward pointers. The lexical analyzer must call and transfer control to some default error recovery routine if no such pattern match are found.

Example 10: Construct an NFA that recognizes NFA accepting $(aa^*) + (ba^*)$.

Solution: The NFA recognizing $(aa^*) + (ba^*)$ is shown in Figure 23:



▲ Figure 23: NFA Accepting $(aa^*) + (ba^*)$

Benefits of using finite automata as acceptors is clearly visible. Finite automata provide a convenient way to describe finite languages and help in deciding whether a string is accepted or rejected. For practical purposes, all finite acceptors need to be made deterministic and minimized as described in the earlier sections. We can use DFA instead of NFA for pattern matching, which is modified version of the method using NFA. The lexical analyzer makes transitions until there is no further moves for current input symbol and identifies a lexeme using the last input position at which the DFA entered an accepting state.

Now, let's learn about optimizing DFA in the next section.

2.9 Optimizing of DFA

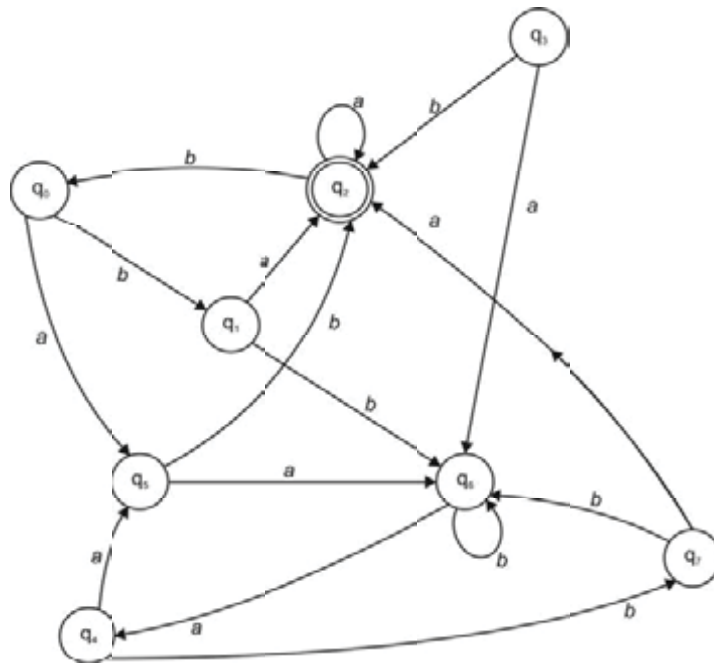
A DFA defines a unique language, but the converse is not true. For a given language, we can construct many recognizers that accept it. To understand minimization, one needs to identify *dead states*, *inaccessible states*, and *indistinguishable states*.

All those non-final states that transit to themselves for all input symbols in the alphabet set Σ are called *dead states*; on the other hand, all those states that can never be reached from initial state are called *inaccessible states*. Two states q and r of DFA are called *indistinguishable* if the following two conditions hold:

- $\delta(q, w) \in F \Rightarrow \delta(r, w) \in F$ and
- $\delta(q, w) \notin F \Rightarrow \delta(r, w) \notin F$ for all $w \in \Sigma^*$

where F denotes final state. Let us understand the minimization process using the following example:

Example 10: Minimize the DFA shown in Figure 24:



▲ Figure 24: A DFA

Solution: As already discussed, we need to identify dead states, inaccessible states, and indistinguishable states to minimize the given DFA. We explain it step by step as follows:

- **Step 1:** Identify and remove the unreachable states from the given DFA. It can be seen that q_3 is an unreachable state and therefore it is removed.
- **Step 2:** Construct the transition table with the remaining states after removing the unreachable states (here q_3), as presented in Table 8:

Table 8: Transition Table after Removing Inaccessible State q_3			
Q	Σ		
	a	b	
(start) q_0	q_5	q_1	
q_1	q_2	q_6	
(final) q_2	q_2	q_0	
q_4	q_5	q_7	
q_5	q_6	q_2	
q_6	q_4	q_6	
q_7	q_2	q_6	

- **Step 3:** Now divide the resulting transition table in two sets as follows:
 - i. First set containing rows that start from non-final states, as presented in Table 9(a):

Table 9: Rows that Start from Non-Final States and Rows that Start from Final States

Table 9(a): Rows that Start from Non-Final States

Q	Σ	
	a	b
(start) q_0	q_5	q_1
q_1	q_2	q_6
q_4	q_5	q_7
q_5	q_6	q_2
q_6	q_4	q_6
q_7	q_2	q_6

Second set containing rows that start from final states, as shown in Table 9(b):

Table 9(b): Rows that Start from Final States

q_2	q_2	q_0
-------	-------	-------

- **Step 4:** Find similar rows that transit to same states on symbols a and b. Remove one of them and update the occurrences of source states. Apply the procedure on both the sets obtained before.

Considering set 1 (Table 9(a)), rows 2 and 6 are similar as states q_1 and q_7 transit to the same states on a and b. Therefore, remove one of them. Let us cross (remove) row 6, then q_7 is replaced by q_1 in the rest of the rows, as presented in Table 10:

Table 10: Removing Similar Rows from Set 1

Q	Σ	
	a	b
(start) q_0	q_5	q_1
✓ q_1	q_2	q_6
q_4	q_5	q_7 q_1
q_5	q_6	q_2
q_6	q_4	q_6
✓ q_7	q_2	q_6

The new updated table is presented in Table 11:

Table 11: Set 1 after Removing Row 6 and q_7 is Replaced by q_1

Q	Σ	
	a	b
(start) q_0	q_5	q_1
q_1	q_2	q_6
q_4	q_5	q_1
q_5	q_6	q_2
q_6	q_4	q_6

- **Step 5:** Repeat step 4 on the rows of set 1 till we find similar rows and write the rest. Row 1 and row 3 are similar, so remove one of them and update, as presented in Table 12:

Table 12: Removing Similar Rows (Row 3)		
Q	Σ	
	a	b
$\sqrt{(\text{start})}$ q_0	q_5	q_1
q_1	q_2	q_6
$\sqrt{}$ q_4	q_5	q_1
q_5	q_6	q_2
q_6	q_4 q_0	q_6

Therefore, the minimized set is presented in Table 13:

Table 13: Minimized Set 1		
Q	Σ	
	a	b
(start) q_0	q_5	q_1
q_1	q_2	q_6
q_5	q_6	q_2
q_6	q_0	q_6

No two rows are similar, so stop.

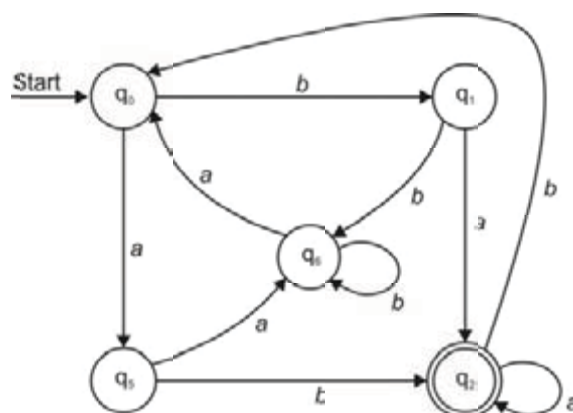
- **Step 6:** Perform the same process for set 2. Since it contains only one row, it is already minimized into the following row:

Q	Σ	
	a	b
(final) q_2	q_2	q_0

- **Step 7:** Combine set 1 and set 2 to obtain the minimized DFA presented in Table 14:

Table 14: Minimized Transition Table Representing Figure 24		
Q	Σ	
	a	b
(start) q_0	q_5	q_1
q_1	q_2	q_6
q_5	q_6	q_2
q_6	q_0	q_6
(Final) q_2	q_2	q_0

The transition diagram for the minimized DFA is shown in Figure 25:



▲ Figure 25: Minimized DFA for Figure 24

Summary

We have learned in this chapter that the compiler in its early stages of translations sees an input program as a string of characters without any structure. The lexical analysis is the first phase, which scans the input text and outputs a sequence of tokens and comments. *Tokens* are basic symbols that can be classified as keyword, identifiers, constants, and operators at large. Lexical analysis, also called *scanning*, does the job of tokenizing and identifying any invalid symbols or strings used in the language. Regular expressions are used to specify token and help to recognize tokens defined in these expressions.

A *token* is a set of strings over a source alphabet. A *pattern* is a rule that describes this set, and a *lexeme* is a sequence of characters matching a pattern. If there is more than one lexeme for a token, we have to save additional information about the token. We associate *attributes* known as token attributes, normally just a pointer into the symbol table. The scanner identifies and separates reserve keywords and user-defined variables as identifiers and recognizes any conflicts, if any. Upon gen-next-token request from the parser, the scanner reads a string of characters to generate token.

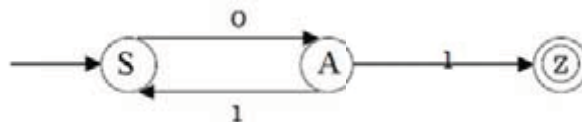
We have discussed that lexical analyzer uses input buffering to discover tokens. Input buffering is extremely useful when tokenizing strings of larger length. Alphabets, string, grammar, and languages generated by these grammars are affecting the working of the lexical analyzer.

Regular expressions are formed for representing sets of strings describing some patterns. The behavior of a lexical analyzer can often be described by a transition diagram that describes strings recognized by it. These diagrams have states, each of which represents something about the history of the characters seen during the current search for a lexeme that matches one of the possible patterns. There are arrows, or transitions, from one state to another, each of which indicates the next possible input characters that cause the lexical analyzer to make that change of state. Finite automata help in defining the behavior of an analyzer. These are a formalization of transition diagrams that include a designation of a start state and one or more accepting states, as well as the set of states, input characters, and transitions among states. Accepting states indicate that the lexeme for some token has been found. A finite automation can be of two types: *deterministic* or *non-deterministic*, where non-deterministic means that more than one transition out of a state may be possible on a same input symbol. Deterministic finite automaton does not permit ϵ transitions and has unique transition through any of the symbols.

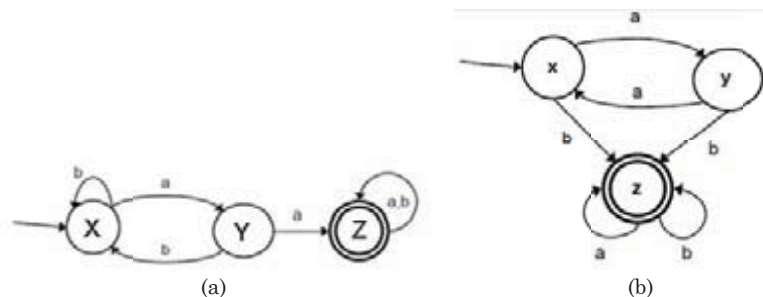
The lexical analyzers are usually implemented using a finite automaton. The analyzers make use of input buffer with two pointers to it, namely, a lexeme beginning and a forward pointer, to find patterns. The lexical analyzer generator constructs a transition table for a finite automaton from the regular expression patterns in the lexical analyzer generator specification. Lex is a popular Lexer. Lexical analyzer generator needs to convert the regular expressions into a single DFA for efficient implementation. It is a two-step process: first to convert regular expressions into an NFA, and second, to convert the NFA into a DFA. For every DFA, there is a minimum-state deterministic machine accepting the same language. To minimize the given DFA, one needs to identify dead states, inaccessible states, and indistinguishable states. The recognizer can be implemented by using either NFA or DFA. DFAs are faster recognizers than NFAs but can be much larger than equivalent NFA. The NFA recognizes the longest prefix of the input that is matched by a pattern.

Review Questions

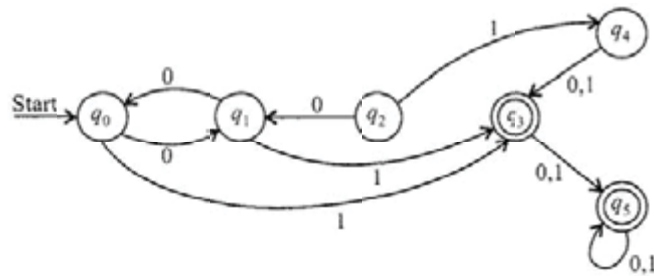
- Q1. What is lexical analysis? Explain the context with suitable illustration.
- Q2. What are the possible types of lexical errors? Explain each in brief.
- Q3. Explain the need of state minimization. Give suitable example to explain the state minimization procedure.
- Q4. What is tokenization? Which phase of the compiler does it?
- Q5. Describe with suitable lock diagram the interaction of scanner and parsers.
- Q6. Define: token, lexeme, and attribute.
- Q7. List and explain what a lexical analyzer is responsible for.
- Q8. What is input buffering? Explain how does it help in recognition of tokens?
- Q9. Define grammar. List and describe the Chomsky classification of grammar.
- Q10. Define regular expression. Derive the regular expression for the following transition diagram:



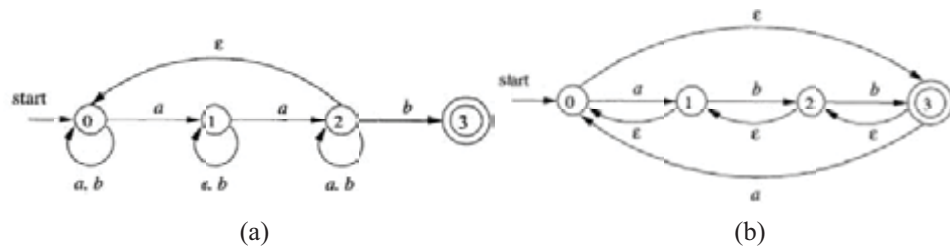
- Q11. What is a regular expression for the FA shown in the following figures?



- Q12. Define finite automata. Differentiate between NFA and DFA.
- Q13. Explain with example pattern matching using NFA.
- Q14. Minimize the DFA given as follows:



- Q15. Compare and contrast NFA and DFA. Which is faster in recognizing tokens?
- Q16. Write a short note on:
- Design of a lexical analyzer
 - Pattern matching using NFA
- Q17. What is a finite automaton? Explain the types of automata with example.
- Q18. Describe the languages denoted by the following regular expressions:
- $a(a+b)^*a$
 - $((\epsilon+a)b^*)^*$
 - $(a+b)^*a(a+b)(a+b)$
- Q19. Convert the following regular expressions to deterministic finite automata:
- $(a+b)^*$
 - $(a^*+b^*)^*$
 - $(a+b)^*abb(a+b)^*$
- Q20. For the NFA of the following figures, indicate all the paths labeled aabb. Does the NFA accept aabb?



- Q21. Given tokens: (1) the keyword *while*, (2) the keyword *when*, and (3) identifiers consisting of strings of *letters* and *digits* beginning with a letter. Construct the following:
- The NFA for these tokens and
 - The DFA for these tokens
- Q22. Construct a combined NFA that recognizes the following tokens.
- 0
 - 011
 - 0^*1^+
- Q23. Formulate the regular definition constructing unsigned integer or floating point numbers.
- Q24. Design a lexical analyzer that can recognize patterns *a*, *abb*, and a^*b^+ .
- Q25. Define: dead states, inaccessible states, and indistinguishable states.

- Q26. List and explain with example the steps for minimization of DFA.
- Q27. Explain the procedure of conversion of NFA to DFA. Give suitable example to illustrate the procedure.
- Q28. Write a short note on Thompson's algorithm.
- Q29. Compare and contrast NFA and DFA. Analyze their capabilities as token recognizer.
- Q30. Describe the role of lexical analyzer in compilation process.
- Q31. What are lexical analyzer generators? What is the benefit of using them? Describe the basic structure of implementing lexical analyzer generators.
- Q32. Construct NFA for the regular expression $a(ab)^*a$. Also convert the NFA to equivalent DFA using suitable method.
- Q33. Write a short note on Lex.

■ Multiple Choice Questions

Q1. A _____ is a rule that describes that set and a _____ is a sequence of characters matching that pattern.

- | | |
|--------------------|--------------------|
| a. lexeme, pattern | b. pattern, lexeme |
| c. pattern, token | d. token, lexeme |

Ans. b

Q2. _____ is set of all possible strings over the alphabet set T of length zero or more.

- | | |
|---------------------|----------------------|
| a. Kleene's closure | b. Positive closure |
| c. Power set | d. None of the above |

Ans. a

Q3. _____ is also called phase structured grammar.

- | | |
|-------------------|------------------------------|
| a. Type-2 grammar | b. Context—sensitive grammar |
| c. Type-0 grammar | d. Type-3 grammar |

Ans. c

Q4. Consider the following:

- | | |
|--|----------------|
| i. A DFA defines a unique language but converse is not true. | |
| ii. DFAs are faster recognizers than NFAs | |
| a. False, false | b. True, false |
| c. False, false | d. True, true |

Ans. d

Q5. In _____ there is always a unique transition for every state-character combination.

- | | |
|-----------------------|----------------------|
| a. DFA | b. NFA |
| c. Regular expression | d. None of the above |

Ans. a

03

Parsing Theory

OBJECTIVES

- ↗ Gaining insight into the role of parsers in compilation process
- ↗ Understanding top-down and bottom-up parsing algorithms
- ↗ Exploring the details of constructing operator-precedence parsers and LR parsers
- ↗ Understanding ambiguous grammars and methods to overcome it.
- ↗ Learning about parser generators and automatic generation of parsers
- ↗ Learning about using ambiguous grammars

It is very uncommon to build a full compiler by adding the appropriate actions to productions. It is preferred to build the compiler in stages. Conceptually, syntactic understanding of program structure and semantic generation of the compiled code are critical activities in the stages of building a compiler. In the last chapter, we have already described how the lexical structure of tokens could be specified using regular expression and how to use them to construct lexical analyzer to recognize them. This chapter discusses about syntactic aspects of the programming language. Once a program is lexically analyzed and stream of tokens are obtained, then comes the challenge of parsing them into formal structure. This is achieved by using formal syntax rules described by Context-Free Grammars (CFGs) and languages. A CFG, as discussed in Chapter 2, describes the specification of allowable syntactic structure of the language.

Syntax analysis phase implements syntax rules that are used for specifying programming languages and recognizing the fact that a source text in the program adheres to the rules of a given grammar. Syntax analysis is also known as parsing, which simply means “to recognize.” Parser routines check and validate an input text as valid or invalid according to the grammar rules.

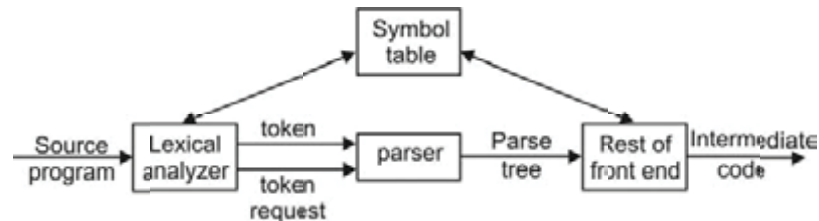
The *parser* (also called recognizer) by virtue of its property, given a grammar, has source text as input and outputs success or failure as answer. Success corresponds to those cases where the input source text satisfies the rules of the grammar, while failure for other cases of possible invalid inputs. Syntax analysis is now a mature area of language and computing theory, with many methods being devised to implement parsers. The usefulness of a parser is recognized if it is capable of generating error messages explaining the negative outcomes. In fact, all parsers must have such error diagnostic facility built in them. To be more precise, a parser must perform the following tasks:

1. Carry out context-free syntax analysis and validate language constructs used for programming
2. Interact with scanner and ask for token as and when needed
3. Guide context-sensitive analysis
4. Assist in translation by producing parse tree from which intermediate code can be generated, which enables speedy target code generation
5. Flag meaningful error messages and attempt to correct them wherever possible

In this chapter, you learn about parsing and its types, namely, top-down parsing and bottom-up parsing. Then you learn about LR parsers. The chapter further discusses about parser generators and explores the using of ambiguous grammar.

3.1 Types of Parsing: Top-Down and Bottom-Up Parsing Algorithms

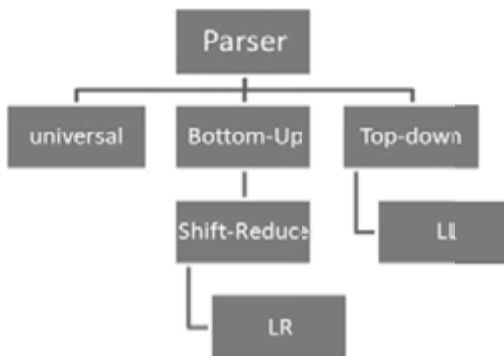
The parser obtains tokens from scanner programs (the lexical analyzer) and validates them against the syntax rules. Figure 1 shows the schematics of a parser interaction with the rest of the phases of the compiler during compilation process:



▲ Figure 1: Parser Interaction during Compilation

Parsing a program generates hierarchical correspondence between the program and the syntax rules. Practically, all programming languages are based on CFGs; parsing is essentially either deriving a string from non-terminal or reducing a string to a non-terminal. There are two fundamental strategies to parsing: *top-down parsing* and *bottom-up parsing*. A third approach called *universal parsing* also exists, but it is not practiced as it is inefficient. In the theory of language verification and representation, the top-down and bottom-up techniques have strong presence. These parsers are not universal, i.e., there are examples of grammars that cannot be used with them.

Top-down parsing is simple strategy, while more advanced algorithms are used with bottom-up parses. In the following section, we explore each of the two parsing and elaborate on the theory of parsing methodologies. Figure 2 shows the types of parsers:



▲ Figure 2: Types of Parsers

The parse tree representation describes the steps in parsing and provides a better understanding of the process. Interestingly, it will be seen later that parse trees are poor intermediate representation for source strings. It contains too much information, which the compiler, in many cases, would not prefer to have with it. Another representation called *Abstract Syntax Tree* (AST) represents the same source string more economically and describes the structure, rather exact sequence of derivation. Parse trees are unique for a source string, while ASTs are not. In practice, compiler may prefer ASTs over parse tree for code generation purpose. However, for the detailed recognition of strings, parse trees are more useful, which have more information.

In the next section, let's learn about top-down parsing.

3.2 Top-Down Parsing-Naïve Approach

Top-down parsing starts with the distinguished start symbol of the grammar and expands one of the remaining non-terminals by replacing it with the Right-Hand Side (RHS) of one of its productions until the desired string is obtained. For a valid source string α , the top-down derivation sequence is as follows:

$$S \Rightarrow \dots \Rightarrow \dots \Rightarrow \dots \Rightarrow \alpha$$

where S is the exclusive start symbol of the grammar. The naïve top-down algorithm can be outlined as follows:

1. Set Current Sentential Form (CSF) to the exclusive symbol of G . That is, initially, the parse tree is just the start symbol, which is the root of the tree.
2. Let us choose the leftmost non-terminal NT in CSF (a non-terminal in the frontier).
3. Choose a production with NT as the Left-Hand Side (LHS) and expand the sentential form (This expands the tree by making the RHS the children of the LHS).
4. Exit with success if $CSF = \alpha$ (the desired output string).
5. Repeat step 2 until the CSF contains only terminals (frontier contains all terminals).
6. Output error otherwise.

This top-down parse produces a leftmost derivation of the sentence. Top-down parsing is also known as *left-to-left* parsing or *LL parsing*.

Let us consider the following example:

Example 1: Consider a simple grammar that recognizes strings containing number of a 's followed by at least one b .

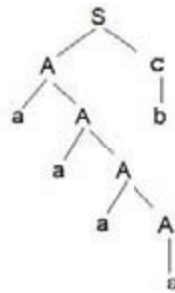
$$\begin{aligned} S &\rightarrow AC \\ A &\rightarrow aA \mid \epsilon \\ C &\rightarrow b \mid bc \end{aligned}$$

Generate the top-down derivation for the string $aaab$.

Solution: Begin with the start symbol (S in this case) and at each step, expand one of the remaining non-terminals by replacing it with the RHS of one of its productions, shown as follows:

$S \rightarrow AC$	(using rule $S \rightarrow AC$)
$\Rightarrow aAC$	(using rule $A \rightarrow aA$)
$\Rightarrow aaAC$	(using rule $A \rightarrow aA$)
$\Rightarrow aaaAC$	(using rule $A \rightarrow aA$)
$\Rightarrow aaa\epsilon C$	(using rule $A \rightarrow \epsilon$)
$\Rightarrow aaab$	(using rule $C \rightarrow b$)

Figure 3 shows the leftmost derivation (parse) tree for $aaab$:



▲ Figure 3: Leftmost Derivation (Parse) Tree for String aaab

Note that the aforementioned procedure will land up into problem for those cases where the grammar has more than one production for a non-terminal. We could choose a production for substitution that may generate a string, which does not match the desired string α . All other alternatives have to be tried until a matching string is derived or it is concluded that the string α cannot be generated. Therefore, it requires generating all possible strings from the grammar and matching the string under question among them before concluding an unsuccessful parse.

Alternatively, top-down parsing makes use of *continuation check*, which determines whether or not the current sequence of derivations will ever lead to a successful parse. It is based on the fact that for a sentential form $\beta A \delta$ with β being string of n terminals that has already been verified, the expansion of non-terminal A will lead to a configuration β . Therefore, a successful parse must at least match terminals from β to that of α . The continuation check compares $(n+m)$ symbols in the source string. If the new contributions from A include m terminals, then these m symbols must match to the right of β . Therefore, incremental check could be more appropriate, which will compare only m symbols as n symbols are already verified.

■ Top-Down Parsing Based on Prediction and Backtracking

Efficient formulation of top-down parsing requires a marker that would keep track of position in the string that has been matched and what is next that needs to be checked. This is usually done by introducing a marker called *String Source Pointer* (SSP) or String Source Marker (SSM) that keeps track of matched and unmatched symbols in the source string. We now systematically select one of the RHS alternatives of a production for the non-terminal. The choice of any RHS alternatives of a production is a *prediction* because we never know if it will lead to a successful parse. Once the predictions are made and substitution is done, continuation check is applied incrementally, and SSP is incremented if the check succeeds and parsing continues. On the other hand, if the prediction fails the continuation check, SSP resets to its original position before the rejected prediction is made. This phenomenon is called *backtracking*. Parsing resumes with any other prediction, if any.

With the knowledge of the configuration of the parser, a decision can be made to go with one particular production. This choice may result into full or partial match or even lead to a path that has no match and hence the dead end. If the current choice leads to a dead end with no further moves possible, the parser backtracks to the decision point that was last found to be consistent by moving backward through the input. The parser now continues from this point, making a different choice. These predictions, choice, evaluation, and backtracking continue until they either found the production that was the appropriate one or ran out of choices. For example, consider the following grammar rules:

$X \rightarrow bab \mid bY$
 $Y \rightarrow d \mid cY$

To parse the string bcd, one can construct Table 1, showing the possible actions during the parsing:

Table 1: Parsing Actions to Recognize String bcd		
Stack content	Input sequence	Action taken
X	bcd	Try $X \rightarrow bab$
bab	bcd	Match found – ‘b’, pop symbol ‘b’
ab	cd	Dead-end, backtrack to restore configuration
X	bcd	Try $X \rightarrow bY$
bY	bcd	Match found – ‘b’, pop symbol ‘b’
Y	cd	Try $Y \rightarrow d$
d	cd	Dead-end, backtrack to restore configuration
Y	cd	Try $Y \rightarrow cY$
cY	cd	Match found – ‘c’, pop symbol ‘c’
Y	d	Try $Y \rightarrow d$
d	d	Match found – ‘d’, pop symbol ‘d’
ϵ	ϵ	Parsing successful!

Backtracking during parsing also introduces few problems. The backtracking approach may be good for a small grammar; however, as the number of non-terminals increases, it results into combinatorial explosion, making the approach slow and impractical. The semantic actions cannot take place during prediction and need to be delayed until prediction is part of a successful parse. Moreover, precise error reporting is not possible and any mismatch merely triggers backtracking. Non-backtracking top-down parsers called predictive parser have been developed to address this issue, described later in the chapter.

■ Recursive Productions

Grammar often has its productions defined in term of itself. The non-terminal on the LHS of the production occurs on the RHS as well. This is called *recursive* production. If the recursive non-terminal occurs on the left in the RHS of the production, e.g., $A \rightarrow a \mid Ab$, the production is said to be *left recursive*. On the other hand, another type called *right-recursive* production has its recursive non-terminal on the right. For example, $A \rightarrow a \mid bA$ is right recursive.

Let us now learn about eliminating left recursion. It can be clearly seen that left-recursive grammar is not suitable for top-down parsing because the parser would enter into infinite loop of prediction making. Therefore, for efficient and feasible top-down parsing, the productions must get rid of left recursion. Note that all left-recursive rules ensure that expressions associate on the left. To eliminate left recursion, we introduce new non-terminal, say X' , and append to the end of all non-left-recursive productions for X . The expansion rule for the new non-terminal X' is essentially the reverse of the original left-recursive rule. In general, if we have productions of the following form:

$$X \rightarrow X\alpha_1 \mid \dots \mid X\alpha_n \mid \beta_1 \mid \dots \mid \beta_m$$

then to remove left recursion, we rewrite the rules as follows:

$$\begin{aligned} X &\rightarrow \beta_1 X' \mid \dots \mid \beta_m X' \\ X' &\rightarrow \alpha_1 X' \mid \dots \mid \alpha_n X' \mid \epsilon \end{aligned}$$

We now consider the immediate left-recursive grammar for non-terminal A and remove left recursion as follows:

$$A \rightarrow AB \mid AC \mid a \mid b$$

The equivalent grammar after removing left recursion is as follows:

$$\begin{aligned} A &\rightarrow aA' \mid bA' \\ A' &\rightarrow BA' \mid CA' \mid \varepsilon \end{aligned}$$

Consider the following example:

Example 2: Convert the following left-recursive rules on X to equivalent non-recursive form:

$$S \rightarrow Sa \mid Sb \mid AB \mid C \mid DEF$$

Solution: We introduce a new non-terminal say S' and append to the end of all non-left-recursive productions for S . The rules for S' are reverse of the original left-recursive rules, shown as follows:

$$\begin{aligned} S &\rightarrow ABS' \mid CS' \mid DEFS' \\ S' &\rightarrow aS' \mid bS' \mid \varepsilon \end{aligned}$$

■ Left Factoring

The parser usually reads tokens from left to right and takes immediate decision about which production from the grammar to expand based on the first token available. If there are two or more grammar rule choices that share a common prefix string, the problem of ambiguity arises. The parser for this kind of grammar requires backtracking. The leftmost tokens of several constructs may be the same; therefore, the decision to choose one of the available rules becomes difficult. For example, the rules

$$A \rightarrow \alpha\beta \mid \alpha\gamma$$

share common prefix α and one has to decide between $A \rightarrow \alpha\beta$ and $A \rightarrow \alpha\gamma$. To overcome this ambiguous situation, we can use *left factoring*. Left factoring separates out the part common to the conflicting grammar rules and delays the choice between them. To left factor a grammar, we bring together all productions sharing the same LHS, including those that begin with the same symbols on the RHS. Now combine the common prefixes from the conflicting productions into a single production and append a new non-terminal symbol to the end of this new production. Finally, create a new set of production rules using this new non-terminal for each of the suffixes to the common production. In simple words, we factor out the common prefix from the options into a shared rule that they will use and then add a new rule to manage tokens. Therefore, the left factoring for the rules:

$$A \rightarrow \alpha\beta \mid \alpha\gamma$$

generates a set of new grammar rules as follows:

$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta \mid \gamma \mid \varepsilon \end{aligned}$$

Left factoring can reduce the need for backtracking and improve the performance of pattern matching code. It can also simplify rules and help to perform semantic actions and enable precise error reporting during real-time parsing.

Let us consider the following example:

Example 3: Consider the following example often used in programming language grammars:

```
if-stmt → if (exp) statement | if (exp) statement else statement | other |
...
```

Use left factoring and rewrite the preceding grammar rules.

Solution: The common prefix is `if (exp) statement`. The presence of common prefixes on the RHS would lead to difficulties in parser implementation. The parser does not know which production to use in such situations. To resolve this, we left factor this grammar and rewrite the productions to delay the decision about which of the alternatives to choose until we have seen enough of the input to make the appropriate choice; therefore, the modified grammar is as follows:

```
if-stmt → if ( exp ) statement else-part | other | ...
else-part → else statement | ε
```

Often, a grammar may not appear to have left recursion or left factors at the first look. These issues may be hidden, which will interfere with parsing. Such hidden cases need to be first exposed, which is usually done by substitution for grammar symbols and then the usual procedure follows. For example, consider the following set of grammar rules:

```
A → da | acB
B → abB | daA | Af
```

When we substitute the expansions for A carefully into the third production of B, we have

```
A → da | acB
B → abB | daA | daf | acBf
```

This will make the parser inconsistent as it exchanges the original third production of B for several new productions, one for each of the productions for A. This clearly shows the overlap, which can be then overcome using left factoring as hereunder:

```
A → da | acB
B → aM | daN
M → bB | cBf
N → A | f
```

The next section discusses about recursive descent and predictive parser.

■ Recursive Descent and Predictive Parser

Recursive descent parsers are a general class of top-down parser that uses a set of recursive procedures to recognize the input, which may require backtracking. However, most programming language constructs do not require backtracking. Recursive descent parsers can be implemented by modifying the left-factored grammar and making repeated occurrences of strings more explicit. The recursive descent parser has several small procedures, one for each non-terminal in the grammar. Procedures corresponding to the non-terminal on the LHS of the productions are called during the parse of a sentence. For productions with recursive structure, the procedures are called recursively. For example, consider the following left-factored form of grammar for simple arithmetic expression involving addition and multiplication operators:

$$\begin{aligned}
 E &\rightarrow TE'' \\
 E'' &\rightarrow +E \mid \varepsilon \\
 T &\rightarrow VT'' \\
 T'' &\rightarrow *T \mid \varepsilon \\
 V &\rightarrow \langle id \rangle
 \end{aligned}$$

Implementing recursive descent parser would modify it to

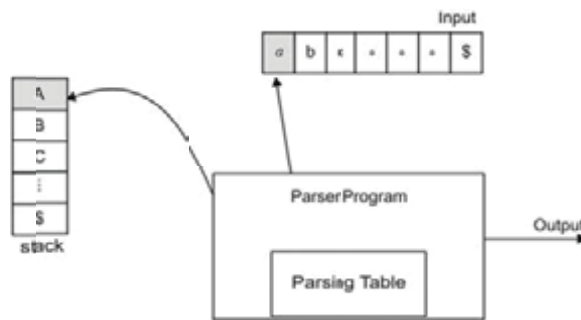
$$\begin{aligned}
 E &\rightarrow T\{+T\}^* \\
 T &\rightarrow V\{*V\}^* \\
 V &\rightarrow \langle id \rangle
 \end{aligned}$$

where $\{\dots\}^*$ denotes zero or more occurrences of enclosed specification. The structure of the resulting program closely resembles to that of the grammar it recognizes. Note that if the non-terminal corresponds to terminal atoms only, the procedure will simply read them. For all other non-terminal building blocks, the procedures will recursively call upon the procedure designed to parse the non-terminal. These recursive procedures examine a sentence in top-down manner in more and more detail until individual tokens are recognized.

Now, let us learn about predictive parsers.

A *predictive parser* is an efficient recursive descent parser that does not require backtracking. Predictive parsers are directed by their input and maintain stack of activation records. These parsers rely on the information available about the first symbols that can be generated by the RHS of production rules of the given grammar. Advance availability of the information about symbols that can be generated by the grammar rules help in building efficient syntax analyzers.

The general model of a predictive parser containing an input, a stack, a parsing table, and an output is shown in Figure 4:



▲ Figure 4: Model of a Predictive Parser

In the preceding figure, input denotes the string to be parsed and output reveals whether the parse is success or not. Stack holds the sequence of grammar symbols used during parsing the input string and parsing table $T[A, a]$, which is a two-dimensional array that has entries describing what action to take. Here, A is non-terminal and a is terminal symbol. Stack contains the start symbol and special symbol such as $\$$ or $\#$ at the bottom of the stack. Input string is also appended with $\$$ for recognizing success conditions.

The parser program evaluates the symbol, say A , on the top and an input symbol a from the input and consults the parsing table to decide what action to take. The most basic actions in consultation with parsing table can be described as follows:

- If $A = a = \$$, announce successful parse and exit.
- If $A = a \neq \$$, pop off A from the stack and advance the input symbol to next symbol.

If A is non-terminal, consult entry $[A, a]$. This would be either a production or an error entry. If the entry is a production, say $A \rightarrow XYZ$, replace A on the stack with XYZ with Z on the top of the stack. Execute the semantic action associated with this production. For error entries, the parser would call error recovery routines to handle the situation.

A predictive parser chooses to apply a production solely on the basis of the information available about the next input symbol and the current non-terminal being processed. The grammar must take some specific form and place restrictions on how to process the input. Predictive parsers can be designed for $LL(k)$ grammars, which are CFGs allowing to use some positive integer k that assists the parser to choose among various productions by examining only the next k tokens of input. The integer k can be understood as a long-distance vision parameter or a lookahead operator that enables us to see the next symbols (called lookahead symbols) coming up in the input. For practical purposes, k is usually 1 or 2. These lookahead symbols unambiguously determine the sequence of grammar rules to be applied in processing the given input. All predictive parsers have linear runtime.

Let's learn about LL parsers next.

■ LL Parsers

LL parsers are table-driven top-down parsers. In this parser, the first L in the name refers to the fact that we scan the input from left to right and the second L reveals that we create a leftmost derivation during the parsing. More general notations of LL parser is writing it as $LL(k)$, where k signifies lookahead. All $LL(k)$ grammars exclude all ambiguous grammars as well as all those that contain left recursion. Note that any CFG can be transformed into an equivalent grammar that has no left recursion, but removal of left recursion does not always yield an $LL(k)$ grammar. Working with LL grammars requires computing FIRST, FOLLOW, and *lookaheads* symbols, which are described as follows:

1. Let $G:(N,T,P,S)$ be a CFG and $x \in T^*$, then

$$First_K(x) = \left\{ \begin{array}{ll} v & ; \text{ if } length(x) \leq k \\ u & ; x = uv \text{ and } length(u) = k \end{array} \right\}$$

Note that $First_K(x) \subseteq T^*$.

2. The function definition for $FIRST_K$ can be extended to languages as well, i.e.,

$$FIRST_k(L) = \{FIRST_k(x) \mid x \in L\}$$

3. Let G be a CFG and $A \rightarrow \alpha \in P$, then

$$k \text{ lookahead}(A \rightarrow \alpha) = \{FIRST_k(w) ; \exists \beta \in (NUT)^* \text{ and } u \in T^* \text{ such that}$$

$$S \Rightarrow^* uA\beta \Rightarrow^* u\alpha\beta \Rightarrow^* uw \in T^*\}$$

4. The grammar is said to be strong $LL(K)$ if for each distinct pair of productions, $A \rightarrow \alpha$ and $A \rightarrow \beta$ in P with the same LHS, that is, for A

$$k\text{-lookahead}(A \rightarrow \alpha) \cap (A \rightarrow \beta) = \emptyset$$

The condition for G to be strong $LL(k)$ guarantees that only one of the cases will be executed, and this is only the choice that leads to successful parse.

5. Let $A \in N$, then

$$FOLLOW_*(A) = \{w ; w \in T^* \text{ and } S \Rightarrow^* uAw \text{ for some } u \in T^*\}$$

and

$$\text{FOLLOW}_K(A) = \text{FIRST}_K(\text{FOLLOW}_*(A))$$

6. Using the definition of FOLLOW, we can redefine k-lookahead ($A \rightarrow \alpha$) as

$$k\text{-lookahead}(A \rightarrow \alpha) = \text{FIRST}_k(L(\alpha) \cdot \text{FOLLOW}_*(A))$$

Note that FOLLOW is computed for non-terminals that give the terminal symbols to the right of the non-terminal. On the other hand, lookahead is always computed for production of G and determines the contribution of the production once it is substituted. For practical purposes, we put \$ in FOLLOW of exclusive start symbol of the grammar.

Let us consider the following example:

Example 4: Let G be a CFG defined by $(\{S, A\}, \{x, y\}, P, S)$, where P consists of

$$S \rightarrow yAx$$

$$A \rightarrow yAy$$

$$A \rightarrow x$$

Find 3-lookahead of all the productions and verify if the grammar is strong LL(3).

Solution: Using the preceding definition, we have

$$3\text{-lookahead}(S \rightarrow yAx) = \{yxy, yyx, yyy\}$$

$$3\text{-lookahead}(A \rightarrow yAy) = \{yxy, yyx, yyy\}$$

$$3\text{-lookahead}(A \rightarrow x) = \{xx, xyx, xyy\}$$

The grammar is strong LL(3) since $3\text{-lookahead}(A \rightarrow yAy) \cap 3\text{-lookahead}(A \rightarrow x) = \emptyset$.

LL(1) Parsers

Informally, an LL(1) has no left-recursive productions and has been left factored. These are LL parsers with $k=1$, which means these parsers have one lookahead associated with them. This requires computing FIRST_1 and FOLLOW_1 . For simplicity, we will write them as FIRST and FOLLOW, respectively. As discussed earlier, implementing LL parsers require construction of parsing table T, which can be outlined as follows:

1. Repeat steps 2 and 3 for each production $A \rightarrow \alpha$ of the grammar.
2. For each terminal $a \in \text{FIRST}(\alpha)$, enter $A \rightarrow \alpha$ to $T[A, a]$.
3. If $\epsilon \in \text{FIRST}(\alpha)$, enter $A \rightarrow \alpha$ to $T[A, b]$ for each $b \in \text{FOLLOW}(A)$. If $\epsilon \in \text{FIRST}(\alpha)$ and $\$ \in \text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $T[A, \$]$.
4. Set all undefined entries of parsing table T to error.

The table contains unique rows for each terminal and non-terminal in the grammar and a column for each terminal. Additionally, a special symbol \$ is also present, which is used to identify successful parse. The table entry $T[A, a]$ indicates what prediction should be made if A is the leftmost non-terminal in the sentential form and terminal a is the next input symbol. The entries of the form $T[a, a]$, where a is a terminal that signifies a match condition and triggers pop operation from the stack. For easy references to the productions, the table entries can store additionally the production number with $A \rightarrow \alpha$ entries. A grammar whose parsing table has unique entries is said to be LL(1), i.e., no entry in the parsing table will have multiple entries. If the table has multiple-defined entries, the grammar is ambiguous, and to make it suitable for LL parsing, we can eliminate left-recursion and apply left factoring.

Consider the following example:

Example 5: Consider a CFG defined by $(\{S, A\}, \{a, b\}, P, S)$, where

$$S \rightarrow aAS \mid b$$

$$A \rightarrow a \mid bSA$$

Compute FIRST, FOLLOW, and 1-lookaheads, and show that G is strong LL(1) and also parse string *abbab*.

Solution: We have the following production rules:

1. $S \rightarrow aAS$
2. $S \rightarrow b$
3. $A \rightarrow a$
4. $A \rightarrow bSA$

Then

$$\text{FIRST}(S) = \{a, b\}$$

$$\text{FIRST}(A) = \{a, b\}$$

$$\text{FOLLOW}(S) = \{a, b, \$\}$$

$$\text{FOLLOW}(A) = \{a, b, \$\}$$

Using the preceding definition, the 1-lookaheads are as follows:

$$\text{1-lookahead}(S \rightarrow aAS) = \{a\}$$

$$\text{1-lookahead}(S \rightarrow b) = \{b\}$$

$$\text{1-lookahead}(A \rightarrow a) = \{a\}$$

$$\text{1-lookahead}(A \rightarrow bSA) = \{b\}$$

Now,

$$\text{1-lookahead}(S \rightarrow aAS) \cap (S \rightarrow b) \Rightarrow \{a\} \cap \{b\} = \emptyset$$

and

$$\text{1-lookahead}(A \rightarrow a) \cap (S \rightarrow bSA) \Rightarrow \{a\} \cap \{b\} = \emptyset$$

Therefore, G is strong LL(1).

Table 2 presents the LL(1) parsing table:

Table 2: LL(1) Parsing Table			
(NUT) $\cup$ \$	1-Lookahead		
	a	b	\$
S	$S \rightarrow aAS, 1$	$S \rightarrow b, 2$	ERROR
A	$A \rightarrow a, 3$	$A \rightarrow bSA, 4$	ERROR
a	POP	ERROR	ERROR
b	ERROR	POP	ERROR
\$	ERROR	ERROR	ACCEPT

Moves by LL(1) predictive parser: Look for $[X, x]$ in the parsing table, where X is the stack top and x is the first symbol of the input string (which is potentially 1-lookahead). If a match is found, replace the stack top with the RHS of the production entry. Table 3 presents the parsing steps during the recognition of string *abbab*:

Table 3: Paring of String abbab

Stack contents	Input string	Output (production sequence to generate string)	Comments
S\$	abbab\$	1	Replace $S \rightarrow aAS$
aAS\$	abbab\$	1	Pop a
AS\$	bbab\$	14	Replace $A \rightarrow bSA$
bSAS\$	bbab\$	14	Pop b
SAS\$	bab\$	142	Replace $S \rightarrow b$
bAS\$	bab\$	142	Pop b
AS\$	ab\$	1423	Replace $A \rightarrow a$
aS\$	ab\$	1423	Pop a
S\$	b\$	14232	Replace $S \rightarrow b$
b\$	b\$	14232	Pop b
\$	\$	14232	ACCEPT 14232

The production sequence 14232 leads to a successful parse of the string abbab, which can be verified by applying the respective production in sequence, i.e.,

$$S \Rightarrow aAS \Rightarrow abSAS \Rightarrow abbAS \Rightarrow abbaS \Rightarrow abbab.$$

Let us consider the following example:

Example 6: Consider a CFG defined by $(\{E, E', T, T', F\}, \{a, (,), +, *\}, P, E)$, where

$$E \rightarrow T E'$$

$$E' \rightarrow +T E'$$

$$E' \rightarrow \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow *F T'$$

$$T' \rightarrow \epsilon$$

$$F \rightarrow (E)$$

$$F \rightarrow a$$

Construct the LL(1) parsing table.

Solution: We have the following production rules:

$$E \rightarrow T E'$$

$$E' \rightarrow +T E'$$

$$E' \rightarrow \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow *F T'$$

$$T' \rightarrow \epsilon$$

$$F \rightarrow (E)$$

$$F \rightarrow a$$

Then

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{ (, a \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

$$\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$ \}$$

$$\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +,), \$ \}$$

$$\text{FOLLOW}(F) = \{ +, *,), \$ \}$$

The 1-lookaheads are as follows:

$$1\text{-lookahead}(E \rightarrow T E') = \{ (, a \}$$

$$1\text{-lookahead}(E' \rightarrow + T E') = \{ + \}$$

$$1\text{-lookahead}(E' \rightarrow \epsilon) = \{ \lambda,) \}$$

$$1\text{-lookahead}(T \rightarrow F T') = \{ (, a \}$$

$$1\text{-lookahead}(T' \rightarrow * F T') = \{ * \}$$

$$1\text{-lookahead}(T' \rightarrow \epsilon) = \{ \lambda,), + \}$$

$$1\text{-lookahead}(F \rightarrow (E)) = \{ (\}$$

$$1\text{-lookahead}(F \rightarrow a) = \{ a \}$$

Now we can verify that

$$1\text{-lookahead}(E' \rightarrow + T E') \cap 1\text{-lookahead}(E' \rightarrow \epsilon) = \{ + \} \cap \{ \lambda,) \} = \emptyset$$

$$1\text{-lookahead}(T' \rightarrow * F T') \cap 1\text{-lookahead}(T' \rightarrow \epsilon) = \{ * \} \cap \{ \lambda,), + \} = \emptyset$$

$$1\text{-lookahead}(F \rightarrow (E)) \cap 1\text{-lookahead}(F \rightarrow a) = \{ (\} \cap \{ a \} = \emptyset$$

Therefore, G is strong LL(1).

Table 4 presents the LL(1) parsing table:

Table 4: LL(1) Parsing Table for the Given Example						
(N $\cup$ T) $\cup$ \$	1-lookahead					
	a	(	)	+	*	\$
E	$E \rightarrow T E', 1$	$E \rightarrow T E', 1$	ERROR	ERROR	ERROR	ERROR
E'	ERROR	ERROR	$E' \rightarrow \epsilon, 3$	$+ T E', 2$	ERROR	$E' \rightarrow \epsilon, 3$
T	$T \rightarrow F T', 4$	$T \rightarrow F T', 4$	ERROR	ERROR	ERROR	ERROR
T'	ERROR	ERROR	$T' \rightarrow \epsilon, 6$	$T' \rightarrow \epsilon, 6$	$T' \rightarrow * F T', 5$	$T' \rightarrow \epsilon, 6$
F	$F \rightarrow a, 8$	$F \rightarrow (E), 7$	ERROR	ERROR	ERROR	ERROR
a	POP	ERROR	ERROR	ERROR	ERROR	ERROR
(	ERROR	POP	ERROR	ERROR	ERROR	ERROR
)	ERROR	ERROR	POP	ERROR	ERROR	ERROR
+	ERROR	ERROR	ERROR	POP	ERROR	ERROR
*	ERROR	ERROR	ERROR	ERROR	POP	ERROR
\$	ERROR	ERROR	ERROR	ERROR	ERROR	ACCEPT

You have learned about top-down parsing. Now, let's move on and learn about bottom-up parsing.

3.3 Bottom-Up Parsing-Naïve Approach

A bottom-up parse performs reverse operation to that of a top-down parse, i.e., a top-down parser tries to expand non-terminals on the RHS, while a bottom-up parser tries to replace (reduce) RHS with non-terminals. Bottom-up parsing begins with the sentence of terminals and applies reduction, i.e., a production in reverse. *Reduction* replaces a substring that matches the RHS of some production with the non-terminal on the left. The input sentence is valid if it can be reduced to the exclusive start symbol, say S. The process of finding substrings and replacing them with LHS continues until we reduce to the start symbol. The substring that matches one of the RHS of the productions is called *handle*. It works by identifying terminal symbols first and then combining them successively to produce non-terminals. The bottom-up parser proceeds in a left-to-right manner and prints the rightmost derivation of the sentence during the parse.

The following example shows how a bottom-up parser works in general. Given the grammar rule as follows:

$$Z \rightarrow bMb$$

$$M \rightarrow (L \mid a$$

$$L \rightarrow Ma)$$

The sequence of reductions and handles for parsing a string input $b((aa)a)b$ for this grammar rule is presented in Table 5:

Table 5: Reductions during Bottom-Up Parse of String $b((aa)a)b$	
Current sentential form	Handle and production
$b((aa)a)b$	Handle is a ; use rule $M \rightarrow a$ for reduction
$b((\underline{Ma})a)b$	Handle is Ma); use rule $L \rightarrow Ma$ for reduction
$b(\underline{(L)a})b$	Handle is $(L$; use rule $M \rightarrow (L$ for reduction
$b(\underline{Ma})b$	Handle is Ma); use rule $L \rightarrow Ma$ for reduction
$b(\underline{L})b$	Handle is $(L$; use rule $M \rightarrow (L$ for reduction
$\underline{bMb}$	Handle is bMb ; use rule $Z \rightarrow bMb$ for reduction
Z	Exclusive start symbol found. Parsing is successful!

Bottom-up approach is more powerful than top-down methods. It basically analyzes the unknown data relationships in the sentential form and attempts to identify the handles, and then infers higher order structures from them. This is similar to building a tree upward toward the start symbol rather than the usual top-down way. The construction of a bottom-up parser is more complex and difficult to hand code for large grammars; however, there are excellent parser generators that can do the job once they are provided with input specifications.

■ Shift-Reduce Parsers

The most common and powerful bottom-up parser discussed in theory is *shift-reduce* parsers. Analogous to table-driven predictive parsers, these parsers also employ stack to keep track of the position in the parse and determine what to do next. The stack holds the grammar symbols while awaiting reduction. During the parse process, the input is examined for tokens and the parser either shifts (pushes) them onto a stack or reduces the elements at the top of the stack, replacing RHS by the LHS of a production.

The parser may construct a table to determine what action to do next. The possible actions that this parser can take are shift, reduce, accept, and error reporting. The *shift* action pushes the token onto the stack, while a *reduce* action removes handle from the stack and pushes to the corresponding non-terminal on the stack. The *accept* entry for parser symbolizes the successful parse of a sentence that corresponds to a state when stack contains only the distinguished symbol and the input is empty. For all other cases, the entry is *error*, which means the input sentence is not valid according to the given grammar. The basic ideology behind shift-reduce parsing can be described in the following steps:

1. Set the current sentential form to the sentence to be parsed.
2. Until the sentential form is the start symbol(s), perform the following steps:
 - i. Scan through the input from left to right until we recognize handle that corresponds to the RHS of one of the production rules of the grammar
 - ii. Perform reduction, i.e., replace the handle with the LHS of the production rule whose RHS is the handle
3. If the current sentential form is start symbol(S), then exit with success.

Else, print error and exit.

Steps i and ii correspond to the shift and reduce actions; hence, the parser is named shift-reduce parser. On stack-based implementation of these parsers, shift corresponds to pushing the current input symbol onto the stack, while reduce pops the handle off the stack and replaces it with the terminal on the LHS of the corresponding rule. The following example illustrates the shift-reduce action during a bottom-up parse:

Example 7: Given the grammar productions:

$$\begin{aligned} S &\rightarrow E\$ \\ E &\rightarrow E+T \mid E-T \mid T \\ T &\rightarrow T*F \mid T/F \mid F \\ F &\rightarrow \text{num} \mid \text{id} \end{aligned}$$

Parse the input `id-num*id$`.

Solution: We rewrite the grammar productions as follows:

0. $S \rightarrow E\$$
1. $E \rightarrow E+T$
2. $E \rightarrow E-T$
3. $E \rightarrow T$
4. $T \rightarrow T*F$
5. $T \rightarrow T/F$
6. $T \rightarrow F$
7. $F \rightarrow \text{num}$
8. $F \rightarrow \text{id}$

The parsing actions are shown in Table 6:

Table 6: Bottom-Up Parse of String id-num*id\$

	Stack content	Input	Action
1.	NULL	id - num * id \$	Shift id
2.	id	- num * id \$	Reduce id by production rule 8
3.	F	- num * id \$	Reduce F by production rule 6
4.	T	- num * id \$	Reduce T by production rule 3
5.	E	- num * id \$	Shift –
6.	E -	num * id \$	Shift num
7.	E - num	* id \$	Reduce num by production rule 7
8.	E - F	* id \$	Reduce F by production rule 6
9.	E - T	* id \$	Shift *
10.	E - T *	id \$	Shift id
11.	E - T * id	\$	Reduce id by production rule 8
12.	E - T * F	\$	Reduce F by production rule 4
13.	E - T	\$	Reduce T*F by production rule 2
14.	E \$		Shift' \$
15.	S		(Reduce E by rule 0) Accept

The preceding example clearly shows that the parser for a stream of tokens develops the list of productions (but in reverse order), which builds the parse tree. Generally, there are two types of bottom-up parsers, namely, *LR parsers* and *precedence parsers*. LR parser reads input from left to right and produces a rightmost derivation. Precedence parsers make use of precedence or priority between grammar symbols for identifying symbols for reduction. We elaborate on these in later sections.

■ Precedence Parsers

The parsing mechanisms of bottom-up parsers require determining handles (the RHS of production) from the sentential form and reducing them with the LHS of the production. The sentential form may at times contain more than one substring that resembles the RHS of productions. Moreover, during the left to right scan over the sentential form, the question is how to identify these handles. These problems can be solved using the concept of precedence relations among the grammar symbols. *Precedence relations* among the symbols signify which symbol has high priority and should be reduced first. The precedence relations can only be set between two adjacent symbols found on the RHS of the grammar productions.

There are three types of precedence relations used in parsing theory, namely, *less than*, *greater than*, and *equal* precedence relations. *Handle* is a group of symbols from the sentential form that is enclosed between less than and greater than relations and reduced with the LHS of the production. In fact, a handle is a prime phrase that is reduced first. Grammar symbols that are part of the handle always have higher precedence than those adjacent symbols that are not part of the handle. Two adjacent grammar symbols with the handle always have equal precedence. Note that from the parse tree perspective, leaf symbols down in the tree have higher precedence than those which are on the higher level and therefore ought to be reduced first. Two adjacent leaf symbols at the same level in the tree have equal precedence and are reduced together. *Simple precedence parsers* and *operator*

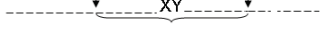
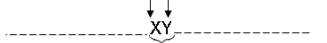
precedence parsers are based on these relations and help in validating the input sentence. We explore these two types of precedence parsers in the following subsections.

■ Simple Precedence Grammar and Parsing

Simple precedence parsers are type of bottom-up parser that works with CFGs but uses only simple precedence grammars. A grammar G is *simple precedence grammar* if for all its symbols X and Y (could be terminal and non-terminals), a unique relation exists between X and Y . In general, for a sentential form $\dots XY\dots$, the possible precedence includes greater than ($\cdot >$), less than ($\cdot <$), and equal ($\cdot \doteq$), which are described as follows:

- $X \cdot > Y$: Means symbol X should be reduced before Y during the bottom-up parse
- $X \cdot < Y$: Means symbol X will be reduced later than Y during the bottom-up parse
- $X \cdot \doteq Y$: Means both X and Y have equal precedence and will be reduced together

Note that the precedence of a symbol depends on its position in the parse tree. The lower it is in the tree, the more precedence it has to its adjacent ones in upper level. Let us assume that during derivation at some stage, we obtain a sentential form $\dots XY\dots$ by using the last derivation of the form $A \rightarrow \beta$. Therefore, $\beta \rightarrow A$ must be the first reduction in a bottom-up parse of the string $\dots XY\dots$. The precedence with respect to handle and position of symbols is summarized in Table 7:

Table 7: Rules for Precedence Relations between X and Y		
Precedence relations notations	Symbol position in the handle	Remark
$X \cdot > Y$		X is in handle and Y is not in the handle (if $\beta \equiv \dots X$)
$X \cdot < Y$		Y is in handle and X is not in handle (if $\beta \equiv Y\dots$)
$X \cdot \doteq Y$		Both X and Y are parts of the handle adjacent to each other (if $\beta \equiv \dots XY\dots$)
$X \cdot \doteq Y$		Handle contains only X and Y adjacent to each other (if $\beta \equiv XY$)

The implementation of the simple precedence parser is quite analogous to any other generic bottom-up parser and uses stack to store a viable prefix of a sentential form from a rightmost derivation. Precedence relations $\cdot <$, $\cdot \doteq$, and $\cdot >$ control the shift and reduce actions of the parser. As aforementioned, adjacent symbols within $\cdot <$ and $\cdot >$ define the *handle*---a unit for reduction. The handle in a sentential form is identified by searching for the precedence relations $\cdot < s_1 \cdot \doteq s_2 \cdot \doteq \dots \doteq s_r \cdot >$ in the input string, where each s_i can be either a terminal or a non-terminal symbol. In the given relation, s_1 and s_r are the first and last symbols of the substring unit to be reduced.

Computing Simple Precedence Relations--Method I

A table called *Simple Precedence Matrix* (SPM) is maintained to store all possible relations between the symbols of the grammar. Entries that do not have any of the three relations ($\cdot <$, $\cdot \doteq$, and $\cdot >$) are treated as error. To compute these precedence relations for a grammar $G: (N, T, P, S)$, we perform the following tasks:

1. Define FIRST and LAST sets (FIRST set contains the first relations and LAST set contains the last relation with the first and last symbols of the RHS of the productions for the corresponding LHS)
2. Define three sets $FIRST^+$, $LAST^+$, and $FIRST^*$ as follows:
 - a. $FIRST^+(X) = \{Y \mid X \Rightarrow^+ Y\alpha\}$
 - b. $LAST^+(X) = \{Y \mid X \Rightarrow^+ \alpha Y\}$
 - c. $FIRST^*(X) = (FIRST^+(X) \cup \{X\})$
3. For each production rule $A \rightarrow \alpha \in P$, perform the following tasks:
 For each two adjacent symbols X and Y in α
 - i. Enter into SPM with $X \dot{=} Y$
 - ii. Enter into SPM with $X < \cdot FIRST^+(Y)$
 - iii. Enter into SPM with $LAST^+(X) \cdot > FIRST^*(Y)$
 Entries not filled in denote error conditions.
4. SPM construction is completed

Note that for the purpose of parsing, we use end marker symbol \$, which has less than precedence than any other symbol of the grammar. In $\$ < \cdot FIRST^+(S)$ and $LAST^+ \cdot > \$$, S is the initial non-terminal and $\$$ is the end marker. Some implementations install relations for $\$$ also in the SPM; otherwise, it could be taken care during parsing itself. Also, note that $FIRST^*(X)$ can also be optimized if we use $FIRST^*(X) \equiv (FIRST^+(X) \cup \{X\}) \cap \{T\}$ because parsing is to be performed on a sentence that is made up of only terminals. Let us illustrate this through the following example:

Example 8: Consider a CFG with the following grammar rules:

$$S \rightarrow aSSb \mid c$$

Generate the SPM containing precedence relations between the grammar symbols.

Solutions: Compute $FIRST^+$, $LAST^+$ and $FIRST^*$ sets as follows:

$$\begin{aligned} FIRST^+(a) &= FIRST^+(b) = FIRST^+(c) = \phi \\ FIRST^+(S) &= \{a, c\} \\ LAST^+(a) &= LAST^+(b) = LAST^+(c) = \phi \\ LAST^+(S) &= \{b, c\} \\ FIRST^*(a) &= \{a\} \\ FIRST^*(b) &= \{b\} \\ FIRST^*(c) &= \{c\} \\ FIRST^*(S) &= \{a, c\} \end{aligned}$$

Compute precedence relation for production $S \rightarrow aSSb$; take pairs on the RHS as follows:

- Relations between a and S
 Equal precedence ($\dot{=}$): $a \dot{=} S$
 Less-than precedence ($<\cdot$) using definition $a < \cdot FIRST^+(S)$: $a < \cdot a$, $a < \cdot c$
- Relations between S and S
 Equal precedence ($\dot{=}$): $S \dot{=} S$
 Less than precedence ($<\cdot$) using definition $S < \cdot FIRST^+(S)$: c
 Greater than precedence ($\cdot >$) using definition $LAST^+(S) \cdot > FIRST^*(S)$:

$b \cdot > a, b \cdot > c, b \cdot > S, c \cdot > a, c \cdot > c, c \cdot > S$

□ Relation between S and b

Equal precedence ($\doteq$): $S \doteq b$

Greater than precedence using definition $LAST^+(s) \cdot > FIRST^*(b)$: $b \cdot > b, c \cdot > b$

Compute precedence relation for production $S \rightarrow c$:

Since there is only one symbol, no new relations are added to SPM.

Compute relations for end marker \$ using definitions $\$ \cdot < FIRST^+(S)$ and $LAST^+(S) \cdot > \$$

$\$ \cdot < a, \$ \cdot < c$

$b \cdot > \$, c \cdot > \$$

Otherwise, treat \$ to have less than precedence ($\cdot <$) with all symbols of the grammar and all symbols to have greater than precedence ($\cdot >$) than \$.

Populating the precedence relations in the matrix, we have the required SPM presented in Table 8:

Table 8: SPM for Example 8					
	S	a	b	c	\$
S	$\doteq$	$\cdot <$	$\doteq$	$\cdot <$	error
a	$\doteq$	$\cdot <$	error	$\cdot <$	error
b	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$
c	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$	$\cdot >$
\$	error	$\cdot <$	error	$\cdot <$	error

Computing Simple Precedence Relations --Method II

Computing precedence relations using method-I requires computing FIRST and LAST relations sets and corresponding $FIRST^+$, $FIRST^*$, and $LAST^+$ sets for each symbols. For large grammar, this method may become complex and difficult. Alternatively, these can be represented in Boolean matrix form, which makes them even easier for computer implementations. Therefore, FIRST and LAST are $n \times n$ Boolean matrices storing entry 1 for the first and last relations between LHS and corresponding RHS, where n is the total number of symbols of the grammar.

FIRST matrix maintains first relations (has 1 entry) between LHS and corresponding first symbol of RHS for each of the productions of the grammar. Similarly, LAST matrix contains relations (1 entry) between LHS and corresponding last symbol of RHS for each of the productions of the grammar. The general algorithm to compute precedence relations $\cdot <$, $\doteq$, and $\cdot >$ can be outlined as follows:

1. Compute FIRST and LAST matrices
2. Compute $FIRST^+$ and $LAST^+$ using Warshall algorithm to compute the transitive closure of the two matrices
3. Compute matrix $FIRST^* = I + FIRST^*$, where I is the identity matrix
4. Compute precedence relations matrices using the following definitions:
 - a. ($\doteq$) : Enter 1 for each location (X,Y) in the matrix, where X and Y are adjacent symbols on the RHS of the productions
 - b. ($\cdot <$) : ($\doteq$)($FIRST^+$)
 - c. ($\cdot >$) : ($LAST^+$)^T($\doteq$)($I + FIRST^+$) or ($LAST^+$)^T($\doteq$)($FIRST^*$)
 - d. All other entries could be set as 0, as it denotes error

4. Superimpose ($\stackrel{*}{=}$), ($\prec$), and ($\succ$) to get the required SPM

Let us illustrate this through the following example. Note that the discussions use label for rows and columns of matrices for notational convenience only.

Example 9: Generate the SPM for the previous problem using matrix method for the following grammar rules:

$$S \rightarrow aSSb \mid c$$

Solution: Construct FIRST and LAST matrices as follows:

$$EQUAL(\stackrel{*}{=}) = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 1 & 0 & 1 & 0 \\ a & 1 & 0 & 0 & 0 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array} \quad (1)$$

$$FIRST = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 0 & 1 & 0 & 1 \\ a & 0 & 0 & 0 & 0 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array}$$

$$FIRST^+ = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 0 & 1 & 0 & 1 \\ a & 0 & 0 & 0 & 0 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array}$$

$$FIRST^* = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 1 & 1 & 0 & 1 \\ a & 0 & 1 & 0 & 0 \\ b & 0 & 0 & 1 & 0 \\ c & 0 & 0 & 0 & 1 \end{array}$$

$$(\prec) = (\stackrel{*}{=})(FIRST^+) = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 1 & 0 & 1 & 0 \\ a & 1 & 0 & 0 & 0 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array} \times \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 0 & 1 & 0 & 1 \\ a & 0 & 0 & 0 & 0 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array} = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 0 & 1 & 0 & 1 \\ a & 0 & 1 & 0 & 1 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array} \quad (2)$$

$$LAST = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 0 & 0 & 1 & 1 \\ a & 0 & 0 & 0 & 0 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array}$$

$$LAST^+ = \begin{array}{c|cccc} & S & a & b & c \\ \hline S & 0 & 0 & 1 & 1 \\ a & 0 & 0 & 0 & 0 \\ b & 0 & 0 & 0 & 0 \\ c & 0 & 0 & 0 & 0 \end{array}$$

$$(\cdot >) = (LAST^+)^T (\dot{=}^*) (FIRST^*) = \begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} \end{array} \times \begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} 1 \\ 1 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 1 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} \end{array} \times \begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} 1 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 1 \\ 1 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \\ 0 \end{array} & \begin{array}{c} 1 \\ 0 \\ 1 \\ 0 \end{array} \end{array}$$

$$(\cdot >) = (LAST^+)^T (\dot{=}^*) (FIRST^*) = \begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \\ 1 \end{array} & \begin{array}{c} 0 \\ 0 \\ 1 \\ 1 \end{array} \end{array} \quad (3)$$

Rewriting equations (1), (2), and (3) with the corresponding precedence symbols (replace entries 1 with $\dot{=}$, $<$, and $>$), we have

$$\begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} \dot{=} \\ \dot{=} \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} \dot{=} \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} \end{array} (\cdot <) = \begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} < \\ < \\ 0 \\ 0 \end{array} & \begin{array}{c} 0 \\ 0 \\ 0 \\ 0 \end{array} & \begin{array}{c} < \\ < \\ 0 \\ 0 \end{array} \end{array} (\cdot >) = \begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} 0 \\ 0 \\ \cdot > \\ \cdot > \end{array} & \begin{array}{c} 0 \\ 0 \\ \cdot > \\ \cdot > \end{array} & \begin{array}{c} 0 \\ 0 \\ \cdot > \\ \cdot > \end{array} & \begin{array}{c} 0 \\ 0 \\ \cdot > \\ \cdot > \end{array} \end{array} \quad (4)$$

Now superimposing all three precedence from equation (4), we obtain SPM as follows:

$$SPM = \begin{array}{ccccc} & S & a & b & c \\ \begin{array}{c} S \\ a \\ b \\ c \end{array} & \begin{array}{c} \dot{=} \\ \dot{=} \\ \cdot > \\ \cdot > \end{array} & \begin{array}{c} < \\ < \\ \cdot > \\ \cdot > \end{array} & \begin{array}{c} \dot{=} \\ 0 \\ \cdot > \\ \cdot > \end{array} & \begin{array}{c} < \\ < \\ \cdot > \\ \cdot > \end{array} \end{array}$$

Bottom-Up Parsing Algorithm Using SPM

The simple precedence parsing makes use of stack structure and precedence relation table constructed before. The steps for bottom-up parse using SPM can be outlined as follows:

1. Compute SPM as discussed before.
2. Prepare stack (push \$ on the stack) and input string (append \$ at the end).
3. While not stack equals to \$\$ and input equals to \$ (where S is the exclusive start symbol), search SPM for the relationship between top of the stack content and next token symbol from the input.
 - a. If the relationship is $\dot{=}$ or $<$ (then shift), then perform the following steps:
 - Push the relationship on the stack (Note: If \$ is top of stack, then push $<$ on the stack because \$ has less precedence than any other symbol of the grammar.)
 - Push the next token symbol from the input on the stack
 - Remove the next token symbol from the input
 - b. If the relationship is $>$ (then reduce), then perform the following steps:
 - Search for the production to reduce: Match the elements from the pivot on the stack on the RHS of the productions (pivot on the stack has elements from the nearest $<$ from the top of the stack)
 - If match found, then
 - Pop off the pivot contents from the stacks
 - Search the SPM for relationship between the non-terminal (the LHS) of the matched production and the element on the stack top
 - If valid relation found, then

- Push the relationship of the stack
 - Else
 - Exit with message precedence relation does not exist--unsuccessful parse
 - Push the non-terminal (the LHS) on the stack
 - c. Else
 - Exit with message precedence relation does not exist--unsuccessful parse
 - 4. If stack equals to \$\$ and input next token equals to \$, then
 - Exit with message 'Parsing successful'!
- Entries of the precedence table left unfilled denote error entries and can be denoted by a '0' or '-' or 'error' entry in the cells.

Example 10: Consider the SPM generated in Example 9, which is given as follows:

$$SPM = \begin{array}{c|ccc} & S & a & b & c \\ \hline S & \cdot & < & \cdot & < \\ a & \cdot & < & 0 & < \\ b & \cdot > & \cdot > & \cdot > & \cdot > \\ c & \cdot > & \cdot > & \cdot > & \cdot > \end{array}$$

Parse the input string *aaccbcb*. \$ has less precedence than any other symbol of the grammar.

Solution: Rewriting the grammar production rules, we have the following two productions:

- $S \rightarrow aSSb$
- $S \rightarrow c$

Table 9 presents the steps of parsing string *aaccbcb* using SPM:

	Stack contents	Precedence (between TOS symbol and Next token input)	Next token input	Input string	Remarks
0.	\$	<	a	aaccbcb\$	Shift and push <a on the stack
1.	\$<a	<	a	accbcb\$	Shift and push <a on the stack
2.	\$<a<a	<	c	ccbcb\$	Shift and push <c on the stack
3.	\$<a<a<c	>	c	cbcb\$	Reduce by $S \rightarrow c$, update precedence relation between TOS symbol and S, push S.
4.	\$<a<a $\cdot$ S	<	c	cbcb\$	Shift and push <c on the stack
5.	\$<a<a $\cdot$ S<c	>	b	bc b\$	Reduce by $S \rightarrow c$, update precedence relation between TOS symbol and S, push S.

Table 9: Parsing Steps for aacbcbb Using SPM

	Stack contents	Precedence (between TOS symbol and Next token input)	Next token input	Input string	Remarks
6.	$\$ < a < \cdot a \equiv S \equiv S$	$\equiv$	b	bcbb\$	Shift and push $\equiv b$ on the stack
7.	$\$ < a < \cdot a \equiv S \equiv S \equiv b$	$\cdot >$	c	cb\$	Reduce by $S \rightarrow aSSb$, update precedence relation between TOS symbol and S, push S.
8.	$\$ < \cdot a \equiv S$	$< \cdot$	c	cb\$	Shift and push $< \cdot c$ on the stack
9.	$\$ < \cdot a \equiv S < \cdot c$	$\cdot >$	b	b\$	Reduce by $S \rightarrow c$, update precedence relation between TOS symbol and S, push S.
10.	$\$ < \cdot a \equiv S \equiv S$	$\equiv$	b	b\$	Shift and push $\equiv b$ on the stack
11.	$\$ < \cdot a \equiv S \equiv S \equiv b$	$\cdot >$	\$	\$	Reduce by $S \rightarrow aSSb$, update precedence relation between TOS symbol and S, push S.
12.	\$S		\$		Accept aacbcbb Parsing successful!

■ Operator Precedence Parsing

An *operator precedence parser* is another class of bottom-up parser that uses operator precedence rules to recognize valid expressions. Operator precedence parsers are designed to work with operator grammars. *Operator grammars* are those grammars that do not permit two adjacent non-terminals on their RHS of the production rules, i.e., non-terminals must be separated by at least one terminal symbols. Terminal symbols for that reason are called operators, thus giving rise to the name operator grammar. Formally, an operator grammar does not contain production of the form $U \rightarrow \dots N_1 N_2 \dots$, where U , N_1 , and N_2 are the non-terminals of the production.

Notably, operator precedence parsers are simple shift-reduce parsers that also have the capability of parsing a subset of LR(1) grammars, which are discussed in later sections. They can parse all LR(1) grammars where two consecutive non-terminals never appear on the RHS of any production rule. LR(1) is practiced at large, but has some features that make it useful within larger designs. It is simple in design and uses operator table at runtime, which gives the parser flexibility to parse languages that allow change in their operators while parsing.

The operator precedence parser constructs and uses operator precedence table that contains all possible precedence relations between operator symbols used in the grammar. More importantly, this parser differs from simple precedence parsers. Simple precedence parsing method defines precedence relations between all symbols of the grammar including terminal and non-terminal of the grammar, whereas operator precedence parsing defines and uses precedence relations only between terminals of

the grammar (the operators). Three precedence relations, namely, equal, less than, and greater than, are denoted by $\equiv$, $\prec$, and $\succ$, respectively, and can be defined as follows:

1. Equal precedence ($\equiv$): $T_1 \equiv T_2$ if and only if there exists a production rule
 $U \rightarrow \dots T_1 T_2 \dots$ or $U \rightarrow \dots T_1 N T_2 \dots$
2. Less than ($\prec$): $T_1 \prec T_2$ if and only if there exists a production rule
 $U \rightarrow \dots T_1 N_1 \dots$ such that $N_1 \Rightarrow^+ T_2 \dots$ or $N_1 \Rightarrow^+ N_2 T_2 \dots$
3. Greater than ($\succ$): $T_1 \succ T_2$ if and only if there exists a production rule
 $U \rightarrow \dots N_1 T_2 \dots$ such that $N_1 \Rightarrow^+ \dots T_1$ or $N_1 \Rightarrow^+ \dots T_1 N_2$

where $T_1, T_2 \in T$ (terminal set or operators) and $N_1, N_2 \in N$ (the non-terminal set).

We can construct the operator precedence table, also known as Operator Precedence Matrix (OPM), holding the precedence relations for the given grammar using the preceding rules. Alternatively, we can also use Boolean matrices to construct the relation matrix similar to what we have discussed in method-II for simple precedence parsing. In addition to the FIRST and LAST matrices holding the first and last relations for the production, we define FIRSTTERM and LASTTERM relations, which will be used to construct the OPM.

Let G be an operator grammar. Let T and N be the terminal and non-terminal symbols, then we define $U\text{-FIRSTTERM}\text{-}T$ for some production $U \rightarrow \alpha$ if and only if $\alpha = T\dots$ or $\alpha = NT\dots$. This means $U\text{-FIRSTTERM}\text{-}T$ holds only for the terminals on the RHS of the production that are either in the first or second position. Similarly, we define $U\text{-LASTTERM}\text{-}T$ for some production $U \rightarrow \alpha$ if and only if $\alpha = \dots T$ or $\alpha = \dots TN$. This means $U\text{-LASTTERM}\text{-}T$ holds only for the terminals on the RHS of the production, which are either in the last or second from the last position. Using these terms, we define the precedence relation for the operator grammar as follows:

1. Equal precedence ($\equiv$) is the entry for two terminals on the RHS separated by a single non-terminal
2. Less than precedence ($\prec$) = $(\overset{\cdot}{\equiv})(\text{FIRST}^*)(\text{FIRSTTERM})$
3. Greater than precedence ($\succ$) = $((\text{LAST}^*)(\text{LASTTERM}))^T(\overset{\cdot}{\equiv})$

Note that $(\overset{\cdot}{\equiv})$ is the usual equal relation matrix similar to what is used during the construction of simple precedence parsing to hold the first relation between LHS and the first symbol on the RHS including terminal and non-terminal. On the other hand, $(\equiv)$ is the equal matrix corresponding to operator grammar holding equal precedence only between operators (terminals) of the grammar. One obtained the three precedence matrices and superimposed these to get the OPM. Note that in the final relation matrices, the Boolean 1 entries are replaced by symbols $\equiv$, $\prec$, and $\succ$ for equal, less than, and greater than precedence relations, respectively. Each empty entry (or Boolean zero) is filled with a pointer to an error routine.

Moreover, note that once the operator precedence table is constructed, there must be unique precedence symbol at each entry and all entries must be only for terminal operator symbols. No entry should correspond to non-terminals. If so, then either the grammar is not an operator grammar or there must be some mistake in computing the OPM.

Let us now understand the operator precedence parsing algorithm. Parsing algorithm for operator grammar is similar to simple precedence grammar, except the fact that precedence is evaluated between terminals (operators) only. Non-terminals are meta-symbols that can take any value. Therefore, non-terminals do not affect the parsing process as such and the parsing is controlled by the operators of the grammar.

The operator precedence relations from OPM allow delimiting the handles in the right sentential forms during the parsing process. The $\prec$ relation marks the left end, while $\succ$ marks the right end of the handle and $\equiv$ appears in the interior of the handle. The operator precedence parsing also makes

use of shift-reduce mechanism and utilizes stack structure. The steps for bottom-up parse using OPM can be outlined as follows:

1. Compute the OPM with precedence relations
2. Prepare stack (push \$ on the stack) and input string (append \$ at the end)
3. Initialize next token = first input token
4. While top terminal on the stack $\neq$ \$ and input $\neq$ \$, search OPM for the relationship between topmost terminal on the stack and next token
 - If the relationship is $\Rightarrow$ or $\Leftarrow$ (then shift),
 - shift the next token onto the stack and advance input
 - Else, if the relationship is $\Rightarrow$, then
 - Find the shallowest pair of terminals x and y on the stack such that $x \Leftarrow y$, where x is deeper terminal
 - Pop everything above x off the stack
 - Push N (the general non-terminal) on the stack
5. If the topmost terminal on the stack = \$ and input = \$, then
 - Message 'Parsing successful'
 - Exit
 - Else
 - Message 'precedence undefined... Parsing unsuccessful'
 - Exit

The algorithm uses end marker \$, which is assumed to have less than precedence than all terminals of the grammar, and all terminals have greater than precedence than \$.

Let us illustrate this through the following example:

Example 11: Given the operator with production rules as follows:

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow (E) \mid a$$

Construct the operator precedence matrix containing precedence relations and parse the string $a*a$.

Solution: Construct equal precedence ($\Rightarrow$) matrix such that $T_1 \Rightarrow T_2$ if and only if there exists a production rule:

$$U \rightarrow \dots T_1 T_2 \dots \quad \text{or} \quad U \rightarrow \dots T_1 N T_2 \dots$$

$$\text{Equal precedence}(\Rightarrow) = \begin{array}{c|cccccccc} & E & T & F & + & * & (&) & a \\ \hline E & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ T & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ F & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ + & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ * & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ (& 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ a & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

Compute less than precedence $(\prec) = (\equiv)(FIRST^*)(FIRSTTERM)$

	<i>E</i>	<i>T</i>	<i>F</i>	+	*	(	)	<i>a</i>		<i>E</i>	<i>T</i>	<i>F</i>	+	*	(	)	<i>a</i>		<i>E</i>	<i>T</i>	<i>F</i>	+	*	(	)	<i>a</i>
<i>E</i>	0	0	0	1	0	0	1	0	<i>E</i>	1	1	1	0	0	1	0	1	<i>E</i>	0	0	0	1	0	0	0	0
<i>T</i>	0	0	0	0	1	0	0	0	<i>T</i>	0	1	1	0	0	1	0	1	<i>T</i>	0	0	0	0	1	0	0	0
<i>F</i>	0	0	0	0	0	0	0	0	<i>F</i>	0	0	1	0	0	1	0	1	<i>F</i>	0	0	0	0	0	1	0	1
= +	0	1	0	0	0	0	0	0	×	+	0	0	0	1	0	0	0	×	+	0	0	0	0	0	0	0
*	0	0	1	0	0	0	0	0	*	0	0	0	0	1	0	0	0	*	0	0	0	0	0	0	0	0
(	1	0	0	0	0	0	0	0	(	0	0	0	0	0	1	0	0	(	0	0	0	0	0	0	0	0
)	0	0	0	0	0	0	0	0	)	0	0	0	0	0	0	1	0	)	0	0	0	0	0	0	0	0
<i>a</i>	0	0	0	0	0	0	0	0	<i>a</i>	0	0	0	0	0	0	0	1	<i>a</i>	0	0	0	0	0	0	0	0

	<i>E</i>	<i>T</i>	<i>F</i>	+	*	(	)	<i>a</i>
<i>E</i>	0	0	0	0	0	0	0	0
<i>T</i>	0	0	0	0	0	0	0	0
<i>F</i>	0	0	0	0	0	0	0	0
($\prec$) = +	0	0	0	0	1	1	0	1
*	0	0	0	0	0	1	0	1
(	0	0	0	1	1	1	0	1
)	0	0	0	0	0	0	0	0
<i>a</i>	0	0	0	0	0	0	0	0

Therefore,

	<i>E</i>	<i>T</i>	<i>F</i>	+	*	(	)	<i>a</i>	
<i>E</i>	0	0	0	0	0	0	0	0	
<i>T</i>	0	0	0	0	0	0	0	0	
<i>F</i>	0	0	0	0	0	0	0	0	
($\prec$) = +	0	0	0	0	$\prec$	$\prec$	0	$\prec$	...(2)
*	0	0	0	0	0	$\prec$	0	$\prec$	
(	0	0	0	$\prec$	$\prec$	$\prec$	0	$\prec$	
)	0	0	0	0	0	0	0	0	
<i>a</i>	0	0	0	0	0	0	0	0	

Compute greater than precedence $(\succ) = ((LAST^*)(LASTTERM))^T(\equiv)$

	<i>E</i>	<i>T</i>	<i>F</i>	+	*	(	)	<i>a</i>
<i>E</i>	0	1	1	0	0	0	0	0
<i>T</i>	0	0	1	0	0	0	0	0
<i>F</i>	0	0	0	0	0	0	1	1
LAST = +	0	0	0	0	0	0	0	0
*	0	0	0	0	0	0	0	0
(	0	0	0	0	0	0	0	0
)	0	0	0	0	0	0	0	0
<i>a</i>	0	0	0	0	0	0	0	0

Therefore,

$$\begin{array}{c}
 \begin{array}{c} E \quad T \quad F \quad + \quad * \quad (\quad) \quad a \\
 E \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 T \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 F \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 ((LAST*(LASTTERM))^T = + \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 * \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 (\quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
) \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 a \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \end{array}
 \end{array}$$

$$\text{⌞} = [((LAST*(LASTTERM))^T(\dot{=}))]$$

$$\begin{array}{c}
 \begin{array}{c} E \quad T \quad F \quad + \quad * \quad (\quad) \quad a \\
 E \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 T \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 F \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 = + \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \times \\
 * \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 (\quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
) \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 a \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \end{array}
 \end{array}$$

$$\begin{array}{c}
 \begin{array}{c} E \quad T \quad F \quad + \quad * \quad (\quad) \quad a \\
 E \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 T \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 F \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 \text{⌞} = + \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 0 \\
 * \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \\
 (\quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
) \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \\
 a \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \end{array}
 \end{array}$$

Now, superimpose ($\oplus$), ($\otimes$), and ($\oslash$) matrices and obtain OPM as follows:

$$\begin{array}{c}
 \begin{array}{c} E \quad T \quad F \quad + \quad * \quad (\quad) \quad a \\
 E \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 T \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 F \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 + \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 0 \\
 * \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \\
 (\quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
) \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \\
 a \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \end{array}
 \end{array}$$

$$\begin{array}{c}
 \begin{array}{c} + \quad * \quad (\quad) \quad a \\
 + \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \\
 * \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \\
 (\quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \\
) \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \\
 a \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \quad \text{⌞} \end{array}
 \end{array}$$

Table 10 shows the steps in parsing $a+a*a$ using OPM generated for the preceding example:

Table 10: Parsing of String $a+a*a$				
Stack contents	Precedence (between operator on TOS and input terminal)	Next token input terminal	Input string	Remarks
\$	$\prec$	a	$a+a*a\$$	Shift token on the stack and advance input
$\$a$	$\succ$	$+$	$+a*a\$$	As $\$ \prec a \succ +$, reduce using production $N \rightarrow a$
$\$N$	$\prec$	$+$	$+a*a\$$	Shift token on the stack and advance input
$\$N+$	$\prec$	a	$a*a\$$	Shift token on the stack and advance input
$\$N+a$	$\succ$	$*$	$*a\$$	As $+ \prec a \succ *$, reduce using production $N \rightarrow a$
$\$N+N$	$\prec$	$*$	$*a\$$	Shift token on the stack and advance input
$\$N+N*$	$\prec$	a	$a\$$	Shift token on the stack and advance input
$\$N+N*a$	$\succ$	$\$$	$\$$	As $* \prec a \succ \$$, reduce using production $N \rightarrow a$
$\$N+N*N$	$\succ$	$\$$	$\$$	As $+ \prec * \succ \$$, reduce using production $N \rightarrow N*N$
$\$N+N$	$\succ$	$\$$	$\$$	As $\$ \prec + \succ \$$, reduce using production $N \rightarrow N+N$
$\$N$		$\$$	$\$$	Accept $a+a*a$ Parsing Successful!

Note that $\$ \prec T$ and $T \succ \$$ for all T , where T is a terminal (operators of the grammar). Although not explicitly constructed in the OPM, parsing process assumes and uses these relations by default. Moreover, as we are concerned with only terminals called operators during parsing, we use general non-terminal N for all non-terminals during the parse.

Although operator precedence parsing is simple in design and powerful enough for handling expressions in programming languages, it cannot handle the unary minus and works for only small class of grammars. It expects the lexical analyzer to handle the unary minus. Operator precedence parsers run entirely on a small table (OPM) with information about the operators. Notably, operator precedence grammar can construct the parse tree even for ambiguous grammars.

Next, let's discuss about precedence functions.

■ Precedence Functions

It is observed that for grammar with large number of grammar symbols, the size of the precedence matrix/table becomes large. This leads to sparse entries, leaving most of the cells without any precedence relations that result in wastage of memory. We must avoid such wastage and improve runtime efficiency. Therefore, the precedence parsers may not store the entire precedence table; instead, they can be implemented by using *precedence functions* that map terminal symbols to

integers. *Precedence functions* are functions that output integer values corresponding to each grammar symbols, and therefore, the precedence relations between the symbols of the grammar can be implemented by numerical comparison. Note that not every table of precedence relations has precedence functions; however, in practice, for most grammars, such functions can be designed.

The integer values are much easier to compare and thus results in fast computation of handle during the parsing process. Formally, the information in the matrix can be equivalently represented by two functions f and g such that the following conditions hold:

$$X = Y \text{ implies } f(X) = g(Y)$$

$$X < Y \text{ implies } f(X) < g(S)$$

$$X > Y \text{ implies } f(X) > g(S)$$

for all symbols X and Y of the grammar. This brings down the storage requirements from $n*n$ locations to $2*n$ locations and this process is called *linearization* of the precedence matrix. In the following subsections, we outline the methods to compute these precedence functions.

Computing Precedence Function—Method I

The precedence functions for a given simple precedence matrix can be obtained using the following steps:

1. Draw a directed graph with $2*n$ nodes labeled $f_1, f_2, \dots, f_n$ and $g_1, g_2, \dots, g_n$.
2. Draw a directed arc from f_i to g_j if $S_i > S_j$.
3. Draw the directed edge from g_j to f_i if $S_i < S_j$.
4. Assign an integer number to each node of the graph, which is equal to the nodes accessible from that node counting itself. The number assigned to f_i is $f(S_i)$ and g_j if $g(S_j)$.
5. Test the computed functions f and g for inconsistencies, if any. If any inconsistency exists, then precedence functions do not exist; else, f and g are valid functions.

Note that f_i and g_j correspond to the respective grammar symbols in the order written in the precedence table. Let us consider the following example:

Example 12: Consider the SPM shown in Table 11 for the grammar definition:

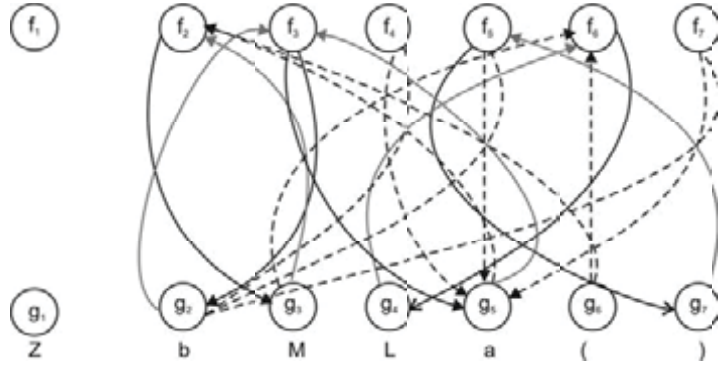
- $Z \rightarrow bMb$
- $M \rightarrow (L$
- $M \rightarrow a$
- $L \rightarrow Ma)$

Note that as mentioned earlier, the blank entries denote ‘error’ entries, i.e., no precedence relation exists for those entries.

Table 11: SPM for the Grammar in Example 12							
	Z	b	M	L	a	(	)
Z							
b			$\underline{\cdot}$		$<\cdot$	$<\cdot$	
M		$\underline{\cdot}$			$\underline{\cdot}$		
L		$\cdot>$			$\cdot>$		
a		$\cdot>$			$\cdot>$		$\underline{\cdot}$
(			$<\cdot$	$\underline{\cdot}$	$<\cdot$	$<\cdot$	
)		$\cdot>$			$\cdot>$		

Compute the precedence functions and parse the string $b(aa)b$.

Solution: Using the linearization algorithm mentioned before, we construct the directed graphs shown in Figure 5:



▲ Figure 5: Precedence Graph for SPM Presented in Table 11

Counting the unique nodes accessible from each node (counting itself) in the edge direction, we have

Nodes accessible from $f_1 = \text{count}\{f_1\} = 1$

Nodes accessible from $f_2 = \text{count}\{f_2, g_3, f_6, g_4\} = 4$

Nodes accessible from $f_3 = \text{count}\{f_3, g_2, g_5, f_2, f_6, g_3, g_4\} = 7$

Nodes accessible from $f_4 = \text{count}\{f_4, g_2, g_5, f_3, f_6, f_2, g_3, g_4\} = 8$

Nodes accessible from $f_5 = \text{count}\{f_5, g_2, g_5, g_7, f_3, f_2, f_6, g_3, g_4\} = 9$

Nodes accessible from $f_6 = \text{count}\{f_6, g_4\} = 2$

Nodes accessible from $f_7 = \text{count}\{f_7, g_2, g_5, f_3, f_2, f_6, g_3, g_4\} = 8$

Nodes accessible from $g_1 = \text{count}\{g_1\} = 1$

Nodes accessible from $g_2 = \text{count}\{g_2, f_3, g_5, f_2, f_6, g_3, g_4\} = 7$

Nodes accessible from $g_3 = \text{count}\{g_3, f_6, f_2, g_4\} = 4$

Nodes accessible from $g_4 = \text{count}\{g_4, f_6\} = 2$

Nodes accessible from $g_5 = \text{count}\{g_5, f_3, f_6, g_2, f_2, g_3, g_4\} = 7$

Nodes accessible from $g_6 = \text{count}\{g_6, f_6, f_2, g_4, g_3\} = 5$

Nodes accessible from $g_7 = \text{count}\{g_7, f_5, g_2, g_5, f_3, f_2, f_6, g_3, g_4\} = 9$

Therefore, the precedence functions corresponding to grammar symbols $\{Z, b, M, L, a, (,)\}$ is presented in Table 12:

Table 12: Table of Precedence Functions							
	Z	b	M	L	a	(	)
f	1	4	7	8	9	2	8
g	1	7	4	2	7	5	9

Note that the functions are not unique, i.e., there can be infinite number of such functions f and g if any exist at all. Also, there are many precedence matrices for which no precedence functions exists.

Computing Precedence Functions—Method II

It has been aforementioned that for computer implementation, computation can be more simple using Boolean matrices during computation of such relations. Therefore, we outline the computation of precedence function using Boolean matrix method. Note that this method is actually the same as what has been discussed in method I, the only difference being in the paths represented by Boolean matrices.

The steps for computing precedence functions for a given precedence matrix are as follows:

1. Construct a Boolean matrix containing greater than and equal relations denoted by GE, i.e., $GE = [\cdot>, \dot{=}]_{n \times n}$.
2. Construct a Boolean matrix containing less than and equal relations denoted by LE, i.e., $LE = [<\cdot, \dot{=}]_{n \times n}$.

3. Construct a Boolean matrix B such that $B = \begin{pmatrix} \text{○} & [GE] \\ [LE]^T & \text{○} \end{pmatrix}_{2n \times 2n}$.
4. Compute a reflexive transitive closure $B^* = (B^+ + I)$.
5. Count the number of 1's in the first n row to get f_i . Similarly, count the number 1's in the next n-rows to get g_i .

Parsing Using Precedence Functions

Parsing using precedence function has simplified comparison that makes use of only integer comparison from the precedence function table. During the parse process, we compare two adjacent token symbols on their f and g values. The integer comparison is performed on f value of first and g of second of the two adjacent symbols of the input token.

The method works well for both simple precedence grammar and operator grammar and can use the parsing algorithm discussed earlier for respective cases. When implemented using stack, we compare the f(top of stack) symbol or the topmost operator as the case may be (for SPG or OPG) with g(first input token). The handle is enclosed between < and > values computed by f and g. Again, we assume that the end marker \$ has least precedence than any other symbol of the grammar.

While working for operator grammars, we use the general non-terminal N to perform reduction; on the other hand, for simple grammar, actual non-terminals are used. Basically, parsing remains the same with the only change in comparing integer rather than precedence notations, which work fast for machine implementations. Let us consider the following example:

Example 13: Consider the grammar and simple precedence matrix discussed in Example 12. Construct the precedence function using method II.

Solution: Construct the Boolean GE (greater-than-equal to) matrix. We consider all entries, which contain $\dot{=}$ and $\cdot>$.

The Matrix with Boolean GE is:

$$\begin{array}{c}
 \begin{array}{cccccc}
 & Z & b & M & L & a & (&) \\
 Z & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 b & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
 M & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 L & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 a & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\
 (& 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
) & 0 & 1 & 0 & 0 & 1 & 0 & 0
 \end{array} \\
 \text{GE} =
 \end{array} \quad \dots(1)$$

Now construct the Boolean LE (less than-equal to) matrix. We have to consider all entries containing $\dot{=}$ and $<$. The Boolean LE matrix is:

$$\begin{array}{c}
 \begin{array}{cccccc}
 & Z & b & M & L & a & (&) \\
 Z & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 b & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\
 M & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 L & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 a & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 (& 0 & 0 & 1 & 1 & 1 & 1 & 0 \\
) & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array} \\
 \text{LE} =
 \end{array} \quad \dots(2)$$

Next the construct $(\text{LE})^T$ as presented below:

$$\begin{array}{c}
 \begin{array}{cccccc}
 & Z & b & M & L & a & (&) \\
 Z & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 b & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
 M & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\
 L & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
 a & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\
 (& 0 & 1 & 0 & 0 & 0 & 1 & 0 \\
) & 0 & 0 & 0 & 0 & 1 & 0 & 0
 \end{array} \\
 (\text{LE})^T =
 \end{array} \quad \dots(3)$$

Then construct the matrix $B = \begin{pmatrix} \text{O} & [\text{GE}] \\ [\text{LE}]^T & \text{O} \end{pmatrix}_{2n \times 2n}$, containing LE and LE^T as shown below:

$$\begin{array}{c}
 \begin{array}{cccccc}
 & Z & b & M & L & a & (&) & Z & b & M & L & a & (&) \\
 Z & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 b & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
 M & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 L & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 a & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\
 (& 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
 Z & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 b & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 M & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 L & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 a & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 (& 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
) & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array} \\
 \text{B} =
 \end{array} \quad \dots(4)$$

Now construct the matrix B^+ using Warshall algorithm (new entries are shown within circle), as presented in below:

$$B^+ = \begin{matrix} & \begin{matrix} Z & b & M & L & a & (&) \end{matrix} & \begin{matrix} Z & b & M & L & a & (&) \end{matrix} \\ \begin{matrix} Z \\ b \\ M \\ L \\ a \\ (\\) \end{matrix} & \begin{matrix} \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix} \end{matrix} & \begin{matrix} \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \end{matrix} \end{matrix} \quad \dots(5)$$

Further, construct the matrix $B^* = (I + B^+)$, the reflexive transitive closure, as shown below:

$$B^* = \begin{matrix} & \begin{matrix} Z & b & M & L & a & (&) \end{matrix} & \begin{matrix} Z & b & M & L & a & (&) \end{matrix} \\ \begin{matrix} Z \\ b \\ M \\ L \\ a \\ (\\) \end{matrix} & \begin{matrix} \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 1 \end{pmatrix} \end{matrix} & \begin{matrix} \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \end{matrix} \end{matrix} \quad \dots(6)$$

By counting the first n-rows, we get f, and by counting the next n-rows, we get g; hence, the precedence function for the preceding matrix is given in Table 13:

Table 13: Table with Precedence Function							
	Z	b	M	L	a	(	)
f	1	4	7	8	9	2	8
g	1	7	4	2	7	5	9

Example 14: Given the precedence function for the simple precedence grammar:

- $Z \rightarrow bMb$
- $M \rightarrow (L$
- $M \rightarrow a$
- $L \rightarrow Ma)$

in Table 14:

Table 14: Simple Precedence Function

	Z	b	M	L	a	(	)
f	1	4	7	8	9	2	8
g	1	7	4	2	7	5	9

Parse the string $b((aa)a)b$.

Solution: The step in parsing using SPF in Table 14 for the given grammar is presented in Table 15:

Table 15: Parsing of String $b((aa)a)b$ Using SPF

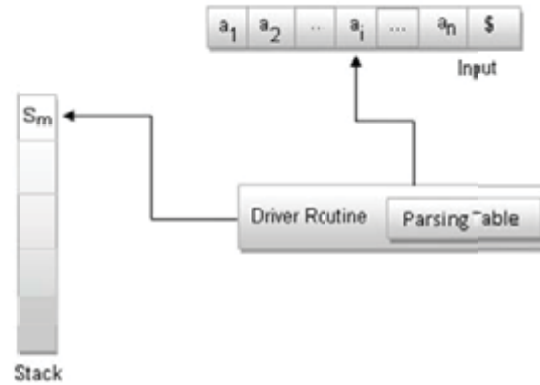
Stack contents	Comparison of f and g	Next token input terminal	Input string	Remarks
\$	f(\$)<g(b)	b	b((aa)a)b\$	Shift
\$<b	f(b)< g('(')	(	((aa)a)b\$	Shift
\$<b<(	f('(')<g('')	(	(aa)a)b\$	Shift
\$<b<(<(	f('(')<g(a)	a	aa)a)b\$	Shift
\$<b<(<(<a	f(a)>g(a)	a	a)a)b\$	Reduce using M→a
\$<b<(<(<M	f(M)=g(a)	a	a)a)b\$	Shift
\$<b<(<(<M=a	f(a)=g('')	)	)a)b\$	Shift
\$<b<(<(<M=a=)	f('')>g(a)	a	a)b\$	Reduce using L→Ma)
\$<b<(<(<=L	f(L)>g(a)	a	a)b\$	Reduce using M→(L
\$<b<(<M	f(M)=g(a)	a	a)b\$	Shift
\$<b<(<M=a	f(a)>g('')	)	)b\$	Shift
\$<b<(<M=a=)	f('')>g(b)	b	b\$	Reduce using L→Ma)
\$<b<(<=L	f(L)>g(b)	b	b\$	Reduce M→(L
\$<b=M	f(M)=g(b)	b	b\$	Shift
\$<b=M=b	f(b)<g(\$)	\$	\$	Reduce using Z→bMb
\$<Z		\$	\$	Accept Parsing successful

You have learned about top-down and bottom-up parsing. Now, let's learn about LR parsers in the next section.

3.4 LR Parsers

LR parsers are an important class of efficient bottom-up recognizers that scan inputs from left-to-right and construct rightmost derivation in reverse for large class of CFGs. These can be used for most of the programming languages based on CFGs. Note that these are more general than any other type of table-driven shift-reduce parsers and have the capability to identify as well as report semantic errors in early stage of parsing for a given input. LR parsers often show better performance and recognition capability as compared to other parsers, specifically predictive LL parsers.

LR parsers contain driver routine and parsing table. The functionality of driver routine is more or less same in all kinds of LR parsers, while parsing table changes from parser to parser. The parsing tables contain all the information that drives the parser. The general schematic of an LR parser is shown in Figure 6:



▲ Figure 6: Schematics of a General LR Parser

The LR parser encompasses a parsing table, a driver routine, a stack, and an input token. The stack maintains a sequence of the form $s_0X_1s_1X_2...X_is_m$, where X_i is the grammar symbol and s_m is the state on top. The parser during its course of action reads the state symbol on top of the stack and the current input symbol, and determines the shift-reduce parsing decision in consultation with the parsing table. The structure and content of the stack control the process and determine the handle to reduce in a sentential form. Parsing depends on the sequence of tokens on the stack and contextual information available at the current position. These parsers not only shift and push tokens, but also purposely push states onto the stack that describes what is on the stack so far. These states provide the current left context, which is the important feature of LR parsers. The state on top of the stack together with the lookahead information enables us to decide on whether to reduce the handle or to shift a new state on top of the stack for the next input token.

The design of an LR parser contains a parsing table, a driver routine, a stack, and an input token. The parsing table contains all the information that drives the parser. It has ACTION and GOTO stubs. ACTION defines shift, reduce, and accept moves of the parser on terminal inputs, while GOTO defines the new state to be pushed on top of the stack after a reduction of the non-terminal X . The ACTION stub entries $ACTION[s_m, a]$ control parser action and enable the parser to decide what to do when the state on top of the stack is s and next input token is terminal a . The probable actions that a parser for entry $ACTION[s_m, a]$ would take are as follows:

1. Shift a state s onto the stack
2. Reduce the handle β on top of the stack ($A \rightarrow \beta$)
3. Accept the input
4. Report an error

The GOTO stub with entries $GOTO[S, X]$ defines the new state to be pushed on top of the stack after a reduction of the non-terminal X . It determines that the state s is on top of the stack and grammar symbol and produces a state. The GOTO table entries are basically the transition functions of a DFA that recognizes the viable prefixes of the given grammar. Many a time conflict occurs when the parser cannot decide whether to either shift or reduce the top of stack known as a shift/reduce conflict. Alternatively, conflict also arises in some cases when the parser cannot determine whether to reduce the top of stack using one of the two possible productions. This is called a reduce-reduce conflict.

■ LR Configuration and Tracing

A convenient notation to describe the configuration of an LR parser is a pair whose first component depicts the stack contents and second component is the token inputs. The typical configuration of an LR parser at any instant could be described as follows:

$$(s_0 X_1 s_1 X_2 s_2 \dots X_m s_m, a_i a_{i+1} \dots a_n \$)$$

This again essentially represents the right-sentential form $X_1 X_2 \dots X_m a_i a_{i+1} \dots a_n$ of shift-reduce parser with embedded states on the stack. The current configuration of the LR parser determines the next move of the parser by consulting table entry $\text{ACTION}[s_m, a_i]$ for input a_i from the token and state s_m from the stack top. Note that we are just looking at the *state* here and no symbol below it. The possible changes in the configuration after each possible move are as follows:

1. If $\text{ACTION}[s_m, a_i] = \text{shift } s$, then the parser shifts input a_i and state s on the stack, resulting into the new configuration of the parser as

$$(s_0 X_1 s_1 X_2 s_2 \dots X_m s_m a_i s, a_{i+1} \dots a_n \$)$$

where a_{i+1} is the next input under the parser head with current state s on the stack top.

2. If $\text{ACTION}[s_m, a_i] = \text{reduce } A \rightarrow \beta$, then the parser executes a reduce move and removes top $2r$ entries from the stack, where r is the length of β . This exposes s_{m-r} on top of the stack. It then pushes A and s on the stack, where A is the LHS of production $A \rightarrow \beta$ being reduced and $s = \text{GOTO}[s_{m-r}, A]$. Note that the current input symbol is not changed in a reduce move and the new parser configuration becomes

$$(s_0 X_1 s_1 X_2 s_2 \dots X_{m-r} s_{m-r} A s, a_i a_{i+1} \dots a_n \$)$$

This reduce move can be associated with semantic action, thus enabling the parser to take appropriate action as the parsing progresses.

3. If $\text{ACTION}[s_m, a_i] = \text{accept}$, declare parsing successful.
4. If $\text{ACTION}[s_m, a_i] = \text{error}$, i.e., if the table entry is blank, then the parser has determined a syntax error and we can never arrive at a sentential form with a handle to reduce.

■ LR Parsing Algorithm

The LR parsing algorithm takes an input string w and parsing table with all possible ACTION and GOTO entries for a given grammar G . The output is success if w is in $L(G)$, otherwise outputs failure with an error indication. The general steps of LR parsing can be outlined as follows:

1. Initialize stack with s_0 , the initial state, and $w\$$ in the input buffer.
2. Repeat step 3 while stack or input buffer not empty
3. Consult the parsing table for entries $[s, a]$, where s is the state on top of the stack and a is the current symbol from the input token.
 - If $\text{ACTION}[s, a] = \text{shift } s'$, then
 - Push symbol a followed by ' s ' on top of the stack
 - Advance input head to obtain next symbol
 - Else if $\text{ACTION}[s, a] = \text{reduce } A \rightarrow \beta$, then
 - Pop off $2r$ top symbols from the stack ($r = |\beta|$)
 - Push A on the stack
 - Push the entry $\text{GOTO}[s', A]$ on the stack
 - Take the semantic action as desired
 - Else if $\text{ACTION}[s, a] = \text{accept}$, then

- ▶ Return with success.
- Else
 - ▶ Report Error.

Note that productions are numbered and notation r_i as well as s_i are used in the table, The entry r_i denotes the reduction by production number i , while s_i means shift state and stack i , respectively. In the following example, we show how to use the parsing table and determine the sequence of moves of a LR parser.

Example 15: Consider the following grammar rules and parsing table for simple expressions:

- $E \rightarrow E + T$
- $E \rightarrow T$
- $T \rightarrow (E)$
- $T \rightarrow id$

Table 16: Parsing Table with Possible ACTION and GOTO Entries							
State on top of stack	Action					GOTO	
	id	+	(	)	\$	E	T
0	s4		s3			1	2
1		s5			accept		
2	r2	r2	r2	r2	r2		
3	s4		s3				
4	r4	r4	r4	r4	r4	6	2
5	s4		s3				
6		s5		s7			8
7	r3	r3	r3	r3	r3		
8	r1	r1	r1	r1	r1		

Parse the string $id + (id)$ with the help of Table 16.

Solution: The sequence of moves in the LR parsing of $id + (id)$ using the LR algorithm with the above ACTION and GOTO table is presented in Table 17:

Table 17: Parsing Sequence for String $id + (id)$				
Current stack contents	Current symbol	Input token	Parser action	Remark
s_0	id	$id + (id)\$$	$ACTION[s_0, id] = S_4$	Stack id and s_4 onto stack, move ahead in input.
$s_0 \text{ id } s_4$	+	$+(id)\$$	$ACTION[s_4, +] = r_4$	Reduce by production 4. Pop two(length of RHS) entries from stack. Push T and $GOTO[s_0, T] = s_2$ on the stack.
$s_0 \text{ T } s_2$	+	$+(id)\$$	$ACTION[s_2, +] = r_2$	Reduce by production 2. Pop two(length of RHS) entries from stack. Push E and $GOTO[s_0, E] = s_1$ on the stack.

Table 17: Parsing Sequence for String $id + (id)\$$

Current stack contents	Current symbol	Input token	Parser action	Remark
$s_0 E s_1$	+	$+(id)\$$	$ACTION[s_1, +] = s_5$	Stack + and s_5 on to stack. Move ahead in input.
$s_0 E s_1 + s_5$	(	$(id)\$$	$ACTION[s_5, (] = s_3$	Stack '(' and s_3 on to stack. Move ahead in input.
$s_0 E s_1 + s_5 (s_3$	id	$id)\$$	$ACTION[s_3, id] = s_4$	Stack id and s_4 on to stack. Move ahead in input.
$s_0 E s_1 + s_5 (s_3 id s_4$	)	$)\$$	$ACTION[s_4,)] = r_4$	Reduce by production 4. Pop two (length of RHS) entries from stack. Push T and $GOTO[s_3, T] = s_2$ on the stack.
$s_0 E s_1 + s_5 (s_3 T s_2$	)	$)\$$	$ACTION[s_2,)] = r_2$	Reduce by production 2. Pop two (length of RHS) entries from stack. Push E and $GOTO[s_3, E] = s_6$ on the stack.
$s_0 E s_1 + s_5 (s_3 E s_6$	)	$)\$$	$ACTION[s_6,)] = s_7$	Stack ')' and s_7 on to stack. Move ahead in input.
$s_0 E s_1 + s_5 (s_3 E s_6) s_7$	\$	\$	$ACTION[s_7, \$] = r_3$	Reduce by production 3. Pop six (length of RHS) entries from stack. Push T and $GOTO[s_5, T] = s_8$ on the stack.
$s_0 E s_1 + s_5 T s_8$	\$	\$	$ACTION[s_8, \$] = r_1$	Reduce by production 1. Pop six (length of RHS) entries from stack. Push E and $GOTO[s_0, E] = s_1$ on the stack.
$s_0 E s_1$	\$	\$	$ACTION[s_1, \$] = \text{accept}$	Declare successful parse.

In the following section, let's explore the types of LR parsers.

■ Types of LR Parsers

A grammar is said to be LR if a left-to-right shift-reduce parser is able to recognize handles of right-sentential forms when they appear on top of the stack. LR parsers come in three flavor, namely, $LR(k)$, *simple LR(k)*, and *lookahead LR(k)*. Simple LR(k) and lookahead LR(k) are also abbreviated as SLR(k) and LALR(k), respectively, where k represents the number of tokens of lookahead used during the parsing. These k -lookaheads provide much of the power to LR parsers and enable them to take action based on contextual information provided by them. All types of LR parsers work in similar manner utilizing the ACTION and GOTO table entries, but differ in the construction of the table and their relative size to each other.

The strength of the parser is determined by the value of k in any type of LR parser. Specifying k give the parser some sort of futuristic power of determining the validity of the input token. For practical purposes, k is either 0 or 1; although $k > 1$ can very well be used, it would make the parser complex and bulky. The most simple of these parsers come in picture with $k=0$ of which LR(0) is the simplest of all the LR parsing methods. LR(0) parsers do not use any lookaheads and parse the token without using any lookahead at all. It is also the weakest of all types and not used much in practice because of its limitations.

Increasing the value of k vastly increases the parsing power, but the parser becomes more complex and the resulting parsing tables are more bulky. LR(0) parsers have limited capability in parsing grammar. LR(1) parsers, on the other hand, can parse most unambiguous CFGs, but at the same time, LR(1) parsing table for a typical programming language will have thousands of states as compared to the few hundred states needed for LR(0). Alternatives to overcome the bulky parser size are SLR(1) and LALR(1) parsers, which also use one token of lookahead but employ techniques that keep the table size as small as that of LR(0).

SLR(k) parsers are better than LR(0), but not as capable as that of a full LR(k) in terms of the number of grammars for which it can be applied. LALR(k) parses have advantage over SLR(k) in handling grammars, but not quite as many as LR(k).

The task of any LR parser is to identify a handle from the stack top that is to be reduced. Bottom-up LR recognizers are efficient in recognizing a handle than predicting a production in top-down parsing. During LR(k) parsing techniques, we must decide upon which production to exercise, having seen only k symbols of the RHS. On the other hand, an LR(k) grammar can postpone its decision until it analyzes the tokens corresponding to the entire RHS. In addition to the RHS, it can wait for k more tokens of lookahead.

■ LR(0) Parsing Table

Construction of an LR parsing table requires identifying the possible states and transitions among them. The table construction requires us to compute LR(0) configurations or an items. These LR items are nothing but grammar rules with a special marker 'dot' at some position on the RHS of the production. For example, given a production $A \rightarrow XYZ$, it has the following four possible items:

- $A \rightarrow \bullet XYZ$
- $A \rightarrow X \bullet YZ$
- $A \rightarrow XY \bullet Z$
- $A \rightarrow XYZ \bullet$

The position of the dot in the preceding LR items tells how far we have seen in parsing the production. Everything to the left of the dot has already been seen (explored) and has been pushed onto the parsing stack. Symbols to the right of the dot indicate symbols that are yet to be seen. The next input token symbol can be found by calculating the FIRST set of the immediate symbol after the dot or in the following set if that symbol is nullable.

Items of the form $A \rightarrow XYZ \bullet$ indicate that the entire configuration is on the stack and possibly we have obtained the handle that can be reduced. Items of the form $A \rightarrow \epsilon$ can generate only one item $A \rightarrow \bullet$. During the parsing, an item with a dot in the middle of the configuration indicates to shift a token symbol that follows the dot to continue. For example, given a configuration or an item $A \rightarrow X \bullet YZ$, it signifies that element of FIRST(Y) or Y itself is next to be shifted. Given the productions $Y \rightarrow u \mid w$, the item is equivalent to writing as:

- $A \rightarrow X \bullet YZ$
- $Y \rightarrow \bullet u$
- $Y \rightarrow \bullet w$

This simply means that we have already recognized X and anticipate next input to be a sequence derivable from YZ . Since Y has potential to generate u and w , it is equivalent to expect the sequence to be derivable from either u or w .

We can group these three items together into a set called *configuration set* of the LR parser. The configuration set represents the states of the LR parser and there is one state for each configuration set. The action of adding equivalent configurations to create a configuration set is called *closure*. Each state encompasses all the items corresponding to each of the possible paths concurrently being explored at that point in the parse. This can be very well modeled as a finite automaton moving from one state to another through transitions on symbols of the CFG. Each time we perform a shift during the parse of the input, we follow a transition to a new state. Note that the configuration set or the closure may have new items added to the closure if a non-terminal follows the dot and productions are defined for the same.

As already discussed, LR parsers make use of parsing table which has ACTION and GOTO entries. ACTION is defined for terminal, while GOTOs are moves for a non-terminal that is encountered during the parse. LR parsers have requirement that it must be augmented to work with. *Augmenting* a grammar has effect of adding new symbols to the grammar such that the exclusive symbol of the grammar does not appear on the RHS of the production. This would have the general effect of introducing a new exclusive symbol, say S' , and an exclusive production. The parser would announce the acceptance only when it is about to reduce by. In order to construct the canonical LR(0) set of item, we use functions CLOSURE and GOTO defined as follows:

Computing CLOSURE set (or configuration set)

Consider a set of items I for a grammar G . The set of items CLOSURE(I) is computed from I using the following rules.

1. Every item $A \rightarrow \alpha \bullet B \beta$ in I is in the CLOSURE(I).
2. If B begins with a terminal, we are done with this production.
3. If B is a non-terminal, then add all productions with B on the left side, with the dot at the start of the right side. For example, if $B \rightarrow \gamma$ is a production, then add $B \rightarrow \bullet \gamma$ to the CLOSURE(I) if it is not already in the CLOSURE set.
4. Repeat steps 2 and 3 for any productions added in step 3. Continue until you no new CLOSURE items can be created.

The GOTO entries of the parsing table are other important information that we need to build. These are the transitions between the CLOSURE (configuration set with set of items) sets that represent states of the automaton. We can simply refer CLOSURE as set of items for short. The successor function computes the new configuration or CLOSURE sets and uses for populating the parsing table entries. It is defined in terms of the set of items and a grammar symbol X .

Given a CLOSURE set I and a grammar symbol X , the successor function successor(I, X) computes the new CLOSURE set I' . This successor function defines the state (CLOSURE set) the parser moves to on recognizing symbol X . The CLOSURE set I' is obtained by moving the dot after X and adding resulting new configuration in I' and any other items introduced by the CLOSURE property discussed before. Formally, the successor(I, X) is defined to be the closure of the set of all items $[A \rightarrow \alpha X \bullet \beta]$ such that $[A \rightarrow \alpha \bullet X \beta]$ is in I .

The new set of items I' is the state we move to on encountering the symbol X in state I . Naturally, if I is the set of items that are valid for some viable prefix, then $\text{successor}(I, X)$ will be a set of items valid for viable prefix.

As an example, given the item $E \rightarrow E \bullet + T$ from simple expression grammar, we obtain the successor item set I' on $+$, i.e., $\text{successor}(I, +) = I'$, if we add the following configuration in I' :

$$E \rightarrow E \bullet + T$$

We then add new item by performing closure on this set:

$$E \rightarrow E \bullet + T$$

$$T \rightarrow \bullet (E)$$

$$T \rightarrow \bullet \text{id}$$

Creating complete CLOSURE set C

Given the initial closure set I_0 , we can create the complete family closure sets C by following the steps mentioned as follows:

1. Start with C containing the initial closure set I_0 , derived from the configuration $S' \rightarrow \bullet S$. Note that S is the exclusive symbols of the original grammar, whereas S' corresponds to the new exclusive symbol in the augmented grammar G' .
2. For each closure set I in C and each grammar symbol X such that $\text{successor}(I, X)$ is not empty, add $\text{successor}(I, X)$ to C .
3. Repeat step 2 until no more closure sets can be added to C .

Construction of LR(0) parsing table

The LR(0) parsing table construction algorithm has the following steps:

1. Augment the grammar and obtain G' if grammar is not already augmented.
2. Construct the complete closure sets $C = \{I_0, I_1, \dots, I_n\}$, for G' . Each set represents a state which is collection of items.

The parsing actions for the state i (state i is determined from I_i) are defined as follows:

1. If $A \rightarrow \alpha \bullet$ is in I_i then set $\text{ACTION}[i, a]$ to reduce $A \rightarrow \alpha$ for all input (Note that $A \neq S'$)
2. If $S' \rightarrow \bullet S$ is in I_i then set $\text{ACTION}[i, \$]$ to accept
3. If $A \rightarrow \alpha \bullet a \beta$ is in I_i and $\text{successor}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to shift j (a is a terminal)

The GOTO transitions for state i are constructed for all non-terminals A of the grammar using the following rules:

1. If $\text{successor}(I_i, A) = I_j$, then set $\text{GOTO}[i, A] = j$
2. All entries not defined are errors
3. The initial state is the one constructed from the closure set containing $S' \rightarrow \bullet S$.

We see that the shifts in the ACTION table and the entries in the GOTO table are just transitions to new states of the automaton. For reduce entries, the states are popped off the stack to match the symbols of the matching right-side. The configuration of the stack at any time enables us to keep track of what has already been seen and what we are in the middle of processing. The shift states entries preserve the possible options onto the stack until we get to the end of a production in one of the states (we have the states where the dot ($\bullet$) is at the end of a production). Once reached to such state, we are ready to reduce. The handle is popped off the stack and replaced by the nonterminal for the handle. What is left on the stack is what we have yet to process.

The next example shows how to build an item set, perform closure, and compute the successor function.

Example 16: Consider the grammar rules for simple expressions once again. Compute the LR(0) parsing table and GOTO graph for the following:

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow (E)$$

$$T \rightarrow \text{id}$$

Solution: Parsing with ACTION and GOTO tables requires constructing all the LR item sets and successor functions for the expression grammar. Initially, we start with an LR item with a dot before the exclusive symbol and change to item with a dot after the exclusive symbol. This is equivalent to shifting and reducing an entire sentence of the grammar. To achieve this, we augment the grammar so that the exclusive symbol appears on the right-hand side of a production. The augmented grammar that we will work with becomes:

$$E' \rightarrow E$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow (E)$$

$$T \rightarrow \text{id}$$

We initially start with closure set I_0 , which is the closure of $E' \rightarrow \bullet E$. This leads to construction of I_0 :

$$E' \rightarrow \bullet E$$

$$E \rightarrow \bullet E + T$$

$$E \rightarrow \bullet T$$

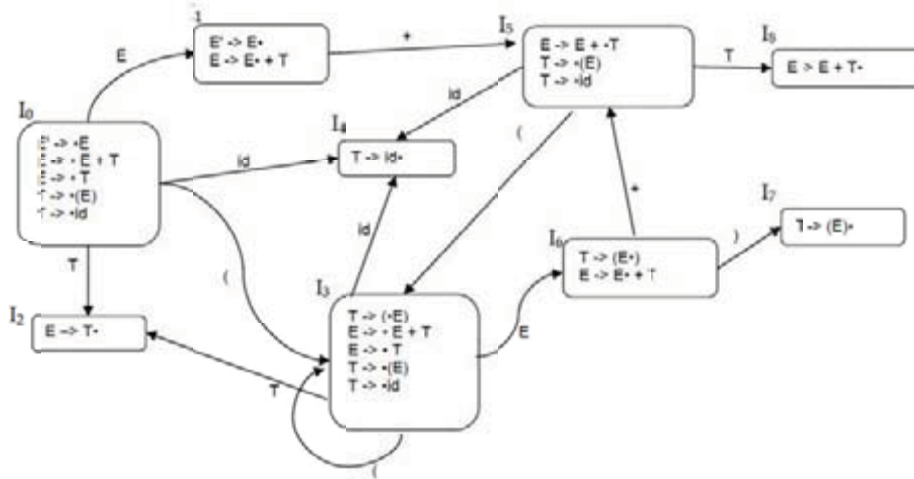
$$T \rightarrow \bullet (E)$$

$$T \rightarrow \bullet \text{id}$$

Following the parsing table construction algorithm, we obtain the other closure sets as follows:

Closure Sets	Successor state	Closure Sets	Successor state
$I_0:$ $E' \rightarrow \bullet E$ $E \rightarrow \bullet E + T$ $E \rightarrow \bullet T$ $T \rightarrow \bullet (E)$ $T \rightarrow \bullet \text{id}$	I_1 I_2 I_3 I_4	$I_4:$ successor (I_0, id) $T \rightarrow \text{id} \bullet$ reduce 4	
$I_1:$ successor (I_0, E) $E' \rightarrow E \bullet$ $E \rightarrow E \bullet + T$	accept I_5	$I_5:$ successor ($I_1, +$) $E \rightarrow E + \bullet T$ $T \rightarrow \bullet (E)$ $T \rightarrow \bullet \text{id}$	I_8 I_3 I_4
$I_2:$ successor (I_0, T) $E \rightarrow T \bullet$	reduce 2	$I_6:$ successor (I_3, E) $T \rightarrow (E \bullet)$ $E \rightarrow E \bullet - T$	I_7 I_5
$I_3:$ successor ($I_0, ($) $T \rightarrow (\bullet E)$ $E \rightarrow \bullet E + T$ $E \rightarrow \bullet T$ $T \rightarrow \bullet (E)$ $T \rightarrow \bullet \text{id}$	I_6 I_6 I_2 I_3 I_4	$I_7:$ successor ($I_4,)$) $T \rightarrow (E) \bullet$ reduce 3	
		$I_8:$ successor (I_5, T) $E \rightarrow E + T \bullet$ reduce 1	

During construction of the LR parser, we are required to construct complete closure set of items. Although the order of defining and numbering the closure sets is not important, what is important to note is that all the possible sets are included. The relationship between the closure sets can be described with *transition diagram* or *GOTO-graph*, as shown in Figure 7:



▲ Figure 7: DFA Showing Transitions among Closure Sets (GOTO-Graph/Successor-Graph)

The complete LR(0) parsing table for our simple grammar, which we have already used is presented in Table 18:

State on top of stack	Action					GOTO	
	id	+	(	)	\$	E	T
0	s4		s3			1	2
1		s5			accept		
2	r2	r2	r2	r2	r2		
3	s4		s3				
4	r4	r4	r4	r4	r4	6	2
5	s4		s3				
6		s5		s7			8
7	r3	r3	r3	r3	r3		
8	r1	r1	r1	r1	r1		

Limitations of LR(0) parsing

The LR(0) method may appear to be a strategy for creating a parser that can handle any context-free grammar, but this is not true. LR(0) parsers do not use any lookaheads and must therefore determine action in each state without looking at any further input symbols, i.e., by what has been seen so far. Moreover, the LR(0) table allows each state to only shift or reduce, i.e., the LR(0) closure set of items cannot have both shift and reduce items. Furthermore, the set must contain exactly one reduce item. Therefore, grammar to be LR(0) must satisfy the following conditions:

1. For any closure set with item $A \rightarrow \alpha \bullet x \beta$, there is no item of the form $B \rightarrow w \bullet$ in that set. In other words, the successor function from that set either shifts to a new state or reduces, but not both (i.e., no shift-reduce conflict).
2. There is at most one item of the form $A \rightarrow \alpha \bullet$ in each closure set. This means successor function has at most one reduction (no reduce-reduce conflict state).

LR(0) parsers are too weak and very few grammars meet the requirements to be LR(0). This is because we decide on what action to take without using any information about the upcoming input. Inclusion of lookaheads could increase the strength and capabilities of the LR parser and overcome such conflicts. Parsing with one token lookahead include- SLR(1), LR(1), and LALR(1), which we have discussed in the sections ahead.

■ SLR(1) Parsers

It has been aforementioned that LR(0) parser can be applied to limited grammars as they use no lookahead tokens. This limits us to make decisions using only what has already been read, without even glancing at what comes next in the input. The parser would work for much larger class of grammars if we could glance at the next token and use that as part of the decision-making. In this move, we introduce SLR(1) parsers.

SLR stands for simple LR. SLR(1) parsers share same LR(0) closure sets and table structure as well as parser operation. It is an improvement in LR(0) constructs to handle conflict situations. This improvement comes in assigning table actions. The SLR(1) parser in contrast to the basic LR(0) parser reduces only if the next input token is a member of the follow set of the non-terminal being reduced. This means that we do not reduce on all inputs to that what we did in LR(0). Instead, we selectively reduce only when the next input token symbol is among the follow sets of the non-terminal on the LHS. We now present the algorithm to construct the SLR(1) parsing table which has similar construction to that of LR(0) with few variations that account for the change in reduce moves. The SLR(1) table construction has the following steps.

1. Augment the grammar and obtain G' if grammar not already augmented.
2. Construct the complete closure sets $C = \{I_0, I_1, \dots, I_n\}$, the collection of LR(0) closure sets for G' . Each set represents a state, which is a collection of items.
3. The parsing actions for the state i (state i is determined from I_i) are defined as follows:
 - a. If $A \rightarrow \alpha \bullet$ is in I_i , then set ACTION[i, a] to reduce $A \rightarrow \alpha$ for a in FOLLOW(A) (Note that $A \neq S'$)
 - b. If $S' \rightarrow S \bullet$ is in I_i , then set ACTION[$i, \$$] to accept
 - c. If $A \rightarrow \alpha \bullet a \beta$ is in I_i and $\text{successor}(I_i, a) = I_j$, then set ACTION[i, a] to shift j (a is a terminal).

The GOTO transitions for state i are constructed for all non-terminals A of the grammar using the rule:

1. If $\text{successor}(I_i, A) = I_j$, then set GOTO[i, A] = j
2. All entries not defined are errors
3. The initial state is the one constructed from the closure set containing $S' \rightarrow \bullet S$.

Note that SLR(1) parser allows both shift and reduce items in the same state and multiple reduce items as well. The SLR(1) parser is consistent and will be able to decide upon which action to take, provided follow sets are disjoint.

A grammar is said to be SLR(1) if it satisfies the following conditions for each closure set:

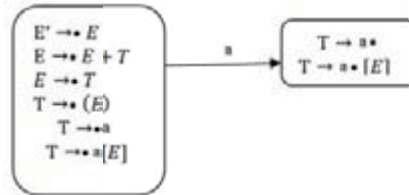
1. For any closure set with item $A \rightarrow \alpha \bullet X \beta$ with terminal a , there is no item of the form $B \rightarrow w \bullet$ in that set with a in the FOLLOW $\{B\}$. This overcomes shift-reduce conflict on any state. This means the successor function for X from that set either shifts to a new state or reduces, but not both.
2. For any two complete items $A \rightarrow \alpha \bullet$ and $B \rightarrow \gamma \bullet$ in the closure set, it must hold the condition: Follow(A) $\cap$ Follow(B) = ϕ , i.e., the follow sets must be disjoint. This solves reduce-reduce conflict on any state. If we have more than one non-terminal that could be reduced from this set, we must distinctively decide which of these to reduce using only one token of lookahead.

Note that all LR(0) grammars are SLR(1), but the reverse is not true. The parsing capability of the parser increases to large class of grammars with the addition of just one symbol of lookahead and use of the follow set.

Example 17: Consider the following grammar. It shows how SLR(1) parsers can overcome shift-reduce conflicts for grammar to support addition of arrays .

$$\begin{aligned} E' &\rightarrow EE' \rightarrow E \\ E &\rightarrow E+T \mid T \\ T &\rightarrow (E) \mid a \mid a[E] \end{aligned}$$

The first two LR(0) closure sets for this grammar, if a is the first token of the input, is as follows:

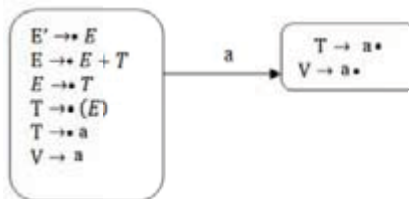


For an LR(0) parser, the set on the right results into a shift-reduce conflict. SLR(1) overcomes this conflict by computing the FOLLOW(T) = {+,), [, \$ } and reduce only on these tokens. The input [will shift and with no conflict. Therefore, the grammar is not LR(0) but SLR(1).

Example 18: The following grammar is simplified expression grammar, which is SLR(1) but not LR(0).

$$\begin{aligned} E' &\rightarrow E \\ E &\rightarrow E+T \mid T \mid V=E \\ E &\rightarrow (E) \mid a \\ V &\rightarrow a \end{aligned}$$

The first two LR(0) closure sets for this grammar, if a is the first token of the input, is as follows:



For an LR(0) parser, the set on the right results into a shift-reduce conflict. SLR(1) overcomes this conflict by computing FOLLOW(T) = {+,), \$ } and FOLLOW(V) = {=}. This distinguishes which reduction to apply depending on the next input token. Therefore, the grammar is not LR(0), but SLR(1).

Example 19: Let us consider an expression grammar shown as follows and construct SLR(1) parsing table and translate to no conflict situations:

$E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$

Solution: We augment the grammar as per the requirement of LR grammar and obtain the resulting grammar G' as follows:

$E' \rightarrow \bullet E$
 $E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$

Now we compute the set of closure items as follows:

Closure Sets	Successor state	Closure Sets	Successor state
$I_0: E' \rightarrow \bullet E$ $E \rightarrow \bullet E + T$ $E \rightarrow \bullet T$ $T \rightarrow \bullet T * F$ $T \rightarrow \bullet F$ $F \rightarrow \bullet (E)$ $F \rightarrow \bullet id$	I_1 I_2 I_2 I_3 I_4 I_5	$I_5: \text{successor}(I_1, id)$ $F \rightarrow id \bullet$	reduce 6
$I_1: \text{successor}(I_0, E)$ $E' \rightarrow E \bullet$ $E \rightarrow E \bullet + T$	accept I_6	$I_6: \text{successor}(I_1, +)$ $E \rightarrow E + \bullet T$ $T \rightarrow \bullet T * F$ $T \rightarrow \bullet F$ $F \rightarrow \bullet (E)$ $F \rightarrow \bullet id$	I_9 I_9 I_3 I_4 I_5
$I_2: \text{successor}(I_0, T)$ $E \rightarrow T \bullet$ $E \rightarrow T \bullet * F$	reduce 2 I_7	$I_7: \text{successor}(I_2, *)$ $T \rightarrow T * \bullet F$ $F \rightarrow \bullet (E)$ $F \rightarrow \bullet id$	I_{10} I_4 I_5
$I_3: \text{successor}(I_0, F)$ $T \rightarrow F \bullet$	reduce 4	$I_8: \text{successor}(I_4, E)$ $F \rightarrow (E \bullet)$ $E \rightarrow E \bullet + T$	I_{11} I_6
$I_4: \text{successor}(I_0, ($ $T \rightarrow (\bullet E)$ $E \rightarrow \bullet E + T$ $E \rightarrow \bullet T$ $T \rightarrow \bullet T * F$ $T \rightarrow \bullet F$ $F \rightarrow \bullet (E)$ $F \rightarrow \bullet id$	I_8 I_9 I_2 I_2 I_3 I_4 I_5	$I_9: \text{successor}(I_6, T)$ $E \rightarrow E + T \bullet$ $T \rightarrow T \bullet * F$	reduce 1 I_7
		$I_{10}: \text{successor}(I_7, F)$ $T \rightarrow T * F \bullet$	reduce 3
		$I_{11}: \text{successor}(I_8,))$ $F \rightarrow (E) \bullet$	reduce 5

We compute FOLLOW in order to decide on the reduce action for items in the set.

We have,

$$\square \text{ FOLLOW}(E) = \{\$, +,)\}$$

$$\square \text{ FOLLOW}(T) = \{+, *,), \$\}$$

$$\square \text{ FOLLOW}(F) = \{+, *,), \$\}$$

The successor function defines the shift, accept, and reduce conditions. Note that the reduce will be performed only on those entries which are in the FOLLOW of the nonterminal on the LHS of the production being reduced. Construction of DFA is left to the reader.

The SLR(1) parsing table is presented in Table 19:

Table 19: SLR Parsing Table for Example 19									
State	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s ₅			s ₄			1	2	3
1		s ₆				accept			
2		r ₂	s ₇		r ₂	r ₂			
3		r ₄	r ₄		r ₄	r ₄			
4	s ₅			s ₄			8	2	3
5		r ₆	r ₆		r ₆	r ₆			
6	s ₅			s ₄				9	3
7	s ₅			s ₄					10
8		s ₆			s ₁₁				
9		r ₁	s ₇		r ₁	r ₁			
10		r ₃	r ₃		r ₃	r ₃			
11		r ₅	r ₅		r ₅	r ₅			

Limitations of SLR(1) parsers

The SLR(1) is powerful but still under operates in some cases. The state i calls for reduction by $X \rightarrow \alpha$ for input symbol a if the item set contains $X \rightarrow \alpha \bullet$ and $a \in \text{FOLLOW}(X)$. The SLR(1) states only tell us about the sequence on top of the stack, not what is below it on the stack. Situations may arise when a viable prefix $\beta\alpha$ may be on the stack that cannot be followed by a in a right sentential form. Thus, reduction by $X \rightarrow \alpha$ is invalid on a . We improve the parser and rule out these invalid reductions by associating more information with the states of the parser. One such method leads to construction of LR(1) parsers.

■ LR(1) Parsers

LR or *canonical LR* parsing incorporates the aforementioned required extra information into the state by redefining closure sets by attaching contextual information as an added component. This extra information is terminal symbol, which is expected to be found in the input token. The canonical LR(1) parser splits states based on different lookahead sets. The general LR(1) configurations is written as:

$$A \rightarrow X_1 \dots X_i \bullet X_{i+1} \dots X_j, a$$

This has little different interpretation. It means states corresponding to $X_1 \dots X_i$ is on the stack and have to push states corresponding to $X_{i+1} \dots X_j$ on the stack and before reduce. Also reduce will be valid only if the token following X_j is the terminal a , where a is called the lookahead of the configuration.

Especially, when $k=1$, the LR parser has contribution from lookahead when LR(1) configurations with a dot at the right end, viz.,

$$A \rightarrow X_1 \dots X_j \bullet, a$$

The parser performs reduce we have states corresponding to $X_1 \dots X_j$ on the stack and when the next symbol is a . Therefore, we write the general form of the items as $[A \rightarrow \alpha \bullet \beta, a]$, where $A \rightarrow \alpha \beta$ is the production and a is the lookahead, either a terminal or \$ (end marker). The lookahead a has no effect if $\beta \neq \epsilon$. Hence, we reduce by $A \rightarrow \alpha$ only if the next token is exactly a for which item $[A \rightarrow \alpha \bullet, a]$ is on the top of the stack. Contrary to this, SLR(1) parser would have reduced if the next token was any of those in $\text{Follow}(A)$ leading to undesired operations in many situations.

If we have more than one symbol in the lookahead, the item set written as $[A \rightarrow \alpha \bullet, a/b/c]$ says we reduce α to A only if the next token is either a , b , or c . Formally, we define the LR(1) item $[A \rightarrow \alpha \bullet \beta, a]$ to be valid for viable prefix γ , for some derivation $S \xRightarrow{*} Aw \xRightarrow{rm} \delta \alpha \beta w$, where

$$\gamma = \delta \alpha \text{ and}$$

$$a \text{ is } \text{FIRST}(w) \text{ or } a = \$ \text{ if } w = \epsilon$$

Viable prefixes are those prefixes of right sentential forms that can appear on the stack of a shift-reduce parser. The method for constructing the closure sets of LR(1) is fundamentally the same as that for SLR with few changes. The closure and successor functions have now lookahead effects. We compute the CLOSURE of an LR(1) item set I as follows:

Repeat the following until no more new items can be added to state I :

1. For each item of the form $[A \rightarrow \alpha \bullet B \beta, a]$ in I
2. For each production $B \rightarrow \eta$ in G' , and for each terminal $b \in \text{FIRST}(\beta a)$
Add $[B \rightarrow \eta, b]$ in I if not already there.

The successor function for the item set I and symbol X is basically similar to LR(0), but in addition, we now propagate the lookahead effect when computing the transitions. Let i be the item set with $[A \rightarrow \alpha X \bullet \beta, a]$ such that $[A \rightarrow \alpha \bullet X \beta, a]$ is in I . The successor(I, X) is the closure of item set i .

Construction of canonical LR parsing table:

To construct the canonical LR parsing table, follow the following steps:

1. Augment the grammar to obtain G' if not already augmented
2. Construct the complete closure set $F = \{I_0, I_1, \dots, I_n\}$ defining the collection of LR(1) items for G
3. Parsing action for state i (determined from I_i) is determined as follows:
 - a. If $[A \rightarrow \alpha \bullet a \beta, b]$ is in I_i and $\text{successor}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to "shift j "
 - b. If $[A \rightarrow \alpha \bullet, a]$ is in I_i , then set $\text{ACTION}[i, a]$ to reduce $A \rightarrow \alpha$
 - c. If $[S' \rightarrow S \bullet, \$]$ is in I_i , then set $\text{ACTION}[i, \$]$ to 'accept'

The GOTO entries of the table are computed as follows:

1. If $\text{successor}(I_i, A) = I_j$, then set $\text{GOTO}[i, A] = j$.
2. Entries not defined are made 'error'
3. The initial state of the parser is the one constructed from $[S' \rightarrow S \bullet, \$]$.

Introducing a lookahead component to LR(1) configurations improves the decisions-making capability of the LR parser to that of SLR(1) parsers. However, this raises the number of distinct configurations sets, which increases the size of the parsing table with GOTO and ACTION entries by a considerable amount.

A grammar is said to be an LR(1) grammar if it satisfies the following conditions for each item set:

1. If the set contains an item $[A \rightarrow \alpha \bullet a \beta, b]$ where a is a terminal, then there is no item in the set of the form $[B \rightarrow \beta \bullet, a]$, thereby overcoming the shift-reduce conflict for any state. This means that the successor function for a either shifts to a new state or reduces, but not both.
2. The lookaheads for all reducible items within the set must be disjoint. This means we cannot have within the set any two items $[A \rightarrow \alpha \bullet, b]$ and $[B \rightarrow \beta \bullet, b]$, thereby overcoming reduce-reduce conflict on any state. For two or more than two non-terminal that could be reduced from the set, it must be possible to uniquely decide which one among them to be used by looking at the next input token.

Remember that every SLR(1) grammar is a canonical LR(1) grammar, but the canonical LR(1) parser may have more states than the SLR(1) parser. Canonical LR(1) use more precise knowledge of appropriate lookahead within a particular context and thus are capable of resolving conflicts that would occur in SLR(1).

Example 20: Consider the augmented grammar given as follow:

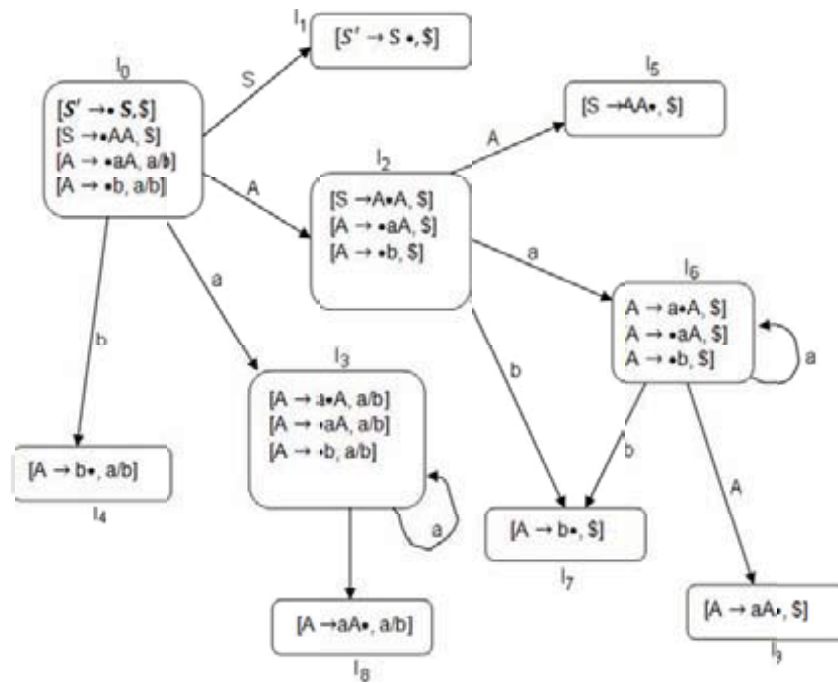
- $S' \rightarrow S$
- $S \rightarrow AA$
- $A \rightarrow aA$
- $A \rightarrow b$

Construct the LR(1) parsing table if possible. Parse the string baab using the table if it exists.

Solution: The complete sets of closure item sets are as follows:

Closure sets	Successor state	Closure sets	Successor state
$I_0: [S' \rightarrow \bullet S, \$]$ $[S \rightarrow \bullet AA, \$]$ $[A \rightarrow \bullet aA, a/b]$ $[A \rightarrow \bullet b, a/b]$	$\begin{matrix} 2 \\ 3 \\ 4 \end{matrix}$	$I_2: \text{successor}(I_1, A)$ $[S \rightarrow AA \bullet, \$]$	reduce 1
$I_1: \text{successor}(I_0, S)$ $[S' \rightarrow S \bullet, \$]$	accept	$I_6: \text{successor}(I_1, a)$ $[A \rightarrow a \bullet A, \$]$ $[A \rightarrow \bullet aA, \$]$ $[A \rightarrow \bullet b, \$]$	I_0 I_6 I_7
$I_2: \text{successor}(I_0, A)$ $[S \rightarrow A \bullet A, \$]$ $[A \rightarrow \bullet aA, \$]$ $[A \rightarrow \bullet b, \$]$	$\begin{matrix} 5 \\ 6 \\ 7 \end{matrix}$	$I_7: \text{successor}(I_1, b)$ $[A \rightarrow b \bullet, \$]$	reduce 3
$I_3: \text{successor}(I_0, a)$ $[A \rightarrow a \bullet A, a/b]$ $[A \rightarrow \bullet aA, a/b]$ $[A \rightarrow \bullet b, a/b]$	$\begin{matrix} 8 \\ 3 \\ 4 \end{matrix}$	$I_8: \text{successor}(I_1, A)$ $[A \rightarrow aA \bullet, a/b]$	reduce 2
$I_4: \text{successor}(I_3, b)$ $[A \rightarrow b \bullet, a/b]$	reduce 3	$I_9: \text{successor}(I_1, A)$ $[A \rightarrow aA \bullet, \$]$	reduce 2

Figure 8 shows the DFA transitions among closure sets:



▲ Figure 8: DFA Showing Transitions among Closure Sets (Successor Graph)

The LR(1) parsing table is presented in Table 20:

Table 20: LR(1) Parsing Table for Example 20					
State	ACTION			GOTO	
	a	b	\$	S	A
0	s ₃	s ₄		1	2
1			accept		
2	s ₆	s ₇			5
3	s ₃	s ₄			8
4	r ₃	r ₃			
5			r ₁		
6	s ₆	s ₇			9
7			r ₃		
8	r ₂	r ₂			
9			r ₂		

Table 21 shows the steps for parsing the input string baab\$:

Table 21: Steps in Parsing String baab\$		
Current stack	Remaining input string	Parser action
s_0	baab\$	Shift state s_4
s_0s_4	aab\$	Reduce 3 ($A \rightarrow b$), goto s_2
s_0s_2	aab\$	Shift s_6
$s_0s_4s_6$	ab\$	Shift s_6
$s_0s_4s_6s_6$	b\$	Shift s_7
$s_0s_4s_6s_6s_7$	\$	Reduce 3 ($A \rightarrow b$), goto s_9
$s_0s_4s_6s_6s_9$	\$	Reduce 2 ($A \rightarrow aA$), goto s_9
$s_0s_4s_6s_9$	\$	Reduce 2 ($A \rightarrow aA$), goto s_5
$s_0s_2s_5$	\$	Reduce 1 ($S \rightarrow AA$), goto s_1
s_0s_1	\$	Accept baab. Parsing is successful.

We show the configuration states of the parser on the stack.

■ LALR(1) Parser

The basic idea behind construction of LALR parsers, also called *lookahead LR* parsers, is to minimize the number of states in an LR(1). LALR(1) parser combines the features of SLR(1) parsers and LR(1) parsers. These parser implementations merge similar states and reduce the total states to that of SLR(1), but still preserve some of the control of the LR(1) lookaheads.

Construction of LALR(1) parsing table

LALR(1) parsing tables are constructed in the same way as canonical LR(1) with lookaheads determining on which item to reduce. If there are no mergable states, then LALR(1) is identical to that of canonical LR(1). Merging of states reduces the number of rows of the table considerably. The resulting LALR(1) has fewer reduce actions than what would have been in an unmerged version.

There are two methods to construct LALR(1) parsing table, which are as follows:

1. Brute-force method
2. Step-by-step construction

The brute-force method first constructs the LR(1) table and then merges the sets manually for all possible cases, resulting into a compressed version of the LR(1) table. The steps of a brute-force method of constructing LALR(1) tables are as follows:

1. Construct a set of all canonical LR(1) states $C = \{I_0, I_1, \dots, I_n\}$ --the collection of sets of LR(1) items.
2. Merge states together that are identical (with same number of items and the same productions) differing only in lookaheads. The lookahead on newly constructed merged states is the union of the lookahead from each state being merged.
3. The successor function for the newly constructed state of LALR(1) is the union of the successors of the merged states.
4. Next the ACTION and GOTO entries in the LALR(1) table are populated similar to that of canonical LR(1) parser.

Example 21: Consider the grammar and LR(1) set of items from Example 20 discussed earlier. Construct the LALR(1) parsing table for the same.

0. $S' \rightarrow S$
1. $S \rightarrow AA$
2. $A \rightarrow aA$
3. $A \rightarrow b$

Solution: We have the LR(1) sets of items as follows:

Closure sets	Closure sets
$I_0:$ $[S' \rightarrow \bullet S, \$]$ $[S \rightarrow \bullet AA, \$]$ $[A \rightarrow \bullet aA, a/b]$ $[A \rightarrow \bullet b, a/b]$	$I_5:$ $\text{successor}(I_2, A)$ $[S \rightarrow AA\bullet, \$]$
$I_1:$ $\text{successor}(I_0, S)$ $[S' \rightarrow S\bullet, \$]$ 	$I_6:$ $\text{successor}(I_2, a)$ $[A \rightarrow a\bullet A, \$]$ $[A \rightarrow \bullet aA, \$]$ $[A \rightarrow \bullet b, \$]$
$I_2:$ $\text{successor}(I_0, A)$ $[S \rightarrow A\bullet A, \$]$ $[A \rightarrow \bullet aA, \$]$ $[A \rightarrow \bullet b, \$]$	$I_7:$ $\text{successor}(I_2, b)$ $[A \rightarrow b\bullet, \$]$
$I_3:$ $\text{successor}(I_0, a) =$ $[A \rightarrow a\bullet A, a/b]$ $[A \rightarrow \bullet aA, a/b]$ $[A \rightarrow \bullet b, a/b]$	$I_8:$ $\text{successor}(I_3, A)$ $[A \rightarrow aA\bullet, a/b]$
$I_4:$ $\text{successor}(I_3, b)$ $[A \rightarrow b\bullet, a/b]$	$I_9:$ $\text{successor}(I_6, A)$ $[A \rightarrow aA\bullet, \$]$

We observe that states 3 and 6 can be merged because they have the same number of items, the core of each item is identical, and they differ only in their lookahead sets. Similarly, 4 and 7, and 8 and 9 are also identical. We can merge these virtually identical states and obtain the LALR(1) table. This introduces new state I_{36} , I_{47} , and I_{89} , and states 3, 4, 6, 7, 8, and 9 can be removed from the Table 27.

We have

- $I_{36}:$
- $[A \rightarrow a\bullet A, a/b/\$]$
 - $[A \rightarrow aA, a/b/\$]$
 - $[A \rightarrow b, a/b/\$]$
- $I_{47}:$
- $[A \rightarrow b\bullet, a/b/\$]$
- $I_{89}:$
- $[A \rightarrow aA\bullet, a/b/\$]$

Therefore, the resulting LALR(1) table is presented in Table 22:

Table 22: LALR(1) Table					
State	ACTION			GOTO	
	a	b	\$	S	A
0	s ₃₆	s ₄₇		1	2
1			accept		
2	s ₃₆	s ₄₇			5
36	s ₃₆	s ₄₇			89
47	r ₃	r ₃	r ₃		
5			r ₁		
89	r ₂	r ₂	r ₂		

This method saves space but does not optimize in time during the construction of LALR parser. Alternatively, a more efficient strategy for building the LALR(1) parser states is *step-by-step merging*. We merge the closure sets as we go, rather than waiting till the end to find the identical ones. Usual method of LR(1) is followed, but when a new set is getting generated, we check to see if it is identical to any other states already there. If yes, merge the newly constructed set with the existing one; else, you add it in normal way.

Some useful properties about LALR grammars are as follows:

1. All LALR(1) grammars are subset of LR(1) and a superset of SLR(1) grammars. In other words, if a grammar is not LR(1), it is definitely not LALR(1). Conflict in LR(1) will reflect in the LALR(1).
2. A grammar that is LR(1) may or may not be LALR(1), depending on whether merging introduces conflicts.
3. All SLR(1) grammars are also LALR(1).
4. A grammar that is not SLR(1) may or may not be LALR(1), depending on whether the more precise lookaheads resolve the SLR(1) conflicts.

Many a times when we merge two states of the parser while generating LALR(1) table, we may land up into reduce-reduce conflict. For example, the following grammar results into reduce-reduce conflict during merger of states.

$$X' \rightarrow X$$

$$X \rightarrow aBc \mid bCc \mid aCd \mid bBd$$

$$B \rightarrow e$$

$$C \rightarrow e$$

The LR(1) configuration sets are as follows:

I ₀ : [X' → •X, \$] [X → •aBc, \$] [X → •bCc, \$] [X → •aCd, \$] [X → •bBd, \$]	I ₅ : [X → aC•d, \$] I ₆ : [B → e•, c] [C → e•, d] I ₇ : [X → bC•c, \$]
I ₁ : [X' → X•, \$]	I ₈ : [X → bB•d, \$]
I ₂ : [X → a•Bc, \$] [X → a•Cd, \$] [B → •e, c] [C → •e, d]	I ₉ : [B → e•, d] [C → e•, c] I ₁₀ : [X aBc•, \$]
I ₃ : [X b•Cc, \$] [X b•Bd, \$] [C •e, c] [B •e, d]	I ₁₁ : [X aCd•, \$] I ₁₂ : [X bCc•, \$] I ₁₃ : [X bBd•, \$]
I ₄ : [X aB•c, \$]	

When we try to merge identical states I₆ and I₉, which only differ in their lookaheads, we have:

I₆₉: [C → e, c/d]
[B → e, d/c]

This merging results into problem. The new merged set allows a reduction to either B or C when the next token is c or d. This is a reduce-reduce conflict, which is an undesired effect of merging LR(1) states. If a grammar encounters such a conflict, then the grammar is not LALR(1). Conflicts can be avoided because an action cell with conflicting actions found in SLR(1) or LR(0) table may have a unique entry in an LALR(1) once invalid reduce actions have been eliminated. Despite of this, LALR(1) is used at large because these parsers have overcome the limited capacity of SLR(1) and LR(0) grammars as well as expensive memory needs of LR(1). LALR(1) features power of the specific lookaheads and table size. It is the method used by the Yacc parser generator.

Let's move on and learn about parser generators.

3.5 Parser Generators

Parsers have the central role to play in the process of compilation. Generating efficient parsers is a challenging task. Parsers may be hand written, but not the preferred choice of many. Alternatively, it can be automatically generated by using a parser generator. These parser generators use description in the form of CFG that forms valid syntactical structures. Parser generators can be utilized to build language parsers of various capacity including parsers for simple desk calculators and complex programming languages.

Yacc is a popular parser generator program used to generate LALR parser for grammar for its specifications. Given the valid syntactical structures to the Yacc, it produces a program that will recognize the input as per the specified grammar rules. Backus Naur Form (BNF) grammar is suitable to express context-free languages and used for most constructs in modern programming languages.

Yacc provides support for specification of semantic routines and semantic stack manipulation. Internally, it keeps two stacks in memory. This includes a stack called *parse stack* that holds terminals and non-terminals representing the current parsing state. The other stack is called *value stack* that contains values for element in the parse stack.

The general structure of the input files for Yacc has three parts of the following form:

```
...C and parser definition...
%%
...Grammar rules and actions...
%%
...C subroutines...
```

The first segment in the Yacc input file contains the specification of the exclusive start symbol of the grammar and listing of expected tokens. It may also contain the specification of the order of precedence and associativity of operators of the grammar, which provides increased flexibility usage of CFGs. Operators such as addition and subtraction are specified to be left associative and of lowest precedence, while exponentiation is specified as right associative and of highest precedence.

An example of Yacc definition segment is given as follows:

```
%start program
%token LET INTEGER IN
%token SKIP IF THEN ELSE END WHILE DO READ WRITE
%token NUMBER
%token IDENTIFIER
%left '-' '+'
%left '*' '/'
%right '^ '
.
.
.
%%
...Grammar rules and actions...
%%
...C subroutines...
```

The second segment in the Yacc input file contains CFG for the required language. This section uses semicolons to separate productions instead of '::=', specifies empty production as left empty, and writes non-terminals in all lower case and the multi character terminals in uppercase. It can be well explained by simplified specification of the expression grammar including precedence as follows:

```
C and parser definitions
%%
program : LET declarations IN commands END ;
declarations : /* empty */
| INTEGER id seq IDENTIFIER '.' ;
id seq : /* empty */
| id seq IDENTIFIER ',' ;
commands : /* empty */
| commands command ';' ;
.
.
.
```

```
.
.
;
exp : NUMBER
    | IDENTIFIER
    | exp '<' exp
    | exp '=' exp
    | exp '>' exp
    | exp '+' exp
    | exp '-' exp
    | exp '*' exp
    | exp '/' exp
    | exp '^' exp
    | '(' exp ')';
%%
C subroutines
```

The last segment of the Yacc file is filled with C code and has a `main()` routine. The body of the `main()` contains a call to function `yyparse()`, which is the driver routine for the parser. In addition, the error reporting during parse is handled by including the function `yyerror()`. Simple examples of the function `main()` and `yyerror()` are:

```
C and parser declarations
%%
Grammar rules and actions
%%
main( int argc, char *argv[] )
{ extern FILE *yyin;
.
.
.
.
.
.
.
yyparse ();
}

yyerror (char *s) /* Called by yyparse on error */
{
printf ("%s\n", s);
}
```

Note that the parse tree is implicitly constructed during the parse and internal representation of the structure of the program is built based on the RHS of the grammar rules. On successful recognition of the RHS, it is reduced to the corresponding LHS. Parsing is said to be successful when the entire program has been reduced to the exclusive start symbol of the grammar.

3.6 Using Ambiguous Grammar

Presence of ambiguity in the grammar results in problem of choice between ‘shift’ and ‘reduce’ actions during the parse. It will also lead to conflicting entries in the LR parsing table. Consider the following grammar that allows multiplying and adding numbers:

1. $E \rightarrow E + E$
2. $E \rightarrow E * E$
3. $E \rightarrow id$

It can be used to generate expressions such as $a + b * c$, given as follows:

```

E → E * E           (rule 2)
  → E * c           (rule 3)
  → E + E * c       (rule 1)
  → E + b * c       (rule 3)
  → a + b * c       (rule 3)
    
```

To recognize the expression, we need to parse the expression in reverse order (bottom-up or shift-reduce parsing) and it should lead to single nonterminal, as shown in the following derivation:

1	. x + y * z	shift	
2	x . + y * z	reduce (rule3)	
3	E . + y * z	shift	
4	E + . y * z	shift	
5	E + y . * z	reduce (rule3)	
6	E + E . * z	shift	
7	E + E * . z	shift	
8	E + E * z .	reduce (rule3)	
9	E + E * E .	reduce (rule2)	emit multiply
10	E + E .	reduce (rule1)	emit add
11	E .	accept	

Note that in step 6, we could have reduced instead of shifting by applying rule 1. This would lead to addition having higher precedence over multiplication. This situation is called *shift-reduce* conflict, which is a result of the ambiguous structure of the grammar as there are more than one derivations leading to the expression $a + b * c$. The following grammar is an example of *reduce-reduce* conflict:

- $E \rightarrow T$
- $E \rightarrow id$
- $T \rightarrow id$

This problem can be solved by disambiguating the grammar by rewriting the grammar or supply Yacc (the parser generator) with clear directives to indicate the precedence of operators. Yacc is capable of handling such situations of conflict. It has default actions for shift-reduce and reduce-reduce conflicts. For shift-reduce conflicts, Yacc will shift, while for reduce-reduce conflicts, it will use the first rule in the listing. Yacc has built-in facilities to issue warning message for conflicts.

An example of disambiguation is discussed as follows:

Example 22: Consider the given grammar. Find if the grammar is ambiguous. If yes, disambiguate it and construct the parsing table without ambiguous entries.

$$E \rightarrow E \mid E - E \mid E * E \mid E / E \mid -E \mid (E) \mid id$$

Solution: To disambiguate, we introduce new non terminal E' and augment the grammar as follows:

$$E' \rightarrow E$$

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid -E \mid (E) \mid id$$

We compute the LR(0) items, which are as follows:

$I_0 :$	$E' \rightarrow \bullet E$ $E \rightarrow \bullet E + E$ $E \rightarrow \bullet E - E$ $E \rightarrow \bullet E * E$ $E \rightarrow \bullet E / E$ $E \rightarrow \bullet - E$ $E \rightarrow \bullet (E)$ $E \rightarrow \bullet id$	$I_1 :$	$E' \rightarrow E \bullet$ $E \rightarrow E \bullet + E$ $E \rightarrow E \bullet - E$ $E \rightarrow E \bullet * E$ $E \rightarrow E \bullet / E$	$I_2 :$	$E \rightarrow - \bullet E$ $E \rightarrow - \bullet E + E$ $E \rightarrow - \bullet E - E$ $E \rightarrow - \bullet E * E$ $E \rightarrow - \bullet E / E$ $E \rightarrow - \bullet - E$ $E \rightarrow - \bullet (E)$ $E \rightarrow - \bullet id$
$I_3 :$	$E \rightarrow (\bullet E)$ $E \rightarrow (\bullet E + E$ $E \rightarrow (\bullet E - E$ $E \rightarrow (\bullet E * E$ $E \rightarrow (\bullet E / E$ $E \rightarrow (\bullet - E$ $E \rightarrow (\bullet (E)$ $E \rightarrow (\bullet id$	$I_4 :$	$E \rightarrow id \bullet$	$I_5 :$	$E \rightarrow E + \bullet E$ $E \rightarrow E + \bullet E + E$ $E \rightarrow E + \bullet E - E$ $E \rightarrow E + \bullet E * E$ $E \rightarrow E + \bullet E / E$ $E \rightarrow E + \bullet - E$ $E \rightarrow E + \bullet (E)$ $E \rightarrow E + \bullet id$
$I_6 :$	$E \rightarrow E - \bullet E$ $E \rightarrow E - \bullet E + E$ $E \rightarrow E - \bullet E - E$ $E \rightarrow E - \bullet E * E$ $E \rightarrow E - \bullet E / E$ $E \rightarrow E - \bullet - E$ $E \rightarrow E - \bullet (E)$ $E \rightarrow E - \bullet id$	$I_7 :$	$E \rightarrow E * \bullet E$ $E \rightarrow E * \bullet E + E$ $E \rightarrow E * \bullet E - E$ $E \rightarrow E * \bullet E * E$ $E \rightarrow E * \bullet E / E$ $E \rightarrow E * \bullet - E$ $E \rightarrow E * \bullet (E)$ $E \rightarrow E * \bullet id$	$I_8 :$	$E \rightarrow E / \bullet E$ $E \rightarrow E / \bullet E + E$ $E \rightarrow E / \bullet E - E$ $E \rightarrow E / \bullet E * E$ $E \rightarrow E / \bullet E / E$ $E \rightarrow E / \bullet - E$ $E \rightarrow E / \bullet (E)$ $E \rightarrow E / \bullet id$
$I_9 :$	$E \rightarrow - E \bullet$ $E \rightarrow - E \bullet + E$ $E \rightarrow - E \bullet - E$ $E \rightarrow - E \bullet * E$ $E \rightarrow - E \bullet / E$	$I_{10} :$	$E \rightarrow (E \bullet)$ $E \rightarrow (E \bullet + E$ $E \rightarrow (E \bullet - E$ $E \rightarrow (E \bullet * E$ $E \rightarrow (E \bullet / E$	$I_{11} :$	$E \rightarrow E + E \bullet$ $E \rightarrow E + E \bullet + E$ $E \rightarrow E + E \bullet - E$ $E \rightarrow E + E \bullet * E$ $E \rightarrow E + E \bullet / E$
$I_{12} :$	$E \rightarrow E - E \bullet$ $E \rightarrow E - E \bullet + E$ $E \rightarrow E - E \bullet - E$ $E \rightarrow E - E \bullet * E$ $E \rightarrow E - E \bullet / E$	$I_{13} :$	$E \rightarrow E * E \bullet$ $E \rightarrow E * E \bullet + E$ $E \rightarrow E * E \bullet - E$ $E \rightarrow E * E \bullet * E$ $E \rightarrow E * E \bullet / E$	$I_{14} :$	$E \rightarrow E / E \bullet$ $E \rightarrow E / E \bullet + E$ $E \rightarrow E / E \bullet - E$ $E \rightarrow E / E \bullet * E$ $E \rightarrow E / E \bullet / E$
$I_{15} :$	$E \rightarrow (E) \bullet$				

The SLR parsing table corresponding to the item sets and follows of E are shown in Table 23:

Table 23: SLR Parsing Table for Example 22									
State	ACTION								GOTO
	+	−	*	/	(	)	id	\$	E
0		s ₂			s ₃		s ₄		1
1	s ₅	s ₆	s ₇	s ₈				accept	
2		s ₂			s ₃		s ₄		9
3		s ₂			s ₃		s ₄		10
4	r ₈	r ₈	r ₈	r ₈		r ₈		r ₈	
5		s ₂			s ₃		s ₄		11
6		s ₂			s ₃		s ₄		12
7		s ₂			s ₃		s ₄		13
8		s ₂			s ₃		s ₄		14
9	r ₆ ,s ₅	r ₆ ,r ₆	r ₆ ,s ₇	r ₆ ,s ₈		r ₆		r ₆	
10	s ₅	s ₆	s ₇	s ₈		s ₁₅			
11	r ₂ ,s ₅	r ₂ ,s ₆	r ₂ ,s ₇	r ₂ ,s ₈		r ₂		r ₂	
12	r ₃ ,s ₅	r ₃ ,s ₆	r ₃ ,s ₇	r ₃ ,s ₈		r ₃		r ₃	
13	r ₄ ,s ₅	r ₄ ,s ₆	r ₄ ,s ₇	r ₄ ,s ₈		r ₄		r ₄	
14	r ₅ ,s ₅	r ₅ ,s ₆	r ₅ ,s ₇	r ₅ ,s ₈		r ₅		r ₅	
15	r ₇	r ₇	r ₇	r ₇		r ₇		r ₇	

Note that FOLLOW(E) = {+, −, *,), \$}.

It can be clearly seen that there are shift-reduce conflict entries in the table. Some observation about the conflict entries are presented in Table 24:

Table 24: Conflict Resolution for Ambiguous States			
State	Input Symbol	Conflict	Remark
9	+	Reduce by E → − E or shift	Since ‘−’ is unary, action must be reduce.
	−	Reduce by E → − E or shift	
	*	Reduce by E → − E or shift	
	/	Reduce by E → − E or shift	
11	+	Reduce by E → E+ E or shift	Assuming ‘+’ is left associative, action must be reduce.
	−	Reduce by E → E+ E or shift	Assuming ‘+’ is left associative, action must be reduce
	*	Reduce by E → E+ E or shift	Since precedence of ‘*’ is more than ‘+’, action must be shift
	/	Reduce by E → E+ E or shift	Since precedence of ‘/’ is more than ‘+’, action must be shift
12	+	Reduce by E → E− E or shift	Since ‘−’ is left associative, action must be reduce
	−	Reduce by E → E− E or shift	Since ‘−’ is left associative, action must be reduce
	*	Reduce by E → E− E or shift	Since precedence of ‘*’ is more than ‘−’, action must be shift
	/	Reduce by E → E− E or shift	Since precedence of ‘/’ is more than ‘−’, action must be shift

Table 24: Conflict Resolution for Ambiguous States

State	Input Symbol	Conflict	Remark
13	+	Reduce by $E \rightarrow E * E$ or shift	Since precedence of '*' is more than '+', action must be reduce
	–	Reduce by $E \rightarrow E * E$ or shift	Since precedence of '*' is more than '–', action must be reduce
	*	Reduce by $E \rightarrow E * E$ or shift	assuming '*' left associative, action must be reduce
	/	Reduce by $E \rightarrow E * E$ or shift	Since '*' is left associative, action must be reduce
14	+	Reduce by $E \rightarrow E / E$ or shift	Since precedence of '/' is more than '+', action must be reduce
	–	Reduce by $E \rightarrow E / E$ or shift	Since precedence of '/' is more than '–', action must be reduce
	*	Reduce by $E \rightarrow E / E$ or shift	assuming '/' is left associative, action must be reduce
	/	Reduce by $E \rightarrow E / E$ or shift	Since '/' is left associative, action must be reduce

The conflicts can be resolved by following the associativity and precedence in descending order as follows:

unary –
 *, /
 +, –

Assuming all operators are left associative, the parsing table after removal of ambiguity can be obtained as shown in Table 25:

Table 25: Parsing Table after Removal of Ambiguity

State	ACTION								GOTO
	+	–	*	/	(	)	id	\$	
0		s ₂			s ₃		s ₄		1
1	s ₅	s ₆	s ₇	s ₈				Accept	
2		s ₂			s ₃		s ₄		9
3		s ₂			s ₃		s ₄		10
4	r ₈	r ₈	r ₈	r ₈		r ₈		r ₈	
5		s ₂			s ₃		s ₄		11
6		s ₂			s ₃		s ₄		12
7		s ₂			s ₃		s ₄		13
8		s ₂			s ₃		s ₄	r ₆	14
9	r ₆	r ₆	r ₆	r ₆		r ₆			
10	s ₅	s ₆	s ₇	s ₈		s ₁₅			
11	r ₂	r ₂	s ₇	s ₈		r ₂		r ₂	
12	r ₃	r ₃	s ₇	s ₈		r ₃		r ₃	
13	r ₄	r ₄	r ₄	r ₄		r ₄		r ₄	
14	r ₅	r ₅	r ₅	r ₅		r ₅		r ₅	
15	r ₇	r ₇	r ₇	r ₇		r ₇		r ₇	

Summary

In this chapter, we have learned that the compiler in its early stages of translations considers an input program as a string of characters without any structure. The lexical analysis is the first phase, which scans the program and generates tokens. Syntax analysis follows lexical analysis phase that has the responsibility to parse source text as input and output success or failure. Parse means to recognize. The parser obtains tokens from scanner programs (the lexical analyzer) and validates them against the syntax rules. Derivation is a sequence of applications of the rules of a grammar that produces a finished string of terminals. It can take two forms---either leftmost or rightmost derivation. Leftmost derivation or simply left derivation always replaces the left-most non-terminal first in the sentential form during the derivation. Right derivation does reverse of it. Most programming languages are implemented using CFG. A grammar is ambiguous if and only if there is at least one string that can be generated by more than one distinct derivation tree. Ambiguity can be overcome by embedding into parser the knowledge of precedence and associativity, and force the parser to construct the tree on the right instead of left.

Top-down and bottom-up parsers are two prominent types of parsing techniques used in language verification. Top-down parsing starts with the distinguished start symbol of the grammar and expand one of the remaining non-terminals by replacing it with the right side of one of its productions until the desired string is obtained. Bottom-up parser starts with a sentence and applies reduction until exclusive symbol is found. LL parsers are top-down parser, while LR is bottom-up parser.

Simple precedence and operator precedence parsing are based on defining and using precedence relations among grammar symbols. These precedence relations identify handle within the sentential form. Handle is a prime phrase that is reduced with LHS of the production during parsing. For all precedence grammars, the precedence relation must be unique. Often, precedence matrices contain sparse entries wasting lot of memory. These are linearized using precedence function. Precedence function generates integer outputs for each symbol of the grammar, which can be compared easily, thus making parsing efficient and economical. Simple precedence grammar and operator precedence grammar are based on shift reduce parsing. Simple precedence parsing defines relations over all the symbols of grammar, while operator precedence defines relations only over terminals called operators.

The LR parser encompasses a parsing table, the driver routine, a stack, and the input token. The parsing tables contain all the information that drives the parser. It has ACTION and GOTO stubs. ACTION defines shift, reduce, and accept moves of the parser on terminal inputs. GOTO defines the new state to be pushed on top of the stack after a reduction of the non-terminal X. The task of any LR parser is to identify a handle from the stack top that is to be reduced. LR parsers can come in many flavors, namely, LR, SLR, and LALR parsers. These parsers can be associated with lookahead and hence also called LR(k), SLR(k), and LALR(k) parsers. For practical purpose, we use $k=1$. Constructions of parsers require defining FIRST and FOLLOW sets.

Construction of all LR parsers requires computing closure sets and successor function. Increasing the value of k vastly increases the parsing power, but the parser becomes more complex and the resulting parsing tables are more bulky. LR(0) parsers have limited capability in parsing grammar. LR(1) parsers, on the other hand, can parse most unambiguous. Alternatives to overcome the bulky parser size are SLR(1) and LALR(1) parsers, which also use one token of lookahead but employ techniques that keep the table size as small as LR(0).

LR(0) parsers do not use any lookaheads and must therefore determine action in each state without looking at any further input symbols. LR(0) parsers are too weak and very few grammars meet the requirements to be LR(0). Inclusion of lookaheads increases the strength and capabilities of the LR parser and overcomes conflicts. SLR stands for simple LR. SLR(1) parsers shares same LR(0) closure sets and table structure as well as parser operation. It is an improvement in LR(0) constructs to handle

conflict situations. The SLR(1) parser in contrast to the basic LR(0) parser reduce only if the next input token is a member of the follow set of the non-terminal being reduced.

The canonical LR(1) parser splits states based on different lookahead sets. Introducing a lookahead component to LR(1) configurations improves the decisions-making capability of the LR parser to that of SLR(1) parsers. Canonical LR(1) parser uses more precise knowledge of appropriate lookahead within a particular context and thus is capable of resolving conflicts that would occur in SLR(1).

The objective behind construction of LALR parsers, also called lookahead LR parsers, is to minimize the number of states in an LR(1). LALR(1) parser merges similar states and reduces the total states, somewhat similar to that of SLR(1) but still retains some of the power of the LR(1) lookaheads. There are two methods to construct LALR(1) parsing table---Brute-force method and step-by-step construction. The brute-force method first constructs the LR(1) table and then merges the sets manually for all possible cases, resulting into a compressed version of the LR(1) table. On the other hand, step-by-step merging method merges the closure sets on the go, rather than waiting till the end to find the identical ones. LALR(1) is used at large because these parsers have overcome the limited capacity of SLR(1) and LR(0) grammars as well as expensive memory needs of LR(1) and uses lookahead well to resolve conflicts.

Parsers may be hand written or automatically generated through the use of automatic parser generators. Yacc is an example of a parser generator program. On valid syntactical structures to the Yacc, it produces a program that will recognize the input as per the specified grammar rules. It includes support for specification of semantic routines and semantic stack manipulation. The general structure of the input files for Yacc has three sections, which include definitions, grammar rules and action, and C subroutines. The parse tree is implicitly constructed during the parse and internal representation of the structure of the program is built based on the RHS of the grammar rules. Ambiguity in the grammar leads to conflict entries in the parse table. It can be overcome by incorporating disambiguating grammar and using precedence and associativity rules. Yacc handles situations of conflicts through its default action. For shift-reduce conflicts, Yacc will shift, while for reduce-reduce conflicts, it will use the first rule in the listing. It also has facilities to issue warning messages.

Review Questions

- Q1. What is a handle? Explain.
- Q2. Elaborate on shift-reduce parsing mechanism.
- Q3. Differentiate between the parsing methodologies used in top-down and bottom-up parsing.
- Q4. What are lookaheads? State its significance in parsing.
- Q5. Write short note on:
 - i. Left factoring
 - ii. Predictive parsers
- Q6. List and describe in brief the naïve algorithm for bottom-up and top-down parsing.
- Q7. What are predictive parsers? Briefly describe the general structure of a predictive parser.
- Q8. Define FIRST and FOLLOW sets. Compute $FIRST_2$ and $FOLLOW_2$ for the following:

$$S \rightarrow aABe$$

$$A \rightarrow Abc$$

$$A \rightarrow b$$

$$B \rightarrow c$$

- Q9. Explain with a suitable block diagram the structure of LR parsers.
- Q10. Briefly describe the various types of LR parsers.
- Q11. What are the limitations of SLR(1) parsers? How do we overcome them?
- Q12. State and explain the characteristics of LALR parsers.
- Q13. Write short note on:
- Reduce-reduce conflict
 - Shift-reduce conflict
 - Precedence functions
- Q14. Prove that the grammar with the following set of productions is ambiguous:
- $\text{string} \rightarrow \text{string} + \text{string}$
 - $\text{string} \rightarrow \text{string} - \text{string}$
 - $\text{string} \rightarrow 0|1|2|3|4|5|6|7|8|9$
- Q15. Find if the following grammar is ambiguous:

```
<if_stmt> → if <logic_expr> then <stmt> | if <logic_expr> then <stmt> else <stmt>
<stmt> → <if_stmt> | <any non-if statement>
```

- Q16. Consider the following grammar:

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <expr> | <expr> * <expr> | ( <expr> ) | <id>
```

Find if it is ambiguous. If yes, draw the left and right derivation trees.

- Q17. Find which of the following is/are ambiguous:
- $$A \rightarrow (L)$$
- $$L \rightarrow x|L, L$$
- $$A \rightarrow (xR)$$
- $$R \rightarrow \epsilon \mid ,xR$$
- Q18. Find if left factoring for the following grammar rules are necessary for a top-down parser. If yes, write the new rules.
- $$\text{statement} \rightarrow \text{identifier} := \text{exp} \mid \text{identifier} (\text{exp-list}) \mid \text{other}$$
- Q19. Consider the following grammar rules:
- $S \rightarrow Tu \mid wx$
 - $T \rightarrow Sq \mid vvS$
 - $\text{statement} \rightarrow \text{assign-stmt} \mid \text{call-stmt} \mid \text{other}$
 $\text{assign-stmt} \rightarrow \text{identifier} := \text{exp}$
 $\text{call-stmt} \rightarrow \text{identifier} (\text{exp-list})$
- Find if each contain left recursion. If yes, remove them.
- Q20. What are the problems of using backtracking approach in implementing parsers? Explain how they can be overcome.

- Q21. Define strong LL grammars. Give example to support the answer.

Consider the following grammar:

$$S \rightarrow iCtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$C \rightarrow b$$

Construct the LL(1) parsing table and find if it is strong LL(1).

- Q22. Consider the following grammar rules. Compute FIRST and FOLLOW. Also make the grammar suitable for LL(1) parsing by removing the left recursion if any.

$$S \rightarrow AB$$

$$A \rightarrow Ca \mid \varepsilon$$

$$B \rightarrow BaAC \mid c$$

$$C \rightarrow b \mid \varepsilon$$

- Q23. Given the language:

$$E \rightarrow T \mid T+E$$

$$T \rightarrow F \mid F*T$$

$$F \rightarrow (E) \mid 0 \dots 9$$

Show step-by-step parsing for the input $5*(1+3)$ using shift-reduce method.

- Q24. Given the CFG with production rules

$$E \rightarrow E + T' \mid T'$$

$$T' \rightarrow T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E') \mid \text{id}$$

$$E' \rightarrow E$$

Parse the string $\text{id}*(\text{id}+\text{id})$ using simple precedence parsing with the following SPM:

	E	E'	T	T'	F	+	*	(	)	id	\$
E											
E'											
T											
T'											
F											
+											
*											
(											
)											
id											
\$											

- Q25. Given a CFG with the following rules of the grammar:

$$Z \rightarrow bMb$$

$$M \rightarrow (L$$

$$M \rightarrow a$$

$$L \rightarrow Ma)$$

Generate the SPM and parse the string $b(aa)b$.

- Q26. Consider an operator grammar:
 $S \rightarrow aAde$
 $A \rightarrow Abc$
 $A \rightarrow b$
Construct the operator precedence table.
- Q27. Generate the matrix of precedence relations for the simple grammar if it exists:
 $S \rightarrow aABe$
 $A \rightarrow Abc$
 $A \rightarrow b$
 $B \rightarrow c$
- Q28. Differentiate between simple grammar and operator grammar.
- Q29. Compute the operator precedence matrix and precedence function for the following grammar if it exists. +, *, /, -, id, num, (and) are terminal symbols.
 $G \rightarrow E$
 $E \rightarrow E + T \mid E - T \mid T$
 $T \rightarrow T * F \mid T / F \mid F$
 $F \rightarrow \text{num} \mid \text{id} \mid (E)$
- Q30. Find if the following grammar is LR(0) or SLR(1). Construct the parsing table if possible.
 $X \rightarrow a$
 $X \rightarrow ab$
- Q31. Find if the following grammar is LR(1).
 $S' \rightarrow S$
 $S \rightarrow xXd \mid yYd \mid xYe \mid yXe$
 $X \rightarrow c$
 $Y \rightarrow c$
- Q32. Construct the LALR(1) parser if possible for the following grammar using
i. step-by-step merging
ii. brute-force method
 $S' \rightarrow S$
 $S \rightarrow V = E$
 $E \rightarrow F \mid E + F$
 $F \rightarrow V \mid \text{int} \mid (E)$
 $V \rightarrow \text{id}$
- Q33. What are parser generators? Give an example of a parser generator and explain its working.
- Q34. What is Yacc? Give the detailed structure of the Yacc input file.
- Q35. How does Yacc handle conflicts? Explain.
- Q36. What is ambiguity of grammar? How do we deal with it?
- Q37. Write a short note on automatic generation of parsers.
- Q38. Differentiate between top-down and bottom-up approach of parsing.

- Q39. Write short note on shift-reduce parsing.
- Q40. Explain how the use of precedence and associativity help to resolve parsing action conflicts. Find if the following grammar has parsing action conflicts. If any, then suggest methods to resolve them.
- $$E \rightarrow E + T \mid T$$
- $$T \rightarrow T * F \mid F$$
- $$F \rightarrow (E) \mid \text{id}$$

■ Multiple Choice Questions

Q1. State true or false

- i. A grammar that is LR(1) is also LALR(1).
 - ii. All SLR(1) grammars are also LALR(1).
- | | |
|-----------------|----------------|
| a. True, true | b. False, true |
| c. False, false | d. True, false |

Ans. b

Q2. _____ parser reduce only if the next input token is a member of the follow set of the non-terminal being reduced.

- | | |
|-----------|------------|
| a. LR(0) | b. LR(1) |
| c. SLR(1) | d. LALR(1) |

Ans. c

Q3. State true or false

- i. In the LR parsing table, ACTION is defined for terminal, while GOTOs are moves for a non terminal that are encountered during the parse.
 - ii. LL is a top-down, whereas operator precedence grammar is bottom-up parser.
- | | |
|----------------|-----------------|
| a. True, true | b. False, false |
| c. True, false | d. False, true |

Ans. a

Q4. _____ is a group of symbols from the sentential form that is enclosed between 'less-than' and 'greater-than' relations.

- | | |
|-----------|--------------|
| a. Item | b. Lookahead |
| c. Follow | d. Handle |

Ans. d

Q5. Non-backtracking top-down parsers are also called _____.

- | | |
|----------------------|----------------------------|
| a. Predictive parser | b. Recursive decent parser |
| c. Both a and b | d. Nether a nor b |

Ans. a

04

Syntax-Directed Translation and Error Recovery

OBJECTIVES

- ↗ Gaining insight into syntax-directed definitions
- ↗ Understanding the construction and role of syntax tree in recognition
- ↗ Learning about synthesized and inherited attributes
- ↗ Understanding the concepts of bottom-up evaluation of attributed definitions
- ↗ Learning about syntax directed translation scheme
- ↗ Learning about error detection and recovery

We have learned in the previous chapters that grammars provide specifications to implement the recognizer for a given language. All computer languages in principle follow a hierarchic structure to address the issues of efficient implementations with multiple and varied levels of context. If the input to the recognizer satisfies the grammar rules, it is accepted; otherwise, it is rejected. Defining the legal syntax and hence the parser for languages accepted by the recognizer is possible by writing a grammar that describes how elements of a language get broken down into sub-elements. It must be understood that a compiler not only recognizes the syntactical constructs, but also determines the meaning of those syntactical constructs for the programming language being implemented.

When writing a compiler, routines must be developed to perform semantic actions against some pieces of input recognized by the recognizers. Code generation is much influenced by the compilers capabilities of interpreting semantics (the meaning) of units of program specification and thus has great significance in the process of compilation. This phase that is responsible for implementing the semantics of the program specifications is known as semantic analysis. This phase often works together with syntax analysis in many compiler implementations. Syntax analysis validates the structure of the program, but semantic analysis provides meaning to it and helps to carry out the intended meaning. Program translation and implementation of semantics associated with the rules of program specifications are important issues in compiler writing and are greatly influence by the syntactic structure of the program. In this view, the program translation is also known as syntax-directed translation. Syntax-directed translation of programs associates semantics with the production rules used in the grammar, thereby guiding the action to be performed on recognition of each construct.

In this chapter, you learn about the various aspects of program translation and understand the idea behind interaction of parsers with semantic routines. This chapter explores the role of Syntax-Directed Definitions (SDDs) and Syntax- Directed Translation Schemes (SDTSs) in compilation process. It also explains the concept of attributed definitions and their evaluation. Later sections of the chapter introduce you to the concepts of error detection and error recovery, which has a vital role in successful implementation of translation and compiling.

4.1 Syntax-Directed Program Evaluation

Use of parse trees is of prime importance in implementing SDD and SDTS. The syntax-directed program translation has as its input a stream of tokens, from which it constructs the parse tree. This parse tree is traversed in some order and semantic rules are evaluated at the nodes of the parse tree. The evaluation of these semantic rules may lead to generation of various forms of intermediate code, updation of symbol table, and issuing of diagnostic error messages or some other important actions that will assist in compiling of program. The input token stream is translated by evaluating the semantic rules. In general, the syntax-directed translation and evaluation of inputs to the semantic analyzer can be described as shown in Figure 1:



▲ Figure 1: Conceptual View of Syntax-Directed Semantic Analysis

Dependency graphs are used to represent the dependencies established between attributes by the semantic rules. These dependency graphs direct and control the evaluation order for the semantic rules. Evaluation of the semantic rules defines the values of the attributes at the nodes in a parse tree for the given input string. The values for attributes associated with the symbols of the productions can be obtained by evaluating the rules over several passes by traversing the parse tree for the input. The process of computing the attributes at the nodes is called *annotating* or *decorating* the parse tree. For each symbol X , the dependency graph has a node for each attribute associated with X . An edge from node A to node B means that the attribute of A is required to compute the attribute of B . A parse tree that shows the values of attributes at each node is called an *annotated parse tree*. Formally, *attributes* are values that are associated with grammar symbols. We can write $X.a$ to represent the attribute a of symbol X . Attributes can also be seen as fields in a record or structure, and translation and semantic evaluation of the program objects require determining these attributes.

4.2 Syntax-Directed Definitions

Semantic analysis and hence the translation of source program are greatly influenced by the parse tree and its parsing process. Syntax-directed translation of the program can either be a separate phase of a compiler or it may be implemented by augmenting additional information with the conventional grammar to control the semantic translation. Grammars with such additional information are called *attributed grammars*. This additional information, also known as *attribute*, is associated with each grammar symbol and describes its properties. An attribute has a name and an associated value, and represents anything including a string, a number, a type, a memory location, or an assigned register—whatever information required.

Now the translation of source language constructs is performed in terms of attributes of their syntactic components. The semantic rules associated with the grammar rules help in computing the values, thereby helping in the implementation of meaning of the syntactic constructs of the program. Attaching attributes with grammar symbol is also sometimes called translation of that symbol and can use fields of various types. SDDs are used to specify translations of the intermediate language representations (mostly ASTs) generated by the syntax analyzer for its stream of input tokens. An SDD is nothing but a context-free grammar which has associated attributes and semantic rules attached to its grammar symbols and the productions, respectively. These attributes can be of two types—*synthesized* and *inherited* attributes, which we will describe later in the chapter.

An SDD is itself a high-level specification for translation that encompasses itself the order in which the translations must occur. An example of an SDD may associate semantic translation for infix to postfix translation with the production as follows:

PRODUCTION	SEMANTIC RULE
$E \rightarrow E_1 * E_2$	$E.code = E_1.code E_2.code '*'$

SDDs are useful for specifying translations and are special form of abstract specification. Note that a semantic rule merely specifies the way in which the attribute of grammar symbols is related to each other, but does not specify when the attributes are to be computed or the data structures which would hold the attribute values. In an SDD, a production $A \rightarrow \alpha$ is associated with a set of semantic rules of the form $b := f(c_1, c_2, \dots, c_k)$, where f is a function, and either of the following condition must hold:

1. A has the synthesized attribute b and $c_1, c_2, \dots, c_k$ are attributes associated with the grammar symbols of the production or
2. One of the grammar symbols on the Right-Hand Side (RHS) of the production has b as its inherited attribute, and $c_1, c_2, \dots, c_k$ are attributes associated with the grammar symbols of the production

In either case, we say that the attribute b depends on attributes $c_1, c_2, \dots, c_k$. We discuss about synthesized and inherited attributes in the following discussions.

There are two types of attributes, namely, *synthesized* and *inherited* attributes, we might encounter while implementing syntax-directed translations.

Synthesized attributes are the attributes of a unit that are calculated from the attributes of its child units. The attributes follow bottom-up propagation, i.e., passed bottom-up in a parse tree; in other words, Left-Hand Side (LHS) attributes of the production rules are computed from attributes of the symbols on the RHS of the production rule. In most cases, the attributes of terminal symbols are supplied by the scanner and the synthesized attributes are ones that are built up for the non-terminals and passed bottom-up in the tree. For example,

$$X \rightarrow Y_1 Y_2 \dots Y_n$$

$$X.a = f(Y_1.a, Y_2.a, \dots, Y_n.a)$$

where the syntax $X.a$ refers to the attribute associated with symbol X . In general, the attribute value of a unit is determined from the attribute values of the children of the node. The following specification is an example to illustrate the concept:

$$E \rightarrow E_1 + E_2 \quad \{E.val := E \rightarrow E_1.val + E_2.val\}$$

Note that the specification written after the instance descriptions of the production specify the semantic actions associated with the production. To put in simple term, a synthesized attribute at a parse-tree node N is determined from attribute values at the child nodes of N and at N itself, and can be evaluated during a single bottom-up traversal of a parse tree.

Inherited attributes are attributes that are derived from attributes of its parent or its siblings. These attributes are passed down the tree, i.e., the attributes of RHS of a production are derived from the attributes of LHS symbols (or other right-side attributes). Inherited attributes pass information about the context further down in the tree. Inherited attributes are useful for expressing the dependence of a program construct on the context in which it appears. For example, inherited attributes may be used to determine whether the address or the value of the identifier is needed. Keeping track of whether an identifier appears on the left or the right side of an assignment may help in it. It is always possible to rewrite an SDD with inherited attributes, shown as follows:

$$X \rightarrow Y_1 Y_2 \dots Y_n$$

$$Y_k.a = f(X.a, Y_1.a, Y_2.a, \dots, Y_{k-1}.a, Y_{k+1}.a, \dots, Y_n.a)$$

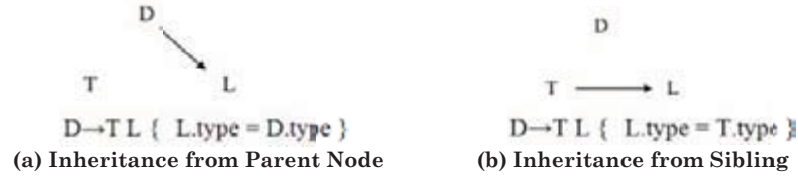
A more simple illustration of the concept can be explained with the following production and the actions associated with them:

$$S \rightarrow ABC \quad \{ A.val := 2 * S.val \}$$

Another example is the type attribute

$$D \rightarrow T L \quad \{ D.type = T.type; L.type = D.type \}$$

Figure 2: illustrates an example of inherited attribute:



▲ Figure 2: SDD with Inherited Attributes

Note that the translation of non-terminal on the right is defined in terms of non-terminal on the LHS.

An SDD that uses synthesized attributes exclusively is called an *S-attributed* definition. S-attributed definition is annotated by evaluating the semantic rules for the attributes at each node in a bottom-up fashion, going from the leaves to the root node. Therefore, a translation scheme is S-attributed if the following two conditions hold:

- Every grammar has synthesized attributes only
- All actions occur on the right side of production

S-attributed definitions can be evaluated through post-order traversal of parse tree. The synthesized attributes during the bottom-up parse keep attribute values of grammar symbols on stack and evaluate attribute values at each reduction. Synthesized attributes are often evaluated during an LR parse.

It is important to note that the evaluation rule for an synthesized attribute will always have the LHS of the production on the LHS of the evaluation rule, e.g.,

$$E :- E_1 '+' E_2 \quad \{ E.val = E_1.val + E_2.val \}$$

On the other hand, an evaluation rule for an inherited attribute will always have a symbol from the RHS of the production on the LHS of the evaluation rule, e.g.,

$$L :- L_1, id \quad \{ L_1.type = L.type \}$$

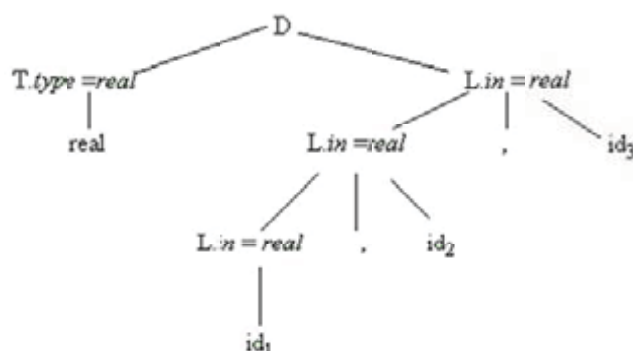
Many real grammars have mixed attributes, i.e., involving attribute from symbols on the LHS of the CFG for some rule and some the RHS of the CFG rule.

Example 1: This example is a declaration generated by the non-terminal *D* in the SDD with keyword *int* or *real*, followed by a list of identifiers. *T* has the synthesized attribute *type*. The production $D \rightarrow TL$ has semantic rule $L.in := T.type$, which sets inherited attribute $L.in$ to the type in the declaration and help rules to pass this type down the parse tree. Productions for *L* have rules that call procedure *addtype* and associate the type of each identifier in the symbol table. Identifiers have entry pointed to by attribute *entry*. Table 1 presents the SDD with inherited attribute *L.in*:

Table 1: SDD with Inherited Attribute <i>L.in</i>	
Production	Semantic rules
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$

Table 1: SDD with Inherited Attribute $L.in$	
Production	Semantic rules
$T \rightarrow \text{real}$	$T.type := \text{real}$
$L \rightarrow L_1 \text{ id}$	$L.in := L.in$
	$addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

Consider a sentence id_1, id_2, id_3 . The parse tree showing inherited attributes at each node labeled L is shown in Figure 3:



▲ Figure 3: Parse Tree with Inherited Attributes

It can be seen in Figure 3 that the value of $L.in$ determines the type of identifiers id_1 , id_2 , and id_3 . It can also be seen that the value of $L.in$ is determined from the value of the attribute $T.type$ followed by $L.in$ top-down manner at the three L. Also, the call to procedure *addtype*, as described earlier, for each L node updates the symbol table with the fact that the identifiers should be of type real.

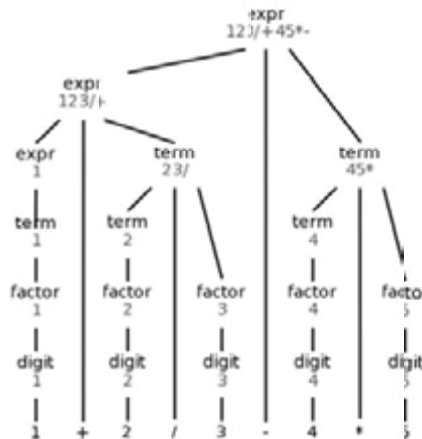
Example 2: This example illustrates the use of synthesized attributes for an infix to postfix translation problem. (Postfix notations of arithmetic expressions have their operators written after operands. For example, an infix expression $A+B$ has its equivalent postfix expression $AB+$. Postfix expressions are mostly used by computing machines for expression translation.)

The SDD, i.e., semantic rules together with the grammar is presented in Table 2:

Table 2: Use of Synthesized Attributes for Infix to Postfix Translation	
Production	Semantic Rule
$\text{expr} \rightarrow \text{expr1} + \text{term}$	$\text{expr.code} := \text{expr1.code} \parallel \text{term.code} \parallel '+'$
$\text{expr} \rightarrow \text{expr1} - \text{term}$	$\text{expr.code} := \text{expr1.code} \parallel \text{term.code} \parallel '-'$
$\text{expr} \rightarrow \text{term}$	$\text{expr.code} := \text{term.code}$
$\text{term} \rightarrow \text{term1} * \text{factor}$	$\text{term.code} := \text{term1.code} \parallel \text{factor.code} \parallel '*'$
$\text{term} \rightarrow \text{term1} / \text{factor}$	$\text{term.code} := \text{term1.code} \parallel \text{factor.code} \parallel '/'$
$\text{term} \rightarrow \text{factor}$	$\text{term.code} := \text{factor.code}$
$\text{factor} \rightarrow \text{digit}$	$\text{factor.code} := \text{digit.code}$
$\text{factor} \rightarrow (\text{expr})$	$\text{factor.code} := \text{expr.code}$

Table 2: Use of Synthesized Attributes for Infix to Postfix Translation	
Production	Semantic Rule
digit $\rightarrow$ 0	digit.code := '0'
digit $\rightarrow$ 1	digit.code := '1'
digit $\rightarrow$ 2	digit.code := '2'
digit $\rightarrow$ 3	digit.code := '3'
digit $\rightarrow$ 4	digit.code := '4'
digit $\rightarrow$ 5	digit.code := '5'
digit $\rightarrow$ 6	digit.code := '6'
digit $\rightarrow$ 7	digit.code := '7'
digit $\rightarrow$ 8	digit.code := '8'
digit $\rightarrow$ 9	digit.code := '9'

Figure 4 shows the annotated tree with synthesized attributes:



▲ Figure 4: Annotated Tree with Synthesized Attributes

The next section discusses about construction of syntax trees.

■ Analysis of SDD

Parse trees are heavily relied for implementing translation schemes. These parse trees determine the order and precedence of operations when performing complex interdependent actions. To be more precise, if an attribute b at a node in a parse tree depends on some attribute c , then the semantic rule for b at that node must be evaluated after the semantic rule that defines c . We use directed graphs called dependency graphs to show the interdependencies between the inherited and synthesized attributes of the nodes of the parse tree.

To construct a dependency graph for a parse tree, recalling the definition of synthesized attributed, we use semantic rule of the form $b = f(c_1, c_2, \dots, c_k)$ and introduce a dummy synthesized attribute b for each semantic rule that consists of a procedure call. There is a node corresponding to each attribute in the graph and an edge to the node for b from c if attribute b depends on attribute c . The dependency graph for a given parse tree is constructed using the following pseudo code:

```

for each node x in the parse tree do
  for each attribute a of the grammar symbol at x do
    construct a node in the dependency graph for a;
  for each node x in the parse tree do
    for each semantic rule  $b := f(c_1, c_2, \dots, c_k)$  associated with the
      production used at x do
        for  $i := 1$  to  $k$  do
          construct an edge from the node for  $c_i$  to the node for  $b$ ;

```

Let us suppose $S \rightarrow AB$ is a production and $S.s := f(A.a, B.b)$ be the semantic rule associated with this production. This rule clearly suggests that the synthesized attribute $S.s$ depends on the attributes $A.a$ and $B.b$. Using this production requires three nodes $S.s$, $A.a$, and $B.b$ in the dependency graph with an edge to $S.s$ from $A.a$ since $S.s$ depends on $A.a$ and an edge to $S.s$ from $B.b$ since $S.s$ also depends on $B.b$. If the production $S \rightarrow AB$ has a semantic rule $A.i := g(S.s, B.b)$ associated with it, then there will be an edge to $A.i$ from $S.s$ and also an edge to $A.i$ from $B.b$, since $A.i$ depends on both $S.s$ and $B.b$.

Topological sort can be found useful and used in determining the appropriate order in which the semantic rules associated with the nodes in a parse tree can be evaluated. This implies the dependent attributes $c_1, c_2, \dots, c_k$ in a semantic rule $b := f(c_1, c_2, \dots, c_k)$ must be available at a node before the evaluation action function f . Translations based on definitions can be made precise with known evaluation order. The grammar guides generation of parse tree for the input under question. Once the dependency graph is constructed, the topological sort of the dependency graph guides the evaluation order for the semantic rules and translation of the input string.

In general, there are many methods for evaluating semantic rules, including parse tree method, rule-based method, and the obvious method. *Parse tree method* uses topological sort of the dependency graph to determine the evaluation order, but may fail when the dependency graph for the given parse tree under construction has a cycle. In *rule-based method*, the order in which the attributes associated with the production are evaluated is predetermined. The semantic rules for each production are hand analyzed or need specialized tools for its analysis. In *obvious method*, the evaluation order is chosen without considering the semantic rules and hence a restricted class of SDD can be implemented. For example, the order of evaluation is forced by the parsing method rather than semantic rules if translation takes place during parsing.

Rule-based and oblivious methods can be more efficient in their use of compile time and space in many cases, as they need not explicitly construct the dependency graph at compile time.

While evaluating expressions, semantic analysis needs to perform static type checking as part of its essential activities. Performing type checking helps to:

- i. Detect incompatible operations that result into serious errors arising out of application of an operator to an incompatible operand and
- ii. Facilitate generation of intermediate code for expressions and statements based on semantic evaluations

Note that the notion of types and the rules for assigning types to language constructs are defined by the specifications of the source language. For practical reasons, all programming language implementations have type and structure associated with the expressions. Most language implementations support types of two kinds, basic and constructed. Examples of basic types are integer, real, character, Boolean, and enumerated, while array, record, set, and pointer are examples of constructed types. Basic types are primitive structures, while constructed types are non-primitive ones.

Expressing the type of a language construct is an important and critical issue. For basic types, it is straightforward, but for the other type, it is non-trivial. A convenient form for expressing the type of language is the use of type expressions. Type expression is a linear form of representation that can

easily capture structure of a type. Directed Acyclic Graph (DAG) or tree representations may also be used for type expression. A basic type is a type expression. Boolean, char, integer, and real are examples of type expressions. For the purposes of type checking, two more basic types- *type_error* and *void* are used. A type name is a type expression and a type constructor applied to type expressions is a type expression. The constructors array, product, record, pointer, and function can be used to create constructed types. We can also define a type system, which is a collection of rules for assigning type expressions to linguistic constructs of a program. A type checker can be used to implement a type system. The type checker checks for type compatibility, which is essentially finding out when two type expressions are equivalent. It requires comparing the type expressions.

Note that if two type expressions are equivalent, then appropriate type is easily determined; otherwise, we return type error. This requires a definition of the notion of equivalence of type expressions. Formally, there are two types of distinct equivalence with reference to names in type expressions. These are name equivalence and structural equivalence. *Name equivalence* assumes each type name as a distinct type. We say two expressions are name equivalent if and only if they are identical. If type names are replaced by the expressions defined by them, then we get the notion of *structural equivalence*. We say two expressions are structurally equivalent provided they represent structurally equivalent type expressions after the substitution of the type names. Expressions are also known to be structurally equivalent if the expressions have same basic type or are formed by using the same constructor to structurally equivalent types. In addition, to type checks, a compiler must perform several other checks on the source program including flow-of-control, uniqueness, and name-related checks.

In the next section, let us discuss about construction of syntax trees.

4.3 Construction of Syntax Trees

As already discussed, an Abstract Syntax Tree (AST) or simply a syntax tree is an abstract representation of the syntactic structure of the program's language constructs and can be assumed to be the condensed form of parse tree representation. SDDs have great influence on creation and usage of syntax trees. Syntax trees when used as an intermediate form can help in dissociating translation activities from parsing steps. It is seen that the grammar used may not reflect the natural hierarchical structure of the language constructs. The order in which information about a construct becomes available and the order in which nodes in a parse tree are considered may be different. Syntax trees are found useful for translation routines invoked in these situations.

Notably, each node in the syntax tree corresponds to a language construct and the child nodes of this node represent the meaningful components of the construct. For example, a node in the syntax tree representing an expression $E_1 * E_2$ has label '*' and two child nodes represented by the sub-expressions E_1 and E_2 , respectively. Also remember that operators and keywords are associated with the interior nodes of the tree and cannot appear as leaf node. Syntax trees have simplified structure with chains of single productions being collapsed.

Although both parse tree and syntax tree representation are equally important in compiler implementation, parse trees have edge over parse trees. The comparison between the two is provided in Table 3:

Table 3: Comparison of Parse Tree And Syntax Tree	
Parse tree	Syntax tree
Interior nodes of the tree represent non-terminals and leaves represent terminals.	Interior nodes represent "operators," while leaves represent operands.
It is rarely constructed as a data structure.	Usually used for tree representing of a program.

Table 3: Comparison of Parse Tree And Syntax Tree

Parse tree	Syntax tree
It represents the concrete syntax of a program and tell us exactly how a string was parsed.	It represents the abstract syntax (semantics) of a program.
It contains detailed and more information than often needed.	Contains simplified structure of a parse tree with limited information. Syntax trees are parse trees without non-terminals.

Many compiler implementations prefer syntax tree as an intermediate representation. The reason behind it is that syntax trees are compact and more closely correspond to the meaning of the various program constructs than other representations. In order to create syntax tree, we need facility to create nodes for each element of the syntax. It generally follows bottom-up approach and hence the nodes are generated bottom up. Therefore, as each node in the tree is created, it already has information about its children (if any). Moreover, use of syntax tree ensures encapsulation and safety of data as access to information in the node is available only by invocation. Note that the access to information in the node is view based. The structure of syntax trees is such that it makes it possible to iterate throughout the entire tree, walk through the child of a node, and display and attach additional views to a node.

Syntax trees can be used to implement bottom-up parsers that motivate the construction of syntax trees in bottom-up fashion. The construction of a syntax tree for a given expression resembles to that of the translation of the expression to its postfix form.

The construction of the syntax tree starts with the construction of sub-trees for the sub-expressions in which nodes are created for each operator and operand. The child nodes of an operator node represent the roots of the nodes representing the sub-expressions containing the operands of that operator. The nodes on the syntax tree are records with several information fields, which include label field and pointer fields. The label of the node often denotes operator. Each node may also have additional field, which, during translation, may hold the values of attributes attached to the node. Constructing syntax tree requires the use of several functions that are used to create the nodes in them, including interior operator nodes and leaf nodes for identifiers and literals. A call to the function creates the node and returns a pointer to a newly created node. The functions used to create the nodes of syntax trees for expressions with binary operators can be something similar to the following functions:

- ❑ **makeknode(op, l_ptr, r_ptr):** Creates an operator node with label **op** and two fields containing pointers to left and right nodes, respectively.
- ❑ **makeleaf(id, name):** Creates an identifier node with label **id** and a field containing name, a pointer to the symbol-table entry for the identifier.
- ❑ **makeleaf(num, val):** Creates a number node with label **num** and a field containing *val*, the value of the number.

A call to any of the aforementioned functions will create the node as required and then return the pointer which points to that node.

Construction of syntax tree is clearer from the following example:

Example 3: Consider the following grammar that implements simple arithmetic expression $id+id*id$ involving '+' and '*':

$$E \rightarrow E + E \mid E * E \mid id$$

The grammar extended with semantic actions and node associated with attribute **t** can be given as follows:

$$a. \quad E_1 \rightarrow E_2 * E_3 \quad \{ E_1.t = \text{makenode}('*', E_2.t, E_3.t) \}$$

- b. $E_1 \rightarrow E_2 + E_3$ $\{ E_1.t = \text{makenode}('+', E_2.t, E_3.t) \}$
 c. $E \rightarrow \text{id}$ $\{ E.t = \text{makeleaf}(\text{id}, \text{id.name}) \}$

The bottom-up construction of the syntax tree is clearly understood from Table 4:

Table 4: Bottom-Up Construction of Syntax Tree for Example 3		
STACK CONTENTS	INPUT	REMARK
\$	id + id * id\$	
\$0 id2	+ id * id\$	C: makeleaf(id, id, name ₁)
\$0 E1	+ id * id\$	
\$0 E1 + 3	id * id\$	
\$0 E1 + 3 id2	* id\$	C: makeleaf(id, id, name ₂)
\$0 E1 + 3 E5	* id\$	
\$0 E1 + 3 E5 *4	id\$	
\$0 E1 + 3 E5 *4 id2	\$	C : makeleaf (id, id, name ₃)
\$0 E1 + 3 E5 *4 E6	\$	B : makenode('*', E.t, E.t)
\$0 E1 + 3 E5	\$	A : makenode('+', E.t, E.t)
\$0 E1	\$	

This can be easily expressed in the form of Table 5:

Table 5: Table Entries to Construct Syntax Tree					
	+	*	id	\$	E
0			2		1
1	3	4		acc	
2	C	C		C	
3			2		5
4			2		6
5	A	4		A	
6	B	B		B	

Example 4: Construct the syntax-directed definition to assists in constructing the syntax tree for production of the following grammar:

$$E \rightarrow E_1 + T \mid E_1 - T \mid T \mid (E) \mid \text{id} \mid \text{NUM}$$

The SDD for construction of syntax tree would look similar to Table 6:

Table 6: SDD to Construct Syntax Tree for Example 4	
Production	Semantic rules
$E \rightarrow E_1 + T$	$E.\text{node} = \text{makenode}('+', E_1.\text{node}, T.\text{node})$
$E \rightarrow E_1 - T$	$E.\text{node} = \text{makenode}('-', E_1.\text{node}, T.\text{node})$
$E \rightarrow T$	$E.\text{node} = T.\text{node}$
$T \rightarrow (E)$	$T.\text{node} = E.\text{node}$
$T \rightarrow \text{id}$	$T.\text{node} = \text{makeleaf}(\text{id}, \text{id.name})$
$T \rightarrow \text{NUM}$	$T.\text{node} = \text{makeleaf}(\text{NUM}, \text{NUM.val})$

Note that we can attach semantic actions to the grammar and perform actions during parsing, thereby saving the construction of the parse tree. Most modern compilers make multiple passes that make use of syntax trees rather parse trees.

4.4 Bottom-Up Evaluation of S-Inherited Attributes

SDDs define semantic rules for determining inherited attributes. The actions specified corresponding to the rules help to effectively manipulate the attributes to perform the translation. The attributes associated with grammar symbols provide much needed information during semantic analysis. The value of each inherited attribute depends only on the attributes of the symbols to its left and also the inherited attributes of the non-terminal on the LHS of the production.

There is a close continuous interaction between the evaluation of synthesized attribute and parser actions. The parser maintains stack to store the values of the synthesized attributes associated with the grammar symbols and evaluates them as the inputs are being parsed. For each reduction, new synthesized attributes are calculated from the attributes on the stack corresponding to the symbols on the RHS of the production being reduced.

Implementing translator for an S-attributed definition using LR parsers with a stack is very popular. Stack maintains the information about the sub-trees being already processed (parsed). A simple implementation of parser stack (for production $S \rightarrow ABC$) is shown in Figure 5:

	state	value
	:	:
	.	.
	A	A.a
	B	B.b
Stack top →	C	C.c
	:	:
	.	.

▲ Figure 5: Synthesized Attributes on the Parser Stack for Production $S \rightarrow ABC$

The parser stack contains fields including state, value, and other information field. The *state* entry is a pointer to LR parsing table, while *value* hold the corresponding value associated with the sub-tree associated with S. If $S.s = f(A.a, B.b, C.c)$ is the semantic rule associated with the production, then before the reduction, the value of the attribute C.c is $val[top] = C.c$. Similarly, the values of attributes B.b and A.a are $val[top-1]$ and $val[top-2]$, respectively. Immediately after the reduction, the top of stack is decremented by 2 and state S is pushed on the top of stack (in place of A), and the synthesized value S.s occupies the value in $val[top]$.

4.5 L-Attributed Definitions

An SDD is said to be *L-attributed* if each inherited attribute of X_j , $1 \leq j \leq n$, on the RHS of the production $A \rightarrow X_1 X_2 \dots X_n$ depends only on:

- the attributes of the symbols $X_1, X_2, \dots, X_{j-1}$ to the left of X_j
- the attributes inherited of A

Top-down translation to implement any L-attributed definition based on an LL(1) grammar need left recursion elimination. The definition tells us that every S-attributed scheme is also an L-attributed

scheme. The unified approach with syntax-directed translation constructs a parse tree or a syntax tree and computes the values of attributes at the nodes of the tree by visiting the nodes of the tree. The "L-attributed translations" (L for "left") can virtually perform all translations during parsing, without building an explicit tree. The attributes can be evaluated in depth-first order. On the other hand, "S-attributed translations" can be performed easily in connection with a bottom-up parse.

For every L-attributed definition, the information flows from left to right in the syntax tree. For evaluating L-attributed definitions, we can use depth first traversal of the parse tree, which is recursively implemented, and evaluate inherited attributes for each child m of n from left to right in the tree. The pseudo code for depth-first procedure for evaluating inherited attributes can be outlined as follows:

```

Call depth_first(node n)
{
  For each child m of n, from left to right
    Evaluate inherited attributes of node m
  Call depth_first(node m)
}
Evaluate synthesized attributes of n

```

As already explained, a translation scheme can be developed for specifying translations during parsing that associates attributes and semantic actions with each symbol and production of the grammar respectively. Often, translation scheme that involves inherited and synthesized attributes is required. In such cases, the inherited attribute for a symbol on the RHS of a production must be computed in an action before that symbol, and an action should not refer to a synthesized attribute of a symbol to the right of the action. Moreover, synthesized attribute for the non-terminal on the LHS can be computed only after computing all the attributes referenced by it. We can construct translation schemes satisfying these conditions. We can use recursive decent parser to evaluate attributes of L-attributed grammar, which has no inherited attributes that depend on synthesized attributes. Inherited attributes are treated analogous to parameters and synthesized attributes to return values.

Implementing *top-down translation* adapts to top-down and left-right flow of information, but requires handling instances of left recursion. All left recursions must be eliminated from the translation scheme since an action will be executed at the time when a symbol in the same position would be expanded.

Bottom-up translation with L-attributed definitions passes information upward in the tree rather than down the tree. Implementing bottom-up translation requires several modifications to address various issues in translation, some of which are discussed as follows:

- **Move attribute rules from the middle to the end of productions:** Introduces new non-terminals and productions. New productions have the form $M \rightarrow e$ for each newly introduced non-terminal symbol M . For example,

$$A \rightarrow B \{ \text{print}("A"); \} C$$
 is modified into

$$A \rightarrow BMC$$

$$M \rightarrow \{ \text{print}("A"); \}$$
- **Inherit attributes on the parser stack:** Means translation scheme that inherits attributes by copy rules can inherit values using stack organization. For example, for a production of the form $Z \rightarrow AB$, the bottom-up parser reduces AB from the stack and replaces them by Z . If A has a synthesized attribute $A.s$ and stored with A on the stack, it can be inherited directly by B provided the copy rule is of the form $B.i = A.s$. This works well when the grammar allows the attribute value to be predicted.

- **Replacement of inherited attributes by synthesized attributes:** Avoids inherited attributes by changing the grammar whenever possible. For example, consider the following grammar for which id-list cannot have type using synthesized attributes alone:

$$\begin{aligned} D &\rightarrow L:T \\ T &\rightarrow \text{integer}|\text{char} \\ L &\rightarrow L, \text{id}|\text{id} \end{aligned}$$

Equivalently, the grammar can be rewritten with id-list associated with a type using synthesized attributes alone as follows:

$$\begin{aligned} D &\rightarrow \text{id } L \\ L &\rightarrow ,\text{id } L_1 && \{L.s=L_1.s;\} \\ L &\rightarrow :T && \{L.s=T.s;\} \\ T &\rightarrow \text{integer} && \{T.s=\text{integer};\} \\ T &\rightarrow \text{char} && \{T.s=\text{char};\} \end{aligned}$$

4.6 Syntax-Directed Translation Scheme

An SDTS is a CFG that has program fragments, called semantic *actions*, associated with it and found helpful in implementing translations. Therefore, SDTSs are used in compiler implementation where the source language translation is completely driven by the parser. With reference to syntax-directed definition, a semantic rule specifies how different attributes are related, but it can also be looked as a prescription for computing an attribute. Therefore, when dealing with SDTS, we call semantic rules as actions.

A SDTS is also called translation scheme. A translation scheme attaches program fragments to the productions rules of a grammar, which are executed on the use of these productions during syntax analysis. Collectively, the execution of all the fragments induced by the syntax analysis in some order produces the translation of the program. The order in which the action is executed fairly depends on the position of a semantic action in the production. Note that the semantic actions may reside at any position in a production. We generally enclosed semantic actions within curly braces. For example, the action may occur at the end, after all the grammar symbol, as shown in the following translation scheme:

$$E \rightarrow E_1 * E_2 \quad \{ \text{print '*' } \}$$

We use semantic rules to define values for attributes associated with the non-terminal symbols in the productions. The values of the attributes can be easily computed constructing parse tree for the input, thereby making a sequence of passes over it and evaluating some or all of the rules over each pass. A *translation scheme* is an improvement of an SDD, which specifies the exact point in time when the actions are to be executed by embedding the actions within the RHS of the productions. Consider the following translation scheme:

$$A \rightarrow X_1 \dots X_i, \{ \text{action} \} X_{i+1} \dots X_n$$

The action is executed after completion of the parse for X_i and before the beginning of X_{i+1} .

In syntax-directed translation, semantic rules (or simply rules) are attached with productions, i.e., instead of giving semantic rules for each production (and thereby generating annotations), we can embed program fragments called *semantic actions* within the productions themselves. We can implement syntax-directed translation in either a top-down or a bottom-up. Note that the place at which the actions are placed in the production determines the order in which they are performed.

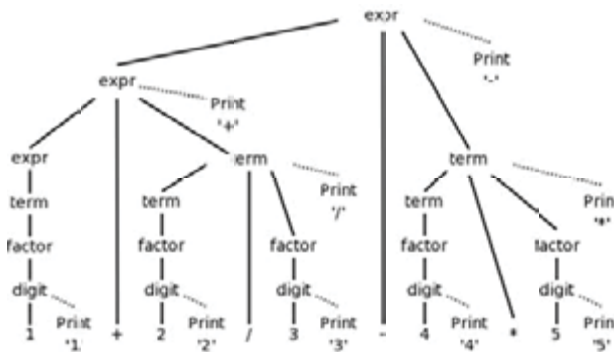
Example 5: The following example defines the semantic actions for evaluating arithmetic expressions. Rules are enclosed within braces shown on the right.

```

expr → expr + term    { print('+') }
expr → expr - term    { print('-') }
term → term / factor   { print('/') }
term → factor          { null }
digit → 0              { print('0') }
      :
      :
digit → 9              { print('9') }

```

We can attach the semantic action to the nodes of the tree. This is clear from Figure 6 which shows semantic actions performed during for syntax-directed translation of $1+2/3-4*5$, where the semantic actions are attached to the nodes of the tree with a distinctive, dotted line:



▲ Figure 6: Semantic Action during for Syntax Directed Translation of $1+2/3-4*5$

The aforementioned formalism can be used to define the output to generate for each input construct. For an output action η associated with the grammar production $S \rightarrow ABC$, the action η is executed by the syntax analyzer on recognition of substring w with derivation of the form $S \Rightarrow ABC \Rightarrow^* w$. The output action η is performed when ABC is reduced to S in a bottom-up parser. With top-down parser implementations, the action is taken when S , A , B , or C is expanded. As mentioned earlier, the attribute/translation field of the grammar symbol can be used to describe many properties as desired. It may be associated with names, viz., $A.VAL$, $A.TRUE$, and so on. If we have productions with several instances of same symbol on the RHS, then to distinguish one production from another, we may use subscripts and each of the non-terminals has a single synthesized attribute called VAL . For example, the semantic action associated with a production $E \rightarrow E * E$ can be written as shown in Table 7:

Table 7: Semantic Action Associate with Production $E \rightarrow E * E$		
Production	Production with instance description	Semantic action
$E \rightarrow E * E$	$E \rightarrow E_1 * E_2$	$\{E.VAL := E \rightarrow E_1.VAL * E_2.VAL\}$

Note that the semantic action is enclosed between braces. This translation scheme could be used to implement simple calculators. For more complex compiler implementation, it performs an action that generates code to perform the required operation. The ability of grammar to associate quantities with programming constructs—for example, values and types with expressions, is heavily utilized for implementing semantic phase of the compiler. We associate attributes with grammar symbols and attach semantic rules to the productions of the grammar. These rules define how the attributes are computed at the nodes of the parse.

Now, let's learn about error detection and recovery.

4.7 Error Detection and Recovery

A good program is one which is free from inconsistencies both at syntactic and semantic levels. We call these inconsistencies as error in the source program. A high-quality and effective compiler therefore depends on its ability to detect such errors, communicate them to users, and recover from them as much as possible. Compiler must find all errors and handle them efficiently before proceeding to code generation phase.

In general, all compilers have error analysis integrated with the compilation process and do not treat them as add-on feature. During error analysis, a compiler must perform the following tasks:

1. Detect errors as many errors as possible in different phases of the compiler.
2. Report all possible errors in a desired form.
3. Recover from errors.
4. Repair of errors. Note that error repair is an attempt of correction, so that the compiled program is executable.

Broadly speaking, errors are classified into lexical error, syntax error, and semantic errors. This classification is based on the fact that most errors are detected during these phases. Errors in token formation, illegal characters, etc., are examples of lexical errors, while errors in structure, missing operators, unbalanced parenthesis, etc., constitute syntax errors. Semantic and logical errors include operands whose types are incompatible, undeclared variables, incompatible arguments to function or procedures, etc.

■ Error Detection

It has been aforementioned that errors are detected as the compiler proceeds with its phases. During its lexical phases, it matches the input with specifications of input class, which are typically regular sets. If after some processing, the lexical analyzer concludes that the input does not match to any valid token formations; it would invoke error recovery routing to take remedial action if possible.

During syntax analysis, table-driven parsers are found to be of great use. These table-driven parsers have explicit entries for erroneous situations. Top-down parser such as LL(1) and bottom-up parsers including LR, LALR, and LR are examples of table-driven parsers. The valid prefix property of these table-driven parsers saves the effort, as the part of the string that has been parsed successfully will never need to be reexamined on encountering an error. LR(1) parsers have immediate error detection property, which makes them suitable for programming language implementations.

■ Error Reporting

Reporting of detected error is equally important. The desirable features of error reporting are as follows:

- ☐ Report errors in terms of source program entities
- ☐ Provide clear and unambiguous error messages
- ☐ Localize the source of error as close as possible to the point of occurrence
- ☐ Avoid redundant (cascading) error messages
- ☐ Notify errors to the users through an appropriate error message, wherever possible, including the errors induced due to the error recovery

Following these guidelines, the compiler will be able to complement the process of efficient program development and enable generation of better code.

With table-driven parsers, the error situations are explicitly known; hence, reporting of such errors is easy. In these cases, appropriate error messages are associated for each error point, and on

encountering the error condition, parser invokes a routine to flash the error message. In addition, the users may be provided with useful information about the tokens that are acceptable at the error point.

There are many approaches that can be helpful in correcting errors. In the simplest approach, the error entries of each state can be examined one by one. The constructed message effectively indicates the valid tokens of the state. The basic error handling approach is as follows:

For each state i of the parsing table, perform the following steps:

1. For each error entry of state i , create a routine e_i , which would be invoked whenever an error is encountered in state i
2. Construct the set of valid tokens, say T , for state i as follows:

$$T = \{t \mid \text{Action}[i, t] \neq \text{error}\}$$
3. The error routine e_i reports about the location of the error and issue a message stating that symbols expected are " T "

Note that in the situation where two states i and j have the same error entries, the same error routine may be used for each of them.

A little improvement in this approach may reduce the number of error entries. If a state has one reduce entry, say r_j , then all error entries of the state are replaced by r_j , while if the state has several reduce entries, anyone may be selected for replacing the error entries. For a state that has only shift and error entries, no change in the table entry is required.

The next section discusses about adhoc and systematic methods.

4.8 Adhoc and Systematic Methods

There are several methods for recovery from syntax errors of which very few have actually been implemented and used. Panic mode, exhaustive error recovery, and adhoc methods are few of them. *In panic mode*, a synchronizing token has to be defined for each important construct of the language. The parser would discard the input symbols until a "synchronizing" token is encountered. The synchronizing token may be delimiters similar to **end** for a list of statements enclosed in a **begin-end** statement, etc. In panic mode, we examine the left context at the point of error and discard tokens from the input till the matching (synchronizing) token is encountered. *Exhaustive error recovery* method comprises examining each error possibility, finding the cause, and then deciding on an appropriate recovery. Although the method can be employed in principle for the table-driven parsers, the task is complex and tedious.

We now learn about adhoc method error handling for LR parsers.

■ Adhoc Methods for LR Parsers

The undefined entries in the parsing table for LR parsers can be regarded as error. During parsing, the parser would detect error (table entry left blank) for some state-input combination. We can fill each such blank entry in the action field with pointer to the appropriate routine that will take action once selected by the compiler. These action include insertion, deletion, transposition, or alteration of input symbols from the stack. These changes in the stack are such that the parser arrives at a state from which it can proceed further toward successful parse or at least flash message of appropriate nature, including unsuccessful parse. A symbol is removed or shifted and stack would shrink if the end of the input is reached.

Consider an example grammar given as follows:

$E' \rightarrow E$

$E \rightarrow E+E \mid E * E \mid \text{id}$

The LR(0) items and SLR parsing table with error entries are computed as follows:

$I_0 :$	$E' \rightarrow \bullet E$	$I_1 :$	$E' \rightarrow E \bullet$	$I_2 :$	$E \rightarrow id \bullet$
	$E \rightarrow \bullet E + E$		$E \rightarrow E \bullet + E$		
	$E \rightarrow \bullet E * E$		$E \rightarrow E \bullet * E$		
	$E \rightarrow \bullet id$				
$I_3 :$	$E \rightarrow E + \bullet E$	$I_4 :$	$E \rightarrow E * \bullet E$	$I_5 :$	$E \rightarrow E + E \bullet$
	$E \rightarrow E \bullet + E$		$E \rightarrow E \bullet * E$		$E \rightarrow E \bullet + E$
	$E \rightarrow E \bullet * E$		$E \rightarrow E \bullet id$		$E \rightarrow E \bullet * E$
	$E \rightarrow \bullet id$				
$I_6 :$	$E \rightarrow E * E \bullet$				
	$E \rightarrow E \bullet + E$				
	$E \rightarrow E \bullet * E$				

Follow(E) = {+, *, \$}

Table 8 shows the SLR parsing table with error entries:

Table 8: SLR Parsing Table with Error Entries					
State	ACTION				GOTO
	id	+	*	\$	E
0	s ₂	e ₁	e ₁	e ₁	1
1	e ₂	s ₃	s ₄	accept	
2	e ₂	r ₃	r ₃	r ₃	
3	s ₂	e ₁	e ₁	e ₁	5
4	s ₂	e ₁	e ₁	e ₁	6
5	e ₂	r ₁	s ₄	r ₁	
6	e ₂	r ₂	r ₂	r ₂	

The table contains two error routines, namely, e₁ and e₂, to be called on various situations.

e₁ is called from states 0, 3, and 4 on seeing operators or end of string marker '\$' while id was expected.

e₂ is called from states 1, 2, 5, and 6 on seeing id while expecting an operator.

On erroneous input id+*\$, the sequence of configurations of the parser is as shown in Table 9:

Table 9: Configuration of the Parser and Action During Parse of Input id+*\$		
Stack	Input	Error messages/ Action taken
0	id + *\$	Perform shift
0id2	+ *\$	Reduce by production $E \rightarrow id$
0E1	+ *\$	Shift
0E1+3	*\$	"id expected", push id and 2 on the stack
0E1+3id2	*\$	Reduce by production $E \rightarrow id$
0E1+3E5	*\$	Shift
0E1+3E5*4	\$	"id expected", push id and 2 on the stack

Table 9: Configuration of the Parser and Action During Parse of Input id+*\$

Stack	Input	Error messages/ Action taken
0E1+3E5*4id2	\$	Reduce by production $E \rightarrow id$
0E1+3E5*4E6	\$	Reduce by production $E \rightarrow E * E$
0E1+3E5	\$	Reduce by production $E \rightarrow E + E$
0E1	\$	Accept

It is clearly seen that the parsing process continues, recovering from syntactical errors, flashing appropriate errors, and doing modifications to the stack in order to recover from errors.

■ Systematic Methods

Several methods have been suggested for error recovery of which some are more effective than others. Systematic method of error recovery provides an efficient approach for handling errors. It encompasses the following fundamental methods:

1. Suspend the normal parsing upon reaching an error configuration.
2. Change the error configuration and arrive at a different configuration. This can be done by modifying either the stack contents or the input buffer.
3. Resume normal parsing from the new configuration.

Systematic error recovery methods have to address many issues while implementing the methods aforementioned. Issues such as how to characterize the new configuration and how to identify the best candidate among several configurations are important ones. Ordering candidate configurations in terms of their cost will prove to be more useful in choosing the recovery and repair mechanisms. The parser generator **Yacc** has features that provide support for error recovery.

Summary

In this chapter, you have learned that understanding and implementing semantics associated with the rules or production are important issues with compiler writing. Association of semantic rules with these productions can be studied using *SDDs* and *translation schemes*. An SDD is a generalization of context-free grammar with attributes attached to grammar symbols and semantic rules attached to the productions. SDDs are useful for specifying translations and are special form of abstract specification.

Attributes attached with each grammar symbol describes the properties of the symbol and can be of two types- synthesized and inherited. *Synthesized* attributes are those attributes that are passed up a parse tree, while *inherited attributes* are attributes that are passed down a parse tree. SDD that uses synthesized attributes exclusively is said to be an *S-attributed* definition. Top-down translation to implement any L-attributed definition is based on an LL(1) grammar and need left recursion elimination. Bottom-up translation with L-attributed definitions passes information upward in the tree rather than down the tree and requires several modification, such as moving attribute rules to the end of the production, inheriting attributes on the parser stack, and replacing inherited attribute by synthesized attributes.

A *syntax tree* is a condensed form of parse tree useful for representing language constructs. It is created with the help of SDDs. The interior nodes of the tree are operators and represent label for results, while leaf nodes correspond to identifiers and literal values.

Parse trees are heavily relied for implementing translation schemes. These parse trees determine the order and precedence of operations when performing complex interdependent actions. Different methods for evaluating semantic rules include parse tree, rule-based, and obvious methods. In case of

semantic analysis, we perform type checking that helps to detect incompatible operations and facilitates generation of intermediate code for expressions and statements based on semantic evaluation. Determining name and structural equivalence are also important activities during semantic translation for implementing type expressions.

An SDTS is a CFG with program fragments, called semantic *actions*, embedded within production bodies. SDTSs are useful for implementing translations. A *translation scheme* is a refinement of an SDD in which the exact point in time when the actions are to be executed is specified by embedding the actions within the RHS of the productions. Attaching attributes with grammar symbol is itself also sometimes called translation of that symbol and can use fields of various types. *Dependency graphs* are used to represent the dependencies established between attributes by the semantic rules. These dependency graphs direct and control the evaluation order for the semantic rules.

Compilers have error analysis integrated with the compilation process and do not treat them add-on feature. During error analysis, it must detect, report, recover, and repair errors. Errors are classified into lexical error, syntax error, and semantic errors, which are detected during these phases. There are several methods for recovery from syntax errors. In panic mode method, a synchronizing token has to be defined for each important construct of the language. The parser would discard the input symbols until a “synchronizing” token is encountered. In exhaustive error recovery method, we examine each error possibility, find the cause, and then decide on an appropriate recovery. Adhoc and systematic methods are two popular methods error recovery.

Review Question

- Q1. Consider the following grammar, which generates the simple language of prefix expressions consisting of addition and subtraction of digits between 1 and 3 without parentheses:

$$P \rightarrow + P P \mid - P P \mid 1 \mid 2 \mid 3$$
Generate the table of semantic actions and rules for implementing this translation.
- Q2. Explain the concepts of syntax-directed definitions and translations with respect to semantic analysis.
- Q3. What are synthesized and inherited attributes? Explain with example.
- Q4. Define S-attributed and L-attributed definitions for inherited attributes.
- Q5. What are syntax trees? How are they different from parse trees?
- Q6. Write a short note on evaluation of semantic rules.
- Q7. Develop the syntax-directed translation scheme for implementing desktop calculators for assignment statements with mixed data types.
- Q8. Write short note on:
 - a. Type checking
 - b. Semantic actions
- Q9. Develop syntax-directed translation scheme for infix-postfix translation.
- Q10. What is dependency graph? What is its significance in semantic analysis and development of translation scheme?
- Q11. Discuss the issues with top-down and bottom-up translation of inherited attributes.
- Q12. What is type checking? Why is it important?
- Q13. How is translation of S-attributed definitions implemented? Explain.
- Q14. What is a parser stack? State its significance in implementing translation schemes.
- Q15. Write short note on types of attributes of syntax directed definitions.
- Q16. What is error recovery? Explain.

- Q17. What are the different types of error? How does the compiler handle them? Give example to explain the concept.
- Q18. Write a short note on systematic method of error recovery.
- Q19. What is panic mode? Explain.
- Q20. Explain with example the adhoc method of error recovery for LR parsers.
- Q21. Consider the following grammar:
 $E \rightarrow E + E \mid E * E \mid E - E \mid E/E \mid a$
- Q22. Write the semantic actions for the grammar and construct the syntax tree for the string $a*a-a$. Also show the steps during bottom-up recognition of the string.
- Q23. What is the advantage of using syntax tree for program translation over parse trees?
- Q24. Write a short note on evolution of semantic rules.
- Q25. Differentiate between parse tree and syntax tree.
- Q26. Explain the role of syntax-directed translations in generation of code.

■ Multiple Choice Questions

Q1. The process of computing the attributes at the nodes is called _____ the parse tree.

- | | |
|---------------|-----------------|
| a. Annotating | b. Pruning |
| c. Decorating | d. Both a and c |

Ans. d

Q2. _____ attributes are those attributes that are passed up a parse tree.

- | | |
|-----------------|--------------------|
| a. Inherited | b. Synthesized |
| c. Both a and b | d. Neither a nor b |

Ans. b

Q3. Consider the following statements:

- | | |
|--|-----------------|
| i. The translation specified by syntax directed definition can be made precise and evaluation order can be determined. | |
| ii. Semantic actions may occur at any position in a production body | |
| a. True, false | b. False, true |
| c. True, true | d. False, false |

Ans. c

Q4. Consider the following statements:

- | | |
|---|-----------------|
| i. The interdependencies among the inherited and synthesized attributes at the nodes in a parse tree can be shown by dependency graphs | |
| ii. Rule based evaluation methods suggests the order in which the attributes associated with that production are evaluated is determined at run-time. | |
| a. True, false | b. False, true |
| c. True, true | d. False, false |

Ans. a

Q5. S-attributed definition can be implemented using _____ parsers.

- | | |
|-----------------|--------------------|
| a. LL | b. LR |
| c. both a and b | d. neither a nor b |

Ans. b

05 Intermediate Code Generation

OBJECTIVES

- ↗ Understanding the role and types of intermediate forms in program compilation
- ↗ Exploring the various techniques of linear and non-linear representations of codes
- ↗ Learning about syntax-directed translation mechanism of various constructs
- ↗ Learning about the various implementation of three-address statements

The design of modern compiler uses phases and passes for processing the input program and generates the output object program. Although the lexical and syntactical phases of the compiler do the task of verifying the construct of the program, a compiler cannot generate the output code immediately with inputs in their original form. The compiler has much more to do before producing the final object code. The compiler, in most cases, translates the source program into an intermediate form rather than generating the output object program. Generating the object program would be a difficult task for most of the high-level specification. The semantic phase or a specially designed code generator works in synchronization with semantic analysis and translates input specifications in the form of parse trees into an Intermediate Representation (IR). IRs are independent of the underlying computer architecture. The later phases of the compiler generate machine code from it, thereby making the task of retargeting the compiler to computer architecture easier to handle.

One important reason for using IR comes from the fact that it is very difficult to generate efficient machine or assembly code because it is difficult to select the most effective addressing mode and registers for implementing high-level source specifications. IR encodes the compiler's knowledge of the program.

This chapter explores the different intermediate forms, wherein you learn about high-level IR and low-level IRs. Then it discusses about syntax-directed translation mechanism and attributed mechanisms.

5.1 Different Intermediate Forms

During the compilation of program from Programming Language (PL), the compiler compiles a program in High-Level Language (HLL) to produce the machine code targeted to some processor. Vastly, all language processors produce an intermediate output that represents the input program in some language or data structure. These representations are known as *IRs* and the code used for the same is called Intermediate Code (*IC*). IC helps in the transformation of source language to the target machine language programs. These ICs are more or less machine independent, and it is easier to generate final code from them. An IR helps to bridge the gap between the source and target languages and can accompany syntax-directed translation. Most implementations of IC use *syntax-directed translation*.

Figure 1 shows the schematics of generating IC:



▲ Figure 1: Intermediate Code Generation

More precisely, the front end of the compiler translates the program into an IR from which the back end generates target object code. Hence, in practice, the compiler has to generate the code twice, that is, from input source form to IR and then from IR to final code. Despite this additional step of IR in compilation, it brings about the following various advantages to the compilation process:

- ❑ Offers better abstraction and cleaner separation of front and back ends of the compiler
- ❑ Helps in retargeting, cross-compiling, and enabling attachment of a back end for the new machine to an existing front end
- ❑ Enables machine-independent optimization, thus reducing overheads of optimization machine-dependent optimization
- ❑ Ensures that IR based on the semantic analysis is much robust

IRs can be classified as structural, linear, and hybrid IRs. *Structural IRs* are graphically oriented and heavily used in source-to-source translator. Syntax trees and Directed Acyclic Graph (DAG) are examples of structural IRs. Three-address codes are *linear IRs* and represent the pseudo code for abstract machine. They use simple and compact data structure with varied level of abstraction and can be rearranged easily. *Hybrid representations* are blend of structural and linear types. Control-flow graphs are hybrid representations. The semantic rules for the construction of three-address code for PL constructs are similar to those of generating syntax tree or postfix notations.

The IRs can also be classified into high-level IRs and low-level IRs. Representations that are close to an HLL specification are called *high-level IR*, while those nearer to assembly are called *low-level IR*. We will discuss about each of these two in the following subsections.

■ High-Level IRs

High-level IRs can preserve information such as structure of object, including loops and if-then-else statements. They resemble more to the source language and maintain notions of subscripts and indices they are compiling.

Examples of most widely accepted high-level intermediate forms are AST, DAG, and postfix code.

Abstract Syntax Trees

We can optimize the parse tree sufficiently to only contain enough information that drives the semantic processing and code generation. This is done by collapsing the intermediate nodes and groupings units, if feasible in the parse trees. *ASTs* are one such high-level IR which corresponds to the procedure's parse tree (the hierarchical structure of the program) with the nodes for most non-terminal nodes being removed. AST may also be simply referred to as syntax trees.

The nodes in the syntax tree symbolize a piece of the program structure and have references to its child sub-trees or possibly also have a reference to its parent. Hence, we can represent the entire program as a sequence of expression trees, where each assignment, procedure call, or jump becomes one individual tree in the forest.

The leaf nodes have no children and usually represent constants and simple variables. Once the grammar that represent leaf nodes is recognized, we store the data immediately in that node and pass it upward so that it can participate in the larger tree. The AST, in practice, drives translation of source program and proves to be very effective for simple specifications. For simple and formal constructs, a

syntax tree can prove to be very effective. For example, for an arithmetic expression, a syntax tree could directly generate assembly language code.

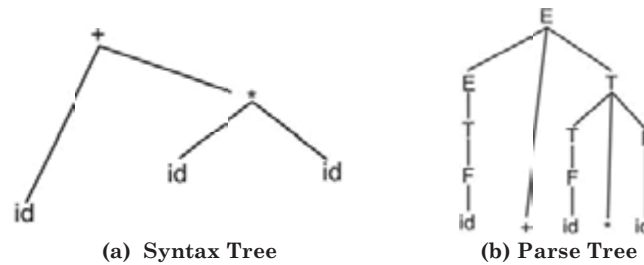
More specifically, a syntax tree shows the structure of a program by abstracting off irrelevant details from a parse tree. The nodes of the tree denote the computation to be performed, while the child of the node represents on what that computation is performed. For expressions, leaves are identifiers and constants; nodes represent operators and child nodes signify operands. Syntax trees for statements use nodes to indicate what kind of statements they are and children correspond to the components of these statements.

The ideological difference between a parse tree and syntax trees is clear from the following example:

Example 1: Consider an expression grammar of the following form:

$$\begin{aligned} E &\rightarrow E+T \mid T \\ T &\rightarrow T*F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Figure 2(a) shows the syntax tree for input string $a*a$ and Figure 2(b) shows the parse tree for input string $a*a$:

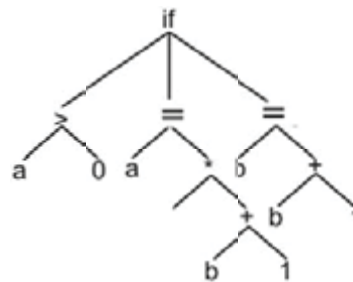


▲ Figure 2: Syntax Tree and Parse Tree for Input String $a*a$

Example 2: Consider the following statements in the source language:

$$\text{if } a > 0 \text{ then } a = 3 * (b + 1) \text{ else } b = b + 1$$

The syntax tree for this statement is shown in Figure 3:



▲ Figure 3: Syntax Tree for Example 2

Generating a code for a procedure or a program represented by entire tree would require first to generate code for each of the sub-trees, store the result in some agreed-upon location, and then combine the results. This would be a fair trial for all simple source programs, such as those containing simple expressions. The general idea behind the construction of a syntax tree for a given input source is to construct it bottom-up using synthesized attributes. Syntax-directed evaluation of expressions

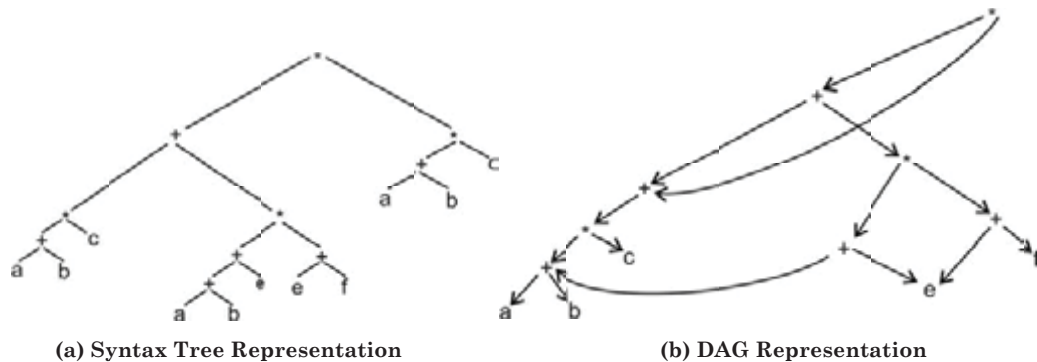
makes use of `makenode(op, l_ptr, r_ptr)`, `makeleaf(id, entry)`, and `makeleaf(num, val)` functions to create the interior and leaf nodes. Each of the interior nodes in the tree is a record with fields for operator and additional fields for pointers to its child nodes. Leaf node stores either constant or identifier and does not have child nodes.

Direct Acyclic Graphs

A DAG is similar to syntax trees, but uses more compact representations. In other words, a DAG is an AST with a unique node for each value. The syntax-directed definition will produce the DAG if the functions `makenode(op, l_ptr, r_ptr)`, `makeleaf(id, entry)`, and `makeleaf(num, val)` return a pointer to an existing node wherever possible, rather than creating new nodes in the tree. An important feature that gives them power over syntax tree is that a node in the DAG can have more than one parent.

A DAG representation provides useful information for later phases that are used during code generation. If the same (sub-) expression appears more than once in a procedure, we can compute its value only once and save its result for future references. This is called common sub-expression elimination. DAGs are used to recognize these common sub-expressions in an expression and thereby avoid duplication in the final code. They are also useful in determining the flow of control and identifying identifiers, which are used inside and outside a basic block. The routines for implementing DAG can check if an identical node has already been constructed or not. If the node already exists, then the new node is not created and all references are made to the existing node thereupon.

The structural difference between syntax tree and DAG is made clear in the construction shown in Figure 4 for an example expression $[(a + b) * c + \{(a + b) + e\} * (e + f)] * [(a + b) * c]$. Figure 4(a) shows the syntax tree representation and Figure 4(b) shows the DAG representation:



▲ Figure 4: Syntax Tree and DAG Representation for the Given Example

It can be seen in Figure 4(a) that there is exactly only one path from a root to each leaf of a tree. It simply states that there is only one route from the start symbol to each terminal in a syntax tree. We can see that some sub-trees are duplicated. DAG does logical optimization to it and shares the common sub-trees. In other words, a DAG is useful where there are multiple, identical portions in a given input. In this sense, DAGs are data structures with more than one path from start symbol to terminals and thus provide an obvious saving in space. They also emphasize on equivalent sections of code, which are useful in the optimization of the code.

Syntax tree and DAG representation can also be stored as array of records, wherein a node is represented by a record. Each record contains field for operation code indicating the label of the node and additional fields indicating the left and right children of the node. Entries of leaf node may contain additional field to store lexical value or a pointer to store symbol table. Each node (record) in the array is associated with an index number called *value number*. All references to nodes are therefore references to value numbers. A node in the array corresponding to a node will be a triple

(**op**, **L**, **R**), where **op** is the label, **L** its left child's value number, and **R** its right child's value number. Proper usage of value number helps to construct expression DAGs efficiently. While constructing a DAG using value number method, we search the array of records for a node with label **op** and child pointers **L** and **R**. If found, return the value number of the matched node; otherwise, create a new node in the array with label **op**, left child **L**, and right child **R**, and return its value number. Figure 5 shows an example of DAG using value numbers:

0	id	a
1	id	b
2	uminus	1
3	*	0 2
4	id	a
5	id	b
6	uminus	5
7	*	4 6
8	+	3 7
9	id	x
10	=	9 8

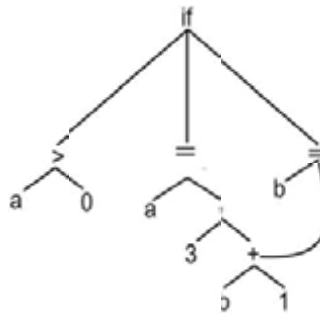
▲ Figure 5: DAG Using Value Number Representations for $x = a * b + a * b$

Let's consider the following example:

Example 3: Consider the following statements in the source language:

```
if a > 0 then a = 3 * (b + 1) else b = b + 1
```

Then the DAG corresponding to it is shown in Figure 6:



▲ Figure 6: DAG for Example 3

Postfix Code

There are three different but equivalent ways of writing expressions, namely, *infix*, *postfix*, and *prefix* notations. The usual high-level convention uses infix form where the operator lies between the operands, e.g., $A+B$. For the purpose of machine translation of the expressions, compiler rather prefers postfix form. *Postfix* notations, also known as *reverse polish* notations, have operators written after their operands, e.g., $AB+$. These are essentially parenthesis-free notations for arithmetic expression and can be seen as a linearized representation of the same AST discussed earlier. Equivalent notation of the infix expression is *prefix notation* that has operators written before their operands, e.g., $+AB$. Postfix equivalent of the input infix expressions is very simple to generate, compact in size, and facilitates easy and straight forward machine code generation. For machine

translation purpose, postfix expressions are also known as postfix code. Some examples of postfix equivalents of the input expressions in infix form are given as follows:

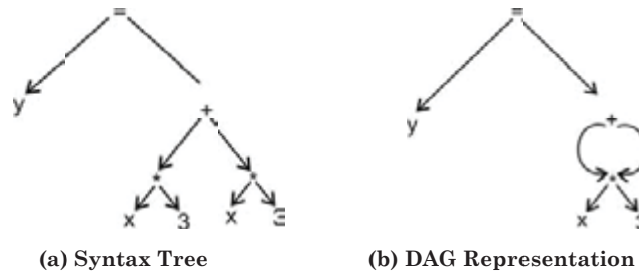
$(a+b)*c \Rightarrow ab+c*$
 $a*(b+c) \Rightarrow abc+*$
 $(a+b)*(c+d) \Rightarrow ab+cd+*$

The IRs can be easily converted from one format to another. Another example of postfix equivalent for the infix expression is as follows:

Infix form: $y = (x * 3) + (x * 3)$

Postfix form: $y x 3 * x 3 * + =$

This is equivalent to the syntax tree shown in Figure 7(a) and the DAG representation shown in Figure 7(b):



▲ Figure 7: Representation of $y = (x * 3) + (x * 3)$ as Syntax Tree and DAG Representation

An efficient implementation of postfix expressions can be done using stacks. A general procedure to construct postfix expression P (initially empty) for infix expression from the source can be outlined as follows:

- ❑ Push left parenthesis '(' on to the stack and add right parenthesis ')' and the end of the input expression in the infix form
- ❑ Read the infix expression from left to right and repeat from step 3 to step 6 for each symbol INEXP until the stack is empty
- ❑ If the symbol read is an operand, add it to the postfix expression P
- ❑ If '(' is read, push it on the stack
- ❑ If the symbol read is an operator ' $\oplus$,' then
 - Repeatedly pop from stack and add to postfix expression P each operator ' $\oplus$ ' that has the same or higher precedence than the current operator
 - Add the operator ' $\oplus$ ' to stack
 - (End of if structure)
- ❑ If the symbol read is right parenthesis ')', then
 - Repeatedly pop from the stack and add to P each operator until the left parenthesis '(' is encountered
 - Remove the left parenthesis '(' [Do not add the '(' to P]

- (End of if structure)
 - (End of step 2 loop)
- Return P as postfix equivalent to input infix expression

When generating final code, the syntax-directed translation uses the property of the stack operation and substitute code leading to final target code.

Let's consider the following example:

Example 4: The expression $X*Y$ would lead to the generation of target code (illustrative only) presented in Table 1:

Table 1: Generation of Target Code			
Original expression	Form output code		
$X*Y$	Code to evaluate X Code to evaluate Y Code to multiply	OR	Code to evaluate X Code to evaluate Y $R_0 \leftarrow \text{POP}$ $R_1 \leftarrow \text{POP}$ $R_2 \leftarrow R_0 * R_1$ PUSH R_2

■ Low-Level IRs

Low-level IRs tend to reveal the underlying architecture of the target machine closely and use explicit addressing schemes and offset information. Low-level IR eliminates structured control flow and generates flat address spaces.

Three-address code is an example of low-level IR. Variants of three-address code such as quadruple, triples, and indirect-triples are also used in compiler implementations.

Three-Address Code

Three-address codes are low-level linear IRs commonly used by most compilers. The format of three-address code resembles to that of assembly code, thus making it more simple to generate target code in assembly or otherwise. It generally allows using an infinite number of temporaries (registers) to store the results of operations.

Three-address codes are preferred representation in most compiler implementations, and support extensive machine-independent code optimization and rearrange themselves in a convenient manner. This makes the final code generation much simple. The three-address code is a sequence of statements typically of the following general form:

$$X := Y \text{ op } Z$$

where X, Y, and Z denote variable names, or constants, or compiler-generated temporaries; **op** denotes any operator, such as a fixed- or floating-point arithmetic operator, or a logical operator on Boolean-valued data. The name three-address comes from the fact that most of the statements contain three addresses: two for operand and one for the result. Three-address codes do not allow built-up arithmetic expressions since only one operator on the right-hand side of a statement is permitted. Thus, a source language expression such as $a + b / c$ might be translated into the following sequence:

$$\begin{aligned} T_1 &:= b / c \\ T_2 &:= a + T_1 \end{aligned}$$

where T_1 and T_2 are temporary names for intermediate values generated by the compiler. The three-address codes simplify complicated arithmetic expressions for the purpose of target code generation and optimization. Temporaries computed by the compiler allow the three-address codes to be easily rearranged and prove to be of great importance in optimizing code. The three-address codes, in fact, are equivalent to linearized versions of graphical syntax tree and DAG representation in which the explicit names correspond to the interior nodes of the graph. The three-address codes for syntax tree and DAG shown in Figure 7 are given as follows:

$T_1 := x * c$	$T_1 := x * 3$
$T_2 := x * c$	$T_2 := T_1 + T_1$
$T_3 := T_1 + T_2$	$y := T_2$
$y := T_3$	

Hence, if we are starting with a DAG (or syntax tree), then the conversion into three-address code is just a topological sort and each three-address operation is assigned a new name for the result (the interior nodes). Let's illustrate this through the following example:

Example 5: Given the following source language statement:

$\text{if } A + 2 > 3 * (B - 1) + 3 \text{ then } C = 0.$

The three-address code corresponding to the preceding expression can be written as follows:

$T_1 = A + 2$
$T_2 = B - 1$
$T_3 = 3 * T_2$
$T_4 = T_3 + 3$
$\text{-if } T_1 \leq T_4 \text{ go to Label } Z$
$C = 0$
Label Z

Implementation of Three-Address Code

The discussions on three-address code in the previous section reveal that it is an abstract form of IC similar to those of syntax tree. A compiler implements these three-address codes as records with fields for storing the operator and the operands. There are three common methods of implementing three-address code, namely, quadruples, triples, and indirect triples. A brief discussion about each of them is given in the upcoming subsections.

Quadruple

Quadruples are the most general form of representing three-address statement. A quadruple (also sometimes called quad for simplicity) uses a record structure with four fields instead of three, namely, OP, ARG1, ARG2, and RESULT. The OP field contains an internal code for the operator, ARG1 and ARG2 store operands, and RESULT stores computed value. The normal implementation of ARG1, ARG2, and RESULT is a pointer to the symbol-table entries for the names represented by these fields. Since the content of RESULT is a computed value, it can also be temporary names created by the compiler and therefore should be entered into the symbol table as and when they are created. Note that unary operations such as $X := -Y$ and $X := Y$ do not use ARG2, so ARG2 must be left blank. The use of quadruple is to assist in performing most machine-independent optimizations including code motion. Consider an assignment statement $X = Y * -Z + Y * -Z$. The IR using quadruple is presented in Table 2:

Table 2: IR Using Quadruple

#	OP	ARG1	ARG2	RESULT
(0)	uminus	Z		T ₁
(1)	*	Y	T ₁	T ₂
(2)	uminus	Z		T ₃
(3)	*	Y	T ₃	T ₄
(4)	+	T ₂	T ₄	T ₅
(5)	:=	T ₅		X

An easy way to construct quadruple is to put OP into the first of the four fields and the addresses (names/constants) into the remaining three. Note that some instructions do not use all the fields, leaving them blank. Many of the operands in the table will be references to entries in tables (e.g., the identifier table). Although simple in design, representation using a quadruple makes use of a lot of temporary names; often, some of them are not required. Quadruple must rearrange code for global optimization.

Triples

Optimization, as will be seen later, deals with reducing requirements of space or time or both of executing programs. Thus, as already mentioned, quad representation requires many entries for name variables including temporaries. Triple is an alternative representation of three-address statements, which saves one complete field present in the quadruples. It eliminates the RESULT field from the table and refers to a temporary value (since results are often temporaries) by the position of the statement that computes it. This avoids entering temporary names into the symbol table, an obvious optimization in space. Therefore, three-address statements can be represented by only three fields, namely, OP, ARG1, and ARG2.

The fields ARG1 and ARG2 are either pointers to the symbol table (for programmer-defined names or constants) or pointers to the triple structure itself (for temporary values). Notions of parenthesized numbers are used to represent pointers into the triple structure, while symbol-table pointers are represented by the names themselves. When the result of the current evaluation occurs as a source operand of a subsequent instruction, we indicate it by writing the value number (with parenthesis) of the current instruction as the operand of the subsequent instruction.

Note that if the RESULT field of a quad is not a temporary, then two triples may be needed. One triple performs the required operation and places the result into a temporary location (which is not a field of the instruction), while the other copies operation from the temporary to the final location.

Triples also permit optimizations such as common sub-expression elimination, but code motion is difficult with it. If, during optimization, the compiler rearranges any of its instructions performance, then undesirable but mandatory extra work is needed with triples since the instruction numbers, which have changed, are used implicitly. Therefore, triple generation must take more care in writing operand references. As an example, the triple representation of the expression $X = Y * -Z + Y * -Z$ is shown in Table 3:

Table 3: Triple Representation of the Expression $X = Y * -Z + Y * -Z$

#	OP	ARG1	ARG2
(0)	uminus	Z	
(1)	*	Y	(0)
(2)	uminus	Z	

Table 3: Triple Representation of the Expression $X = Y * -Z + Y * -Z$

#	OP	ARG1	ARG2
(3)	*	Y	(2)
(4)	+	(1)	(3)
(5)	:=	X	(4)

Indirect Triples

One more implementation of three-address code maintains array of pointers to triples rather than listing the triples themselves. This implementation is called *indirect triples* because of the nature to reference triples indirectly. Table 4 presents an example of indirect triple:

Table 4: Example of Indirect Triple

#	Stmt		#	OP	ARG1	ARG2
(0)	(20)	→	(20)	uminus	Z	
(1)	(21)	→	(21)	*	Y	(20)
(2)	(22)	→	(22)	uminus	Z	
(3)	(23)	→	(23)	*	Y	(22)
(4)	(24)	→	(24)	+	(21)	(23)
(5)	(25)	→	(25)	:=	X	(24)

Now, if it is necessary to reorder instructions, as in many cases, during optimization, the work is minimum, that is, just reordering these pointers. The use of such implementation has the following two advantages:

1. The pointers are smaller than the triples and hence move faster. This is a generic advantage and could be used for quads and many other reordering applications (e.g., sorting large records).
2. Since the triples do not move, the references they contain to past results remain accurate. This is specific to triples (or similar situations).

The fundamental difference between triples and quadruples comes from the fact how much indirection is present in the representation. During target code generation, each name, whether compiler-generated temporary or programmer-defined, will be assigned some run-time memory location for the datum. Quadruples can immediately access the location for name including temporary via the symbol table. The use of quadruple in optimizing compiler is clearly visible when the statements are often moved around. Together with the symbol table quadruples provides an extra degree of indirection between the computation of a value and its use. Moving a statement computing x does not require any change to the statements using that x . On the other hand, when using triple notation, it would be required to change all references to that statement in ARG1 and ARG2, thus posing problem in an optimizing compiler. Indirect triple has no such limitations and the statements can be moved just by reordering the statement list. This makes indirect triples alike quadruple and uses almost the same amount of space. Both quadruple and indirect triple are equally efficient for reordering of code. However, indirect triples can prove to be more efficient as they save some space in comparison to quadruples, specifically when same temporary value is used more than once, which is an important form of optimization.

In the next section, let's discuss about syntax-directed translation and attributed mechanism.

5.2 Syntax-Directed Translation Mechanism and Attributed Mechanisms

Syntax-directed translations are very helpful in generating intermediate and hence final code. They are done by associating and computing set of attributes with the terminal and non-terminals of the grammar with these translations. In the following discussion, we provide the mechanism to generate three-address codes for assignment statements, arrays, and Boolean expression.

■ Three-Address Code Generation for Assignment Statements

Consider the following grammar for assignment of expression to identifier:

$$\begin{aligned} S &\rightarrow \text{id} := E \\ E &\rightarrow E + E \mid E * E \mid - E \mid (E) \mid \text{id} \end{aligned}$$

The S-attributed definition given in Table 5 generates the three-address code for the assignment statements:

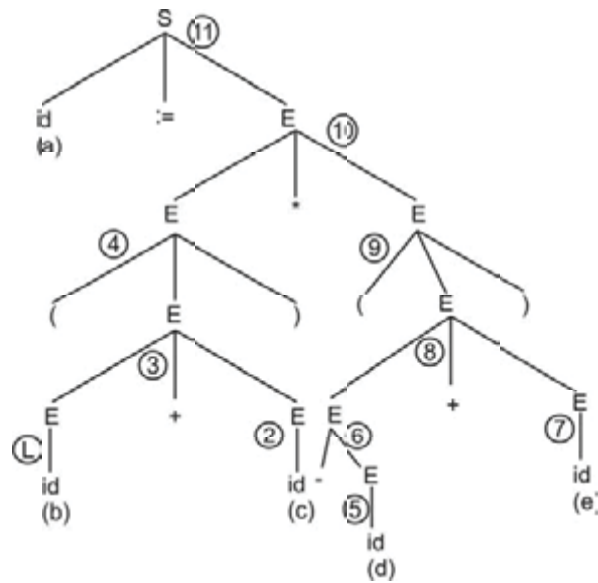
Table 5: Attributed Definitions for Generating Three-Address Code for Assignments	
Grammar rule	Semantic rules
$S \rightarrow \text{id} := E$	$S.code := E.code \parallel \text{gen}(\text{id.place} := E.place)$
$E \rightarrow E_1 + E_2$	$E.place := \text{newtemp}();$
	$E.code := E_1.code \parallel E_2.code \parallel \text{gen}(E.place := E_1.place + E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place := \text{newtemp}();$
	$E.code := E_1.code \parallel E_2.code \parallel \text{gen}(E.place := E_1.place * E_2.place)$
$E \rightarrow -E_1$	$E.place := \text{newtemp}();$
	$E.code := E_1.code \parallel \text{gen}(E.place := \text{'uminus'} E_1.place)$
$E \rightarrow (E_1)$	$E.place := E_1.place;$
	$E.code := E_1.code$
$E \rightarrow \text{id}$	$E.place := \text{id.place};$
	$E.code := \text{' '}$

The synthesized attribute $S.code$ represent the three-address code for assignment, $S.E$ has two attributes, namely, $E.place$ and $E.code$. and S and id have only one attribute each, $S.code$ and id.place , respectively. Note that the attribute $E.place$ designates the name that contains the value of E , while the attribute $E.code$ represents the sequence of three-address statements that evaluate E . The function $\text{newtemp}()$ returns a new temporary storage, while the function $\text{gen}()$ is used to generate the required three-address code. The operator $\parallel$ denotes the concatenation that joins two three-address code segments.

Example 6: Consider the following assignment statement:

$$a := (b + c) * (-d + e)$$

The annotated parse tree is shown in Figure 8:



▲ Figure 8: Annotated Parse Tree

Table 6 shows the corresponding action associated with the reduction sequence in a bottom-up parse:

Table 6: Reduction Sequence and Corresponding Action	
Reduction No.	Action
1	E.place = b
2	E.place = c
3	E.place = T_1
	E.code = $\{T_1 := b + c\}$
	E.place = T_1
	E.code = $\{T_1 := b + c\}$
4	E.place = d
5	E.place = T_2
	E.code = $\{T_2 := \text{uminus } d\}$
6	E.place = e
7	E.place = T_3
	E.code = $\{T_2 := \text{uminus } d, T_3 := T_2 + e\}$
8	E.place = T_4
	E.code = $\{T_1 := b + c, T_2 := \text{uminus } d, T_3 := T_2 + e, T_4 := T_1 * T_3\}$
9	S.code = $\{T_1 := b + c, T_2 := \text{uminus } d, T_3 := T_2 + e, T_4 := T_1 * T_3, a := T_4\}$

■ Three-Address Code Generation for Arrays

Consider the following grammar for assignment statement, where the identifiers are array elements instead of simple variables:

```

S → L := E
E → E + E | (E) | L
L → Elist | id
Elist → Elist, E | id[E]

```

Note that the indices to the arrays could be expressions if so desired as per the grammar definition. The translation scheme associates the following attributes with the non-terminals:

- **L.place**: Holds the name of the variable including name of array
- **E.place**: Holds the value of the expression E
- **L.offset**: Uses for offset of the element within the array (null for simple variables)
- **Elist.array**: Holds the name of the array
- **Elist.place**: Holds value for the index expression
- **Elist.dim**: Stores the current dimension of the array

The translational scheme (semantic rules) to generate the code is as follows:

1. $S \rightarrow L := E$

```

{ if L.offset = null then
  emit(L.place := E.place);
else
  emit(L.place[L.offset] := E.place)
}

```
2. $E \rightarrow E_1 + E_2$

```

{ E.place := newtemp();
  emit(E.place := E1.place + E2.place)
}

```
3. $E \rightarrow (E_1)$

```

{ E.place := E1.place
}

```
4. $E \rightarrow L$

```

{ if L.offset = null then
  E.place := L.place
else
  E.place := newtemp();
  emit(L.place := L.place [L.offset])
}

```
5. $S \rightarrow L := Elist$

```

{ L.place = newtemp();
  L.offset = newtemp();
  emit(L.place := c(Elist.array)); //return constant part of the address
  emit(L.offset := Elist.place * width(Elist.array))
}

```

6. $L \rightarrow \text{id}$

```
{ L.place = id.place;
  L.offset := null;
}
```

7. $\text{Elist} \rightarrow \text{Elist}_1, E$

```
{ t := newtemp();
  m := Elist1.dim + 1;
  emit(t := Elist1.place * limit(Elist1.array, m));
  emit(t := t + E.place);
  Elist.array = Elist1.array;
  Elist.place = t;
  Elist.dim = m;
}
```

8. $\text{Elist} \rightarrow \text{id}[E]$

```
{ Elist.array = id.place;
  Elist.place = E.place;
  Elist.dim = 1;
}
```

■ Three-Address Code Generation for Boolean Expressions

The use of Boolean expressions is rich programming features of all modern programming languages. They are expressing the condition in expressions and help the program to jump from one statement to the other. Boolean expressions evaluate to either *true* or *false*. The translation scheme associates two attributes, namely, $Y.true$ and $Y.false$, with the Boolean expression Y , where $Y.true$ defines the position the program control should reach if Y happen to be true, while $Y.false$ defines the position the program control should reach if Y is false.

Consider the following Boolean grammar:

$$Y \rightarrow Y \text{ or } Y \mid Y \text{ and } Y \mid \text{not } Y \mid (Y) \mid \text{id rel op id} \mid \text{true} \mid \text{false}$$

Note that the rules of Boolean algebra help in optimizing the evaluation of the code. For example, for **or** operation, if the first operand is true, then the entire expression is true and we need not evaluate the other operand. Similarly, the rules of **and** operator is used to design the translation to generate the IC. The translational scheme to generate the three-address code for the preceding grammar is as follows:

1. $Y \rightarrow Y_1 \text{ or } Y_2$

```
{ Y1.true = Y.true
  Y1.false = newlabel()
  Y2.true = Y.true
  Y2.false = Y.false
  Y.code = Y1.code || gen Y1.false ':' || Y2.code
}
```

2. $Y \rightarrow Y_1 \text{ and } Y_2$

```
{ Y1.true = newlabel()
  Y1.false = Y.false
  Y2.true = Y.true
  Y2.false = Y.false
  Y.code = Y1.code || gen(Y1.true ':') | Y2.code
}
```

3. $Y \rightarrow \text{not } Y_1$

```
{ Y1.true = Y.true
  Y1.false = Y.false
  Y.code = Y1.code
}
```

4. $Y \rightarrow (Y_1)$

```
{ Y1.true = Y.true
  Y1.false = Y.false
  Y.code = Y1.code
}
```

5. $Y \rightarrow \text{id}_1 \text{ relop id}_2$

```
{ Y.code = gen('if' id1.place relop id2.place 'goto' Y.true) || gen('goto' Y.false)
}
```

6. $Y \rightarrow \text{true}$

```
{ Y.code = gen('goto' Y.true);
}
```

7. $Y \rightarrow \text{false}$

```
{ Y.code = gen('goto' Y.false);
}
```

Note that the preceding translation will lead to a two-pass procedure, where the first pass computes all the jump targets ($Y.true$ and $Y.false$) and the second pass generates the actual code. This process can be improved into a single phase by modifying the grammar by introducing more attributes for Y and new procedures as follows:

- **Y.truelist:** Refers to the list of positions within the generated three-address code for Y at which Y is certainly true
- **Y.falselist:** Contains a list of all the locations where Y is certainly false

Since the three-address code generated by the translation scheme can be visualized as an array of quadruples, we define the following functions to help in code generation:

- **Createlist(i)**: Specifies a function to create a new list with a single entry *i*---an index into the array of quadruples.
- **Mergelist(list₁, list₂)**: Represents a function to merge *list₁* and *list₂* and return a new list containing *list₁* followed by *list₂*.
- **backpatch(list, target)**: Represents a function to insert the *target* into the quadruple pointed to the entries in the *list*.
- **nextquad()**: Specifies a function that returns the address of the next quadruple to be generated.
- **M.quad**: Refers to an attribute (of variable M) to hold the address of a quadruple. M is a dummy variable introduced to simplify the code generation process.

Now consider the following modified grammar:

$$Y \rightarrow Y \text{ or } MY \mid Y \text{ and } MY \mid \text{not } Y \mid (Y) \mid \text{id relop id} \mid \text{true} \mid \text{false}$$

Therefore, the new set of rules to generate the three-address code that optimizes the process is as follows:

1. $Y \rightarrow Y_1 \text{ or } MY_2$

```

      { backpatch(Y1.falselist, M.quad)
        Y.ruelist = mergeist(Y1.ruelist, Y2.ruelist)
        Y.falselist = Y2.falselist
      }
```
2. $Y \rightarrow Y_1 \text{ and } MY_2$

```

      { backpatch(Y1.ruelist, M.quad)
        Y.ruelist = Y2.ruelist
        Y.falselist = mergelist(Y1.falselist, Y2.falselist)
      }
```
3. $Y \rightarrow \text{not } Y_1$

```

      { Y.ruelist = Y1.falselist
        Y.falselist = Y1.ruelist
      }
```
4. $Y \rightarrow (Y_1)$

```

      { Y.ruelist = Y1.ruelist
        Y.falselist = Y1.falselist
      }
```
5. $Y \rightarrow \text{true}$

```

      { Y.ruelist = makelist(nextquad)
        emit('goto ...')
      }
```
6. $Y \rightarrow \text{false}$

```

      { Y.falselist = makelist(nextquad)
        emit('goto ...')
      }
```
7. $M \rightarrow :$

```

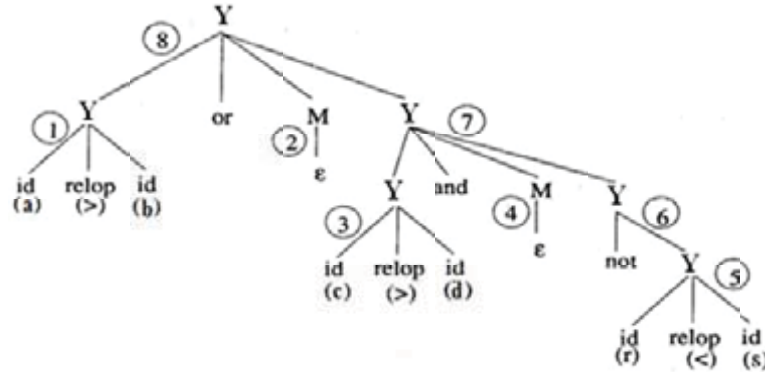
      { M.quad = nextquad }
```

Translation of Boolean expressions to their corresponding code is more clear from the following example.

Example 7: Consider the Boolean expression given as follows:

$a > b$ or $c > d$ and not $r < s$

The parse tree is shown in Figure 9:



▲ Figure 9: Parse Tree for the Given Boolean Expression

Table 7 presents the corresponding action associated with the reduction sequence for the Boolean expression of Example 7:

Table 7: Reduction Sequence and Corresponding Action, Three-Address Code and Final, for the Boolean Expression of Example 7			
Reduction no	Action	Three-address code	Final Code
Reduction 1	Y.Truelist = {1} Y.Falselist = {2}	1: if a > b goto ... 2: goto ...	1: if a > b goto ... 2: goto 3 3: if c > d goto 5 4: goto ... 5: if r < s goto ... 6: goto ...
Reduction 2	M.quad=3		
Reduction 3	Y.Truelist = {3} Y.Falselist = {4}	3: if c > d goto ... 4: goto ...	
Reduction 4	M.quad=5		
Reduction 5	Y.Truelist = {5} Y.Falselist = {6}	5: if r < s goto ... 6: goto ...	
Reduction 6	Y.Truelist = {6} Y.Falselist = {5}		
Reduction 7	Backpatch the list {3} with 5 and modify the quadruple 3	3: if c > d goto 5	
Reduction 8	Backpatch the list {2} with 3 and modify the quadruple 2 Y.Truelist = {1,6} Y.Falselist = {4,5}	2: goto 3	

In the next section, let's learn about translation of conditional and iterative control flow structure.

■ Translation of Conditional and Iterative Control Flow

Each programming language has a set of control structures that detail the direction the program takes. A *control structure* is a set of programming decision-making features that analyze variables and govern the direction in which the sequence of control is based on the given parameters. Constructs such as control transfer, conditional execution, iterative controls, and procedure calls are some control structures found in most languages of PL.

Program generators generate specification of the application domain in some language of the PL domain. These languages encompass many conditional and iterative constructs, which are explained as follows:

- ❑ **Assignment statement:** Represents a single statement to assign some expression to a variable. After the execution of this statement, by default, the control continues to the immediate next statement in the sequence.
- ❑ **if-then-else statement:** Refers to a conditional construct where the control flows either to the *then-part* or to the *else-part* based on the condition met.
- ❑ **while-do loop:** Keeps the control within the loop until the specified condition become false.
- ❑ **begin-end block:** Refers to a group of statements within a block.

Each of these control structures has its own significance in program writing and follows different translation schemes leading to three-address code. The grammar for the preceding types of control statements can be given as follows:

```

S → if Y then S
    | if Y then S else S
    | while Y do S
    | begin L end
    | A /* used for assignment */
L → LS | S

```

In the preceding specifications, we use S, L, Y, and A for denoting a statements, a statement list, a Boolean expression, and an assignment statement, respectively. In order to generate the code for the aforementioned grammar, the grammar can be slightly modified for simplicity. We introduce two new marker non-terminals M and N and the grammar is modified as follows:

```

S → if Y then MS
    | if Y then MSN else MS
    | while MY do MS
    | begin L end
    | A /* used for assignment */
L → LMS | S
M → ε
N → ε

```

We use attributes similar to what we used for Boolean expression. In addition, we also introduce the following attributes to ease the process of three-address code generation:

- ❑ **S.nextlist:** Refers to the list of quadruples having jumps to the quadruple subsequent to S
- ❑ **L.nextlist:** Refers to the list of quadruples having jumps to the quadruple subsequent to L, where *L* represents a group of statements

- **N.nextlist:** Holds the quadruple number that represents the jump after the *then* part of the *if-then-else* statement

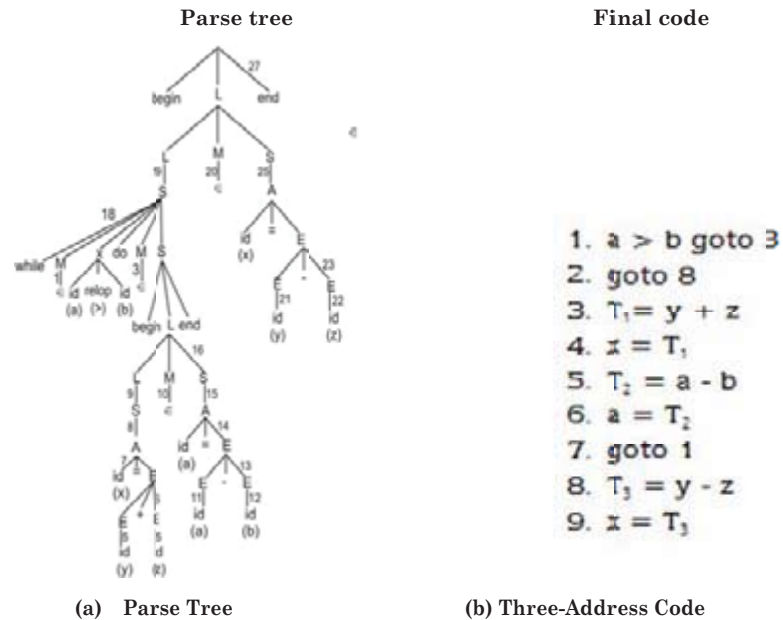
The semantic translations to generate the three-address codes are as follows:

1.	$S \rightarrow \text{if } Y \text{ then } M1$ { backpatch(Y.truelist, M.quad) S.nextlist:=mergelist(Y.falselist, S1.nextlist) }
2.	$S \rightarrow \text{if } Y \text{ then } M1S1N1 \text{ else } M2S2$ { backpatch(Y.truelist, M1.quad) backpatch(Y.truelist, M2.quad) S.nextlist:= mergelist(S1.nextlist,mergelist(N.nextlist, S2.nextlist)) }
3.	$S \rightarrow \text{while } M1Y \text{ do } M2S$ { backpatch(S1.nextlist, M1.quad) backpatch(Y.truelist, M2.quad) S.nextlistS.nextlist: = Y.falselist emit('goto' M1.quad)
4.	$S \rightarrow \text{begin } L \text{ end}$ { S.nextlist: = L.nexlist}}
5.	$S \rightarrow A$ { S.nextlist: = nil}}
6.	$L \rightarrow L1MS$ {(backpatch(L1.nextlist, M.quad) L.nextlist: = S.nextlist)}
7.	$L \rightarrow S$ {L.nextlist: = S.nexlist }
8.	$M \rightarrow \epsilon$ {M.quad=nextquad() }
9.	$N \rightarrow \epsilon$ {N.nestlist = nextquad() emit ('goto...')}

Example 8: Consider the following block of program statements:

```
begin
  while a>b do
    begin
      x=y+z
      a=a-b
    end
```

The parse tree and the three-address code corresponding to the translations are presented in Figure 10:



▲ Figure 10: Parse Tree and Corresponding Three-Address Code Translations

■ Computing l-and r-Values

PLs use variables and array for storage of data items and addresses. Generating code corresponding to them requires translation to find address and value of the data items. Some implementations use *l-value* and *r-value* notations during translation to differentiate address and data of the variables. *l-value* is the location where the value of the expression is stored, and *r-value* is the actual value of the expression. Table 8 presents the rules of translation that can be used by the compiler during translation of arrays:

Table 8: Rules for Translation Of Arrays	
Computing l-value attribute	
$E \rightarrow id$	$E.lval = id.name;$ $E.code = ';$
$E \rightarrow E_1[E_2]$	$E.lval = newtemp();$ $E.lcode = E_1.lcode \parallel E_2.code \parallel E.lval \parallel ':=' \parallel E_1.lval \parallel '+' \parallel E_2.rval$
Computing l-value and r-value attributes	
$E \rightarrow E_1 = E_2$	$E.code = E_1.lcode \parallel E_2.code \parallel gen(' *', E_1.lval ':=' E_2.rval)$ $E.rval = E_2.rval$

Table 8: Rules for Translation Of Arrays

$E \rightarrow E_1 [E_2]$	$E.lval = \text{newtemp}();$ $E.rval = \text{newtemp}();$ $E.lcode = E_1.lcode \parallel E_2.code \parallel \text{gen}(E.lval \text{ ':=' } E_1.lval \text{ '+' } E_2.rval)$ $E.code = E.lcode \parallel \text{gen}(E.rval \text{ ':=' } * E.lval)$
-----------------------------	--

■ Code Generation for Function Calls

All modern programming languages support function (or procedures). Functions allow calling and performing repetitive block of code. Two important activities during the translation of function calls are as follows:

- ❑ **Implementing calling sequence:** Includes the set of actions to be performed (by the caller) during the time of function call
- ❑ **Implementing return sequence:** Includes the set of actions to be performed (by the callee) when returning from the function.

The action performed by the callee and caller during call and return from a function is described in the Table 9:

Table 9: Actions Performed by the Callee and Caller During Call and Return From A Function

Calling/Return sequence	Callee/Caller	Actions performed	Instruction/code
Calling Sequence	Caller	Computes and places actual parameters as desired by the callee	param t
		Stores the current status of the machine (stack pointers and return address), and transfers the control to the callee	call p, n where n is the number of actual parameters and p stand for procedure
	Callee	Save registers, if needed; updates stack and frame pointers to accommodate m bytes of local storage	enter m
Return sequence	Callee	Stores the return value x , if any, to the place as desired by the caller; updates the stack/frame pointers (if needed) and jumps to return address	return x or return
	caller	Stores into x if any value is returned by the callee,	retrieve x

Parameters to a function can be passed either by value or by reference. Let $E_1 \rightarrow (E_2, E_3)$ be the production for defining a function call. The translation of a function call can be given as presented in Table 10:

Table 10: Translation of Function Call	
Production rule	Function calls (Call-by-Value)
$E \rightarrow E_1 (E_2, E_3)$	$E.rval = \text{newtemp}();$ $E.code = E_1.code \parallel$ $\quad E_2.code \parallel$ $\quad E_3.code \parallel$ $\quad \text{gen}(\text{push } E_2.rval)$ $\quad \text{gen}(\text{push } E_3.rval)$ $\quad \text{gen}(\text{call } E_1.rval)$ $\quad \text{gen}(\text{pop } E.rval)$
	Function Calls (Call-by-Reference)
	$E.rval = \text{newtemp}();$ $E.code = E_1.code \parallel$ $\quad E_2.lcode \parallel$ $\quad E_3.lcode \parallel$ $\quad \text{gen}(\text{push } E_2.lval)$ $\quad \text{gen}(\text{push } E_3.lval)$ $\quad \text{gen}(\text{call } E_1.rval)$ $\quad \text{gen}(\text{pop } E.rval)$

■ Translation of Mixed-Mode Expressions

Often, PLs use Boolean expressions that contain arithmetic sub-expressions—e.g., $(A+B)>C$. If $\text{true} = 1$ and $\text{false} = 0$, such expression will have either value 0 or 1. The preferred way would be to use jumping code in such cases. Consider once again the grammar:

$$E \rightarrow E+E \mid E \text{ AND } E \mid E \text{ relop } E \mid \text{id}$$

We see that $E+E$ produces an arithmetic result of which the arguments (operands) can be of mixed type. $E \text{ AND } E$ produces a Boolean result, but both arguments must be Boolean. Use of $E \text{ relop } E$ will produce a Boolean result where the arguments can be again mixed type. Finally, id can be assumed to have type arithmetic. While generating the code, we introduce a synthesized attribute $E.type$, which is either type ‘arith’ or ‘bool’. Boolean expressions are associated with inherited attributes $E.true$ and $E.false$ and used for jumping code. Arithmetic expressions have the synthesized attribute $E.place$ denoting temporaries holding the value of E . We also use variable $nextstat$ to return the index of the next three-address code statement from the sequence. The variable $nextstat$ is incremented by $gen()$ after corresponding three-address code generation. A simple translation for mixed-mode Boolean expression $E \rightarrow E+E$ is summarized in Figure 11:

```

 $E \rightarrow E_1 + E_2$ 
  E.type := arith;
  if E1.type := arith and E2.type := arith then begin
    E.place := newtemp;
    E.code := E1.code | E2.code |
              gen(E.place := E1.place + E2.place)
  end
  else if E1.type := arith and E2.type := bool then begin
    E.place := newtemp;
    E2.true := newlabel; E2.false := newlabel;
    E.code := E1.code | E2.code |
              gen(E2.true := E.place := E1.place + 1) |
              gen('goto' nextstat + 1) |
              gen(E2.false := E.place := E1.place)
  end
  else if ...

```

▲ Figure 11: Translation for Mixed-Mode Boolean Expression $E \rightarrow E + E$

Summary

In this chapter, we have learned that generating the target object code directly from the input source program written in high-level language is difficult. Instead, compiler in most cases translates the source program into an intermediate than generating the output object code. A specially designed code generator that works in synchronization with semantic analysis translates the specifications in the form of parse trees into an IR. The front end of the compiler translates the program into an IP from which the back end generates target object code.

IRs have several advantages and simplified the design of efficient object code. They also help in retargeting target code, offer better abstraction, cross-compilation capability, and facilitate machine-independent optimizations. IRs can be classified into high-level IRs and low-level IRs. Representations that are close to a high-level language specification are called *high-level IR*, while those nearer to assembly are called *low-level IR*. *AST*, *postfix notations*, and *three-address code* are most widely used IRs. Syntax trees and DAGs are high-level IR, while three-address codes scaled down to low-level IRs. Variants of the three-address code include *quadruple*, *triples*, and *indirect-triples*.

ASTs correspond to the procedure's parse tree (the hierarchical structure of the program) with the nodes for most non-terminal nodes removed. A syntax tree shows the structure of a program by abstracting off irrelevant details from a parse tree. Each of the interior nodes in the tree is a record with fields for operator and additional fields for pointers to its child nodes. Leaf node stores either constant or identifier and does not have child nodes. A DAG is an AST with a unique node for each value. An important feature that gives the DAG power over syntax tree is that a node in the DAG can have more than one parent. DAGs are useful in removing common sub-expressions, determining the flow of control, and identifying identifiers that are used inside and outside the basic block.

Postfix notations, also known as *reverse polish notations*, have operators written after their operands, e.g., $AB+$. These are essentially a parenthesis-free notation for arithmetic expression and can be seen as a linearized representation of the same AST.

Three-address codes are low-level linear IRs that resemble to that of assembly code and allow using an infinite number of temporaries (registers) to store the results of operations. The different types of three-address statement are assignment, copy, jump, and procedure calls.

There are three common methods of implementing three-address code: quadruples, triples, and indirect triples. Quadruples are most general form of representing three-address statement. A quadruple, also called quad, uses four fields instead of three for each statement. Triple is a representation of three-address statements, which uses only three fields. Triples also permit optimizations such as common sub-expression elimination, but code motion is difficult with it. *Indirect triples* perform space optimization to triples, but reference triples indirectly. Quadruple has benefit in an optimizing compiler, where statements are often moved around. Both quadruple and indirect triple are equally efficient for reordering of code.

Syntax-directed translations are used for generating three-address code in an efficient way to transform source language to target language. Translations of mixed-mode expression are required for determining the mode and type of sub-expressions and evaluation of the given expression. The translation schemes associate attributes with each symbol of the grammar rules, which commands implementation of semantic rules.

Review Questions

- Q1. What is the significance of intermediate representation in compilation of a source program?
- Q2. List and explain the salient advantages of using intermediate representations.
- Q3. Write a short note on high- and low-level IRs.
- Q4. Compare and contrast syntax trees and directed acyclic graphs as IR.
- Q5. What is the significance of postfix translation schemes? Develop the postfix translation for implementing simple arithmetic expressions.
- Q6. Translate the arithmetic expression $a * - (b + c)$ into
 - a. A syntax tree
 - b. Postfix notation
 - c. Three-address code.
- Q7. Translate the expression $-(a + b) * (c + d) + (a + b + c)$ into
 - a. Quadruples
 - b. Triples
 - c. Indirect triples.
- Q8. List and explain the various types of three-address statements used by compiler for implementing a source program.
- Q9. Define quadruple, triple, and indirect triple. Give example of each.
- Q10. Convert the following program into postfix notation: $a = (-a + 2 * b) / a$.
- Q11. Compare and contrast quadruple, triple, and indirect triple w.r.t. optimizing compiler.
- Q12. Translate the executable statement of the following C program:

```
main ()
{
  int i;
  int at 10 ];
  i = 1;
```

```
while (i <= 10) {
  a[i] = 0; i = i + 1;
```

Q13. Given the following program:

```
int a = 3, b = 8, c = 4, d;
for(j=0; j<=5; j++){
  a = a * (j* (b/c));
  d = a * (j* (b/c));
}
```

Show the intermediate code generated from this source code in the form of quadruples.

Q14. Generate quadruple for the expression: if ($a < b$) then $c = 1$ else $c = 2$.

Q15. Find the postfix representation of the expression: $(A + (B * (C - D)) \uparrow E) * F$.

Q16. Consider the Boolean grammar

$E \rightarrow E_1 \&\& E_2$

$E \rightarrow !E_1$

$E \rightarrow \text{True}$

$E \rightarrow \text{id}$

where ($\&\&$ and $!$ stands for Boolean AND and NOT operators. Generate postfix code for the given Boolean expressions.

Q17. Generate the three address code for the conditional statement

$S \rightarrow \text{if } E, S_1, S_2$

Q18. Generate DAG and three-address representation for the following code:

if ($a < b + c$)

$a = a - c;$

$c = b * c;$

Q19. Develop the semantic actions for productions defining array references given by

$A \rightarrow L := E$

$E \rightarrow E + E \mid (E) \mid L$

$L \rightarrow \text{elist} \mid \text{id}$

$\text{elist} \rightarrow \text{elist}, E \mid \text{id} \mid E$

Q20. What is jumping code? Give example.

Q21. Elaborate on the different forms of intermediate representations.

Q22. Discuss with example the implementation of translation schemes for conditional and iterative conditional flow.

Q23. Write short note on:

a. Translation of mixed-mode expressions

b. Translation of function calls

Q24. What is intermediate code?

Q25. Differentiate between quadruple and triples.

■ **Multiple Choice Questions**

Q1. State true or false:

- i. Low-level IR eliminates structured control flow and generates flat address spaces.
- ii. Control-flow graphs and indirect triples are hybrid representations.
- a. True, True
- b. False, False
- c. True, False
- d. False, False

Ans. c

Q2. The postfix equivalent of the expressions $A*B+C/D$ and $A*(B+C)/D$ are:

- a. $+*AB/CD, ABC+*D/$
- b. $ABCD/+*, AB*CD/+$
- c. $ABCD/+*, AB*CD/+$
- d. $AB*CD/+, ABC+*D/$

Ans. d

Q3. Boolean expressions are evaluated from _____ and the highest precedent among AND, OR, and NOT operators is _____.

- a. Left-to-right, OR
- b. Right-to-left, AND
- c. Left-to-right, NOT
- d. Right-to-left, AND

Ans. c

Q4. _____ nodes of the syntax tree represent results of the operations.

- a. Leaf
- b. Interior
- c. Both a and b
- d. Either a or b

Ans. b

Q5. The references to past results remain accurate in _____.

- a. Quadruple
- b. Triple
- c. Indirect triple
- d. All of the above

Ans. b

06

Runtime Memory Management

OBJECTIVES

- ✧ Understanding runtime memory management and its needs
- ✧ Learning about source language issues in memory management
- ✧ Exploring the techniques of storage organization and storage-allocation strategies
- ✧ Learning about parameter passing and access to non-local names
- ✧ Understanding the implementation of symbol tables
- ✧ Gaining insight onto dynamic storage allocation and its techniques

An indispensable prerequisite for a compiler is storage management that requires allocating and managing the memory used by the program in static and dynamic environments. Elementary data types such as integer, real, or float and logical variables have equivalent data objects at the machine level. The complex structure of modern programming languages require abstractions in the source programming languages, including names, scopes, bindings, data types, operators, procedures, parameters, and flow-of-control constructs. The compiler must take care of them very carefully in the course of compilation to object code. Procedural calls and recursion features require dynamic memory management. All programs have requirements of memory aggregates, which are usually fulfilled using arrays, strings, pointers, and structures. A compiler manages these requirements through *runtime requirements* and solves issues including layout, allocation, and access of storage locations for the program objects. It also assists in linking procedures, passing parameters, and managing controlled interfaces to Operating System (OS).

Code generation phase is greatly affected by storage allocation and access mechanism supported by the data structures in the runtime environment. Before getting into the details of code generation, we need to understand what kind of memory environment we need to provide to support typical program behavior. This depends on the languages being supported and their features such as external reference, recursion, and more. The compiler needs to relate the static source text of a program to the actions that must occur at runtime. Memory management is highly influenced by the size of memory requests made within the source program. Efficient allocation and deallocation of memory is managed using support from runtime libraries of programming languages together with the OS. Calling routines call appropriate routines to allocate memory from the memory chunk to the native program.

The executable machine code generated at the end is completely controlled by the storage allocation techniques used at runtime for the programming language in which the program is coded. When a program is submitted, it is under the control of the OS that allocates space for the program. Once the object code is loaded into a part of the allocated space, the loader makes the OS to jump to the starting point (usually “main”, or else explicitly specified location) and initiate execution. The allocation of data objects is managed by the runtime support package consisting of routines loaded with the generated target code. The dynamic allocation requirements are handled using stack and heap allocation schemes.

This chapter discusses about source language issues. Then it explores the storage organization, wherein you learn about runtime storage organization, allocation of simple data objects, activation and activation trees, Activation Records (ARs), etc. It also explores the storage allocation strategies, which include static allocation, stack allocation, and heap allocation. Next, you learn about management of variable-length blocks, passing parameters, and symbol tables. Further, the chapter discusses about language facilities for dynamic allocation and finally the dynamic storage allocation techniques.

6.1 Source Language Issues

The style and structure of modern programming languages have highly influenced the storage organization and memory management. Modern compilers and many other runtime systems handle data objects of variable size, which are generated in dynamic environment. The task of allocations of memory to such data objects remains a challenging task. The allocation and deallocation of these data objects are done by incorporating runtime support packages, which themselves are supported and influenced by semantic procedures.

The design of runtime and storage management system is greatly influenced by the nature of the source language. The structure and composition of the source language command the requirements of memory management for the program and data objects in the dynamic environment. Symbol tables, strings from the source program, Abstract Syntax Trees (ASTs), register interference graphs, variable-sized arrays in imperative languages, procedures, and incoming messages in distributed languages heavily require dynamic storage organizations. Some of the important source language and runtime issues in memory management are discussed as follows:

- ❑ **Use of procedures:** Nearly all programming languages support the use of procedures, which enable the programmer to reuse the program code. During the call to an executable procedure in the program code, the control is transferred to a specific memory location to execute the procedure body. In addition to transfer of control, management of formal parameters and their corresponding values are important issues. This necessitates controlled access to memory and management of transfer of control between specific memory locations.
- ❑ **Implementing activation tree:** The flow of control between procedures can be designated as tree representations. The execution of the procedure is called activation of that procedure. Call and return from procedures can be therefore illustrated as activation tree. Implementing recursive calls and return are simplified using activation trees.
- ❑ **Maintaining control stack:** The flow of control of the program corresponds to depth-first traversal and control stacks are used to keep track of live activations.
- ❑ **Scope of declaration and binding names:** The portion of the program that declares names is called scope of that name declaration. The declarations can be local or non-local to the procedure, based on the fact whether it occurs with the procedure or outside. Binding is a phenomenon by which the environment assigns memory locations to the names in the program. Static and dynamic binding are two types of binding, which are discussed in detail in later sections of the chapter.

Note that the runtime memory management is greatly concerned with binding variable to its values at proper locations in the memory at appropriate stage of compilation.

Next, let's learn about storage organization.

6.2 Storage Organization

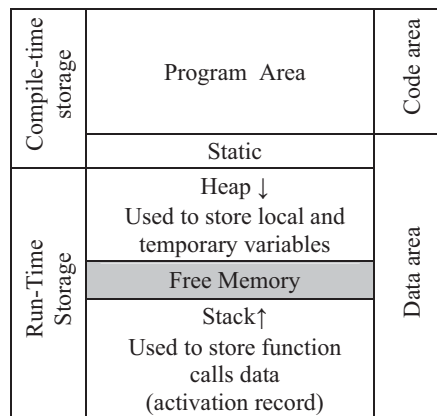
Before a compiler takes up the job of code generation, it must allocate memory for the program objects that have their own logical address space. A program can use data types of different kinds and so the code generator must look into simple storage structures such as variables, pointers, and arrays

to more advanced memory representations including stacks and heaps. For many of these simple data representations, the storage structures can be identified in early stages, while the complex ones require inputs from semantic analyzers.

■ The Runtime Storage Organization

The design of runtime support package is influenced by the semantics of procedures in object program that defines data objects and program area (Figure 1). The storage allocation policies for these runtime objects allocate contiguous bytes to them in the form of blocks, wherein the data type of each object determines the amount of storage needed for each name. The runtime storage is also influenced by addressing and accessing constraints.

A compiler uses static and dynamic memory management principles for fulfilling the storage requirements of a program. Allocation is *static* if it is done at compile time, analyzing only at the source text of the program. On the other hand, the storage allocation is *dynamic* if it can be determined only at runtime. Compilers allocate *static* memory area to executable target code and other data objects such as global constants since their size is known at compile time. Advantage of statically allocating many data objects is that they can be compiled into target code, resulting in fast execution. For other cases, one can optimize the program execution using *dynamic* runtime storage such as stack and heaps, which grow toward each other to maximize space utilization. Stacks are used to maintain ARs, which record information about return addresses and parameters during procedural calls in the program. ARs thus help in determining the flow of control during the program execution. Heaps are allocation structures that distribute ownership of limited memory resources among many program data objects and code. Figure 1 shows the schematics of compile-time and run-time storage arrangement:



▲ Figure 1: Memory Arrangement for the Program

■ Allocation of Simple Data Objects

A compiler may allocate simple variables, such as characters, integers, floats, pointers, and Boolean, sufficiently large memory locations, which can differ from compiler to compiler. Most implementations can store integer types in a binary polynomial form, floats, and doubles in the standard floating point representation. Pointers are implemented as unsigned integers whose memory requirement (either 2 or 4 byte) depends on the range of addresses on the machine. Interestingly, both pointers and unsigned integer store similar data, the only difference being in interpretation. For pointers, the stored data is interpreted as address. The code generator for operational reasons treats addresses fairly similar to integers, although it has much different interpretation than usual integer

arithmetic semantically. The amount of storage required for allocation to name is determined from its type information

The storage organization for local data is determined as the declarations in a procedure are examined at compile time. Variable-length data objects are dealt separately and may be kept outside this organization. Memory allocations for previous declarations are recorded, which help to determine a relative address (offset) of the storage for new local objects with respect to some position on the AR. Arrays are example of data storage, which suit many applications. An array is a contiguous block of elements of same type. The size of an array is at least equal to the size of each element multiplied by the number of elements. The elements are laid out consecutively starting with the first element and working from low memory to high. Given the base address of the array, the compiler can generate code to compute the address of any element as an offset from the base address:

$$\text{arr}[i] = L_0 + k * i,$$

where k represents the size of each element, L_0 is the base address, and i is the index of the element in the array being referenced. In multidimensional arrays implementations, one of two approaches is used: either a true multidimensional array or an array of arrays strategy. A true multi-dimensioned array is one contiguous block, which can be either implemented as row-major form or column-major form. In a row-major configuration (C and Pascal), the rows are stored one after another. In a column-major form, such as in FORTRAN, the columns are stored one after another. For a row-major array of m rows $\times$ n columns integer array, we compute the address of an element as:

$$\text{arr}[i,j] = L_0 + k * (n * i + j)$$

or in a more general form for the array:

$$\text{arr}[i, j] = L_0 + k * ((i - L_1) * (U_2 - L_2 + 1) + (j - L_2))$$

where L_1 and L_2 are lower bounds while U_2 is used for upper bound for referencing elements of a two-dimensional array between upper and lower bounds. The alternative is an array of arrays, i.e., we have an array with one pointer element for each row. Each element of the outer array holds the address of the memory for the elements in the corresponding row. Structures and objects are allocation structures that are laid out by allocating the fields sequentially in a contiguous block, working from low memory to high. The size of a record is equal to at least the sum of the field sizes.

The storage organization highly affects the scope and lifetime of a variable within program's address space for which a variable remains in existence. Variables can be global, static, local, or dynamic. Global variables exist throughout the lifetime of the entire program and can be referenced anywhere, while static exist throughout the lifetime of the entire program but can only be referenced in the procedure (function or module) in which they are declared. Local variables (also called automatic) have smaller scope and exist only until the routine is active in which they are declared. A dynamic variable is created during runtime and its value is obtained during the execution time. Note that the storage layout for data objects of the users object program is strongly influenced by the addressing constraints of the target machine.

Now, let's learn about activation and activation trees.

■ Activation and Activation Trees

All modern programming languages make use of stack or similar data storage extensively. Stacks are linear storage structures based on last-in first-out principle, which are used to manage run-time memory for procedures, functions, or other user-defined methods. For every call to a procedure, the return address and parameter list are pushed on the stack. Space is allocated for each of these and

made them accessible to code areas of the procedures, which may run in totally different memory space. When the procedure terminates, the return address is popped off the stack. The use of stack by a compiler to handle such situations allows the space to be shared by procedure that operates in mutual exclusion. Furthermore, the compiled code for a procedure has always the same relative addresses of its non-local variables, regardless of the sequence in which the calls to procedures were made. In short, the use of stacks is motivated for the following reasons:

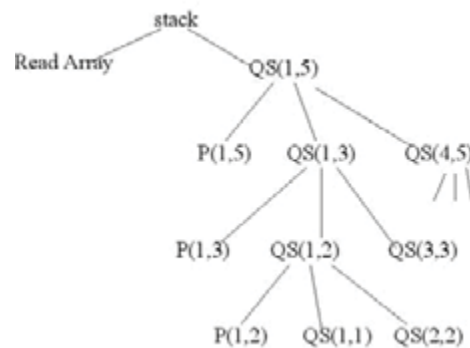
- Space shared by procedure calls rarely exists together
- The relative address of each non-local variable is constant throughout the execution

Note that when a procedure name appears (the procedure call) within an executable statement, the actual parameters are passed to the called procedure through stacks where they are substituted for the formals in the body. The execution of a procedure body is called an *activation* of the procedure. The *lifetime* of an activated procedure can be defined as a consecutive sequence of steps during the execution of a program. Since the flow of control is sequential and returns the control to the point immediately following the place where the procedure was called, this flow of control can be described in the form of *activation tree*. The root of the tree corresponds to the activation of main procedure that initiates the program execution and the interior nodes represent individual activations thereafter.

In an activation tree representation, each node represents a unique activation. For a node that corresponds to an activation of procedure *proc*, the child of this node corresponds to activations of the procedures called by this activation of *proc*. These activations are always seen in the order from left to right and one child must finish before the activation to its right can begin. The structure of the activation tree and its associated program behavior determine primarily influence the execution order. The sequence of procedure calls and returns may correspond to preorder and postorder traversals of the activation tree. The order of the activations is the order in which they appear along the path to node *N* in the activation tree, starting at the root, and return in the reverse of that order. Figure 2 shows an example of activation tree during the execution of quick sort:

Start Read Array
End Read Array
Start Quick Sort(1,5)
Start Partition(1,5)
End Partition(1,5)
Start Quick Sort(1,3)
Start Partition(1,3)
End Partition(1,3)
Start Quick Sort(1,2)
....
Start Quick Sort(1,1)
End Quick Sort(1,1)
...
End Quick Sort(1,3)
Start Quick Sort(3,5)
....
End Quick Sort(1,5)

(a) Activations during Procedure Calls

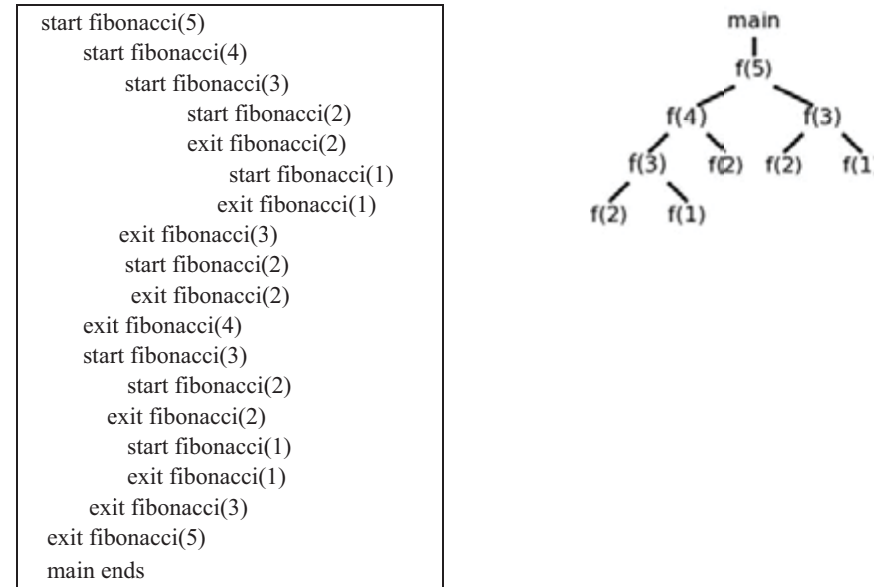


(b) Activation Tree

▲ Figure 2: Activation and Activation Trees for Procedures Calls during Execution of Quick Sort

Let us consider the following example:

Example 1: Consider the Fibonacci series 1,1,2,3,5,8, ... defined by $f(1)=f(2)=1$ and $f(n)=f(n-1)+f(n-2)$ for $n>2$. Figure 3 shows the function calls and corresponding activation tree that result from a main program calling fibonacci (5):



▲ Figure 3: Order of Function Calls and Activation Tree for Computing Fibonacci(5)

■ Activation Records

We use a runtime stack, called a *control stack*, to keep track of *live* procedure activations. Each active procedure call has its own unique stack frame. The control stack is updated for each call and return, viz. push and pop the node for activation, onto the control stack as the activation begins and ends. The entries in the control stack relate to the paths to the root of the activation tree. The live activation on the stack is associated with an AR in which values of local names are kept. It stores information such as the value of the Program Counter (PC) and machine registers when a procedure call occurs. During return from the call, the activation of the calling procedure restarts after restoring the values of relevant registers and setting the program counter to the point immediately after the call. Data objects whose lifetimes are contained in that of activation can be allocated on the stack along with other information associated with the activation. Figure 4 shows the format of an AR:

Actual Parameters
Returned values
Control link
Access link
Saved machine status
Local data
Temporaries

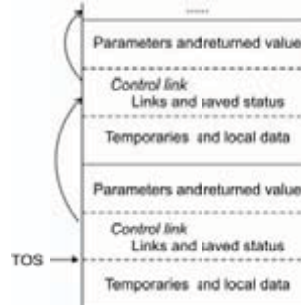
▲ Figure 4: Format of an Activation Record

The number of ARs on the stack at any point of time corresponds to the depth of procedure calls. The root of the activation tree has records at the bottom, while latter activation has its record at the top of the stack. The actual content may vary from language and compiler implementations.

The content of a general AR is shown in Figure 4. *Temporaries* are memory areas for intermediate results generated, such as during expression evaluations. They are used when internal registers are not free or cannot be made available for storage. *Local data* stores data local to activations, while *saved machine status* hold information about return addresses and contents of internal registers during procedure calls. The return address and register values are restored when the control returns to the caller. The *links* connect the ARs by pointing to the ARs of the caller. *Return values* and *parameter* values are also stored on the stacks that can also be placed in registers.

■ Activation Record for Procedure Calls

The use and support for procedures is an important feature of all modern programming languages. Implementing procedure requires managing ARs. The general organization of ARs on the control stack during procedure calls is shown in Figure 5:



▲ Figure 5: Activation Records on the Control Stack during Procedure Call

When a procedure (the caller) calls another procedure (the callee), it triggers the creation of an AR on the stack and fills in the first few fields, while the rest of the field is populated by the callee. The caller computed values, such as arguments to the procedures, are placed at the beginning of the AR, which can be referenced by the caller using fixed offsets. Following this, the AR contains fixed-length items next in the AR. Finally, data items whose size may not be known early enough are placed at the end. These are callee's data items known only at runtime. The stack pointer Top of Stack (TOS) is updated so that it points to temporaries and local data. Fixed-length data can be referenced by fixed offsets (known to the intermediate code generator) from the TOS.

In general, during a calling sequence, the caller evaluates the arguments and stores the return address and the TOS in the callee's AR. It then increments TOS to points to the corresponding point in the callee's AR. Now the callee saves the registers and other (system dependent) information. The callee allocates and initializes its local data and finally begins execution. During return sequence, we restore TOS and the registers. The callee jumps to the return address.

6.3 Storage Allocation Strategies

Allocation strategy is an important factor in efficient utilization of objects memory, defining their scope and lives. In general, a system can allocate space to its data objects using either static, stack, or heap allocations. Static allocation does allocations to the data objects at compile time, while stack and heap allocations are allocation strategies for runtime allocations. Since these data areas have different scope, they follow different allocation strategy described hereunder.

■ Static Allocation

Static memory allocation strategy is compile-time allocation to objects and need not wait till the execution for runtime memory packages. All programs, functions, or procedures that have static declarations for objects can perform *static binding* of these objects name to memory determined at compiler time. Since the memory requirement is fixed, a single copy of static data object is retained for access across the procedure in which it is declared. Advantage of static allocation can also be seen in modular program design, where data must be retained throughout the runtime of the program. For object-oriented programming, it enables a single copy data to be shared between all the objects of the class. All data objects whose memory requirements can be determined at compile time are candidates for static allocation. A static declaration can also have global scope so that the same copy is used throughout a run of code for consistency. This is because objects are bound to the same memory location on activation of the procedure and values of local names are retained across activation of a procedure.

Advantages of Using Static Allocation

Benefits of using static allocation is clearly visible for programs written in controlled environment, i.e., the memory requirements for data objects in the program are fixed and known while writing the program. The storage requirement can be determined from type declarations and memory is set aside for the object name. The address of such allocations is predictable and can be calculated using offsets from one end of the AR for the procedure. The storage for object name is fixed within the AR once the compiler has fixed the relative location of ARs relative to the target code.

Disadvantages of Static Allocation

Static allocation, however, come with few constraints that make programming more constrained and offer little flexibility. Many a times the programmer might not be in a position to know at compile time the object size and constraints, if any, on their location in the memory, which is the basic requirement for static allocation. Compiler must determine where the ARs are placed relative to target code and how they are filled in. It does not permit storage creation dynamically at runtime, and recursive procedures operate in restricted manner as all activations of a procedure have same bindings for local object names. FORTRAN is an example that uses static allocation extensively.

■ Stack Allocation

The stack allocation is based on the principles of control stack in which entries are ARs of some procedure. Based on the LIFO policy, these ARs are pushed and popped on each call (the activations) and return (the end of procedure), respectively. The LIFO principle of operation on stacks make them well suited for recursive calls and parameter passing across procedure calls. Local object names in the procedure are bound to the respective activations and have fresh storages assigned within ARs corresponding to each procedure call. End of the procedure pops off the AR, deleting the storage for local object names too.

Stack allocation is associated with a pointer to the TOS, which is implemented using machine registers. On each activation request, the memory is allocated on the TOS and pointer is incremented by the size of allocation. On execution of return from the procedure, AR is deallocated and TOS is decremented by the same size.

Programming languages allow the use of procedures that give calls to other procedures. Thus, compiler implements execution of such calls by determining calling sequences. A *calling sequence* allocates ARs and data object in them on each call that can be nested in their calls. The code is divided into calling and called procedures (Figure 6). On execution of return statements, the caller is restored and the execution is continued. The allocation scheme works well even for variable-length

data with overhead of maintaining additional pointers that have fixed size. Figure 6 shows the AR on stack during procedure call:



▲ Figure 6: Activation Record on Stack during Procedure Call

Improper deallocation results into problems such as dangling references and memory leaks because other objects may also be referencing these object locations. Therefore, the programmer needs to be more careful while doing deallocations with stack organizations.

■ Heap Allocation

It can be observed that stack allocation strategy may prove to be inefficient in many situations. In particular, when the values of local objects are needed even after the activation has completed or when a called activation outlives the calling procedure, the LIFO principle of stacks does not fit in the requirements of deallocation. An alternative in such situation is storage organization using *heaps*. It allows allocation and deallocation of objects in any order as needed.

Since heaps allow allocation and deallocation in any random order, they are fit for runtime dynamic memory assignments. Space allocation strategy using heaps are optimized for performance by maintaining list of free areas and implementing policies such as *first-fit* and *best-fit* for new object allocations. The heap approach for storage allocation proves to be fast for small storage as heaps use linked structures that can be easily updated.

The global and static arrays stored in the data segment use static allocation strategy, local object and object arrays use stack allocation, and dynamic objects make use of run-time heaps.

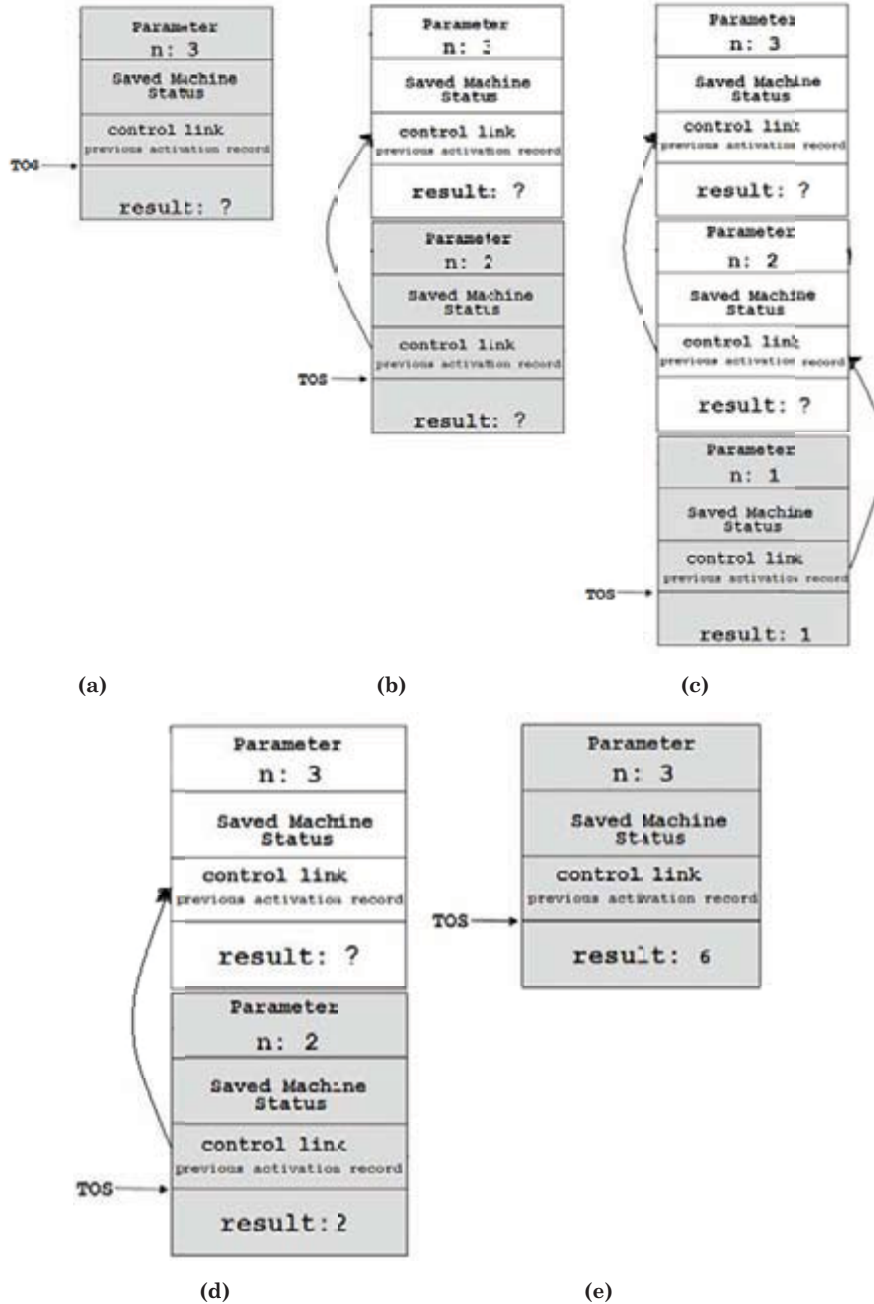
■ Handling Recursive Calls

A procedure is said to be *recursive* if a new activation can be started before an earlier activation of the same procedure has been completed. Procedure activation can be recursive even if call itself directly or indirectly. A procedure *p* is recursive if it calls another procedure *q*, which may in turn call *p* through some sequence of procedure calls. The ARs are created for each recursive call until base criterion occurs.

Each invocation of a procedure has its own instantiation of local variables and recursive calls require several instantiations to exist simultaneously. The local variables are created upon entry to procedure and destroyed when procedure returns. To support recursion, we need to allocate a new AR for every activation of a procedure. Consider the following code fragment for recursive implementation of factorial:

```
int fact(int n) {
    int result;
    if (n<2) result = 1;
    else result = n * fact(n-1);
    return result;
}
```

Figure 7 shows the updates on the control stack for the factorial code for $n=3$:



▲ Figure 7: Activation Record during Recursive Factorial Calls with $n=3$

In the upcoming section, let's learn about management of variable-length blocks.

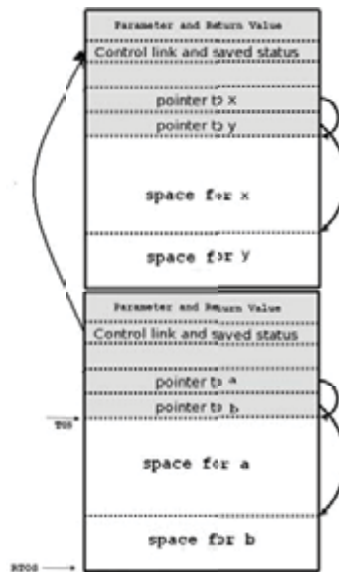
6.4 Management of Variable-Length Blocks

The allocations for data objects that are local to a procedure are not known at compile time. Space allocation for these runtime objects can be done using either heap or stacks. Often, the allocation requirements of data objects within the procedure may be of varied size and moreover need to be determined at runtime. For data obtained by malloc/new methods, it is hard to determine the lifetimes and can be stored in the heap instead of the stack. Arrays and other similar structures whose bounds are determined by the parameters are stack-like in their lifetimes. These are more conveniently allocated space on stacks and avoid the expense of garbage collection.

■ Runtime Allocation Using Stacks

Stacks are useful only for accessing data objects that are local to a procedure (which is in most cases). The data objects on the stack become inaccessible when the control from the procedure returns. Access to such runtime memory allocations is done by introducing pointers. The procedures can access these locations (array of data objects) using addresses pointed by respective pointers which can be determined at compile time. Figure 8 illustrates the use of pointer arithmetic for accessing variable-length data blocks.

Since pointers have fixed size, they can thus be accessed using static offsets. The AR now has additional entries for pointers that point to array of data. Now, for every activation of the procedure, the sizes are known, the pointers are filled in, and the space can be allocated. The modified stack model has now two stack pointers for top instead of one. With this small change, additional pointer called Real-Top-Of-Stack (RTOS) is introduced, which tells from where the new AR should begin. TOS points to the end of the machine-status field and finds local, fixed-length fields of the top AR. Figure 8 shows the AR structure for variable-length blocks:



▲ Figure 8: Activation Record Structure for Variable-Length Blocks

When the current procedure returns, TOS and RTOS are updated. TOS restores values from the saved control link in the current AR, while the new value of RTOS is TOS minus the length of the machine-status, control and access link, return value, and parameter fields. The code for updating TOS and

RTOS can be generated at compile time since the length of entries on the ARs is known at compile time to the caller.

A procedure often needs to access data declared outside its allocated space. This data can be either declared globally outside of all procedures or declared inside another procedure. In Standard languages such as C, the data access is either local to the procedure in question or declared globally. Data access without nested procedures involves fewer issues in implementing access to program variables. The addresses of global variables are known statically at compile time when there is only one source file. Otherwise, linker assists the compiler by resolving the addresses and no further reference to the AR is needed. In either case, the addresses of variables are known prior to execution. The addresses of variables local to the current procedure are found in AR, which is known at compile time and can be computed using constant offset from the TOS. When dealing with variable size arrays, this constant offset refers to a pointer to the actual storage.

When nested procedures are used in the program, complication arises and needs to be handled more carefully. Outermost procedures have nesting depth, while other procedures have nesting depth one more than the nesting depth of the immediately outer procedure. The AR for a nested procedure contains an *access link* that points to the AR of the most recent activation of the immediately outer procedure. The compiler has to manage access links, specially links for procedural parameters, and the programmer remains unaware of it. In many situations, these access links can form long chains, which can be more efficiently handled using the concept of *display*. A display is an array in which entry i points to the most recent (highest on the stack) AR of depth i . In order to maintain the display correctly, we need to save the previous values of the display entries in new ARs.

■ Runtime Allocation Using Heaps

Heaps are non-linear data structure that permits allocation and deallocation of data objects in random order at runtime. They are capable of storing data indefinitely until explicitly de-allocated. It is an internal memory pool created at start-up to dynamically allocate memory as and when needed. It can be used by procedures that require a lot of memory from the stack. An allocation to data object returns a pointer to the allocated area on the heap. It enables to allocate objects whose existence is not tied to the procedure activation that creates them. Heaps are used by memory managers to keep track of all the free space. On allocation request, block of continuous heap memory is assigned to object of requested size. When objects are deallocated, it adds to free list that memory managers use to allocate to new objects. Deallocation introduces free memory blocks interleaved between allocated heap areas.

Memory managers could use either *first-fit* or *best-fit* technique to serve the fresh allocation request. In first-fit allocation, the memory manager identifies and allocates first free area whose size is greater than the requested size. Remaining free area, if any, within this allocation are again added to free list. The inefficient use of memory with this technique is overcome by applying best-fit, wherein the free memory block that wastes minimum is chosen. In either case, maintaining free list is of prime importance. If memory is allocated from the heap, it is available throughout program execution. Allocation for objects from the heap is much slower than obtaining it from the stack. However, heaps are more flexible as compared to stack and allocation can be done any time and deallocated in any order.

Stack data is automatically deallocated when a procedure returns. On the other hand, heaps defined by programming language features such as `new` and `malloc` can be manually as well as dynamically deallocated. Manual deallocation often suffers from problem of memory leaks and dangling references. Memory leak happens when a programmer forgets to deallocate, whereas dangling references are situations where the programmer forgets that he/she has deallocated. Both of these can prove to be highly fatal and therefore automatic method is preferred over manual.

6.5 Passing Parameters

During calls between the procedures, the communication between them can be done using nonlocal names and through parameters of the called procedures. Parameter can be passed to a procedure either by call-by-value, call-by-reference, restore, or call-by-name. A language implementation may use one or many of these methods. The primary differences between parameter-passing methods are based on whether an actual parameter represents an r-value or an l-value. Note that l-value refers to the storage represented by an expression and r-value to the value contained in that storage. The aforementioned different parameter passing methods are explained as follows:

- ❑ **Call-by-value:** The actual parameters are evaluated and their r-values are passed to the called procedure. One important aspect of call-by-values is that the operations on the formal parameters do not affect the values in the AR of the caller. During the procedure call, the formal parameter is treated similar to a local name and the caller computes the actual parameters and places their r-values in the storage for the formal parameters. A procedure called-by-value can influence its caller through the use of nonlocal names or through pointers that are explicitly passed as values.
- ❑ **Call-by-reference:** Call-by-reference is also called call-by-location. In this method, the caller passes a pointer to the memory address of each actual parameter to the called procedure. All reference to formal parameter in the called procedure becomes indirect reference in the target code generated by the compiler. These indirect references managed through the pointer passed to the called procedure. Notably, if the actual parameter happens to be a name variable or some expression with l-value, then the l-value is passed to the procedure. On the other hand, if the actual parameter is an expression, the expression is evaluated and stored in a new storage location and the address of that storage is passed to the procedure.
- ❑ **Copy-restore:** Copy-restore is also known as cop-in-copy-out or value-result, and it is a hybrid of call-by-value and call-by-references methods of parameter passing. In this method of passing parameters, the actual parameters are evaluated ahead of transfer of the control to the called procedure. The l-values of the actual parameters having l-values are determined before the call and then the r-values of the actual parameters are passed to the called procedure similar to the call-by-value method. When control returns from the procedure, the current values of the formal parameters are copied back into the r-values of the actuals, using the l-values computed before the call.
- ❑ **Call-by-name:** Call-by-name method leaves the actual parameters unevaluated and evaluates them anew each time they are needed. The general way of implementing call-by-name is to pass parameterless subroutines called *thunks* to the called procedure. The called procedure will evaluate the l-values and r-values. It is similar to a macro being substituted for a call and actual parameters substituted for formal parameters.

Call-by-value and call-by-reference are widely used parameter passing methods available in most programming languages, including C language. Call-by-name was used in ALGOL programming.

Next, let's learn about symbol table.

6.6 Symbol Table

Symbol table is a form of data organization that stores the names and their associated information which appeared in the source program. The information includes the character string denoting the name, its type, its form, its memory address, and any other attributes, depending on the language used for programming. The simplest structure of an entry in the symbol table is a pair of the form (name, information). For each reference to a new name during the analysis of the source program, the symbol is updated with the new name and its information. The table is searched to see whether that name has

been seen previously. If the name is new, it is entered into the table and information about that name is updated during analysis phases of the compilers. If the name is already present in the symbol table, then only information about it is updated in the table and no new entries are created.

■ Uses of Symbol Table

Symbol table is used in most phases of a compiler, including semantic analysis, code generation, and error detection in addition to earlier phases of compile where it is constructed. The purpose of the usage of symbol table in these phases of the compiler is explained as follows:

- ❑ **Used during semantic analysis:** To verify the uses of names for consistency with their implicit or explicit declaration
- ❑ **Used during code generation:** To find how much and what type of storage must be allocated to a name
- ❑ **Used during error detection:** Used for error-reporting purposes during and postprogram development

■ Content of the Symbol Table

The symbol table has at least two fields containing a name and an information field. There may also be a single or separate table for variable names, labels, procedure names, constants, field name, and other types of names. It is often useful to have more than one table because the amount of space needed per name may vary considerably, depending on the usage of name. The most common data structures used to implement symbol table are linear list, hash tables, and variants of tree structures.

The information in the symbol table is as follows:

- ❑ The character string denoting the name variable. If the identifier is used in more than one block or procedure, then the name of the block or procedure which it belongs may also be specified in the information field.
- ❑ Attributes of the name and information identifying its use, viz., name is a label, a formal parameter, or an array
- ❑ Parameters including the dimensions of the array, and the upper and lower limits on each dimension may also be specified.
- ❑ Address and offset information for the allocation of name variables.
- ❑ Any other relevant information

One of the simplest implementation of symbol table is to use a linear array of records. A record contains number of consecutive words in memory (Figure 9(a)). This method is suitable if there is a control on the upper limits on the length of the identifier. For variable-and dynamic-length requirements, other organization including lists with link pointers can be used. Many implementation store memory links or pointers rather than storing the identifier and its information in the symbol table itself (Figure 9 (b)). When an access to an identifier is requested, it is accessed from the location pointed by the link pointer field in the symbol table. The compiler can allocate space for the information in a relatively small portion of the compiler's data space and set the symbol table pointer to the location of that information. The complete identifier denoting a variable name must be stored to ensure that all uses of the same name can be associated with the same symbol table record. Figure 9(a) shows a simple symbol table organization and Figure 9(b) shows a symbol table organization using pointer field:



(a) Simple Symbol Table Organization (b) Symbol Table Organization Using Pointer Field

▲ Figure 9: Storage of Identifiers in the Symbol Table

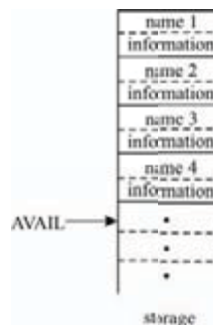
The identifiers used by the programmer denote a specific name that must be preserved in the symbol table. This is essential so that all users of the identifier can be associated with the same symbol table entry and hence the same name. Special care must be taken while working with external names. To work with external names, the compiler finds name *external* and makes a copy of name in another small array of characters. The pointer for the identifier (name) is made to point to the new copy of name.

■ Data Structures for Symbol Table

During compilation, the symbol table is searched every time an identifier is encountered. Data is added if a new name or new information about an existing name is discovered. Some popular symbol table organizations are linear lists, trees, and hash tables, some of which are explained in the following subsections.

List

Linear list organization is the simplest and easiest way to implement symbol tables. It can be constructed using single array or equivalently several arrays that store names and their associated information. During insertion of a new name, we must scan down the list to ensure that it is a new entry. If an entry is found during the scan, it may update the associated information but no new entries are made. To retrieve information about name, we search from the beginning of the array up to the position marked by the pointer AVAIL. AVAIL is the beginning of the available free memory---the empty portion of the array. If the symbol table has n names, the insertion of new name will take effort proportional to n ; and to insert n names with m information, the total effort is $cn(n+m)$, where c is a constant representing the time necessary for a few machine operations. The advantage of using list is that it takes minimum possible space. On the other hand, it may suffer for performance for larger values of n and m . Figure 10 shows the linear list organization of a symbol table:

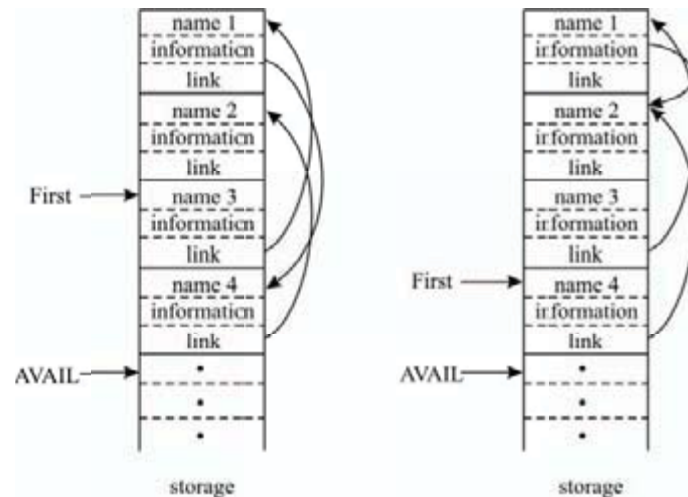


▲ Figure 10: Linear List Organization of Symbol Table

Self-Organizing List

Searching in symbol table takes most of the time during symbol table management. We can save a substantial fraction of time spent in searching the symbol table at the little cost of extra space. Pointer field called LINK is added to each record, and the search is controlled by the order indicated by the LINK. A pointer called FIRST can be used to designate the position of the first record on the linked list, and each LINK field indicates the next record on the list.

Self-organizing list is advantageous over simple list implementation in the sense that name variables that are referenced frequently will be likely to be at the front of the list. If a small set of names is heavily used in a program, then those names will grow to the top. If the access is random, the self-organizing list will cost time and space. Figure 11 shows self-organizing symbol tables:



▲ Figure 11: Self-Organizing Symbol Tables

Search Trees

Symbol tables can also be organized as binary tree organization with two pointer fields, namely, a LEFT and RIGHT, in each record that points to the left and right subtrees, respectively. The nodes of the tree represent records with names and information. The left subtree of the record contains only records with names less than the current records name. The right subtree of the node will contain only records with name variables greater than the current name. Both the left and right subtrees are also search trees. The advantage of using search tree organization is that it proves more efficient in searching operations, which are the most performed operations over the symbol tables.

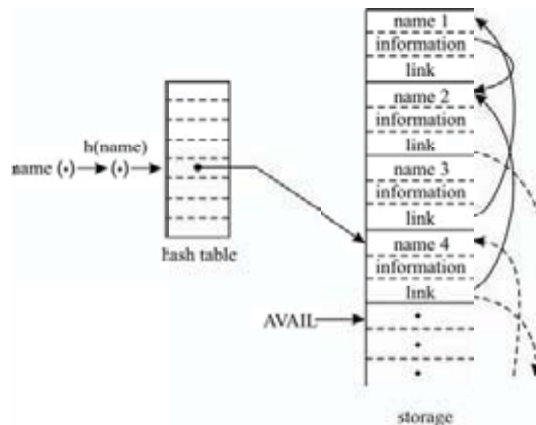
This means that all names $name_j$ will be accessible from $name_i$ by following the link $LEFT_i$ and then following the sequence of the links will precede $name_i$ in alphabetical order. A binary search tree gives better performance compared to list organization at some difficulty in implantation.

Hash Table

Hash table organization is another efficient way of implementing symbol tables that outperform other data structures with its capability of performing m accesses on n names. This data structure has the following two parts:

- A *hash table* that contains a fixed array of m pointers to storage table entries
- *Storage table* entries organized into separate linked lists called buckets

With this organization, each record in the symbol table appears on exactly one of these lists. In order to determine whether *name* variable is already in the symbol table, we use a hashing function h to name such that $h(name)$ maps to an integer between 1 and $k-1$. Hash table has storage space of k words. During a search for the name, $h(name)$ is computed and we search the list corresponding the hash value only. During the insert operation for a new name variable, we create a record in the first available space in the storage table and link that record to the beginning of $h(name)^{th}$ list in the table. Assuming that the size of average list is n/k records long, we reduce the search down to factor of k . Notably, hashing table organization provides best performance over others at the cost of somewhat greater programming effort and some extra space. Figure 12 shows hash organization of symbol table:



▲ Figure 12: Hash Organization of Symbol Table

6.7 Language Facilities for Dynamic Allocation

Support for dynamic allocation of storage for data is one of the important features of modern programming environments. Storage of dynamic allocations usually makes use of heap data structure, wherein allocated data is retained until it is explicitly deallocated. The presence of dangling references and garbage are some of the challenges that require attention. Most modern languages provide facilities to deal with them.

■ Dangling References

If a heap variable is destroyed, any remaining object reference that still refers to it is said to contain a *dangling reference*. This means the storage that has been deallocated is still referred even after its deallocation. Dangling references occur if deallocation occurs before the last reference.

There are many approaches to avoid this problem. The best option is to structure the code itself such that dangling references do not occur. Most modern languages provide inbuilt mechanism to detect and recover from the effect of such dangling references.

■ Garbage

Many program implementations have allocations to data objects that are never referenced. In fact, we can see that these are unreachable data objects, and once an object becomes unreachable, it stays unreachable. These types of data objects are known as *garbage* with no significance and can be thrown away. The memory occupied by them is a waste as none of the program control will visit these locations. Note that garbage exists if the last reference occurs before deallocation. Most modern high-level language implement concept of garbage collection, which is a form of automatic memory

management. Garbage collection identifies the objects that are no longer needed by the program and reclaims their memory allocations. In simple words, garbage collection is a mechanism for memory recycling that recycles the heap spaces of objects that are no longer referenced by the program, thus making space available for subsequent new objects. The technology has been popularized among object-oriented environment for program development such as Java and .NET.

With the popularity of garbage collection, programming has been simplified and freed from the hassle of explicit memory management for every object that the programmer creates. The development of efficient algorithms and increased computational power of processors have made the overhead of implementing garbage collection mechanism significantly negligible. Note that the garbage collector routine must somehow determine which data objects are no longer referenced and allocate their space (by handling them over to memory manger, which keeps track of free spaces) to unreferenced objects. It can be simplified by understanding that references to objects are always to the address of the beginning of the object, with no pointer to memory references within the object. The garbage collector must finalize objects being freed and freeze them, and modify the collection of objects in the heap. Lisp performs garbage collection.

■ Garbage Collection Algorithms

Garbage collection can be achieved in many ways. Any garbage collection algorithm at its core detects garbage objects and then reclaims the heap space occupied by the garbage objects to make them available again to the program for new objects. These algorithms determine whether an object is reachable by set of any other currently reachable objects called roots. Roots are such that they are always accessible to the program, and objects reachable from the roots are considered "live" and classified as non-garbage objects. Any other object is unusable and has no role in future course of program execution apart from unnecessarily holding allocated spaces. The garbage collection broadly uses two basic approaches called *reference counting* and *tracing* to classify program objects into live objects and garbage.

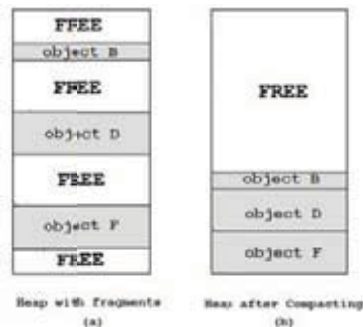
In *reference counting* technique for garbage collection, the system associates a *count* with each object on the heap memory to keep track of its active users. The count is incremented when a user gains access to an object area and decremented after the user finishes using it and no longer needs it for immediate use. When the count drops to zero, the object memory is unreachable and identified as garbage, which can be set free and allocated to some other object if needed. It is quite convincing as it can run in small chunks of time closely interwoven with the execution of the program. The disadvantage of this technique is that reference count technique demands frequent updates on allocation and deallocation. Moreover, it cannot account for cyclical references (two or more objects that refer to one another) and requires every object to reserve a special storage space for maintaining reference counts.

Tracing, also called *mark-and-sweep* technique of garbage collection, traverses all the references pointing to allocated area and *marks* the one in use. This is, in fact, a two-pass memory trace that scans through the tree of references starting with the root nodes, marking in some way the objects that are encountered during the first pass of trace. Marking requires setting flags that can be done as part of object storage itself or by setting flags in a separate storage for the purpose. Any object area after the first trace that remains unmarked is known to be unreachable and can be garbage collected. The second pass, also called sweep, finds all unmarked heap areas and declares them as free. The mark-and-sweep mechanism addresses cyclic references more accurately. It can easily recognize objects that have already been flagged as reachable. A disadvantage of using mark-and-sweep techniques is that the memory becomes fragmented since objects can be removed sporadically throughout the heap, developing unused holes. Tracing also proves to be resource intensive and requires scanning memory irrespective of memory being marked or not.

■ Compacting Collectors

Garbage collection should also account for addressing the critical problem of heap fragmentation that occurs through the course of normal program execution. *Fragmentation* is nothing but development of unused holes that are created as new objects are allocated, while unreferenced objects are freed. The free space of heap memory is left in between portions occupied by live objects. Traditionally, the request from new objects for memory space could not be fulfilled in most cases, as there is not enough contiguous free unused heap space available into which the new object will fit. The allocation for new object could be done only by extending the memory heap, which is not feasible for practical reasons of limited memory. The garbage collectors also have strategy to combat the heap fragmentation problem. Two common strategies are *compacting* and *copying*, which are adopted by tracing collectors to deal with the issue. Both compacting and copying techniques move objects on the fly-and-reduce heap fragmentation.

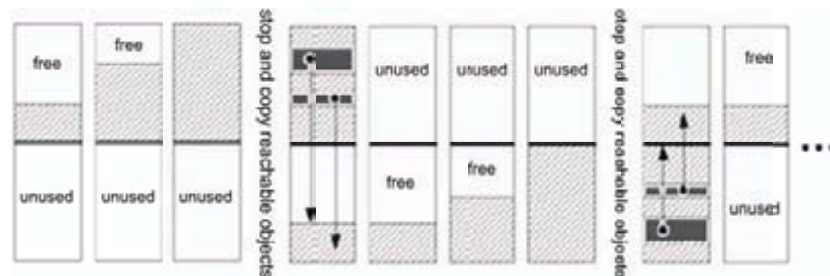
Compacting collectors maintain a clear partition of memory by dividing it into active and free sections. It scans the memory heap to find all reachable (live) objects and slides them over free memory space toward one end of the heap. As a result, the other end becomes large contiguous free memory available for consumption. Since the technique compresses all marked entries to one end, it is also known as *mark-compact* collector. All references to moved objects must be updated to new locations to avoid dangling references. It can be simplified by adding a level of indirection to these object references. Table of object handles is used for indirection whose entries point to actual objects on the memory heap. With this arrangement, all references are made to the table in instead of objects on the heap. The object handles are updated to new location after every object move. All references to the object in the executing program remain unaffected and still refer to table of handles and gain access to new location of the moved objects. The approach achieves heap defragmentation at the cost of some performance overheads. Figure 13(a) shows heap with fragments and Figure 13(b) shows heap after compacting:



▲ Figure 13: Allocation Status of Heap and Heap after Compaction

Copying garbage collectors can dismiss garbage objects without ever scanning them. The technique is based on splitting their allocated memory space into two sections, only one of which is used at any time. The two sections may be termed as active and unused sections. There are no separate mark and sweep phases. Moreover garbage objects are not moved from the memory; rather, they relocate all non-garbage live objects to an entirely new location in the memory called active section. All data objects are allocated space from the active section until all the space in the section has been exhausted. When the section runs out of memory, the program execution stops and the garbage collector traces through the heap for all reachable objects. All reachable objects are copied contiguously to the unused (other) section as discovered during the trace. When the stop and copy procedure is finished, the active section is full of reachable and unreachable objects, and unused section has only copy of reachable objects. The program execution resumes with new active section

(the previously unused section becomes active and vice versa), allocating memory from this new active heap until it too runs out of space. The program execution stops once again. The process is repeated again--the heap is traced, reachable objects are copied again, and the sections are swapped. Note that the active and unused sections in the memory switch places, dismissing the previous section of memory. This technique is also popularly known as stop and copy collector approach. Figure 14 shows the copying collector mechanism:



▲ Figure 14: Copying Collector Mechanism (Stop-and-Copy)

The copying collector approach has excellent allocation speed and eliminates fragmentation, but requires twice the amount of memory to run effectively. Also, the objects that have large lifetime result into swapping overheads. This is because of continuous swapping between active and unused sections for every collector run.

■ Generational Collectors

The analysis of the program execution reveals that there are short-lived and long-lived objects on the heap. Stop and copy approach disadvantageously copies all reachable objects at every collection irrespective of the object lives. A *generational collector* attacks this inefficiency by classifying and garbage collecting the objects by age, wherein younger objects are collected more often than older objects. Generational approach divides the memory heap into two or more sub-heaps, serving one generation of object each. Objects from the youngest generation are garbage collected most often and only a small percentage of these young objects survive their first collection. Objects are usually created in the base (youngest) generation, and if they survive few garbage collections, they are promoted to older generations by moving them to another higher position in the heap. Progressively, the older generations are garbage collected less often than the next younger generation. Importantly, when an older generation object references a younger generation object, some difficulties arise. This situation can be handled by maintaining a list of intergenerational references. The technique improves efficiency of garbage collection and can be applied to mark-and-sweep as well as copying algorithms.

■ Advantages and Disadvantages of Garbage Collection

The primary advantage of garbage collection is that it relieves the burden of freeing allocated memory by identifying it automatically. Moreover, garbage collection helps in ensuring program integrity and security in many programming languages. Programmers are unable to accidentally (or purposely) access (overwrite or even free) system area that would otherwise prove to be fatal and even crash the system.

The likely disadvantage of a garbage collection is that it adds overheads in collection of unusable garbage objects, which can affect program performance. The overheads are clearly visible as to keep track of live objects, it has to execute memory scanner program, and finalize and free garbage objects on the fly. This activity is likely to consume more CPU time than program, explicitly freeing unnecessary memory. Moreover, programmers have less control over the scheduling of CPU time, which is no longer needed.

6.8 Dynamic Storage Allocation Techniques

Techniques of storage deallocation have strong influence on implementation dynamic storage allocation. A deallocation can be of two types: implicit or explicit. Implicit deallocation requires runtime support package to decide when a storage block is no longer needed. For explicit deallocation, the compiler has little to do as it is totally controlled by the programmer.

■ Managing Implicit Deallocation

Implicit deallocation requires cohesive understanding between the user program and the runtime support packages. The support packages must be able to know when storage blocks are no longer in use. This is possible by implementing fixed format of storage blocks. These storage blocks may contain information such as block size, reference count, mark filed, pointers to block, and user information. If the size of the block is fixed, it can be used for recognizing the object boundaries of deallocation and hence used for allocating new objects. Deallocation of storage blocks in use may cause problems including program crash or other unwanted behaviors. Compiler can read the reference count field to determine its status of use and take necessary decisions of deallocation and allocation to new object blocks. Deallocation is primarily managed through reference count and marking technique. If the reference count drops to 0, then the block can be deallocated as it is not being referred by any block. Reference count goes up by one for each new reference to it. Reference counts can be used when pointers between blocks never appear in cycles.

■ Explicit Allocation of Fixed-Sized Blocks

It is the simplest dynamic allocation techniques for block of fixed size. The technique links blocks of fixed size in a list, which allows quick allocation and deallocation. During allocation, the block is obtained from the list of available block, while deallocation adds the block back to the list of available blocks.

The compiler merely treats the block as a block consisting of a link to the next block. Compiler routine does not require knowing the type of object in the block. With no space overheads, the user program can make use of the entire block for its own purposes. Once the block is deallocated, the runtime routine of the compiler uses some space from the block to link it into the list of available storage blocks.

■ Explicit Allocations of Variable-Sized Blocks

Storage space becomes fragmented during allocation and deallocation. Fragmentation is the presence of alternate storage blocks that are free and in use. The problem is more severe in variable-sized blocks. Although enough space is available, allocation of a block larger than any one of the free blocks cannot be done. To deal with variable-sized block, *first-fit* methods can be used, wherein we search for the first free block that is of size greater than the current requirement. Although simple in implementation, this method incurs time overheads because one has to search for a free block that is large enough to hold the data object. *Compacting* adjacent storage blocks can free blocks of memory which can reduce further fragmentations.

Summary

In this chapter, we have learned that compiler has much more things to do besides generating code. It must look into storage requirements for data objects defined in the program. It has to manage memory requirements in static and dynamic environments as well in its course of object code generation. Programming languages that use procedural calls and recursion features require dynamic memory management and bring in the need for runtime storage requirements. Code generation phase is greatly affected by storage allocation and access mechanism supported by the data structures in the runtime

environment. The kind of memory environment during code generation depends on the languages being supported and their features. The compiler needs to relate the static source text of a program to the actions that must occur at run-time. Efficient allocation and deallocation of memory is managed using support from runtime libraries of programming languages together with OS.

Some of the important of source language and runtime issues in memory management include use of procedures, implementing activations, managing control stack, scope declaration, and binding names. The storage organization affects the scope and lifetime of an object variable within a program's address space for which an object remains in existence. Variables can be global, static, local, or dynamic. Global variables exist throughout the lifetime of the entire program and can be referenced anywhere, while static variables exist throughout the lifetime of the entire program but can only be referenced in the procedure in which they are declared. A dynamic variable is created during runtime and obtains its value during execution time. Based on this modern compiler, implementations allow *static* and *dynamic* storage assignments.

Runtime support package is influenced by the semantics of procedures in object program that defines data objects and program area. Allocation is *static* and it is done at compile time, analyzing only at the source text of the program. The storage allocation is *dynamic* if it can be determined only at runtime. Dynamic runtime storage is implemented using stack and heap allocation structures.

The execution of a procedure body is called an *activation* of the procedure. Flow of control can be described in the form of *activation tree*. The root of the tree corresponds to the activation of main procedure that initiates the program execution and interior nodes represent individual activations thereafter. We use a run-time stack, called a *control stack*, to keep track of *live* procedure activations. The live activation on the stack is associated with an AR that keeps values of local names. The number of ARs on the stack at any point of time corresponds to the depth of procedure calls. Contents of AR are temporaries, local data names, and control and access links, and return values. Recursive calls can be implemented by efficiently using control and access links that point to caller AR and so on. Variable-length blocks are implemented by adding fixed size entries for pointers to arrays of data objects. Such implementations have two tops instead of one. Stack data is automatically deallocated when a procedure returns.

Allocation strategy is an important factor in efficient utilization of objects memory, defining their scope and lives. In general, a system can allocate space to its data objects using either static, stack, or heap allocations. Static allocation does allocations to the data objects at compile time, while stack and heap allocations are allocation strategies for runtime allocations. *Static memory allocation* strategy is compile-time allocation to objects and need not wait till execution for run-time memory packages. The stack allocation is based on the principles of control stack whose entries are ARs of some procedure. Efficient runtime allocation can be done using heap allocation that allows to dynamically allocate memory as and when needed. Management of variable-length block can be efficiently done using stacks and heaps.

During calls between the procedures, the communication between them can be done using nonlocal names and through parameters of the called procedures. Parameter can be passed to a procedure either by call-by-value, call-by-reference, restore, or call-by-name. Symbol table is a form of data organization that stores the names and their associated information appeared in the source program. For each reference to a new name during the analysis of the source program, the symbol is updated with the new name and its information. Symbol table has uses in most phases of compiler including semantic analysis, code generation, and error detection in addition to earlier phases of compile, where it is constructed. Symbol table can be implemented using linear array of records, list, self-organizing list, tree organization, and hash table organizations. Each has its advantages over some aspects of compilation and implementation issues.

Deallocation introduces free memory blocks interleaved between allocated heap areas called fragmentation. Heaps can be used by memory managers to keep track of all the free space by mechanism called garbage collection. Many program implementations have allocations to data

objects that are never referenced. These are known as garbage objects. Language facilities for dynamic allocation include provisions for handling dangling references and managing garbage. Garbage collection identifies the objects that are no longer needed by the program and reclaims their memory allocations. The garbage collection broadly uses two basic approaches called *reference counting* and *tracing* to classify program objects into live objects and garbage. In *reference counting* technique for garbage collection, the system associates a *count* with each object on the heap memory to keep track of its active users. When the count drops to zero, the object memory is unreachable and identified as garbage, which can be set free and can be allocated to some other object if needed. *Tracing*, also called *mark-and-sweep* technique, is a two-pass algorithm of garbage collection that traverses all the references pointing to allocated area in the first pass and *marks* the one in use. The second pass, also called sweep, finds all unmarked heap areas and declares them as free.

The garbage collectors also have strategy to combat the heap fragmentation problem. Two common strategies are *compacting* and *copying*. Compacting scans the memory heap to find all reachable (live) objects and slides them over free memory space toward one end of the heap. *Copying* garbage collectors is based on splitting its allocated memory space into two sections termed as active and unused sections. Another technique called *generational collector* attacks this inefficiency by classifying and garbage collecting the objects by age, wherein younger objects are collected more often than older objects.

Allocation strategy is an important factor in efficient utilization of objects memory, defining their scope and lives. System can allocate space to its data objects either static, stack, or heap allocation. Static memory allocation strategy is compile-time allocation to objects and need not wait till execution for run-time memory packages. The stack allocation is based on the idea of control stack whose entries are ARs of some procedure. Based on LIFO policy, these ARs are pushed and popped on each call (the activations) and return (the end of procedure), respectively. Heaps allow allocation and deallocation in any random order. They are fit for runtime dynamic memory assignments. The heap approach for storage allocation proves to be fast for small storage. Dynamic storage allocation technique addresses the issues concerned with implicit and explicit allocations. Explicit allocation of fixed and variable-sized blocks employ different techniques to handle problems caused by deallocation.

Review Questions

- Q1. What is runtime storage organization? How do modern programming languages necessitate using such storage assignments?
- Q2. Write a short note on modern storage organization.
- Q3. Differentiate between static and dynamic allocation techniques. Also describe the structure of runtime memory management.
- Q4. What are activation records? Describe the stack oriented allocation strategy and its operation for maintain activation record.
- Q5. How do we manage storage for programming environment with variable-length data storage and nested calls?
- Q6. Explain how are procedure calls implemented in modern programming languages.
- Q7. Explain with example the allocation and deallocation of activation record for demonstration of Fibonacci sequences.
- Q8. What are heaps? State its advantages and disadvantages in run-time storage organization.
- Q9. What is garbage collection? Explain.
- Q10. Explain various strategies in brief for garbage collection in run-time environment.
- Q11. What is reference counting method of garbage collection?
- Q12. Write short note on tracing.

- Q13. What are compacting collectors? How do they solve fragmentation problem? Explain.
- Q14. What is copying? How is its different from compacting?
- Q15. What are different storage allocation strategies adopted by modern compiling environment?
- Q16. List and describe in brief the various source language issues related to run-time memory management.
- Q17. Explain the storage allocation strategies of run-time memory management.
- Q18. What are symbol tables? Why are they important?
- Q19. Describe in brief the data structures used for symbol table organization.
- Q20. What are the methods of parameter passing? Explain.
- Q21. Write short note on storage allocation techniques.
- Q22. Compare and contrast static and dynamic allocation of program objects.
- Q23. Compare and contrast stack and heap allocations.
- Q24. What is activation? Explain with example.
- Q25. Describe the structure of atypical activation record. Also explain the concept of activation record with suitable example.
- Q26. Explain with suitable illustration the activation record during procedure call and return.
- Q27. State the advantages and disadvantages of static allocations.
- Q28. Explain how the use of activation record facilitates implementing recursive calls.
- Q29. Explain how stacks and heaps can be used to manage allocations of variable length block?
- Q30. Write short note on:
 - a. Garbage collection
 - b. Dangling references
 - c. Implicit deallocation
 - d. Heap as a dynamic memory storage

■ Multiple Choice Questions

- [illegible]

07 | Code Optimization

OBJECTIVES

- ↗ Understanding the need of code optimization in compilation
- ↗ Learning about the various classifications of optimization
- ↗ Learning about global data flow analysis and its significance in optimization.
- ↗ Understanding the importance and techniques of loop optimization.
- ↗ Gaining insight into popular code optimizations

To make the target programs fast, small, and maintainable, a compiler has optimization phase, wherein it applies certain transformations on the source or output of the Intermediate Code (IC) generator. The *code optimization* phase applies sequence of optimizing transformations, which transform the input program to a program that requires fewer resources. The front end of the compiler generally produces an intermediate language version of the source program that can be easily optimized, resulting in a faster and smaller target program. Developers perform optimization on source code, while optimizing compiler does it on IC.

The optimizing compiler tends to generate target program with higher execution efficiency. Notably, these optimizations must follow the following conditions:

- ❑ Optimization must maintain semantic equivalence between the original source and optimized code.
- ❑ Optimization should not compromise with the algorithm of the program. The algorithm remains unchanged as much as possible.
- ❑ Results before and after optimization should remain unchanged.
- ❑ The optimize code should avoid redundancy and produce less code.

There are many factors that affect the optimization activities of a compiler. The machine and Central Processing Unit (CPU) architecture are two such important factors. Factors such as cache size and type and memory transfer rate on the given machine, number of registers, number of functional units, pipelines, and Reduced Instruction Set Computing (RISC)/Complex Instruction Set Computer (CISC) instruction set have more control on what is generated as target code.

In this chapter, you learn about the classification of compiler optimization. Next, the chapter explores the important optimizing transformations. Then it discusses about global data flow analysis. Toward the end, you learn about role of data flow analysis in optimization.

7.1 | Classification of Compiler Optimization

There are various types of optimizing transformation carried out by a compiler. This optimization can be classified into the following categories:

- ❑ **Language-dependent and -independent optimizations:** Most of the High-Level Languages (HLLs) from Programming Language (PL) domain follow similar programming constructs and abstractions. Hence, similar optimization can be applied across the languages of the PL domain.

These optimizations are called *language-independent* optimizations and include constructs such as decision (e.g., if, switch, and case) and looping (e.g., for, while, repeat until, and do while). Some PLs have certain specific features, which do not occur in all the languages, and similar optimization across the languages is not possible. For example, use of pointers and function may not be the same in all languages. When optimizations are more specific to the language and its implementation, they are called *language-dependent* optimizations.

- ❑ **Machine dependency:** Optimization can also be classified into machine-dependent or -independent optimization. *Machine-independent optimization* generally works on the Intermediate Representation (IR), creates more optimal matrix or similar representation, and assists in efficient target code generation. Example of machine-independent optimization includes removing common sub-expression. These optimizations work on IRs and are not concerned with the final machine details. *Machine-dependent optimization*, on the other hand, takes place during or post final code generation phase. These optimizations are greatly concerned with improving execution efficiency of the target code on the machine by optimizing register usage, reducing redundant load, storing operations, and using appropriate addressing modes. All machine-dependent optimizations require the knowledge of machine architecture and its operation.
- ❑ **Scope of optimization:** The result of compiler optimization can be either restricted to a single statement or propagated to the entire program. Therefore, the scope of a particular optimization can be classified as follows:
 - **Peephole optimizations:** Refer to the optimizations that would inspect at instructions to transform them into comparatively simpler and less expensive ones. For example, a multiply instruction can be equivalently implemented as additions, which is an economical alternative. Peephole optimization is discussed in detail in Chapter 8 of this book.
 - **Local optimization:** Specifies the optimization that is restricted to a single basic block.
 - **Loop optimizations:** Refer to the optimizations on loop structure within a program, which provide significant improvement of the program performance. These operate on more than one basic block that makes up a loop.
 - **Global optimizations:** Refer to the optimizations that operate on the complete flow graph of a single procedure. These are basically focussed on a larger section of a program, typically on a loop or procedure/function.
 - **Whole-program optimizations:** Refer to the optimizations that optimize the interactions between procedures within a program.

In the next section, let's explore some of the important optimizing transformations.

7.2 Important Optimizing Transformations

We discuss about some important optimizing transformations in this section. Note that all optimizations have the objectives of producing codes that are smaller, smarter, and closer to the underlying architecture. These include transformations such as *compile-time evaluation*, *Common Sub-Expression Elimination (CSE)*, *variable propagation*, *code movement*, *strength reduction*, *dead code elimination*, and *loop optimization*. Optimizing compilers may make use of few of the aforementioned optimizations, which are explained in the following subsection.

■ Compile-Time Evaluation

Often, the program statements contain name variables and expressions whose values are known at the compile time and used, perhaps repeatedly, in the execution of the program. These program

statements can be treated as constants. For such cases, performing computations at compile time will save space and execution time of the target object program. The optimization transformation in such cases would modify the code such that the expressions are pre-computed at compile time (wherever possible) and used directly in the target code. *Constant folding* and *constant propagation* are two such methods, which are described as follows:

- **Constant folding:** Refers to the replacement of expressions that can be evaluated at compile time by their computed values. Constant folding corresponds to the evaluation of an expression with constant operands during compilation itself. The expression is replaced by a single value.

For example, the statement $A=5+3+A+C$ can be replaced by $A=8+A+C$. Note that the values 5 and 3 are variables at compile time and the sub-expression $5+3$ can be pre-computed and replaced with 8 in the original expression, leading to $A=8+A+C$. This optimization would help us to delete an entry from the quadruple, leading to more compact and faster code.

Note that folding is mainly applied at some stages in initial translation of (during parsing) source program. It is mostly applied to expressions involving arithmetic operations such as $+$, $-$, $*$, $/$, and $\uparrow$.

- **Constant propagations:** Refer to the task of replacing the name variables with constants that have been assigned to them earlier. It is an indirect optimization, which does not lead to any optimization itself but help in other optimization. For example, it may assist in performing folding of an expression given as follows:

```
PI=3.141592
```

```
...
```

```
R=PI/180.0
```

Constant propagation would transform this code into

```
PI=3.141592
```

```
...
```

```
R=3.141592/180.0
```

After applying folding transformation in the preceding code, the code becomes

```
PI=3.141592
```

```
...
```

```
R=0.0174644
```

Note that the constant propagation can only be applied if there are no intervening statements that would modify the value of the variable either directly or in an indirect manner. Direct modifications are possible as a result of assignment statements, while indirect modifications may result from the use of pointers and function calls. Only after global analysis is performed, it can be decided which value can still be folded when a particular block is entered.

■ Common Sub-Expression Elimination

Programmers often write expressions that contain sub-expressions, which compute the same value. Code optimizer must identify the common sub-expression and eliminate it because it would amount to perform the same operation repeatedly, thus wasting much of the CPU time. Once identified, it is evaluated only once and values are substituted at respective sub-expression occurrences. Alternatively, in dynamic environment, the identified common sub-expression is assigned a new temporary variable and all occurrences of the sub-expression are replaced by the new temporary variable. Consider the following example:

```
y = a+b
...
z = a+b - 10
```

The preceding code contains common sub-expression $a+b$ and thus can be optimized by using a new variable `temp` for it and replacing all occurrences of $a+b$ with it as follows:

```
temp= a+b
y = temp
...
z = temp - 10
```

Note that the sub-expression must be evaluated to identical values during any course of execution for being eligible for elimination. It can be identified by a lexical search or using advanced representation such as Directed Acyclic Graph (DAG).

Variable Propagation

Variable propagation is the method of optimization that replaces one variable by another variable holding identical value. This would increase the probability of generating common sub-expressions, which, in turn, can be easily identified and eliminated. Consider the following example:

```
a= b
...
...
y= a*4
z= b*4+10
```

Note that the values of the variables `a` and `b` are same, and since both are not modified, we can replace variable `a` by `b`. This leads to the following modified code:

```
a= b
...
...
y= b*4
z= b*4 +10
```

Finally, we find that the sub-expression $b*4$ happens to be the common sub-expression and will be eliminated, as discussed earlier.

■ Code Movement

Code movement is the method in which code from one part of the program is moved to another part so that the resulting program is more efficient. The modified programs maintain the semantic equivalence even after modification. Generally, code movement is motivated by the following reasons:

- ❑ **To reduce the code space of the program:** Means to reduce the code size by moving the piece of code to earlier parts of the program so that the code generated is substantially small. Consider the following example:

```
if p > q then
  x= a *10
...
```

```

Else
    Y = a*10 +20

```

We note that `a*10` is computed twice. The code can be modified to compute `a*10` only once before the `if` statement. The resulting code is as follows:

```

temp= a*10
if p > q then
    x= temp
...
Else
    Y = temp +20

```

The code for `a*10` is generated only once.

- ❑ **Reduction of execution frequency:** Means to reduce the execution frequency of the code being removed. It is the transformation that identifies the common codes to be evaluated at various places in the program and moves them within the code, such that their frequency of execution is reduced. It can be done in two ways: *code hoisting* and *loop optimizations*. Code hoisting moves the codes to some earlier parts of the program to reduce their frequency of execution. On the other hand, loop optimization moves code statement(s) from the loop body whose values do not depend on the loop iterations. This is done because these are loop-invariant statements, and keeping them in the loop will only invite repetitive executions of the same code repeatedly.

■ Dead Code Elimination

The portion of the program statements that is never visited and executed under any situation is called *dead codes* and can be removed from the program without affecting the program behavior and performance. A variable that is defined but not used in the entire program is considered to be a dead variable.

A simple example for understanding dead code is the use of `if` constructs. When the compiler finds out that the condition inside the `if` statement is never going to be true, the body of the `if` statement will never be executed. For such situations, the compiler can consider it as dead code and completely eliminate it, thus releasing the memory space occupied by the respective code. Consider the following example:

```

flag=false;
...
if(flag)
    a=b+c;
...

```

The data flow analysis reveals that each time the program reaches the `if(flag)` statement, the value of `flag` is always false. This means the statement `a=b+c` would never be executed. Therefore, the test and the assignment statements are considered to be dead code and can be safely removed from the object code.

■ Loop Optimization

Loops are principle sources of optimization because the body of the loop is executed many times, leading to sharing of most of the memory space in the target code. Thus, the improvement in the execution of the statements inside the loop will be the most promising optimization. Some important loop optimization techniques are code motion, induction variable elimination, loop unrolling, loop fission and fusion, loop interchange, and loop reversal, which are explained as follows:

- **Code motion and frequency reduction:** Often, loops contain expressions that retain their values independent of the number of times a loop is executed. Such expressions are called loop invariant expressions. These expressions can be moved and placed before the loop, thereby achieving code motion and frequency reduction as well.

For example, consider the following statements in the while construct:

```
while (i <= limit-2 ) /* assume limit does not change * /
```

The equivalent code after code motion will be

```
t = limit-2;
while (i <= t) /* assume limit or t does not change * /
```

This improves the code significantly as it computes limit-2 only once.

- **Loop unrolling:** It is an optimization technique where the code constituting the loop body is reproduced a number of times. This improves the time efficiency of the execution process. This is because by increasing the number of code through the generation of many copies of the loop body, the loop overhead due to calculations required to modify and test the loop variable will be significantly reduced.

For example, consider the following code statement containing for loop:

```
for( i=1 ;i< 30;i++)
a(j+i)=0.0;
```

The equivalent decomposed form of the loop is given as follows:

	...	
	i=1	←initialization
label0:	if i >30	←test
	then go label1	
	a(j+i)=0.0;	
	i=i+1	←adjustment
	go label0	
label1:	...	
	...	
	...	

This amounts to the generation of the following code sequences:

```
a(j+1)=0.0;
a(j+2)=0.0;
...
...
...
a(j+30)=0.0;
```

Note that the loop structure has completely disappeared. In addition, note that the loop's initial value, limit value, and adjustment value must be known at compile time to perform loop unrolling. Sometimes, partial loop unrolling may also be used as it may save time and increase the amount source statements. Partial unrolling increases the complexity of the space versus time decision.

- ❑ **Loop interchange:** This optimization interchanges the inner loops with the outer loops to improve the nesting structure of the loops, thereby helping in improving the localization of memory references and enabling further loop transformations. Consider the segment of code involving arrays in the following loop:

```
for j= 0 to 50 do
  for i=0 to 50 do
    A[i][j]= ...
```

The code fragment given before makes nonlocal references. The access pattern would be $a[0][0]$, $a[1][0]$, $a[2][0]$, ..., etc. The code can be improved by interchanging the inner and outer loops with better locality of reference. The previous code can be rewritten as follows:

```
for i= 0 to 50 do
  for j=0 to 50 do
    A[i][j]= ...
```

The modified access pattern would be $a[0][0]$, $a[0][1]$, $a[0][2]$..., etc., which may offer better locality of references in many cases.

- ❑ **Loop splitting:** It is the method of simplifying a loop by breaking it into multiple loops. The resulting loops have same bodies but iterate over different, contiguous index range. Loop splitting can successfully eliminate dependencies, thereby facilitating easy target code generation. Consider the following example:

```
for i= i to 100 do
  if i<50 then b1
  if  $50 \leq i \leq 60$  then b2 else b3
end loop
```

After loop splitting, the previous loop splits into three loops as follows:

```
for i= 1 to 49 do
  b1
end loop
for i= 50 to 59 do
  b2
end loop

for i= 60 to 100 do
  b3
```

- ❑ **Loop peeling:** It is a special case of loop optimization that simplifies a loop structure with a first iteration condition by performing the iteration separately before entering the loop. For example, the following loop:

```
for i= i to 50 do
  if i= 1 then b1 else b2
end loop
```

can be simplified to

```
i=1
b1
for i= i to 50 do
    b2
end loop
```

- ❑ **Loop test replacement:** This optimization replaces a loop termination test phrased in terms of one variable by a test phrased in terms of another variable. It is usually useful for strength reduction and often leads to dead code elimination.
- ❑ **Loop fission:** It is all about breaking the loop into multiple loops over the same index range, with each loop taking only a part of the loop's body. This improves the locality of reference. For example, consider the following loop code:

```
for i= i to 50 do
    x[i] = ...
    y[i] = ...
end loop
```

The preceding code fragment can be improved for locality in the reference pattern by rewriting it as follows:

```
for i= i to 50 do
    x[i] = ...
end loop
for i= i to 50 do
    y[i] = ...
end loop
```

Note that both loops access different variables and hence offer better memory reference performances.

■ Strength Reduction

A strength reduction transformation is used to replace an operation with a more efficient operation or series of operations which yield the same result, but using fewer machine clock cycles. For example, multiplication by a power of 2 can be replaced by left shift (i.e., $x * 8$ can be implemented as $x \ll 3$). In addition, operations such as multiplication by small numbers can be replaced by multiple additions.

7.3 Global Data Flow Analysis

Optimizing compilers need techniques for gathering information about possible values of program objects in order to produce efficient code. *Data-flow analysis* is one such technique that gathers information about the possible set of values calculated at various points in a computer program. Such information is especially useful in determining whether the changes made during the optimization are safe. Many optimizations depend on either re-ordering or eliminating statements from the IC representation of the program. Such optimizations must ensure that they leave the behavior of the program unchanged. Hence, it is crucial to determine the range of the program over which the values used remain unchanged and also the range to which they are unused. Data structures known as *flow graphs* are useful in data flow analysis. These are directed graphs, which explain the data dependencies between program elements. Nodes of flow graph represent *basic block*, which contains sequence of code with only one entry point and one exit. Flow graph and basic blocks are discussed in more detail in Chapter 8.

A program's *Control Flow Graph* (CFG) can be used to gather information by an optimizing compiler to identify those parts of the program that assigns value to particular name variable and to which point it might propagate. It can be clearly seen that simply identifying a unique section of code that sets a variable is not enough. An optimizing compiler must also recognize those chains of instructions through which the variable might have been set. This is because the flow of control forms several intertwining paths through the statements of the program during execution.

Data-flow analysis attempts to obtain particular information at each point in a procedure. This information is easily available at the boundaries of basic blocks and can be easily computed. In essence, data flow analysis computes the set of items of data flow information associated with the entities of the flow graph and are defined by compiler writer.

We define some of these set of items in the following discussions and explore concepts that relates to flow graph analysis and its contribution in optimization.

■ Available Expressions

An expression e is said to be *available* at the program point w if and only if the following properties are satisfied:

- Along all the paths reaching w ,
- There is an evaluation point e
- There are no definitions of any operands of e following its last evaluation along the path

An evaluation point refers to a program point containing an evaluation of the expression. Note that an expression e is assumed to be killed by a definition of its operands and not available after such definitions. The advantage of finding available expression is that it can help in identifying and eliminating of common sub-expressions.

Another canonical example of a data-flow analysis is reaching definitions, which is discussed in the following subsection.

■ Reaching Definitions

A definition d of a variable v situated at the program point w_i reaches another program point w_j if and only if there exists some path in which the variable is not defined. Reaching definition can be applied in many optimizing situations, including constant propagation and variable propagation. In order to determine reaching definitions, the following constructions are required:

- Find the list of all points that define each simple variable. A *definition* is a point where a value is being set for that variable.
- Compute the set of generated definitions “generated by B,” say $GEN[B]$. This set contains those definitions that reach their end for block B .
- Compute the set of all definitions “killed by B,” say $KILL[B]$. These are definitions that are outside the block B , which also have definitions inside B .
- Now compute for all blocks B the set $IN[B]$, which is the set of all definitions reaching the point just before B 's first statement.

Having computed the aforementioned information, we are ready to find the definitions reaching a point of any use of A within the block B . For some i^{th} statement being examined, which uses A , if there are some definitions of A within block B before i , then the last definition is the only one that reaches i . Moreover, if there are no definition of A prior to i within the block B , then the definitions reaching i are in $IN[B]$. Computing the set IN requires us to compute for each block the set $OUT[B]$, which itself includes $GEN[B]$, and definitions in $IN[B]$, which persist after block B has terminated.

■ Data-Flow Equations

One of the most widely used methods to perform data-flow analysis over the programs is to use data-flow equations. The data-flow equations are set up for each node of the CFG and solved repeatedly to calculate the output at each node for the inputs until the system stabilizes. The data-flow equations assist in computing the data-flow information and henceforth the optimization of the program.

Data flow equation computes the sum of information at entry and exit of each node of the flow graph. It also determines the information generated or killed within the block. For each block B , we have

$$1. \text{ OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] - \text{KILL}[B])$$

i.e., we say a definition d reaches the end of the block B if and only if it satisfies the following conditions

- a. $d \in \text{IN}[B]$ and is not killed by B or

$$\text{IN}[B] \cup \bigcup_{\forall P \text{ preceding } B} \text{OUT}[P]$$

- b. d is generated in block B but not subsequently redefined there

i.e., a definition d is said to reach the start of a block B if and only if it reaches the end of one of its predecessors.

It is clear from the aforementioned definitions that the entry state of a block in the program is a function of the exit states of its predecessors. Solving these sets of data-flow equations with entry and (or) exit states of the blocks, it is easy to derive properties of the program at the block boundaries.

■ Use-Definition (U-D) Chaining

We define the *use* of a value as any point in the program where the variable (or constant) is used on the Right-hand Side (RHS) of an assignment statement or it is evaluated as an expression. The *definition* of a constant and a variable may be at different points in the program. For a constant, the definition occurs implicitly at the start of the program, while for variables, it occurs at the point where it is the target of an assignment or read statement.

Data interconnections in the program can be expressed in the form that may assist in establishing direct links between the instructions. These direct links are known as *u-d chains* and can be seen between the instructions that produce values and to those that use them. The flow analysis of the program reveals these links and has been found to be very useful in optimizing programs.

Formally, if the definition of variable x precedes its use in the block, then this is the only one reaching it. Otherwise, all definitions of x in $\text{IN}[B]$ form their u-d chain. U-d chain analysis provides much useful information and is found to be useful in the following situations:

- If the only definition of x that reaches this statement involves a constant, then we can substitute that constant for x
- It can be useful in displaying warning messages if no definition of x reaches the point
- Definitions that reach nowhere can be simply eliminated as they form the part of dead code elimination

In the next section, let's discuss about optimizations based on data flow analysis.

7.4 Optimizations Based on Data-Flow Analysis

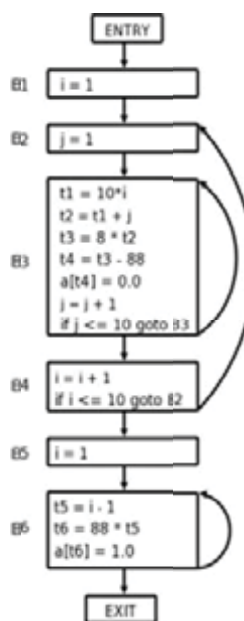
Data-flow analysis across blocks in the flow graph helps in safely performing several of the optimizations in the scope of a single block and across multiple blocks as well. Some of the important optimizations where data flow analysis (including global data flow analysis) is useful are discussed in the following subsections.

■ Loop Detection

Global data flow analysis is very useful in detecting loops. Loops are a principal source of optimization in any optimizing compiler. A *loop* can be defined as a set of CFG nodes L such that the following conditions hold:

- There is a node h in L known as header node, which dominates all nodes in L
- There exists a path from h to any node in L
- h is the only node in L with predecessors not in L
- From any node in L , there exists a path to h .

Importantly, a loop has a single entry with possibility of multiple exit points and contains paths with non-zero length that contain nodes only from L . Figure 1 shows flow graph with loops:



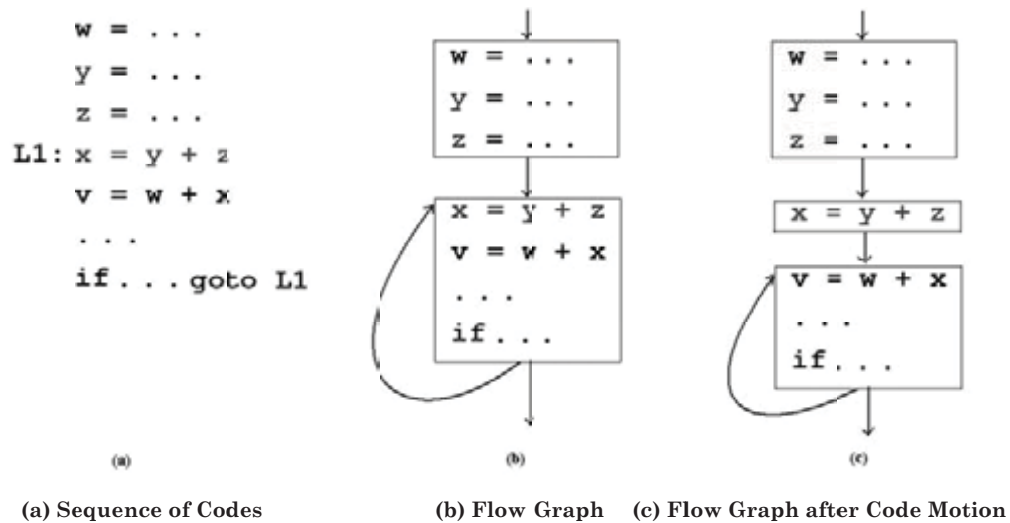
▲ Figure 1: Flow Graph with Loops

A loop L with loop entry E is a collection of basic blocks such that no block other than E has a predecessor outside L and all the blocks in L have a path to E completely inside L .

Consider a flow graph with basic blocks, as shown in Figure 1. The flow graph has three loops: $\{B_3\}$, $\{B_6\}$, and $\{B_2, B_3, B_4\}$. Loops $\{B_3\}$ and $\{B_6\}$ retain the program flow with the basic blocks B_3 and B_6 , respectively, while the loop $\{B_2, B_3, B_4\}$ allows the program flow across basic blocks.

■ Invariant Code Motion

Code motion can be extended to move code in front of preceding blocks if it does not get killed in the intervening blocks. Code motion is especially useful in loop optimization where we take a computation that gives the same results independent of the number of times used in the loop and places it before the loop. Figure 2(a) shows sequence of codes, Figure 2(b) shows a flow graph, and Figure 2(c) shows flow graph after code motion:



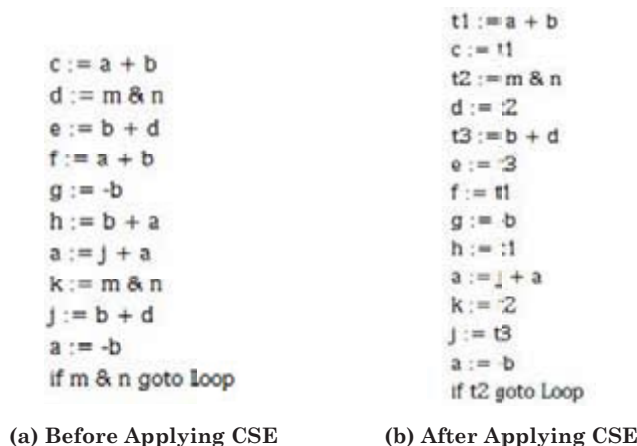
▲ Figure 2: Performing Code Motion

In Figure 2(a), we see that everything that the variable x needs is computed outside the loop, *i.e.*, all definitions of y and z are outside of the loop. Hence, the expression $x = y + z$ can be moved outside the loop.

The input to the code motion algorithm is a loop L that consists of basic blocks. The basic block contains a sequence of 3-address instructions. The algorithm assumes that use-definitions (u-d chains) have already been computed. The output of the algorithm is the set of instructions that compute to the same value for iterations in the loop.

■ Common Sub-Expression Elimination for Data-Flow Analysis

Flow graph analysis helps in finding common sub-expressions by spanning basic blocks and eliminating unnecessary re-evaluations. The idea of common sub-expression is that if an expression is known to be available at a point where it is evaluated, then it need not be recomputed. Figure 3(a) shows example before applying CSE and Figure 3(b) shows example after applying CSE:



▲ Figure 3: Example of CSE

We can clearly see that before applying CSE, 11 instructions, 12 variables, and 9 binary operators were used to perform the desired operation represented by the sequence of code. After applying the CSE, 14 instructions, 15 variables, and 4 binary operators are exercised. Notably, optimization is achieved with reduced number of operators, although numbers of instructions have increased. CSE may also be extended to multiple blocks. If two or more blocks perform same computations and have same values across the block, only one may be retained and become the part of the flow graph. Other blocks do not contribute to the control flow and can be eliminated.

■ Dead Code Elimination

One important technique of code optimization is to remove the codes from the program which has no effect on the program results. The technique is known as dead code elimination. It may be easily performed using global flow analysis if no ultimate use is made of a live variable at the end of the block in which its value is generated. Dead code elimination is the removal of statements from the code sequence (or flow graphs) that defines only one variable and the variable being defined is not in the live set at the exit point. The following algorithm performs dead code elimination from the flow graph:

do {

1. Construct the flow graph from the list of instructions
2. Determine liveness of variables
3. For each node in the flow graph, perform the following:
If the definition set contains only one temporary and if the temporary being defined is not in the live at exit, then remove the node from the flow graph.
4. Generate a list of instructions from the newly modified flow graph

} while (changes);

Dead code elimination has twofold benefits. First, removal of code sequences reduce the program size, thereby achieving the objective of memory space optimization, and second, it offers significant improvement in running time since such removal avoids executing irrelevant operations altogether.

■ Loop Unrolling

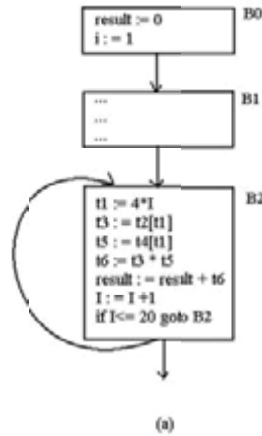
Global data flow analysis is useful in loop unrolling, which is an important feature of parallelizing and optimizing compilers. Loop unrolling can be useful for reducing loop overheads and exposing instruction level parallelism for machines with multiple functional units. With this technique, the loop overheads are reduced by reducing the number of iterations and replicating the body of the loop. Basically, it requires that no value used in iteration k is generated in iteration $k - 1$.

■ Induction Variable Elimination

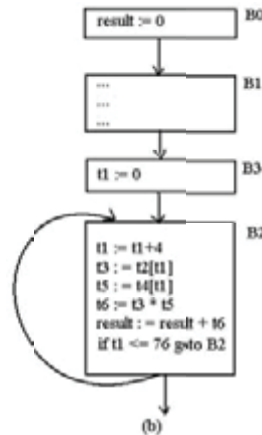
Induction **variable elimination** is an important class of optimization that is applied to flow graphs. Identifying and eliminating induction variables results in reducing the total number of instructions and speeding up the loop execution.

We say a variable i in a loop is an induction variable of the loop if each time i changes value in the loop, it is incremented/decremented by loop-invariant value. Induction variable can be of two types---basic induction variables and derived induction variables. Basic induction variables are variables whose only definitions within loop are of the form $i = i + c$ or $i = i - c$, where c is loop invariant. Derived induction variables are variables that are defined only once within the loop and their values are linear function of some basic induction variable.

Figure 4 gives an example of induction variable elimination. If I is the variable whose value is not required after the loop terminates, then I can be eliminated from blocks B_0 , B_1 , and B_2 entirely by replacing it with t_1 . The value of t_1 forming the arithmetic progression $t_1 = t_1 * 4$ can be replaced by $t_1 := t_1 + 4$ and the test condition $I \leq 20$ goto B_2 can be replaced by statement if $t_1 \leq 76$ goto B_2 . Since t_1 has no value when we enter into B_2 , we may place an assignment statement $t_1 := 0$ in a new block before B_2 . Alternatively, it may also be assigned within B_2 . Figure 4(a) shows a flow graph before induction variable elimination and Figure 4(b) shows a flow graph after induction variable elimination:



(a) Flow Graph Before Induction Variable Elimination



(b) Flow Graph After Induction Variable Elimination

▲ Figure 4: Example Showing Induction Variable Elimination

■ Busy Expressions

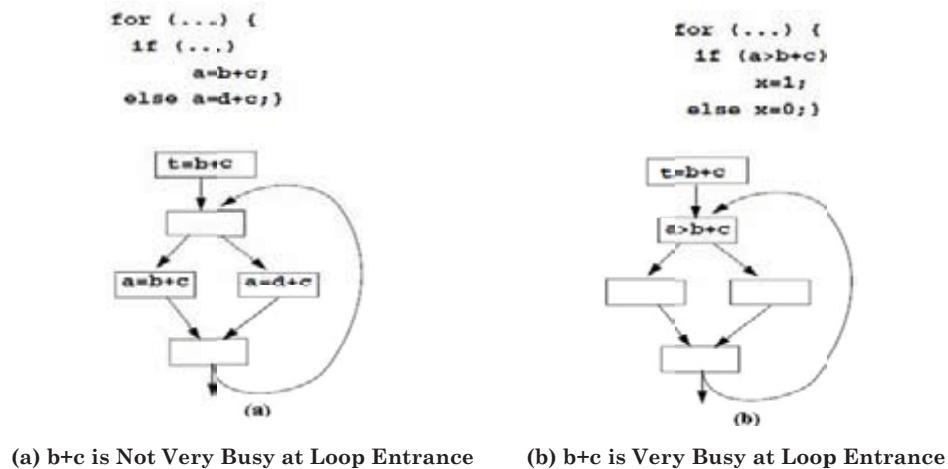
Finding busy expressions is another outcome of data flow analysis. An expression e is said to be *busy* at some program point if and only if it satisfies the following conditions:

- e is evaluated along some path $w_1, w_2, \dots, w_j$ starting at program point w_i and
- There are no definitions for any of the operands of e before its evaluation along that path

Note that if an expression is found busy at some point of the control flow graph, it must be used in some path following that point.

An expression that is busy along all paths starting at w_i is said to be a *very busy expression*. It is a variant of available expression analysis. A very busy expression at a point guarantees that the expression will be computed at some time in the future. This means that the expression must be reached before its value changes. In other words, we can always move the evaluation to that point, which will be a safe transformation. Very busy expression analysis is a form of backward flow analysis. The information about future evaluations is propagated backward to “earlier” points.

Very busy expressions are potential candidates for performing invariant loop code motion. An expression that is invariant in a loop and also very busy can be evaluated outside the loop. The concept of very busy expressions can be well understood with the help of Figure 5(a), which shows that $b+c$ is not very busy at loop entrance, and Figure 5(b), which shows that $b+c$ is very busy at loop entrance:



▲ Figure 5: Analysis of Very Busy Expression

Summary

In this chapter, we have learned that code optimization is an important phase of compilation that generates equivalent but more efficient codes. The objective of an optimizing compiler is to generate target program with higher execution efficiency. There are many types of optimization that a compiler can perform on the source or its IR representations. Code optimization is primarily based on the machine and language dependency of the code.

Machine-independent optimization generally works on the IR, creates more optimal representation, and does not require program to know the internal details of machine architecture. On the other hand, machine-independent optimization takes place during or post final code generation phase and requires fair detail of machine architecture.

When optimizations are more specific to the language and its implementation, optimizations are called language-dependent optimizations. Optimizations classified based on the scope of optimization include peephole optimization, loop optimization, local and global optimization, and whole-program optimization.

Important optimizing transformation include compile-time evaluation, CSE, variable propagation, code movement, strength reduction, dead code elimination, and loop optimizations. Constant folding

and constant propagation are examples of compile time evaluation. An efficient code optimizer must identify the common sub-expression and eliminate it. Variable propagation is the method of optimization that replaces one variable by another variable holding identical value. Code movement is the method in which code from one part of the program is moved to another part so that the resulting program is more efficient.

The portion of the program statements that is never visited and executed under any situation is called dead code and can be removed from the program without affecting the program behavior and performance. A variable that is defined but not used in the entire program is considered to be a dead variable. Loops are principle sources of optimization because the body of the loop is executed many times, leading to sharing of most of the memory space in the target code. Some important loop optimization techniques are code motion, induction variable elimination, loop unrolling, loop fission and fusion, loop interchange, and loop reversal. Loop unrolling is an optimization technique where the code constituting the loop body is reproduced a number of times. Optimizing compilers very often implement strength reduction transformation, which is used to replace an operation with a more efficient operation or series of operations.

Data-flow analysis is a technique that gathers information about the possible set of values calculated at various points in a computer program. Such information is especially useful in determining whether the changes made during the optimization are safe. Data structure known as flow graph is useful in data flow analysis. These are directed graphs that explain the data dependencies between program elements. Nodes of flow graph represent basic block that contains sequence of code with only one entry point and only one exit. A program's CFG can be used to gather information by an optimizing compiler to identify those parts of a program that assigns value to a particular name variable and to which point it might propagate. Data-flow analysis attempts to obtain particular information at each point in a procedure. This information is easily available at the boundaries of basic blocks and can be easily computed.

Global data flow analysis requires computing available expressions, reaching definition, and using definitions. Reaching definition can be applied in many optimizing situations including constant propagation and variable propagation. One of the most widely used methods to perform data-flow analysis over the programs is to use data-flow equations. The data flow-equations are set up for each node of the CFG and solved repeatedly to calculate the output at each node for the inputs until the system stabilizes. Data-flow equations compute the sum of information at entry and exits of each node of the flow graph. They also determine the information generated or killed within the block.

The use of a value is any point in the program where the variable (or constant) is used on the RHS of an assignment statement or it evaluates an expression. The *definition* of a constant and a variable may be at different points in the program. Data interconnections in the program can be expressed in the form which may assist in establishing direct links between the instructions. These direct links are known as *u-d chain*. Data flow analysis across blocks in the flow graph helps in safely performing several of the optimizations in the scope of a single block and across multiple blocks as well. Few such techniques useful in implementing optimizing compiler include-loop detection, invariant code motions, CSE, dead-code elimination, loop unrolling, induction variable elimination, and computing busy expressions. Very busy expression analysis is a form of backward flow analysis. The information about future evaluations is propagated backward to earlier points.

Review Questions

- Q1. What is code optimization? What are the essential features of code optimizations?
- Q2. Differentiate between:
 - a. Machine-dependent and machine-independent optimization
 - b. Local and global optimization

- Q3. Briefly describe the various classifications of compiler optimizations.
- Q4. What is peephole optimization?
- Q5. What is compile-time evaluation? How does it optimize the code performance? Give example of compile-time evaluations.
- Q6. Write short note on constant folding.
- Q7. What is common sub-expression elimination? Give example.
- Q8. What are dead codes? Give example of dead code elimination.
- Q9. Define loop. List and describe in brief the various loop optimization techniques.
- Q10. What are the advantages of global data flow analysis? Give example.
- Q11. What are available expressions? Give example.
- Q12. Write short note on:
- Reaching definitions
 - U-d chain
 - Data-flow equations
- Q13. What are the advantages of computing u-d chains?
- Q14. List and describe the various types of optimizations based on global data flow analysis.
- Q15. What is induction variable elimination? Explain.
- Q16. Write short note on code motion.
- Q17. What is loop fission?
- Q18. Consider the following program that performs matrix multiplication.

```

for (i=0; i<10; i++)
    for (j=0; j<10; j++)
        c[i][j] = 0;
for (i=0; i<10; i++)
    for (j=0; j<10; j++)
        for (k=0; k<10; k++)
            c[i][j] = c[i][j] + a[i][k] * b[k][j];

```

What are the basic blocks in the above code fragment? Construct the flow graph and identify the loops in the flow graph.

- Q19. What is strength reduction? Explain with example.
- Q20. What is loop splitting and loop peeling? Explain.
- Q21. What is loop unrolling? Suggest the suitable unroll modification for the following loops:

```

for (i = 1; i <= 60; i++) a[i] = a[i] * b + c;

```

- Q22. What are dead codes and unreachable codes? Identify them in the following code fragment and optimize it if possible:

```

int global;
void f ()
{
    int i;
    i = 1;
    global = 1;
    global = 2;
}

```

```
    return;  
    global = 3;  
}
```

Draw the flow graphs before and after the optimization.

Q23. What are very busy expressions? Explain.

■ Multiple Choice Questions

Q1. Constant folding is an example of _____.

- a. Compile-time optimization
- b. Run-time optimization
- c. Neither a nor b
- d. Both a and b

Ans. a

Q2. _____ moves the codes to some earlier parts of the program to reduce their frequency of execution.

- a. Constant propagation
- b. Code hoisting
- c. Loop unrolling
- d. None of these

Ans. b

Q3. For a _____ the definition occurs implicitly at the start of the program, for _____ it occurs at the point where it is the target of an assignment or read statement.

- a. Variables, constant
- b. Constant, variables
- c. Pointer, constant
- d. None of these

Ans. b

Q4. State true or false

- i. A loop has a single entry with single exit point.
- ii. Very busy expression analysis is a form of backward flow analysis.
- a. False, false
- b. True, true
- c. True, false
- d. False, true

Ans. d

Q5. State true or false

- i. An expression e is assumed to be killed by a definition of its operands and not available after such definitions.
- ii. A basic block contains sequence of code with only one entry point and only one exit.
- a. False, false
- b. True, true
- c. True, false
- d. False, true

Ans. b

08 Code Generation

OBJECTIVES

- ✧ Learning about the issues in the design of a code generator
- ✧ Understanding the target machine and implementing simple code generator
- ✧ Introducing the concept of basic blocks and flow graphs
- ✧ Gaining insight into register allocation and assignment
- ✧ Understanding DAG and its representation and role in code generation
- ✧ Learning about peephole optimization, dynamic programming code generation algorithm, and code-generator generators

The front end of a compiler prepares the input program and generates Intermediate Representation (IR) for final code generation. The back end of the compiler encompasses code generation phase which converts internal representation of the source code fed to it into a machine executable form. This is usually the final phase performed by the code generator in a compiler model. It takes as input an IR of the source program generated by the front end of the compiler and produces as output an equivalent program in target machine language (Figure 1). This phase is very machine-specific and generates specific code for specific architectures. This is because each architecture has its own set of instructions and must be followed with due care.

Efforts to implement the final phase of code generation may range from trivial to complex, which depends on what form of IR is available to it. Fairly low-level IR would make the task much easier than what would be with high-level IR representations. Moreover, final code generation is also highly influenced by what information is available about the target machine and runtime environment. The quality of code generated can be highly improved by using an optimizing compiler. Although the optimization of the final code is not mandatory as far as the correct working of the compiler is concerned, a good compiler must produce efficient code than just producing a working code. Optimizing compilers aggressively perform machine-independent optimization in IR before generating the code. This would make a great difference in the final code compared to a non-optimizing compiler. We shall also discuss on code optimization in this book.

This chapter explores the major tasks in code generation and the design issues of a code generator. Next, it discusses about target machine and runtime storage management. It explains about basic blocks and flow graphs. Further, you learn about next-use information and simple code generator. The chapter then elaborates on the DAG representation of basic blocks and peephole optimization. You learn about generating code from DAGs. Toward the end, you learn about dynamic programming code generation algorithm and code-generator generators.

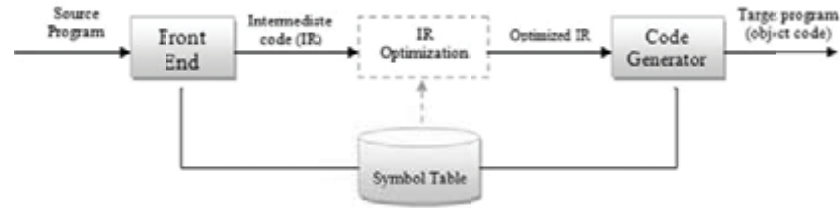
8.1 Major Tasks in Code Generation

The code generation phase has the much crucial responsibility of generating the code for target machine. A good compiler encompasses the following characteristics:

- ❑ **Generates correct output code:** The target program must preserve the meaning of the original specifications, i.e., given an input, it gives the same output both from the target and source programs. Although it is quite difficult to ensure this condition, it can be achieved by maintaining test suite and check.

- ❑ **Ensures quality code generation:** The equivalent target code should effectively use the resources of the target machine such as memory, computing power, etc., avoiding unnecessary overloads and should execute fast on the target machine.
- ❑ **Ensures efficient code generators:** The code generator should take minimum time to generate code. A code generator that takes long time to convert a small piece of code is noteworthy.

Figure 1 shows the input and output of a code generator:



▲ Figure 1: Input and Output of a Code Generator

The rules for executable programs on a given architecture direct the code generation. Some of the important factors that should be considered while writing a code generator include choice of instruction format, register usage, calling conventions, instruction scheduling, memory organization, executable format, etc. The input to the code generator may consist of a parse tree, an Abstract Syntax Tree (AST), or some linear sequence of instructions, typically in an intermediate language such as three-address code. The code generator is designed to process these intermediate forms to generate the target code. In many cases, the nonlinear inputs are converted into a linear sequence of instructions, typically in an intermediate language such as three-address code. This simplifies the task for the code generator. The code generator for some architecture would typically perform the following tasks:

- ❑ **Instruction selection:** Means the compiler must decide upon which instructions to use from the available set.
- ❑ **Instruction scheduling:** Means determining the order in which the instructions will be generated, keeping in mind the optimization issues such as speed reuse of available registers in pipelined machines.
- ❑ **Register allocation:** Refers to the allocation of variables to processor registers for execution. It is greatly influenced by instruction selection and scheduling policies.

Notably, the final code generator handles locations of programmer-defined variables as well as system-generated temporaries and generates the code that is abided by the requirements of the runtime environment. For all modern programming languages that allow usage of functions, code generator must be set up and return from function calls, manage the stack with auxiliary data, and so on.

The code generators are considered to be efficient if they generate target program that preserves the semantic meaning of the source program. In addition to this, the target programs should efficiently utilize the target machine's resources as much as possible. Optimal code generation happens to be a challenge because designing of a good code generator reverts the problem of designing good heuristics.

8.2 Design Issues of a Code Generator

The design of code generators has several restrictions imposed on them. An important criterion for a code generator is to ensure that it produces correct code. The target code must conform to the standards of quality code and should effectively use the resources of the target machine. In practice, often, we produce code that runs faster but not optimal. The kind of code one requires to generate will decide the heuristics for the code generation algorithm. More practically, the issues of the target language and operating system decide the kind and quality of generated code.

Some important issues involving memory management, selection of instructions, register allocation policy, and order of evaluation are at the center of concern of all code-generation algorithms. Correctness and efficiency of code are the central objectives of a good code generator. The design of a code generator should be such that it can be easily implemented, tested, and maintained. In this section, we explore some of the generic issue that must be attended in designing a code generator.

■ Input for Code Generators

The code generator in the program translation is greatly influenced by the input to it and its position in the translation model. All modern compilers generate IRs (language) of the source program. These IRs together with the information from the symbol table are fed to the code generator and object program in target language is generated. The symbol table is used to determine the runtime addresses of the data objects denoted by the names in the IR.

It is assumed that the input is free of errors and it has been validated by the front end, i.e., type checking, syntax, semantics, etc., have been carried out. The input to the code generator may come in many forms, including linear representations such as postfix notation and three-address representations, virtual machine representations such as stack machine code, and graphical representations that include syntax trees and DAGs. Linear representations of three-address code such as quadruples and triples are also often used. When the input to the code generator is typically a parse tree or an AST, it is usually converted into a linear sequence of instructions in intermediate form such as three-address code or variants of it.

Compiler in its initial stages processes the source programs, produces information about program entities, and translates the source program into a reasonably detailed IR. The quantities in the representation have forms such as bits, integers, real, pointers, etc., which can be directly handled by the target machine. It is expected prior to the code generation that the necessary type checking and type-conversion operations have been applied wherever necessary and also any semantic errors would have been detected. Code generation assumes that its input is free of errors or else embeds semantic checking in them if necessary.

■ Output from the Code Generator

The output of the code generator is an object (target) program. The output language of this program can also take various forms, including *absolute machine language*, *re-locatable machine language*, or an *assembly language*. The characteristics of each one these differ from the other and possibly one of the representations may suit over other under specific environment. Note that object program must present the code in such a form, which is either close to the machine representation or easy to set correspondence with it.

Target programs using *absolute machine-language* program have the advantage over other as they are ready programs that can be immediately executed and are mostly placed in a fixed location in the memory. Programs that are small in size and have fixed and predictable memory requirements would suit this output form. The absolute codes are executed quickly than any other form.

Many programming environments allow writing programs in subroutines or modules, wherein each does a specified job. Each of these subroutines can be compiled separately into a set of *re-locatable object modules* (machine language programs), which can be linked together and loaded for execution by a linking loader. The increased expense of linking and loading is compensated by increased flexibility in being able to use subroutines separately and to call other previously compiled programs from another object module. Note that in case the target machine cannot handle relocation automatically, then the compiler must explicitly supplement the relocation information to the loader in order to link the separately compiled program modules.

Finally, the easiest of all forms of target program is *assembly-language program*, which is preferred by most compiler implementations. The task of code generation reduces to generating simpler symbolic instructions and using macro facilities of the assembler to finally generate code for them. The assembler that may be inbuilt in most case would finally generate machine code. Note that producing assembly code does not duplicate the entire task of the assembler, as much of the information would have already been produced by the compiler. Remember that as long as addresses can be calculated from the offsets and other information available with the symbol table, the preferred output of code generator would be to produce re-locatable or absolute programs. In all other case, symbolic assembly output as target language would be the choice.

■ Memory and Reference Management

Memory management for program objects such as names are handled at runtime by the front end and code generator together. These names can be a program variable, a procedure, or even another program itself. Names are bound to their address and space is allocated at runtime by determining the type information available in the symbol table. If machine code is being generated, using three-address statements in many cases, then names (or labels) must be translated to addresses of the instructions. Information about the offset and base address must be computed prior to final code generation. For all statically defined name references, this information would already have been produced by the passes of the compiler in early stages. For other dynamic definition, the code generator coordinates with runtime storage routines and obtains memory locations for them.

Labels, if any, in the code refer to some quadruple numbers in a quadruple array. This helps to deduce the location of the first machine instruction generated for that quadruple. We scan each quadruple to work out the location, simply by maintaining a count of the number of words used for the instructions generated so far. If a reference to a quadruple is encountered in the code and the index of referenced quadruple is less than the current quadruple number, then we may simply generate a jump to the machine location of the first instruction in the code for the referenced quadruple. For other cases, use assembler (that introduces extra pass over the code) that resolves the references through backpatching.

■ Choice of Instruction to Generate

Choice of instructions from the available instruction set for the target machine is a critical factor for efficient implementing of intermediate code. *Instruction selection* is a stage of compilation wherein compiler transforms the IR into relatively low-level IR that is often very close to its final target language. These low-level IRs are usually assembly equivalents of the binary code needed for execution of the high-level source inputs. The quality of the output code is greatly influenced by the instruction set architectures and level at which the codes are produced. If the IR reflects high-level details, then code generator can translate them statement-by-statement basis into corresponding target code. On the other hand, if IR exhibits some of the low-level information about the underlying machine, a more efficient code sequences can be generated by the code generator.

If the efficiency of the target program is not the limiting factor, then the instruction selection is straightforward. A simple way is to use *code production table* to generate the code, usually when the

input IR is high level. These code production tables contain definitions (templates) for all possible operations performed on the target machine. The code generation phase scans each high-level IR statement and translates them to a sequence of machine instructions using standard code templates. This is analogous to treating operators as macro calls and operands as arguments and code production table as macro definition. The use of code production table scales down the code generation by identifying the operation from the three-address code and substituting the template (set of code statements) for operations defined in the code production table. The final code is assembly code programs with instructions suited for the target architecture. A sample code production definition is shown in Figure 2(a) and the input and output from code generator is shown in Figure 2(b):

Operator	Op code	Operands (source, destination)
-	LOAD	&R1, R1
	SUB	&R2, R1
	STORE	R1, &MEM
+	LOAD	&R1, R1
	ADD	&R2, R1
	STORE	R1, &MEM
*	LOAD	&R1, R1
	MUL	&R2, R1
	STORE	R1, &MEM
/	LOAD	&R1, R1
	DIV	&R2, R1
	STORE	R1, &MEM
=	LOAD	&R2, R1
	STORE	R1, OP1

(a) Code Production Definition for -, +, *, /, and = operators

Input (Three address code)	Output (sequence of Assembly codes)	Comment
$T_1 = P + Q$	LOAD P, R1	
	ADD Q, R1	
	STORE R1, T ₁	
$T_2 = T_1 - R$	LOAD T ₁ , R1	//Can be eliminated
	SUB R, R1	
	STORE R1, T ₂	
$Z = T_2$	LOAD T ₂ , R1	//Can be eliminated
	STORE R1, Z	

(b) Input and Output from Code Generator for $Z = P + Q - R$

▲ Figure 2: Code Production Definition and Input and Output from Code Generator

The arithmetic expression $Z = P + Q - R$ is unraveled into sequence of three address code and input to the code generator, with T_1 and T_2 being compiler-generated short-lived temporaries to hold the intermediate results. The output of the code generator is a sequence of expansions in assembly using the code productions, which are handled by inbuilt assembler. Although this model of statement-by-statement code generation is quite simple, it will generate many a times poor code requiring further optimization. This is clearly evident in Figure 2(b), which exhibits redundant loads and stores of temporaries T_1 and T_2 .

It is also visible from the study of instruction set that we can obtain same results for an operation using different instructions. A target machine with a rich instruction set may provide several ways of implementing a given operation. For example, to implement the three address statement $A=A+1$, we can generate (a generic symbolic) assembly statements `INC A` or alternatively generate sequence of assembly statements, as shown in Figure 3:

Input (Three address code)	Output (sequence of Assembly codes)	Efficient code
$X=X+1$	LOAD X, R_1	<code>INC X</code>
	ADD $\#1, R1$	
	STORE $R1, X$	

▲ Figure 3: Instruction Set Alternatives for $X=X+1$

Deciding on which of the instruction to choose would be a difficult decision. The nature of the instruction set of the target machine must be clearly known before the selection of the instructions for the final code. The uniformity and completeness of the instruction set are important factors in designing a good code generator. Therefore, the instruction selection depends upon the following factors:

- ❑ **Instructions used:** To decide which instructions should be used in case there are multiple instructions that do the same job.
- ❑ **Uniformity:** The support for different object/data types, what op-codes are applicable on what data types, etc.
- ❑ **Completeness:** Not all source programs can be converted/translated into machine code for all architectures/machines. E.g., 80x86 doesn't support multiplication.
- ❑ **Instruction speed:** Choose among available instructions that have better performance and consume less Central Processing Unit (CPU) time for execution.
- ❑ **Register allocation:** Instructions involving registers perform usually faster than those involving memory operations. Short lived values during computation can use registers for storage than memory for faster performance.
- ❑ **Evaluation order:** The order in which the instructions will be executed. This increases the performance of the code.

For example, in Figure 3, "`INC X`" consumes less time as compared to the standard set of instructions as that would take almost 3 cycles for each source instruction, but "`INC X`" takes only one cycle. Therefore, "`INC X`" instruction would be preferred in place of the other set of instructions.

Notably, the choice must ensure uniformity in operation across and completeness of the instruction. If not, then exceptions to the general rule require special handling. The design of good code sequence is judged by its speed, but accurate timing information is often difficult to obtain. The size is usually attributed to the cost of operation which the amount of memory an instruction required for its execution. The order in which the instructions will be executed greatly influences the performance of the code. Choice of addressing modes in generating target code also influences the efficiency in various ways. Generating the best sequence of codes for a given three-address construct requires knowledge about the context in which that construct appears and selecting appropriate addressing mode for the same.

■ Register Allocation

It is seen that instructions involving register operands are more efficient than those involving operands in memory. Instructions involving registers are small in size and faster in operation. Therefore, good code must use register instruction as much as possible. Assuming that register addressing modes are available, it is still a complex decision to work with registers in the instruction. During register allocation, decision must be taken to select the set of variables that will reside in the registers at a point in the program. General convention is to store long life time values in registers that are often used. Problem of redundant loads and stores must be accounted and code generator should avoid them as far as possible. The choice of specific registers among all available ones is a fairly complex task, as discussed later in the chapter.

The problem of register allocation and assignment is further complicated due to the fact that target machine may require certain register-usage conventions be observed. Using certain specific registers often generates optimal code both in size and speed of operation. Some machines may require the use of register-pairs for some operations for their operands and results. For example, IBM 360/370 machines for their integer multiplication and division had mandatory requirement of using register pairs. The multiplication instruction has the multiplicand residing in the even register of an even/odd register pair and the product occupying the entire even/odd register pair. Such instruction brings binding to the target code to follow the internal implementation of high-level operations.

■ Choice of Evaluation Order

While evaluating a given expression, the order in which computations are performed greatly influence the efficiency of the target code. Note that some computation orders need fewer registers to hold intermediate results than others. This is because certain computing order would minimize the redundant loads and stores of intermediate results into the memory and registers, thereby reducing the code size and use of less registers. Determining the best order is a difficult task, but the order in which it has been produced by the intermediate code generator may aid in such cases to a great extent.

In the next section, let's learn about target machine.

8.3 The Target Machine

A good code generator requires knowledge of the target machine and its instruction set. Generating a good code for a complete language on that machine requires smart selection of addressing modes with reference to the speed of operation and size in memory. As already discussed earlier, the simple instructions with register operand are found more suitable; however, under dynamic environment, the data may not necessarily be in registers. One has to look into all available instruction formats and needs to choose the appropriate ones.

■ Addressing Modes

Addressing modes refer to the way we access datum. The datum may be in internal registers or may reside in memory. The source and destination of an instruction are specified by combining registers and memory locations with address modes. Most modern computing architectures have the following addressing modes, which describe the address of the operands to operate on:

- ❑ **Immediate addressing mode:** The datum to be operated on is part of the instruction
- ❑ **Direct (absolute) addressing mode:** The instruction contains the effective address of the data as operand to be operated on
- ❑ **Register mode:** The data is in the register and register is specified as operand in the instruction

- ❑ **Register indirect:** The register contains the effective address of the data and is specified as operand in the instruction
- ❑ **Register relative:** The effective address is the sum of displacement and content of base or index register specified in the instruction
- ❑ **Base indexed:** The effective address is the sum of content of base and index register specified in the instruction
- ❑ **Relative base indexed:** The effective address is the sum of displacement and base indexed address

Often, the datum is long enough so that the source and destination fields are not sufficient to hold memory addresses for such data items. Therefore, certain bit patterns in these fields could be used to specify that words following an instruction contain operands and/or addresses in such cases.

Generating code depends more on the architectural details of the machines. While generating code, it may require understanding whether code to be generated is for Reduced Instruction Set Machines (RISCs) or Complex Instruction Set Machines (CISCs). RISC uses many registers, three-address instructions, simple addressing modes, and simple instruction set architecture. CISC, on the other hand, uses fewer registers, two-address instructions, variety of addressing modes, several register classes, and variable-length instructions. These instructions often result in side effects if not properly understood. Note that stack-based machines push operands onto stack and perform operations on the operands at the top of stack.

Assuming that the target machine has n general-purpose registers and available instructions such as load, store, compute, jump, conditional jump, etc., some examples of code generation are given as follows:

```

for a = b + c we can generate
    LOAD  b, R0          // R0 = b
    ADD   c, R0          // R0 = R0 + c
    STORE R0, a          // a = R0
for *p = x we can generate
    LOAD  p, R1          // R1 = p
    LOAD  x, R2          // R2 = x
    STORE R2, 0(R1)      // contents(0 + contents(R1)) = R2

```

The preceding examples are two-address instructions of the following form:

op source, destination

where **op** is an op-code and *source* and *destination* are data fields, which can be specified in various ways using addressing modes supported by the target architectures.

■ Program and Instruction Cost

Implementing any operation involves cost. Cost of an operation is sometimes attributed to space, time, and number of internal operations carried out to achieve the instruction. The runtime cost of a program depends on (among other factors) the cost of the generated instructions and the number of times the instructions are executed. Assuming that all operations take unit time to execute, the cost corresponds to the length of the instruction. Knowing address modes that involve registers have associated less cost (say zero) than those with a memory location or literal in them (say have cost one), the optimized output code can be generated. All memory references are always costly than the register references for the data.

It is clear that minimizing the instruction length will also minimize the time taken to perform the instruction as well; therefore, most optimizing compilers tend to generate register instructions than memory references as their operands wherever possible. Instruction length is affected by the memory size, memory organization, bus structure, CPU complexity, and speed.

Let's learn about runtime storage management next.

8.4 Runtime Storage Management

While generating the code, the code generator must convert the name references to addresses in the target code. These name references can either correspond to variable names or procedure calls used in the program. Each executing program resides in its own logical address space, which is divided into statically determined code areas, code static areas, and dynamically managed areas including heaps and stacks. Code area holds the executable target and code static stores global constants and other compiler data. The size of the code and static area can be determined at compile time. Heaps hold data objects that are allocated and freed during program execution. Stacks manage activation records as they are created and destroyed during procedure calls and returns. The size of the heap as well as stack cannot be usually determined at compile time.

■ Stack Allocation

Semantic stack allocation uses relative addressing for storage in activation records. Note that the address of an activation record for a procedure is not known during compile time. Being relatively addressed, the exact position these records are stored in a register is determined at runtime. Individual data words are accessed by computing offset from the value in register, such as Stack Pointer (SP) and Base Pointer (BP). Offset enabled addressing modes, such as indexed address mode are available in modern architectures for this purpose.

SP is a pointer register that stores the start of the activation record on top of the stack. For every procedure call, the SP is incremented and control is transferred to the called procedure. During a return from the procedure, we decrement SP, indicating the deallocation of the activation record for the called procedure.

The first procedure (or the run-time library code called before any user-written procedure) must initialize SP with

```
LOAD SP, #stackStart
```

where `stackStart` is a known-at-compile-time (even -before-) constant.

The caller increments SP (which now points to the beginning of its AR) to point to the beginning of the callee's AR. This requires an increment by the size of the caller's AR, which, of course, the caller knows.

■ Caller Calling Callee

The code generated for a call without parameters can be:

```
ADD    SP, SP, #caller.ARSize
STORE  *SP, #here+16           // save return address
BR     callee.codeArea
```

■ Callee Returning

The return from a procedure requires writing code for both the caller and callee. The callee transfers control back to the caller with the code `BR *0(SP)` and upon return, the caller restores the SP with the code similar to `SUB SP, SP, caller.ARSize`

A following machine code gives example of fragment of code-generated managing stacks:

```

LOAD    SP, #600
ACTION1
ADD     SP, SP, #msize
STORE  *SP, #152
BR      300
SUB     SP, SP, #msize
ACTION12
HALT

```

8.5 Basic Blocks and Flow Graphs

All optimizing compilers essentially need to gather information concerning the structure and the control flow of the program. This will enable to deal with issues such as detection of loops within the program and instructions that are always executed before and after a given instruction. Such information enables the types of code-improving transformations that a programmer and a compiler writer can use to improve the performance of the code generated.

The program in its source form or at intermediate form can be improved in order to generate a better target code. It consists of determining control flow and performing data-flow analysis followed by the application of transformations, as discussed in this section.

■ Basic Blocks

A *basic block* is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching, except at the end. Alternatively, a basic block is a sequence of consecutive three-address code statements satisfying the following properties:

- ☐ Flow always enters the block at the beginning
- ☐ Flow always leaves the block at the end
- ☐ There are no branches or halts, except at the end

The following algorithm can be used to partition a sequence of three-address statements into basic blocks:

1. We first determine the set of *leaders*, the first statements of basic blocks. The rules we use are as follows:
 - The first statement is a leader
 - b. Any statement that is the target of a conditional or unconditional goto is a leader
 - c. Any statement that immediately follows a goto or conditional goto statement is a leader
2. For each leader, its basic block consists of the leader and all statements, but not including the next leader or the end of the program.

Example 1: Consider a simple arithmetic expression: $a*a + 2*a*a + b*b$. The three-address code for this expression forms a basic block given as follows in Figure 4:

```

t1 := a * a
t2 := a * b
t3 := 2 * t2
t4 := t1 + t3
t5 := b * b
t6 := t4 + t5

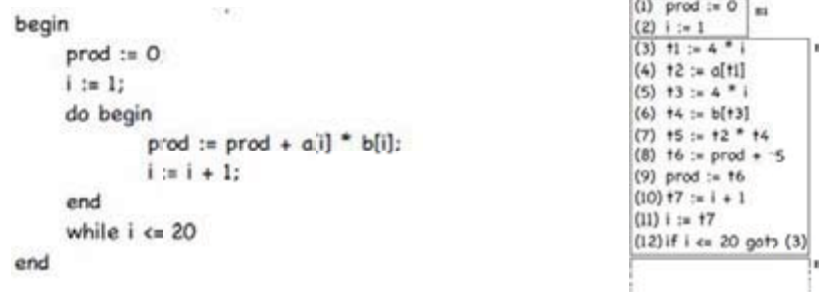
```

▲ Figure 4: Basic Block for $a*a + 2(a*b) + b*b$

Note that there are no jumps in the generated code; hence, the entire code forms one basic block.

The following example gives illustration of constructing basic blocks for more complex code involving jumps.

Example 2: Consider the pseudo code for calculating dot product of vectors $a[]$ and $b[]$ in Figure 5(a). The corresponding three address code and basic block for the above code is given in Figure 5(b):



(a) The Sample Source Code

(b) Three-Address Code and Basic Blocks

▲ Figure 5: Example of Basic Block

If we assume that the three-address statements (or quadruples) perform computation for machine with four byte/word, then the statement 1 is leader by rule 1.1, statement 3 is leader by rule 1.2, and statement following 12 will be leader by rule 1.3. This results in three basic blocks B1, B2, and B3, respectively. B1 runs from statement 1 to 2, B2 runs from statement 3 to statement 12, and B3 starts from statement following statement 12.

While doing program analysis, we often use *maximal basic blocks*. These are basic blocks that cannot be extended by including adjacent nodes without violating the definition of a basic block. It starts at a label or at the beginning of the routine and ends just before a jump or jump-like node or label or the end of the routine. Once all parameters have been evaluated and stored in their required locations, a call to the routine will terminate the basic block. For imperative languages, a basic block exclusively contains expressions and assignments, which occur sequentially.

■ Flow Graphs

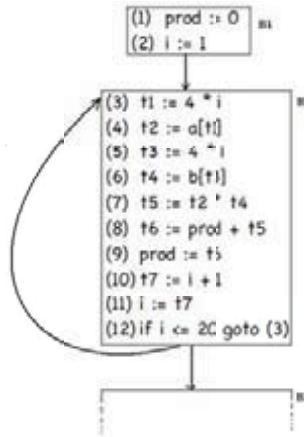
We can add flow control information to the set of basic blocks making up a program by constructing a directed graph called a *flow graph*. The nodes of a flow graph are the basic nodes. The block whose leader is the first statement of the code is designated as initial node of the graph. The flow graph contains directed edge from block B_i to block B_j if B_j immediately follow B_i during some execution sequence. We connect basic blocks with directed edges indicating the possible flows of control.

The rules of drawing the directed arcs between the nodes of the flow graph are as follows.

- Presence of jump statements (conditional or unconditional) from the last statement of B_i to the first statement of B_j

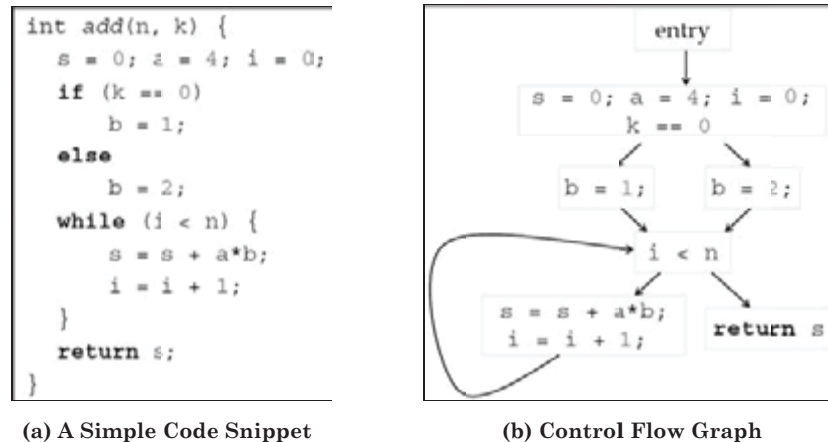
- B_j immediately follows B_i in the order of the program, and B_i does not end in an unconditional jump where B_i is called the predecessor of B_j and B_j is known as successor of B_i .
- Predecessors of block i are the blocks j such that the flow graph has an edge $j \rightarrow i$ and successors of block i are the blocks j such that the flow graph has an edge $i \rightarrow j$.

Flow graph for Figure 5(b) is shown in Figure 6:



▲ Figure 6: Flow Graph for Figure 5(b)

Example 3: Figure 7(a) gives illustration of a code snippet in C and Figure 7(b) shows its control flow graph for a procedure involving conditional computations:



▲ Figure 7: Illustration of a Code Snippet in C and Its Control Flow Graph

Each node in the flow graph represents a basic block which itself is sequence of instructions. Note that there are no branches out of the middle of basic block and no branches into the middle of basic block. Since jumps have been excluded, the control flow inside a basic block cannot contain cycles. The execution of basic block starts with the first instruction and the flow graph includes all executable instructions contained in various basic blocks. The edges in the graph represent control flow within the program (between the basic blocks).

8.6 Next-Use Information

A compiler that can analyze the program and determine the next-use of its names (variables), is said to have better opportunities of optimizing the target code.

Consider a three-address statement that assigns a value to x . Let this statement be the i^{th} statement in the code sequence. If a statement j uses x as its operand and control can flow from statement i to j along a path such that it has no intervening assignments to x , then the statement j is said to have used the value of x computed in the i^{th} statement. Knowing the next-use of names within the basic blocks and across the flow graph will enable the optimal use (or reuse) of register and temporaries.

If a name contained in the storage has no next-use, then the storage can be reallocated to some other name. The storage keeps the name only if it will be used subsequently in some context. The next-use information is used by simple code generators, wherein it is used for register assignments.

We can determine the next-use information by performing a backward pass over each basic block and keeping record (in the symbol table) for each name x the information whether x has a next-use in the block. If the name has no next-use in the block, then we identify whether it is live on exit from that block which is an important issue in optimizing compilers. It can be simply assumed that all non-temporary names are live on exit and all temporary variables are dead on exit from the block. We use the steps of the algorithm described below to which can help to improve the target code.

Algorithm to compute next-use information

As an application, we consider the assignment of storage for temporary names.

Suppose we reach three-address statement i : $x := y \text{ op } z$ in our backward scan. We then do the following tasks:

1. Attach to statement i the information currently in the symbol table regarding the next use and liveness of x , y , and z . If x is not live, we delete the statement.
2. Set x to “not live” and “no next use” in the symbol table.
3. In the symbol table, set y and z to “live” and the next uses of y and z to i .

Note that the order of steps (2) and (3) may not be interchanged because x may be y or z . For three-address statement of the form $x := y$ or $x := \text{op } y$, the steps remain same as before, ignoring z .

Example 4: Consider the statements within the basic block as given as follows. Determine the next-use information of temporary names in the block.

1. $t_1 = a * a$
2. $t_2 = a * b$
3. $t_3 = 2 * t_2$
4. $t_4 = t_1 + t_3$
5. $t_5 = b * b$
6. $t_6 = t_4 + t_5$
7. $X = t_6$

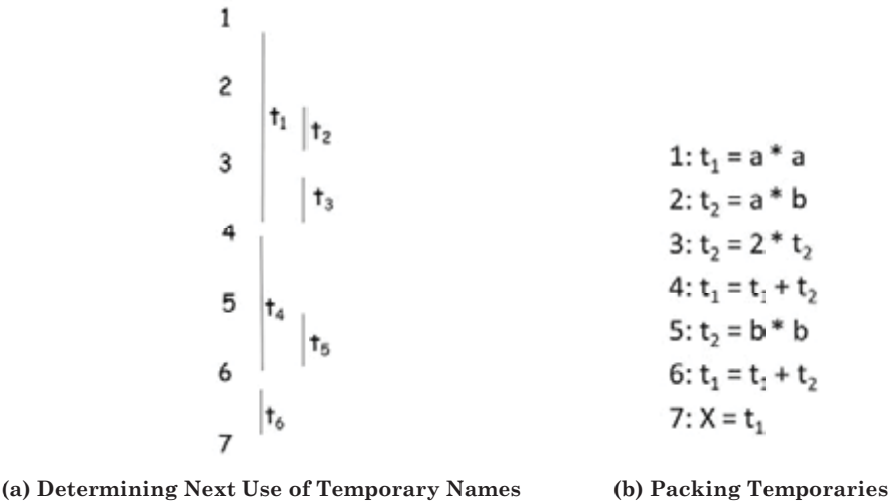
Performing the backward scan of the basic block, we determine the next use of the temporary names as shown in Figure 8:

STATEMENT	Symbol Table		
7: no temporary is live	t_1	dead	Use in 4
6: t_6 :use(7), t_4 t_5 not live	t_2	dead	Use in 3
5: t_5 :use(6)	t_3	dead	Use in 4
4: t_4 :use(6), t_1 t_3 not live	t_4	dead	Use in 6
3: t_3 :use(4), t_2 not live	t_5	dead	Use in 6
2: t_2 :use(3)	t_6	dead	Use in 7
1: t_1 :use(4)			

▲ Figure 8: Next-Use of Names and Symbol Table Entries

A program has most statements either of the form of expression or may be in the form of assignments. We can allocate storage locations for temporaries by examining each in turn and assigning a temporary to the first location in the field for temporaries that does not contain a live temporary. If a temporary cannot be assigned to any previously created location, add a new location to the data area for the current procedure.

The six temporaries in the basic block can be packed into two locations. These locations correspond to t_1 and t_2 , as shown in Figure 9(b). Figure 9(a) shows determining next use of temporary names and Figure 9(b) shows packing temporaries:



▲ Figure 9: Determining Next Use of Temporary Names and Packing Temporaries

In many cases, temporaries can be packed into registers rather than memory locations, which are more useful in many cases. Allocations in the registers lead to efficient code.

The effect of an assignment in a basic block may be local to that block or may be non-local. If the assignment has a local effect, the resulting value is not used anywhere else and the variable is dead at the end of the basic block. For assignments that have non-local effects, the assigned variable is an output variable of the basic block.

8.7 A Simple Code Generator

The code generator generates target code for a sequence of three-address statement. It analyzes each three-address statement, recalls if any of the operands of the statement are currently in use and found in any of the registers, and takes advantage of that fact, if possible. The code generation uses descriptors to keep track of register contents and addresses for name variables.

All optimizing compilers use **register descriptor** to keep track of the status of each register. A register descriptor acknowledges if the register is free or occupied. If occupied, then what is the content? During the code generation when the storage is required, it is consulted for new register (or old registers if the values are already there). Initially, the register descriptor marks all the registers as empty and updates their status as they are assigned across blocks. As the code generation for the block progresses, each register will hold the value of zero or more names at any given time.

An **address descriptor** is used to maintain the information about location (or locations) where the current value of the name (variable) is available at runtime. The current location of name can be a register, a stack location, a memory address, or some set of these. The values in these locations might be the result of the current operation or the previous computation. The values once copied remain where they were unless erased or updates. Address descriptors are useful in determining the accessing method for a name and can be stored in the symbol table.

■ Code Generation Algorithm

There are basically three parts of the simple code generation algorithm- choosing registers, generating instructions, and managing descriptors. The basic input for code generation algorithm is assumed to be a sequence of three-address statements; these three-address statements are organized as basic blocks. A *basic block* is a sequence of successive statements in which the flow of control enters at the beginning and leaves only at the end. There are no possibility of branching, except at the end between the start and end of basic block. Consider the general three-address statement of the form $x := y \text{ op } z$, we describe the simple code generation algorithm in the subsequent discussions. The algorithm makes use of function **getReg()** and descriptors, which are described subsequently. The code generator proceeds as follows:

1. Call a function **getreg()** to find out the location L to store result of the computation $y \text{ op } z$. Usually, L is a register. In some cases, it could also be a memory location, especially when no registers are free.
2. Refer to the address descriptor for y to find y' . It is the possible current location(s) of y . If the value of y is found both in the memory and register, then prefer register. This is because the use of register is faster than memory access. If the value of y is not already in L , generate the instruction $\text{MOV } y', L$ to place a copy of y in L .
3. Now generate the instruction $\text{OP } z', L$, where z' is a current location of z . Again, if the value of z is found both in the memory and register, then prefer register. Update the address descriptor to specify that x is in location L . If L is a register, update its descriptor to indicate that it contains the value of x and remove x from all other register descriptors.
4. Analyze the values of y and z . If the current values of y and/or z have no next uses in the registers and are also not live on exit from the block, then update the register descriptors to show that after the execution of $x := y \text{ op } z$, these registers will not hold the values of y and/or z , respectively.

■ Register Allocation and Assignments

The function **getreg()** is used to determine the location L that holds the value of x for the assignment $x := y \text{ op } z$. The location L is determined through the following steps:

1. If the variable y is found in a register and y is not live and also does not have next use after the execution of $x := y \text{ op } z$, then return the register of y for L . Update the address descriptor of y to reveal that y no longer remains in L or L no longer holds the value of y .
2. Failing (1), choose and return an empty register for L if any one exists.
3. Failing (2), if x has a next use in the block or op is an operator such as indexing that requires a register, find an occupied register R . Copy the value of R in the memory location M (by using instruction $\text{MOV } R, M$) if not already there in location M , update the address descriptor M , and return R . If R holds the value of several variables, an MOV instruction must be generated for each variable that needs to be stored. The register R is chosen such that the datum contained in it either referenced furthest in the future or the one whose value is also in the memory.
4. If no suitable occupied register can be found and also x is not used in the block, select the memory location of x as L .

Example 5: The assignment $d := (a - b) + (a - c) + (a - c)$ might be translated into the following three-address code sequence:

- $t_1 = a - b$
- $t_2 = a - c$
- $t_3 = t_1 + t_2$
- $d = t_3 + t_2$

The generated code, values of the register, and address descriptors during the code generation are given as follows in Table 1:

Table 1: The Final Code and Status Of Register and Address Descriptors for Code in Example 5

Table 1: The Final Code and Status Of Register and Address Descriptors for Code in Example 5			
Statement	Code	Register descriptor	Address descriptor
$t_1 = a - b$	$\text{mov } a, R_0$	R_0 contains t_1	t_1 in R_0
	$\text{sub } b, R_0$		
$t_2 = a - c$	$\text{mov } a, R_1$	R_0 contains t_1	t_1 in R_0
	$\text{sub } c, R_1$	R_1 contains t_2	t_2 in R_1
$t_3 = t_1 + t_2$	$\text{add } R_1, R_0$	R_0 contains t_3	t_3 in R_0
		R_1 contains t_2	t_2 in R_1
$d = t_3 + t_2$	$\text{add } R_1, R_0$	R_0 contains d	d in R_0
	$\text{mov } R_0, d$		d in R_0 and memory

■ Machines Implementation of Conditional Jumps

Conditional jumps can be implemented in the following two ways:

1. Branch if the value of a chosen register meets one of the six conditions: negative, zero, positive, non-negative, non-zero, and non-positive. One way to implement the three-address statement of the form **if** $x < y$ **go to** z is to subtract y from x in register R , then jump to z if the value in the register is negative, that is,

$\text{if } x < y \text{ goto } z$	$\Rightarrow$	$\text{MOV } X, R_0$
		$\text{SUB } Y, R_0$
		$\text{JMP negative } z$

2. Use of a set of *condition codes* that indicate whether the last computation makes the register negative, zero, or positive. The condition codes set by the use of compare (CMP) instruction can be used to make the jump under a designated condition $<$, $=$, $>$, $\geq$, $\leq$, or $\neq$. The compare instruction can set the condition code without actually computing the value. That is, `CMP x, y` sets the condition code to positive if $x > y$. The code to implement if $x < y$ go to `z` using compare instruction is given by:

```
CMP x, y
CJ< z
```

3. Code generation makes use of condition-code descriptor to tell the name that last sets the condition code. Thus, the three-address code

```
x := y + z
if x < 0 goto z
```

generates the code

```
MOV Y, R0
ADD Z, R0
MOV R0, X
CJ< L
```

8.8 The DAG Representation of Basic Blocks

We can convert the AST and the control flow implied in the basic block into a dependency graph called DAG which can then be rewritten to code.

A DAG is a handy data structure that is used to implement transformations on the basic blocks and describe how the value computed by a statement in a basic block is used in subsequent statements of the block. DAGs can be used to determine common sub-expressions (expressions that are computed more than once) within a block. In addition, it is also helpful in determining which names are used inside the block, but evaluated outside the block, and which statements of the block could have their computed value used outside the block. The most general DAG representation for a basic block satisfies the following conditions:

1. Labels on leaf node represent unique identifiers, which are either variable names or constants.
2. The operator applied to a name determines whether the label represents an l-value or r-value of a name; most leaves represent r-values.
3. The leaves represent initial values of names, and we subscript them with 0 to avoid confusion with labels denoting “current” values of names as in (5) below.
4. Interior nodes are labeled by an operator symbol.
5. Nodes are also optionally given a sequence of identifiers for labels. This is because the interior nodes represent computed values and the identifiers labeling a node are deemed to have that value.
6. The nodes whose variables are live on exit from the block are designated as output nodes.

The generation of DAG for a sequence of three-address code in the basic block is simple. Each interior node of the DAG represents a formula in terms of the leaves, that is, the values possessed by variables and constants upon entering the block. Figure 10(a) shows three-address code sequence in the basic block and Figure 10(b) shows its DAG:

```

1.  $t_1 := 4 * i$ 
2.  $t_2 := a[t_1]$ 
3.  $t_3 := 4 * i$ 
4.  $t_4 := b[t_3]$ 
5.  $t_5 := t_2 * t_4$ 
6.  $t_6 := prod + t_5$ 
7.  $prod := t_6$ 
8.  $t_7 := i + 1$ 
9.  $i := t_7$ 
10. if  $i \leq 20$  goto (1)

```

(a) Three-Address Code Sequence in the Basic Block (b) DAG for Figure 10(a)

▲ Figure 10: DAG Representation of Basic Block

In Figure 10(b), the node labeled t_2 represents the formula $a[4*i]$. That is, the value of t_2 is the value of the word whose address is $4*i$ bytes offset from address a .

Making use of DAG for code generation has the following several *advantages*:

- ❑ It is easy to rearrange the order of the final computation sequence compared to what we can do from a linear sequence of three-address statements or quadruples
- ❑ It allows applying algebraic laws to reorder operands of three-address instructions, and sometimes, it hereby simplifies the computation
- ❑ The code generated from DAG is optimal under criteria such as program length or number of temporaries being used
- ❑ Helps in elimination of common sub-expressions and thus avoids repeated execution of repetitive operations whose value does not change
- ❑ Helps to eliminate *dead code*, that is, instructions that compute a value that is never used

It is easier to generate code from data dependencies that form the control flow graphs. During code generation, data dependencies are more important, which are well supported by DAGs. With DAG implementations of the basic blocks, the order of the evaluation of operations in expressions is immaterial as long as the data dependencies inside the expressions are respected. When the intermediate code is itself a parse tree or AST, the algorithm for optimal code generation from a tree is more useful and efficient.

8.9 Peephole Optimization

Redundant instructions may be discarded during the final code generation.

Almost of all the compilers apply some effort to improve the efficiency of the code they produce. This is because code generation based on statement-by-statement transformation may often lead to code which may contain set of redundant instructions and inefficient program constructs. Much of this problem can be handled locally by inspecting the code and modifying inefficient sequences of instructions. The compiler uses method called *peephole optimization* to discover local improvements by looking at a window on the code. This tiny window corresponds to a peephole that moves over the program. The method slides the peephole over the code and examines the contents to match patterns (with a limited set of patterns).

Early peephole optimizers (McKeeman) used limited set of patterns and matched them with exhaustive search over the program. This method used a small window and a small pattern set which

leads to quick executions. Modern peephole optimizers have larger and complex pattern set and produced more systematic approach of optimizing the code. The peephole replaces the instructions by a shorter and (or) faster sequence, whenever possible. The code generation is inherently local.

In simple terms, a peephole optimization is an optimization that is performed over a very small set of instructions in a segment of generated code. Peephole optimizations identify sets of non-performing instructions or that can be replaced by a simple and efficient set of instructions.

Some popular examples of program transformations that are characteristic of peephole optimization are as follows:

1. Elimination of redundant loads and stores
2. Elimination of unreachable code
3. Algebraic simplifications
4. Reducing of strength
5. Use of machine idioms
6. Elimination of multiple jumps

We discuss each of them briefly in the following subsections.

■ Eliminating Redundant Loads and Stores

Code generation by statement-by-statement strategy often produces code with a set of redundant load and store instructions. These codes contain set instruction which transfers the computed value between registers and memory locations. If a value is already in the internal register of the machine and is required subsequently, then loads from the memory will prove to be slower and are redundant themselves. The compiler must save the computed values in the memory only when there are no available registers that keep the value furthermore and require the value the next use in future. We illustrate this with the following example.

Consider a set of arithmetic statements:

```
x = y + z;
a = x + b;
```

A statement-by-statement code generation strategy straightforwardly implements the preceding statements as follows:

1. MOV y, R0 ; Copy y to the register R0
2. ADD z, R0 ; Add z to the register R0, the register R0 is now y+z (x)
3. MOV R0, x ; Copy the register R0 to x
4. MOV x, R0 ; Copy x to the register R0
5. ADD b, R0 ; Add b to the register R0, the register R0 is now x+b [(y+z)+b]
6. MOV R0, a ; Copy the register R0 to a

We can delete instruction (4) because whenever (4) is executed, (3) will ensure that the value of x is already in register R0. Note that if (4) has a label, it is difficult to determine that (3) was always executed immediately before (4) and so we could not remove (4), i.e., instruction 4 can always be removed if it does not have a label.

The abovementioned code can be optimized to

- ☐ MOV y, R0 ; Copy y to the register R0
- ☐ ADD z, R0 ; Add z to the register R0, the register R0 is now y+z or (x)
- ☐ MOV R0, x ; Copy the register R0 to x

- ❑ `ADD b, R0` ; Add `b` to the register `R0`, the register `R0` is now `y+z+b` or `[(x)+b]`
- ❑ `MOV R0, a` ; Copy the register `R0` to `a`

■ Identifying Unreachable Instructions

Many programs contain sequence of code that comprises jumps and use labels. It is often possible that there is an unlabeled instruction in the program that immediately follows an unconditional jump. This instruction can potentially be removed. The reason in doing so is that this statement will be possibly never executed in the program's life. The instruction is called unreachable instruction. The flow of control within the program does not allow reaching to this statement. This provides another opportunity for peephole optimization - the removal of unreachable instructions.

Unreachable code is undesirable for the following reasons:

- ❑ It occupies unnecessary memory that can be used by other program objects if made available
- ❑ It results in unwanted caching of instructions into the CPU instruction cache, thereby decreasing data locality
- ❑ Time and effort may be spent in maintaining and documenting a piece of code which has no effect as it is never executed

Presence of unreachable code in the program may be result of number of reasons. This can include:

- ❑ Presence of complex conditional branches in the program code which makes a set of statements never reachable
- ❑ Result of the internal transformations performed by an optimizing compiler such as common-sub expression elimination

Removal of unreachable code surely optimizes the program in term of size and has been taken seriously by most optimizing compilers. The operation can be repeated in order to eliminate a sequence of instructions, thereby reducing the size of generated program. The following example makes the idea more clear.

Consider a fragment of C code

```
int foo (int a, int b)
{
    int c;
    return a+b;
        c=a*b;
}
```

In the preceding code fragment, the program control flow will never reach the definition `c=a*b`. This is because the function returns before the definition is reached. Therefore, the definition of `c=a*b` can be discarded.

Unreachable code is a form of static analysis and requires control flow analysis to detect code sequences that will never be executed regardless of their values and other conditions at runtime.

■ Simplify Algebraic Expressions

Much of the programs can be optimized by performing algebraic simplifications. For example, if the program contains statements such as:

```
a = a+ 0
a= a *1
```

then these statements can be eliminated because zero is an additive identity and one is a multiplicative identity. Either of the statements bring no change in the values contained in variable `a`.

Benefit of simplifying algebraic expression is clearly visible as it will reduce the amount of target code and thus save memory space and execution time.

■ Strength Reduction

The modern computing environment provides multiple machine instructions that can perform similar computations of which some are considered to be cheaper than others. Many compilers replace most of the expensive operations (as far as possible) by equivalent cheaper ones on the target machine, thereby increasing the efficiency of the code. This phenomenon is called strength reduction. *Strength reduction* is a popular compiler optimization that replaces the expensive operations with equivalent but less expensive ones.

The examples discussed as follows make the concept of strength reduction much clear:

- **Replace exponentiation by multiplication:** For example, x^2 is invariably cheaper to implement as $x * x$ than to call to an exponentiation routine.
- **Replace fixed-point multiplication or division by a power of two as a shift:** Shift operation is cheaper than implementing multiplication or division. For example, $x \ll 1$ is cheaper than $x * 2$ and $x \gg 1$ is cheaper than $x / 2$.

Note that most of a program's execution time is in general spent in smaller sections of the program code, and that code is often inside a loop that is executed over and over. We can improve the program to a great extent by reducing the strength of instruction within the loop, thereby achieving the better optimization objectives.

Consider the following code:

```
c = 8;
for (i = 0; i < N; i++)
{
    y[i] = c * i;
}
```

It can be simplified and replaced with weaker additions as follows:

```
c = 8;
k = 0;
for (i = 0; i < N; i++)
{
    y[i] = k;
    k = k + c;
}
```

■ Use Faster Machine Instructions

The target machine has many instructions that can be implemented directly by the underlying hardware. These are special instructions which when used lead to faster execution. Identifying specific situations in the instruction and using such hardware instructions can reduce execution time significantly. Auto-increment and auto-decrement instructions are among such advanced hardware-oriented instructions. A good optimizing compiler is expected to implement such faster instructions.

Using these modes can greatly improve the quality of the code when pushing or popping a stack. These modes can also be used for implementing statements such as $a = a + 1$.

Consider an example instruction that adds constant 1 to the register R.

ADD #1, R

It can be replaced by

INC R

The latter instruction is equivalent to the former instruction but costs less and more efficient.

■ **Elimination of Multiple Jumps**

The user program often contains multiple jump sequences, and control flow analysis of the program can help you to optimize the program by replacing the jump sequences, as illustrated as follows:

```
goto L1
...
...
L1 : goto L2
```

This sequence of code containing jump sequence can be replaced by

```
goto L2
...
...
L1 : goto L2
```

This simplification leads to time efficient execution of the target program and makes the flow of the program much simpler.

8.10 Generating Code from DAGs

Generating code from DAG has several advantages over linear sequence of quadruples. Many compiler implementations prefer generating code for a basic block from its DAG representation. The code generation algorithm proceeds in two steps. First, we convert the code sequences or AST and the control flow implied in it into a DAG. We then rewrite the DAG to code. There is a clear advantage of using DAG for code generation. It is easier to rearrange the final order of the final computation and thus generate code that is optimal with fewer numbers of temporaries and in term or program size.

■ **Rearranging Order of the Code**

Rearranging the order of the code often results in more efficient and compact code and avoids the use of many temporaries. DAGs are useful in reordering the code and eliminating the common sub-expressions. Thus, the code generated from DAG will be more efficient and compact compared to code generated directly from three-address codes or AST representations.

Let us consider statements within the basic block and corresponding DAG representation, as shown in Figure 11(a) and 11(b), respectively:



(a) Code Sequence within the Basic Block (b) The DAG Representation

▲ Figure 11: Code Sequence within the Basic Block and the DAG Representation

If we assume there are two available registers R0 and R1, then the code corresponding to the three-address statements for Figure 11(a) can be generated using simple code generation algorithm described in Section 8.8. We obtain the code sequence as follows:

```
MOV  a, R0
ADD  b, R0
MOV  c, R1
ADD  d, R1
MOV  R0, t1
MOV  e, R0
SUB  R1, R0
MOV  t1, R1
SUB  R0, R1
MOV  R1, X
```

Consider that the order of the statements is rearranged so that the computation of t_1 occurs immediately before that of X as:

```
t2 = c + d
t3 = e - t2
t1 = a + b
X = t1 - t3
```

Then, the new code sequence (again only R0 and R1 are available) is:

```
MOV  c, R0
ADD  d, R0
MOV  e, R1
SUB  R0, R1
MOV  a, R0
ADD  b, R0
SUB  R1, R0
MOV  R1, X
```

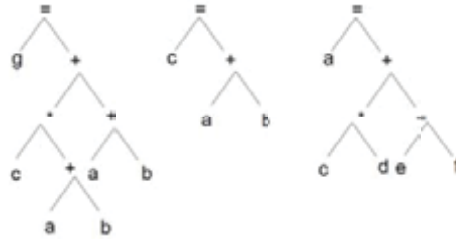
We can clearly see that by changing the order of code, we have saved two instructions: `MOV R0, t1` (which stores the value of R0 in memory location t_1) and `MOV t1, R1` (which reloads the value of t_1 in the register R1), thus saving space as well as execution time.

■ A More Practical Code Generation from DAG

We now describe a practical approach to generate code from DAG. For a given sequence of code, we first convert them to trees, trees to DAG, and finally from DAG to respective codes. This approach accounts for elimination of common sub-expressions and order of evaluation. We explain the code generation process with example. Consider the following code:

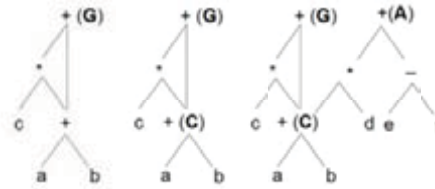
```
g = c * (a + b) + (a + b)
c = a + b
a = (c * d) + (e - f)
```

We first convert code to trees, as given in Figure 12:



▲ Figure 12: Tree Representation of Code Sequences

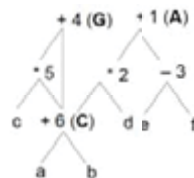
Now convert trees to DAG, as shown in Figure 13:



▲ Figure 13: DAG Representation of Figure 12

Now schedule DAG nodes before attempting to assign registers and code generation, described as follows:

1. The first operator scheduled will be the last one to be evaluated
2. Start from the rightmost root since a DAG can have more than one root
3. Nodes are added to DAG from left to right, so the rightmost root is the last to be created and added to graph
4. Work left-to-right within sub-DAG, scheduling each operator in turn
5. Within a sub-DAG, schedule an operator if all of its parents have been scheduled, as shown in Figure 14:



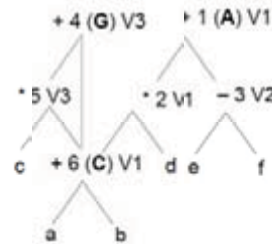
▲ Figure 14: Scheduled Nodes

Next, in the process, determine the virtual registers following the steps described as follows:

Start at node with schedule number 1

- i. Assign the node that currently has the lowest schedule number m and has not already been assigned to a register set to the next sequential virtual register set V_n , where $n = 1, 2, \dots$
- ii. Recursively assign each left node in the tree under m to virtual register set V_n so long as its lowest schedule number parent belongs to V_n
- iii. Continue until all nodes are assigned virtual registers

Figure 15 shows assigning virtual registers:



▲ Figure 15: Assigning Virtual Registers

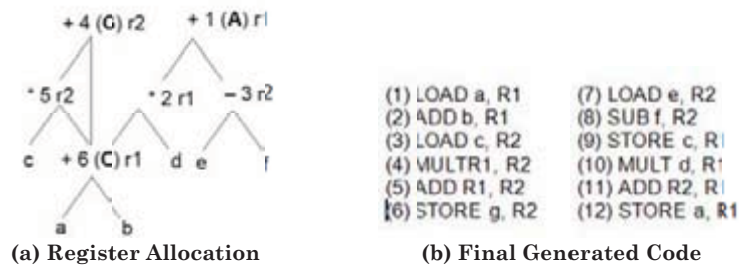
Next determine the virtual register span to assist in final register allocation and code generation. A *virtual register span* is the set of nodes the register is needed for. Note that two virtual registers may be mapped into the same hardware registers if their spans do not overlap. From Figure 15, we observe that V1 is mapped alone to R1 since its span is from node 1 to node 6 and V2 and V3 are mapped into R2 since their spans do not overlap. The virtual register span is shown in Figure 16:

Register	Span
V1	1-6
V2	1-3
V3	4-5

▲ Figure 16: Virtual Register Span

Now generate code from the highest schedule number to the lowest schedule number.

In this process, make sure that the register that is saved is a real register, which is virtual register mapped at that node. Also delay any store operations until code for the lowest schedule number parent is about to be output. Figure 17(a) shows register allocation and Figure 17(b) shows the final generated code:



▲ Figure 17: Register Allocation and Final Generated Code

8.11 Dynamic Programming Code-Generation Algorithm

The dynamic programming code generation is an optimal code generation strategy based on the work of Aho and Johnson algorithm that can be used to generate code for expression on register machines with r -interchangeable registers, load, store, and add instructions. Their algorithm assumes a uniform-register machine with r interchangeable registers $R_0, R_1, \dots, R_{r-1}$ and instructions of the form $R_i := E$, where E can be any expression which contains operators, registers, and memory locations.

Dynamic programming is based on the principle of dividing the problem into sub-problems and then solving them independently. In other words, dynamic programming allows solving an optimization problem by caching sub-problems solutions rather than recomputing them. Dynamic programming code-generation is also based on similar principle that partitions the problem of generating optimal code for an expression E into many subproblems of generating optimal code for the subexpressions of E . In other words, the optimal code for expression $E_1 + E_2$ is the result of combination of optimal code of subexpressions E_1 and E_2 , respectively, followed by code to evaluate the operator $+$. Optimal codes for subexpressions E_1 and E_2 are subproblem themselves, which are solved recursively. Remember that the total cost of the program is the sum of the cost of instructions in the program. The dynamic programming code generation approach tries to keep each partition optimal, thus producing optimal code for the entire program.

Let us consider an expression tree T denoting expression $E = E_1 \text{ op } E_2$, where T_1 and T_2 are trees for subexpressions E_1 and E_2 , respectively, as shown in Figure 18:



▲ Figure 18: An Expression Tree

The dynamic programming algorithm evaluates an expression “contiguously,” i.e., completes the evaluation of T_1 , T_2 , and then evaluates the root T . In general, a contiguous program first evaluates those subtrees of T that need to be computed into memory and then evaluates the remainder of T . The order of evaluation can be either T_1 , T_2 , and then the root, or in the order T_2 , T_1 , and then the root. In either case, it uses the previously computed values from memory whenever necessary and tries to keep the overall code optimal.

For the uniform-register machine, every expression tree can be evaluated optimally by a contiguous program on a uniform-register machine such that given any machine-language program P to evaluate an expression tree T , there is a program P' that computes the same expression satisfying the following properties:

1. P' is of no higher cost than P
2. P' uses no more registers than P
3. P' evaluates the tree in a contiguous fashion

From the discussion, it is clear that for any expression tree T , there is an optimal program that consists of optimal programs for subtrees of the root, followed by an instruction to evaluate the root. This property enables the use of dynamic programming to generate optimal program for T . The code generation using dynamic programming proceeds in the following three stages:

- **Stage I:** In the first stage, the algorithm computes a cost array C for every node n of the tree in a bottom-up fashion such that component $C[i]$ is the optimal cost of computing the subtree S rooted at n into a register, assuming i registers are available for the computation, for $1 \leq i \leq r$. The cost includes whatever cost incurred in loads and stores and cost of computing operators at the root of the tree.

- ❑ **Stage II:** In the second stage, the algorithm traverses tree T using the cost vectors and determines which subtrees of T must be computed into memory.
- ❑ **Stage III:** In this stage, the algorithm traverses each tree, uses the cost vectors and associated instructions, and generates the final target code.

Note that the code for the subtrees computed into memory locations is generated first. The cost vectors for a given tree T can be easily computed bottom-up in time linearly proportional to the number of nodes in T . Also the smallest cost in the vector for the root of T leads to the minimum cost of evaluating T .

8.12 Code-Generator Generators

Code-generator generators are a way toward automating compiler production. Code-generator generators produce code generator, given a description of the intermediate code and the description of machine code that is to be generated. Matching intermediate code to object templates resembles to that of parsing problem and representations of a syntax tree can be defined using CFG that contains actions to emit code. There are many code-generator generators developed in the past, each with some specific features. Some of the well-known code-generator generators are as follows:

- ❑ **Graham and Glanville – 1980:** It was one of the earliest methods, where linearized tree patterns were recognized using a bottom-up shift-reduce parser. An LR(0) grammar was automatically derived from the machine instruction set. One known problem with these generator-generators was that the grammars for this method were highly ambiguous.
- ❑ **Davidson and Fraser – 1984:** This method did not attempt to select good code from the tree directly. First, it locally macro-expands the tree into naive machine instructions. Then, the resulting code was incrementally improved using a set of declarative optimizations, such as a peephole optimizer. This method was used in GNU Compiler Collection (GCC).
- ❑ **TWIG - Aho, Ganapathi and Tjiang – 1989:** It introduced the principles of tree pattern matching and dynamic programming. TWIG uses a table-driven pattern matching, which was more complicated than raw code generation.
- ❑ **BURG - Fraser, Henry and Proebsting – 1992:** It was based on Bottom-Up Rewrite System (BURS) theory in order to move the dynamic programming to compile-compile time. This generator was more complicated, but very efficient. IBURG is a variant of BURG that does the dynamic programming at compile time, using a straight-forward method.

Holistically, the code-generators generator is a special software component that takes as input a specification (a BURG tree-grammar in the case of IBURG) and generates a software module that plugs into the compiler.

Summary

Code generation is the process by which a compiler converts internal representation of the source code fed to it into a machine executable form. It takes as input an IR of the source program generated by the front-end and produces as output an equivalent program in target machine language. All code generators must generate correct and efficient target code. Factors such as form of instructions, register usage, calling conventions, instruction scheduling, memory organization, executable format, etc., are some important factors that influence the code generation process.

Some important issues involving memory management, selection of instructions, register allocation policy, and order of evaluation are at the center of concern of all code-generation algorithms. Instruction selection depends on several factors including instructions used, uniformity, completeness, speed, register allocation, and evaluation order. On the target machine, the choice among available

addressing modes is a challenging task. Code generator must determine the best suited addressing mode and format for the target code for both time and space requirements.

The program in its source form or at intermediate form can be improved in order to generate a better target code. It consists of determining control-flow and performing data-flow analysis. A *basic block* is a sequence of consecutive statements in which the flow of control enters at the beginning and leaves at the end without halt or possibility of branching except at the end. Each basic block is identified with leader statement. Flow graph is a directed graph whose nodes are basic blocks. There are no branches out of middle of basic block and no branches into middle of basic block. Since jumps have been excluded, the control flow inside a basic block cannot contain cycles.

We can determine the next-use information by performing a backward pass over each basic block and keeping record (in the symbol table) for each name x the information whether x has a next use in the block. The information about the next-use of name enables to optimize the target code by eliminating redundant loads and stores. The effect of an assignment in a basic block may be local to that block or may be non-local. If the assignment has a local effect, the resulting value is not used anywhere else and the variable is dead at the end of the basic block. For assignments that have non-local effects, the assigned variable is an output variable of the basic block.

The code-generation uses descriptors to keep track of register contents and addresses for name variables. A register descriptor tells if the register is free or occupied. An address descriptor is used to maintain the information about location (or locations) where the current value of the name (variable) is available at runtime. The current location of name can be a register, a stack location, a memory address, or some set of these. The basic input for code generation algorithm is a sequence of three-address statements, which are organized as basic blocks. It calls function `getreg()` to find out the location L to store result of the computations. L can be register or memory location as determined by the algorithm. Code generation phase takes special care while implementing conditional and unconditional jumps. It may use set of condition codes to ease the process.

DAGs data structure that is used to implement transformations on the basic blocks and describes how the value computed by a statement in a basic block is used in subsequent statements of the block. DAGs can be used to determine common sub-expressions (expressions that are computed more than once) within a block. In addition, it is also helpful in determining which names are used inside the block but evaluated outside the block, and which statements of the block could have their computed value used outside the block. DAG has several advantages which makes it more suitable for code generation as compared to linear three-address code or syntax trees.

Peephole optimization method performs local improvements by looking at a window on the code, inspecting the code, and modifying inefficient sequences of instructions. In essence, peephole optimization is an optimization that is performed over a very small set of instructions in a segment of generated code. Some popular peephole optimization techniques include elimination of redundant load and stores, eliminating unreachable codes, algebraic simplifications, reduction of strength, use of machine idioms, and eliminating sequence of jumps.

Generating code from DAG has several advantages over linear sequence of quadruples or syntax trees. It is easier to rearrange the final order of the final computation and thus generate code that is optimal with fewer numbers of temporaries and in term of program size. The dynamic programming code-generation is an optimal code generation strategy that can be used to generate code for expression on register machines with r -interchangeable registers, load, store, and add instructions. Dynamic programming code-generation is based on the principle that partitions the problem of generating optimal code for an expression E into many subproblems of generating optimal code for the subexpressions of E . The dynamic programming code generation approach tries to keep each partition optimal and hence producing optimal code for entire program. The code-generators generator is a special software component that takes as input a specification and generates a software module that plugs into the compiler.

Review Questions

- Q1. List and explain the major challenges in designing a code generator.
- Q2. What are the characteristics of good compiler? Explain the major tasks in code generation.
- Q3. What are the important design issues of code generators? Briefly describe each one of them with suitable example.
- Q4. Differentiate between absolute machine language and re-locatable machine language.
- Q5. What is the significance of instruction selection in code generation phase? What are the important factors that influence the selection of instructions?
- Q6. How does choice of evaluation order affect the target code generation? How can it be solved?
- Q7. Write short note on addressing modes.
- Q8. What are basic blocks? State their properties.
- Q9. What is a flow graph? State its significance in code generation.
- Q10. Define next-use of names. Give the algorithm to compute next-use information for three-address statements in the program.
- Q11. What are register descriptors and address descriptors? Explain.
- Q12. State and explain the code generation algorithm for sequence three-address statements organized as basic blocks.
- Q13. Explain the process of register allocation and memory assignments for program objects during code generation. Give example to illustrate the concept.
- Q14. Explain with example the code generation for conditional jumps. What are the challenges faced in generating code for program with multiple jump sequences? How do we solve them?
- Q15. Define DAG. What are the advantages of using DAG?
- Q16. What is peephole optimization? How is it performed? Give examples of peephole optimizations.
- Q17. Write short note on elimination of:
 - a. Redundant load and store
 - b. Reduction in strength
 - c. Multiple jump sequences
- Q18. What is unreachable code? Why is it undesirable?
- Q19. What is the benefit of rearranging the order of the code in final code generation? Explain with suitable example.
- Q20. Explain the code generation process using DAG. Describe each step with suitable example.
- Q21. What is virtual register span?
- Q22. What are the benefits of dynamic programming code generation algorithm? How does it generate optimal target code? Explain with example.
- Q23. List and explain the code generation algorithm using dynamic programming.
- Q24. Consider the following quads:
 1. $i = 1$
 2. $j = 1$
 3. $t1 = 10 * i$
 4. $t2 = t1 + j$ // element $[i,j]$

```

5.  t3 = 8 * t2      // offset for a[i,j] (8 byte numbers)
6.  t4 = t3 - 88     // we start at [1,1] not [0,0]
7.  a[t4] = 0.0
8.  j = j + 1
9.  if J <= 10 goto (3)
10. i = i + 1
11. if i <= 10 goto (2)
12. i = 1
13. t5 = i - 1
14. t6 = 88 * t5
15. a[t6] = 1.0
16. i = i + 1
17. if i <= 10 goto (13)

```

Find the set of leaders and basic block for the given sequence of codes.

Q25. Write a short note on code-generator generators. Give examples of code-generator generators.

■ Multiple Choice Questions

Q1. A set of re-locatable object modules can be linked together and loaded for execution by a _____.

- | | |
|--------------------|-------------------|
| a. Absolute loader | b. Linker |
| c. Compiler | d. linking loader |

Ans. d

Q2. _____ refers to the way we access datum.

- | | |
|---------------------------|--------------------------|
| a. Addressing modes | b. Instruction selection |
| c. Instruction scheduling | d. None of these |

Ans. a

Q3. Eliminating redundant load and stores is an example of _____ optimization.

- | | |
|----------------|------------------|
| a. Runtime | b. Peephole |
| c. Algorithmic | d. None of these |

Ans. b

Q4. Instructions that compute a value that is never used is called _____.

- | | |
|------------------|---------------------|
| a. Dead code | b. Hibernating code |
| c. Either a or b | d. Neither a nor b |

Ans. a

Q5. It is easier to rearrange the final order of the final computation in _____.

- | | |
|--------|------------------------------|
| a. AST | b. Linear three-address code |
| c. DAG | d. All of these |

Ans. c

Index

A

- absolute machine-language program, 223
- Abstract Syntax Tree, 140, 180
- access link, 190, 200
- activation, 180, 181, 182, 183, 184, 185, 186, 187, 189, 190, 200, 201, 202, 229
- Activation Records, 180
- activation tree, 180, 182, 183, 184, 185, 200
- address descriptor, 235, 236, 248, 249
- Addressing modes, 227, 250
- allocation strategy, 186, 200, 201
- alphabet, 32, 33, 34, 36, 37, 45, 46, 51, 61, 64, 194
- Analysis, 10, 13, 21, 26, 210, 212
- annotated parse tree, 163
- application programs, 26
- Arden's rule, 38, 39
- array, 17, 18, 31, 139, 140, 156, 157, 162, 163, 165, 168, 172, 177, 179, 180, 182, 189, 190, 192, 193, 200, 209, 224, 246
- Assemblers, 5, 17, 19
- assembly-language program, 224
- attributes, 16, 32, 33, 61, 133, 134, 135, 136, 138, 139, 141, 143, 144, 145, 146, 150, 151, 152, 163, 166, 170, 174, 176, 191
- Automatic code generators, 24

B

- Backtracking, 69
- basic block, 156, 204, 210, 211, 213, 214, 218, 219, 220, 221, 230, 231, 232, 233, 234, 235, 237, 238, 242, 248, 249, 250
- bottom-up parsing, 65, 77, 100, 128
- busy expressions, 218, 220

C

- call-by-name., 191, 200
- call-by-reference, 191, 200
- call-by-value, 191, 200
- calling sequence, 173, 186
- CLOSURE, 106, 107, 114
- Code generation, 15, 199, 223, 237, 239, 247, 248
- code generation, 15, 16, 26, 32, 65, 147, 159, 160, 162, 170, 174, 180, 192, 199, 200, 204, 209, 221, 223, 224, 228, 235, 236, 239, 242, 247, 248, 249
- code generation algorithm, 221, 223, 235, 242, 248, 249
- code hoisting, 207
- Code motion, 208
- code movement, 15, 204, 206
- code optimization, 9, 15, 24, 159, 215, 218, 221
- code production table, 224
- Code-generator generators, 247
- Common Sub-Expression Elimination (CSE), 204
- compile-time evaluation, 204, 219
- compiler pass, 9, 26, 27
- compiler-compilers, 24, 26
- condition codes, 237, 248
- configuration set, 106, 119
- Constant folding, 15, 205, 220
- constant propagation, 205, 211, 218
- context-free grammar, 150
- Control Flow Graph, 211
- control stack, 180, 186, 200, 201, 202
- control structure, 170
- Copying garbage collectors, 201

D

DAG, 140, 154, 156, 157, 158, 160, 175, 177, 206, 221, 223, 237, 238, 242, 243, 244, 248, 249, 250
dangling reference, 190, 201, 202
Data-flow analysis, 210, 211, 212, 218
Data-flow engines, 24
data-flow equations, 212, 218
dead code, 204, 207, 210, 212, 215, 218, 219
Dead code elimination, 215
dead code elimination, 204, 210, 212, 215, 219
dead codes, 207, 219
display, 141, 190, 212
dynamic allocation, 180, 199, 201, 202

E

Empty string, 34
Exhaustive error recovery, 148

F

FIRST, 73, 74, 75, 77, 82, 83, 88, 90, 105, 114, 127, 128, 130
flow graphs, 178, 210, 215, 220, 221
flow-graph analysis, 24
FOLLOW, 73, 74, 75, 77, 110, 111, 113, 125, 127, 128, 130
Fragmentation, 199
frequency reduction, 208

G

garbage, 195, 196, 197, 198, 200, 201, 202
Garbage collection, 196, 201, 202
generational collector, 201
Global optimizations, 9, 204
Grammar, 13, 34, 65, 69, 80, 95, 121, 122, 163

H

Handle, 78, 80, 127, 132
Hash table, 194
heap, 179, 180, 181, 185, 187, 189, 190, 195, 196, 197, 198, 200, 201, 202, 229, 241
heap allocation, 180, 200, 201, 202
high-level IR, 153, 221

High-Level Language, 153

I

identifier, 13, 14, 15, 16, 17, 18, 22, 29, 32, 33, 38, 42, 44, 61, 129, 135, 137, 141, 150, 155, 156, 161, 163, 175, 192, 193, 237
Implicit deallocation, 199, 202
indirect triple, 160, 162, 176, 178
Induction variable elimination, 215
infix, 135, 137, 151, 157, 158, 159
inherited attributes, 135, 143, 144, 145, 150, 151, 174
Instruction selection, 224, 247, 250
Intermediate Code, 153, 159, 161, 163, 171, 173, 177
Intermediate Representation, 153, 204, 221
Interpreters, 5, 28

K

Kleene's closure, 34, 64

L

L-attributed definitions, 144, 150, 151
l-value, 172, 191, 237
LALR(1) parser, 105, 117, 119, 127, 128, 131
language translator, 3, 25, 26, 27, 37
language-independent optimizations, 204
leaders, 230, 250
left factoring, 70, 71, 74, 129
Lex, 11, 12, 13, 21, 22, 28, 29, 30, 31, 32, 33, 35, 37, 39, 41, 42, 43, 44, 45, 47, 49, 51, 53, 54, 55, 56, 57, 59, 61, 62, 63, 64
lexeme, 11, 12, 30, 32, 33, 40, 41, 53, 55, 56, 57, 61, 64
lexemes, 11, 12, 32, 40, 53
lexer, 30, 42
lexical analysis, 13, 16, 24, 26, 27, 33, 39, 127
lexical analyzer, 24, 32, 33, 40, 53, 54, 64, 65, 127, 147
lifetime of an activated procedure, 183
linear grammar, 36, 39
linear IRs, 154, 159, 176
Link and load compiler, 9, 26

linker, 1, 3, 7, 15, 20, 25, 26, 27, 190
 List, 27, 62, 64, 128, 176, 193, 194, 202, 219, 249
 LL parsing, 67, 74
 Load and go compiler, 9, 26, 28
 loader, 1, 2, 3, 7, 9, 15, 17, 20, 21, 25, 26, 27, 224, 250
 Local optimization, 204
 loop, 9, 14, 15, 69, 154, 159, 170, 203, 204, 207, 208, 209, 210, 213, 214, 215, 216, 217, 218, 219, 220, 230, 241
 loop detection, 218
 Loop fission, 210
 Loop interchange, 209
 loop optimization., 204
 Loop optimizations, 204
 loop optimizations, 9, 207
 Loop peeling, 209
 Loop splitting, 209
 Loop test replacement, 210
 Loop unrolling, 208, 215, 218, 220
 Low-level IRs, 159
 LR parsers, 65, 80, 100, 101, 104, 106, 117, 127, 128, 129, 143, 148, 152
 LR(1) parser, 105, 110, 113, 114, 117, 119, 127, 128, 129, 131, 147

M

Machine-dependent optimization, 15, 204
 Machine-independent optimization, 15, 204
 Machine-independent optimizations, 15
 Macro, 17, 18
 mark-and-sweep, 196, 201
 Multi-pass compilers, 9, 26

N

Name equivalence, 140
 next-use information, 221, 233, 248, 249
 non-terminal symbol, 34, 70, 144, 145

O

object program, 3, 5, 25, 26, 153, 181, 182, 200, 205, 223

op-codes, 19
 Optimizing compilers, 9, 26, 204, 210, 218, 221

P

panic mode, 148, 151, 152
 parameter passing, 186, 191, 202
 Parse Tree, 140, 141, 164
 Parser, 22, 24, 30, 65, 66, 71, 73, 74, 78, 80, 100, 103, 104, 110, 113, 117, 120, 128, 143, 148, 149, 150
 parser generator, 24, 65, 78, 120, 128, 131, 150
 Parser generators, 24, 120
 parsing, 12, 13, 24, 30, 44, 65, 66, 67, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 82, 86, 87, 88, 94, 95, 97, 100, 101, 102, 103, 104, 105, 106, 107, 108, 110, 111, 113, 114, 116, 117, 121, 123, 124, 126, 127, 128, 130, 131, 132, 139, 140, 143, 144, 148, 149, 150, 205, 247
 pattern, 15, 17, 22, 30, 31, 32, 41, 42, 43, 44, 53, 54, 55, 56, 57, 61, 62, 63, 64, 70, 210, 228, 238, 239, 247
 Peephole optimizations, 9, 204, 239
 postfix, 135, 141, 151, 154, 158, 159, 175, 176, 177, 178, 223
 Precedence relations, 80
 predictive parser, 69, 71, 73, 75, 78, 128
 prefix, 36, 42, 45, 53, 55, 56, 70, 71, 81, 101, 107, 113, 114, 147, 151, 157
 Preprocessors, 17, 26, 27
 Programs, 29, 223

Q

Quadruple, 160, 161, 162, 176, 178

R

r-value, 172, 191, 237
 re-locatable object modules, 224, 250
 reaching definitions, 211
 Recognizer, 54
 Recursive descent parsers, 71
 recursive procedures, 71, 186
 recursive production, 69, 70, 74

reduce-reduce conflict, 110, 119, 120, 128
Reduction, 78, 164, 169, 207, 210, 241, 249
reference counting, 196, 201
register allocation, 9, 221, 223, 227, 247, 249
register descriptor, 235, 248, 249
regular expression, 24, 37, 38, 39, 40, 53, 64, 65
return sequence, 173
right-recursive production, 69

S

S-attributed definition, 143, 150, 151, 152, 163
scanner, 12, 16, 22, 29, 30, 31, 32, 33, 38, 42, 47, 50, 61, 62, 65, 66, 127, 135, 198
Scanner generators, 24
scanning, 9, 12, 30, 37, 196
semantic actions, 69, 70, 135, 141, 143, 144, 145, 151, 152, 177
semantic analysis, 13, 16, 17, 26, 139, 143, 151, 153, 192, 200
Semantic stack allocation, 229
shift-reduce conflict, 110, 125, 128
shift-reduce parsers, 78, 100
simple precedence grammar, 99
SLR(1) parser, 110, 113, 114, 117, 127, 128, 129
stack allocation, 180, 186, 200, 201, 229
static allocation, 180, 186, 202
static binding, 186
Static memory allocation, 186, 200, 201
Strength reduction, 241
strength reduction, 15, 204, 210, 218, 219, 241
Strings, 34, 42
structural equivalence, 140, 151
Symbol table, 180, 191, 192, 200
symbol table, 9, 16, 27, 30, 32, 33, 42, 134, 160, 161, 162, 180, 191, 192, 193, 194, 195, 202, 223, 224, 233, 235, 248
syntax analysis, 9, 13, 15, 24, 26, 65, 145, 147
Syntax directed translation engines, 24
Syntax tree, 140, 141, 155
syntax-directed translation, 16, 135, 144, 145, 151, 152, 153, 159, 162

synthesized attributes, 135, 143, 144, 145, 150, 152
system programs, 17
Systematic method, 150

T

template matching, 24
terminal symbol, 32, 34, 70, 74, 78, 113, 131, 135, 144, 145
Thompson's algorithm, 64
Three-address code, 159, 176
three-address code, 15, 159, 160, 162, 163, 170, 171, 174, 176, 223, 230, 237, 242, 248, 250
thunks, 191
tokens, 11, 12, 13, 24, 29, 30, 31, 32, 33, 37, 39, 40, 42, 45, 54, 61, 62, 63, 65, 66, 70, 72, 73, 78, 80, 101, 104, 105, 110, 121, 127, 134, 148
top-down parsing, 65, 69, 77, 105, 128
top-down translation, 144
tracing, 196, 197, 201
transition diagram, 60
Transitive closure, 34
translation-linking-loading mode, 5, 25
translator, 1, 2, 3, 7, 17, 19, 23, 24, 25, 26, 27, 29, 37, 143
translator-writing systems, 24, 26
Triple, 161, 162, 176, 178
Type-2 grammar, 64
Type-3 grammar, 64

U

u-d chains, 212, 219

V

value number, 157, 161
variable propagation, 204, 211, 218
very busy expression, 220
virtual register span, 249

Compiler Design

The book *Compiler Design* is a complete text for exploring the design and working of modern compilers. The book is of immense use for the graduate and undergraduate students of computer science, engineering, and information technology. The book uses lucid language along with examples on every topic for better grasp of the concepts. It explains the concepts of compiling and program translation in detail. Various phases of the modern compiler have been discussed in separate chapters, thereby, presenting a clear distinctive role performed by the compiler. The topics in the book have been supplemented with adequate solved examples, review questions, and objective type questions, which will help students and practitioners to sharpen their learning. The book follows systematic approach for explaining the concepts and uses an algorithmic approach wherever needed.

The book is useful for students, system programmers, professionals, and practitioners interested in language design and translation, as well as for those interested in understanding machine architecture and its implementation.

The salient features of this book are:

- Provides solved examples, multiple-choice questions, and review questions at the end of each chapter
- Helps students who are preparing for university and competitive examinations
- Uses easy language to help students grasp important topics
- Provides a summary at the end of each chapter to refresh learning and improve understanding

About the Authors

Rajesh K. Maurya holds the M.Sc., MCA, M.Tech., and M.Phil. degrees. He also holds a Postgraduate Diploma in Higher Education (PGDHE). He is currently Associate Professor at DES's Navinchandra Mehta Institute of Technology and Development, Mumbai, where he is involved in teaching postgraduate courses in computer science, application, and technology. He is the author of various books, such as *Computer Graphics*, *Computer Graphics with Virtual Reality Systems*, and *System Programming and Compiler Construction*. His research interests include information management, image processing, and intelligent machines. He is a member in high-level academic bodies at various levels, including universities and industries.

The proficient team at **Kogent Learning Solutions Inc.** and **Dreamtech Press** has seized the market of engineering textbooks, bringing excellent content in engineering and technical education to the fore. The team at Kogent Learning Solutions Inc., and Dreamtech Press is committed to providing excellence in quality of content by judiciously analyzing the needs of its readers and ensuring dedication of its authors and editors in catering to these needs.

CATEGORY:
*Computer Science/
Engineering/Technology*

USER LEVEL:
Beginner to Intermediate

978-93-5004-215-1



9 789350 042151

Published by:

dreamtech
PRESS

19-A, Ansari Road, Daryaganj, New Delhi-110002

Tel.: 91-11-23284212, 23243075

Fax: 91-11-23243078

Email: feedback@dreamtechpress.com

www.dreamtechpress.com

Blog: <http://dreamtechpress.wordpress.com>

DREAMTECH
ENGINEERING TEXTBOOKS